SPT Interface

for S32R41

Rev.0.8.10/16.12.2024

Chapter 1

Module Index

1.1 Software Modules

Here is a list of all modules:

Chapter 2

Data Structure Index

2.1 Data Structures

Here are the data structures with brief descriptions:

Chapter 3

Module Documentation

3.1 SPT

Modules

- SPT Driver API

  Describes the functions, data types and structures that make up the SPT Driver user interface. Proper usage of the SPT Driver is described in spt_ug.
- SPT Kernels API

  *The **SPT Kernels** are assembly-style computationally-intensive instruction sets intended to be executed by the SPT hardware.*

3.1.1 Detailed Description

3.2 SPT Driver API

Describes the functions, data types and structures that make up the SPT Driver user interface. Proper usage of the SPT Driver is described in spt_ug.

Modules

- SPT Driver Constants

  SPT Driver Constants.
- SPT Driver Data Types

  SPT Driver Data Types.
- SPT Driver Functions

  SPT Driver Functions.

3.2.1 Detailed Description

Describes the functions, data types and structures that make up the SPT Driver user interface. Proper usage of the SPT Driver is described in spt_ug.

3.3 SPT Driver Constants

SPT Driver Constants.

Modules

- SPT Error codes

Macros

- #define SPT_MAX_N_PAR (10u)

  Maximum number of input parameters to an SPT kernel.
- #define SPT_KERNEL_WATERMARK (0x00005253444BULL)

  Kernel code watermarking pattern.
- #define SPT_CMD_RESULT_SIZE (8u)

  SPT command maximum result size.
- #define SPT_CODE_ADDR_ALIGN_BYTES (16u)

  Alignment constraint for start address of the SPT code.
- #define SPT_DATA_ADDR_ALIGN_BYTES (8u)

  Alignment constraint for start address of the SPT data buffers.

3.3.1 Detailed Description

SPT Driver Constants.

3.3.2 Macro Definition Documentation

3.3.2.1 SPT_CMD_RESULT_SIZE

```
#define SPT_CMD_RESULT_SIZE (8u)
```

SPT command maximum result size.

It is used to define the size of the Spt_DriverCmdResType structure.

3.3.2.2 SPT_CODE_ADDR_ALIGN_BYTES

```
#define SPT_CODE_ADDR_ALIGN_BYTES (16u)
```

Alignment constraint for start address of the SPT code.

3.3.2.3 SPT_DATA_ADDR_ALIGN_BYTES

```
#define SPT_DATA_ADDR_ALIGN_BYTES (8u)
```

Alignment constraint for start address of the SPT data buffers.

3.3.2.4 SPT_KERNEL_WATERMARK

```
#define SPT_KERNEL_WATERMARK (0x00005253444BULL)
```

Kernel code watermarking pattern.

It must be used by all SPT kernels which are launched by RSDK SPT Driver, by placing it in a mandatory first instruction: "set #SPT_KERNEL_WATERMARK, WR_0". This helps protect against passing wrong code memory pointers to the SPT command sequencer.

3.3.2.5 SPT_MAX_N_PAR

```
#define SPT_MAX_N_PAR (10u)
```

Maximum number of input parameters to an SPT kernel.

It is used to define the size of Spt_DriverContextType::kernelParList array.

3.4 SPT Driver Data Types

SPT Driver Data Types.

Data Structures

- struct Spt_DspCmdType

  This structure packs the information needed to call a function on the BBE32 .
- struct Spt_DrvParamType

  Datatype describing the parameters to be passed to the SPT kernel. See spt_call_conv for details.
- struct Spt_DriverContextType

  Contains runtime parameters for the SPT Driver.
- struct Spt_DriverInitPlatSpecificType

  Init parameters which are specific to the hardware platform.
- struct Spt_DriverInitType

  Contains initialization parameters for the SPT Driver.
- struct Spt_DriverCmdResType

  Typedef for returning information from Spt_Command(); to be used as argument to the function.
- struct Spt_DriverCommandType

  Structure containing the ID of the command to be served and additional parameters that may be needed for Spt_Command().

Typedefs

- typedef void(∗ Spt_IsrCbType) (rsdkStatus_t isrStatus, uint32 errInfo)

  Definition of callback function type to be called by the SPT interrupt handlers.
- typedef uintptr_t Spt_DriverCommandParamType

Enumerations

- enum Spt_ParamType { SPT_PARAM_TYPE_NOTINIT = 0U , SPT_PARAM_TYPE_ADDR , SPT_PARAM_TYPE_VALUE , SPT_PARAM_TYPE_LAST }

  Specifies the type of parameter to be passed to the SPT kernel.
- enum Spt_DriverOpModeType { SPT_OP_MODE_BLOCK = 0U , SPT_OP_MODE_NONBLOCK }

  Datatype coding the operating modes for the SPT Driver.
- enum Spt_DriverCommandIdType { SPT_CMD_NONE = 0U , SPT_CMD_MEM_ERR_INJECT_EN , SPT_CMD_MEM_ERR_INJECT_DIS , SPT_CMD_TRIGGER_SW_EVENT , SPT_CMD_GEN_DSP_CMD_CRC , SPT_CMD_BBE32_REBOOT }

  Defines the command codes for Spt_Command().

3.4.1 Detailed Description

SPT Driver Data Types.

Persistent data ELF section

Internals of SPT driver make use of persistent data. All persistent data of SPT driver is grouped under .RSDK_SPT_DRV_MEM_PER section. It is application-level linker file responsibility to place this section in a proper memory location, with read/write priviledges.

3.4.2 Typedef Documentation

3.4.2.1 Spt_DriverCommandParamType

```
typedef uintptr_t Spt_DriverCommandParamType
```

3.4.2.2 Spt_IsrCbType

```
typedef void(* Spt_IsrCbType) (rsdkStatus_t isrStatus, uint32 errInfo)
```

Definition of callback function type to be called by the SPT interrupt handlers.

This callback is offered to the user for additional processing during the SPT interrupts.

Parameters

in	*isrStatus*	- the callback receives a status field from the SPT interrupt handler, that can signal either successful execution of the SPT or an error.
in	*errInfo*	- detailed error information associated to each Std_ReturnType code (e.g. content of the corresponding error status register).

Returns

nothing

3.4.3 Enumeration Type Documentation

3.4.3.1 Spt_DriverCommandIdType

`enum Spt_DriverCommandIdType`

Defines the command codes for Spt_Command().

Enumerator

SPT_CMD_NONE	dummy value to avoid empty enum declaration
SPT_CMD_MEM_ERR_INJECT_EN	Enable injection of parity errors in the SPT OPRAM and TRAM. Parity bits will be altered on subsequent write accesses to memory. Then, parity errors will be generated on memory read-back
SPT_CMD_MEM_ERR_INJECT_DIS	Disable injection of parity errors in the SPT OPRAM and TRAM.
SPT_CMD_TRIGGER_SW_EVENT	Trigger and event on the SPT software event line (SW_EVTREG register), to be used by the SPT WAIT instruction
SPT_CMD_GEN_DSP_CMD_CRC	Compute an 8-bit CRC of the DSP command, accounting for length and endiannes to match with the verification on BBE32 side. The command parameter must be a pointer of type Spt_DspCmdType. The result is passed in Spt_DspCmdType::crc field. See the DSP Calling Convention
SPT_CMD_BBE32_REBOOT	

3.4.3.2 Spt_DriverOpModeType

`enum Spt_DriverOpModeType`

Datatype coding the operating modes for the SPT Driver.

Spt_Run() can execute either in blocking (polling) or non-blocking (interrupt) modes.

Enumerator

SPT_OP_MODE_BLOCK	Blocking call (polling on the SPT_CS_STATUS0[PS_STOP] bit)
SPT_OP_MODE_NONBLOCK	Non-blocking call (triggering an interrupt on SPT_CS_STATUS0[PS_STOP] bit)

3.4.3.3 Spt_ParamType

`enum Spt_ParamType`

Specifies the type of parameter to be passed to the SPT kernel.

Enum fields are used to initialize Spt_DriverContextType::kernelParList.

User-space address arguments must be pre-processed by the SPT Driver before passing them to the SPT hardware, hence the need for differentiation between 'address' and 'data' argument types.

Attention

The SPT hardware instructions cannot handle full-width 32bit addresses, instead they work with an implicit memory base address (which is RSDK_SPT_CUBE_BASE_ADDR or RSDK_SPT_OTHER_BASE_ADDR) and an address offset on maximum 23 bits.

See the spt_call_conv for details.

Enumerator

SPT_PARAM_TYPE_NOTINIT	Parameter is not initialized
SPT_PARAM_TYPE_ADDR	Parameter represents a memory address. NOTE that all system memory data addresses which are passed to the SPT must be aligned to an SPT_CODE_ADDR_ALIGN_BYTES boundary
SPT_PARAM_TYPE_VALUE	Parameter represents a scalar value. The SPT driver splits this 32-bit value, then copies the 8 most significant bits into the imaginary part of a Work Register and the remaining 24 bits into the real part of the same WR.
SPT_PARAM_TYPE_LAST	Signals the end of the SPT parameter list. The last member in Spt_DriverContextType::kernelParList must have this type.

3.5 SPT Driver Functions

SPT Driver Functions.

Functions

- Std_ReturnType Spt_Setup (Spt_DriverInitType const ∗const pSptInitInfo)

This function initializes the internal state of the SPT Driver and the hardware registers of the SPT in preparation for running SPT kernel functions.

- Std_ReturnType Spt_Run (Spt_DriverContextType const ∗const sptContext)

  Configures the SPT kernel parameters, triggers SPT execution, feeds back status information to the caller.

- Std_ReturnType Spt_Command (Spt_DriverCommandType const ∗const pSptCommand, Spt_DriverCmdResType ∗const pSptCmdResult)

  Used to handle asynchronous control or status requests, apart from the SPT kernel processing sequence.

- Std_ReturnType Spt_Stop (void)

  This function stops the SPT processing, brings the hardware and the Driver back into reset state, ready for a new initialization.

3.5.1 Detailed Description

SPT Driver Functions.

-

3.5.2 Function Documentation

3.5.2.1 Spt_Command()

```
Std_ReturnType Spt_Command (
            Spt_DriverCommandType const *const pSptCommand,
            Spt_DriverCmdResType *const pSptCmdResult )
```

Used to handle asynchronous control or status requests, apart from the SPT kernel processing sequence.

See Spt_DriverCommandIdType for details about the supported commands.

Parameters

in	*pSptCommand*	- pointer to a structure containing the ID of the command to be served and additional parameters that may be needed.
out	*pSptCmdResult*	- container for the returned information.

Returns

success or error status information.

Precondition

It must be called only after Spt_Setup() has been executed successfully at least once. If the SPT signals a hardware error (through the Spt_DriverContextType::ecsIsrCb), then it is recommended to re-run Spt_Setup() before calling Spt_Command().

3.5.2.2 Spt_Run()

```
Std_ReturnType Spt_Run (
            Spt_DriverContextType const *const sptContext )
```

Configures the SPT kernel parameters, triggers SPT execution, feeds back status information to the caller.

It checks the validity of the input arguments, then parses the SPT Spt_DriverContextType::kernelParList, writes the parameters into the SPT working registers (see spt_call_conv), starts the SPT, then it either waits for SPT completion (blocking mode) or exits immediately (non-blocking mode). See also the spt_rm_running for details.

Parameters

in	*sptContext*	- pointer to the SPT runtime parameters structure

Returns

success or error status information.

Precondition

It must be called only after Spt_Setup() has been executed successfully at least once. If the SPT signals a hardware error (through the Spt_DriverContextType::ecsIsrCb), then it is recommended to re-run Spt_Setup() before calling Spt_Run()

3.5.2.3 Spt_Setup()

```
Std_ReturnType Spt_Setup (
            Spt_DriverInitType const *const pSptInitInfo )
```

This function initializes the internal state of the SPT Driver and the hardware registers of the SPT in preparation for running SPT kernel functions.

It checks the validity of the input arguments, then writes them into the SPT registers. It does not modify the input argument. It also enables boot-up of the BBE32 DSP, if Spt_DriverInitType::dspEn is not set to 0.

Parameters

in	*pSptInitInfo*	- pointer to the SPT Driver initialization structure.

Returns

success or error status information.

Precondition

It must be called in the following situations:

- during the first setup of the SPT Driver (e.g. after boot),

- whenever the system parameters described in Spt_DriverInitType need to be changed
- everytime the Spt_Run() returns an SPT hardware error

3.5.2.4 Spt_Stop()

```
Std_ReturnType Spt_Stop (
             void  )
```

This function stops the SPT processing, brings the hardware and the Driver back into reset state, ready for a new initialization.

It checks the validity of the input arguments, then stops the SPT sequencer, data acquisition and waits until the SPT transitions to 'reset' state.

Returns

success or error status information.

Precondition

It must be called only after Spt_Setup() has been executed successfully at least once, to prepare for reinitialization or to recover from a SPT hardware error.

3.6 SPT Kernels API

The **SPT Kernels** are assembly-style computationally-intensive instruction sets intended to be executed by the SPT hardware.

Modules

- SPT Kernels Constants

  SPT Kernels Constants.
- SPT Kernels Data Types

  SPT Kernels Data Types.
- SPT Kernels Functions

  SPT Kernels Functions.

3.6.1 Detailed Description

The **SPT Kernels** are assembly-style computationally-intensive instruction sets intended to be executed by the SPT hardware.

The *SPT kernels* are designed for the following use-cases:

- 512 samples/chirp, 128 chirps, 4 physical channels
- 256 samples/chirp, 256 chirps, 4 physical channels
- 1024 samples/chirp, 128 chirps, 4 physical channels
- 256 samples/chirp, 256 chirps, 4 physical channels, 2TX TD-MIMO
- 256 samples/chirp, 128 chirps, 4 physical channels, 3TX TD-MIMO
- 1024 samples/chirp, 64 chirps, 4 physical channels, 4TX TD-MIMO
- 1024 samples/chirp, 64 chirps, 4 physical channels
- 2048 samples/chirp, 256 chirps, 4 physical channels
- 512 samples/chirp, 128 chirps, 8 physical channels
- 256 samples/chirp, 256 chirps, 8 physical channels
- 1024 samples/chirp, 128 chirps, 8 physical channels
- 256 samples/chirp, 256 chirps, 8 physical channels, 2TX TD-MIMO
- 256 samples/chirp, 128 chirps, 8 physical channels, 6TX TD-MIMO
- 1024 samples/chirp, 256 chirps, 4 physical channels, 3TX DD-MIMO
- RF Build-in-self-test, 128 samples, 4 channels
- RF Build-in-self-test, 2048 samples, 4 channels

Note

Whatever the use-case, the RSDK Range FFT kernels are designed assuming the size of the SPT acquisition buffer is equal to 2 chirps' worth of data.

The *SPT kernels* are structured in two categories:

- **Initialization kernels**: loading the Twiddle, Window, Beamforming Steering Vector and De-rotation(TD-MIMO only) coefficients in TRAM, once, for a specific use-case:
  - RsdkSptInit512smp128crp4ch()
  - RsdkSptInit256smp256crp4ch()
  - RsdkSptInit1024smp128crp4ch()
  - RsdkSptInit256smp256crp4ch2TxTdMimo()
  - RsdkSptInit256smp128crp4ch3TxTdMimo()
  - RsdkSptInit1024smp64crp4ch4TxTdMimo()
  - RsdkSptInit1024smp64crp4ch()
  - RsdkSptInit2048smp256crp4ch()
  - RsdkSptInit512smp128crp8ch()

  - RsdkSptInit256smp256crp8ch()
  - RsdkSptInit1024smp128crp8ch()
  - RsdkSptInit256smp256crp8ch2TxTdMimo()
  - RsdkSptInit256smp128crp8ch6TxTdMimo()
  - RsdkSptInit1024smp256crp4ch3TxDdMimo()

- **Run-time kernels**, called each Radar frame, for a specific use-case:
  - Range:
    - RsdkSptRange512smp128crp4ch()
    - RsdkSptRange512smp128crp4chAdptv()
    - RsdkSptRange256smp256crp4ch()
    - RsdkSptRange256smp256crp4chAdptv()
    - RsdkSptRange1024smp128crp4chCp4d()
    - RsdkSptRange256smp256crp4ch2TxTdMimoCp4d()
    - RsdkSptRange256smp128crp4ch3TxTdMimo()
    - RsdkSptRange256smp128crp4ch3TxTdMimoAdptv()
    - RsdkSptRange1024smp256crp4ch3TxDdMimoCp4d()
    - RsdkSptRange1024smp64crp4ch4TxTdMimoCp4d()
    - RsdkSptRange1024smp64crp4chCp4d()
    - RsdkSptRange2048smp256crp4chCp4d()
    - RsdkSptRange512smp128crp8ch()
    - RsdkSptRange512smp128crp8chAdptv()
    - RsdkSptRange256smp256crp8ch()
    - RsdkSptRange256smp256crp8chAdptv()
    - RsdkSptRange1024smp128crp8chCp4d()
    - RsdkSptRange256smp256crp8ch2TxTdMimoCp4d()
    - RsdkSptRange256smp128crp8ch6TxTdMimoCp4d()
  - Doppler:
    - RsdkSptDoppler512smp128crp4ch()
    - RsdkSptDoppler512smp128crp4chRcomp()
    - RsdkSptDoppler256smp256crp4ch()
    - RsdkSptDoppler256smp256crp4chRcomp()
    - RsdkSptDoppler1024smp128crp4chCp4d()
    - RsdkSptDoppler256smp256crp4ch2TxTdMimoCp4d()
    - RsdkSptDoppler256smp128crp4ch3TxTdMimo()
    - RsdkSptDoppler256smp128crp4ch3TxTdMimoRcomp()
    - RsdkSptDoppler1024smp256crp4ch3TxDdMimoCp4d()
    - RsdkSptDoppler1024smp64crp4ch4TxTdMimoCp4d()
    - RsdkSptDoppler1024smp64crp4chCp4d()
    - RsdkSptDoppler2048smp256crp4chCp4d()
    - RsdkSptDoppler512smp128crp8ch()
    - RsdkSptDoppler512smp128crp8chRcomp()
    - RsdkSptDoppler256smp256crp8ch()
    - RsdkSptDoppler256smp256crp8chRcomp()
    - RsdkSptDoppler1024smp128crp8chCp4d()
    - RsdkSptDoppler256smp256crp8ch2TxTdMimoCp4d()
    - RsdkSptDoppler256smp128crp8ch6TxTdMimoCp4d()
  - Non Coherent Combing:
    - RsdkSptNcc512smp128crp4ch()

    - RsdkSptNcc256smp256crp4ch()
    - RsdkSptNcc1024smp128crp4chCp4d()
    - RsdkSptNcc256smp256crp4ch2TxTdMimoCp4d()
    - RsdkSptNcc256smp128crp4ch3TxTdMimo()
    - RsdkSptNcc1024smp64crp4ch3TxDdMimoCp4d()
    - RsdkSptNcc1024smp64crp4ch4TxTdMimoCp4d()
    - RsdkSptNcc1024smp64crp4chCp4d()
    - RsdkSptNcc2048smp256crp4chCp4d()
    - RsdkSptNcc512smp128crp8ch()
    - RsdkSptNcc256smp256crp8ch()
    - RsdkSptNcc1024smp128crp8chCp4d()
    - RsdkSptNcc256smp256crp8ch2TxTdMimoCp4d()
    - RsdkSptNcc256smp128crp8ch6TxTdMimoCp4d()
  - 3D FFT:
    - RsdkSpt3Dfft512smp128crp4ch()
    - RsdkSpt3Dfft256smp256crp4ch()
    - RsdkSpt3Dfft1024smp128crp4chCp4d()
    - RsdkSpt3Dfft256smp256crp4ch2TxTdMimoCp4d()
    - RsdkSpt3Dfft256smp128crp4ch3TxTdMimo()
    - RsdkSpt3Dfft1024smp64crp4ch4TxTdMimoCp4d()
    - RsdkSpt3Dfft1024smp64crp4chCp4d()
    - RsdkSpt3Dfft2048smp256crp4chCp4d()
    - RsdkSpt3Dfft512smp128crp8ch()
    - RsdkSpt3Dfft256smp256crp8ch()
    - RsdkSpt3Dfft1024smp128crp8chCp4d()
    - RsdkSpt3Dfft256smp256crp8ch2TxTdMimoCp4d()
    - RsdkSpt3Dfft256smp128crp8ch6TxTdMimoCp4d()
  - Peak Search 1D:
    - RsdkSptPeakSearch1024smp64crp1D()
    - RsdkSptPeakSearch2048smp256crp1D()
  - Peak Search 2D:
    - RsdkSptPeakSearch512smp128crp()
    - RsdkSptPeakSearch256smp256crp()
    - RsdkSptPeakSearch1024smp128crp()
    - RsdkSptPeakSearch256smp128crp()
    - RsdkSptPeakSearch1024smp64crp()
    - RsdkSptPeakSearch2048smp256crp()
  - Digital BeamForming and Direction of Arrival:
    - RsdkSptDbfDoa64Beams4Ch128Peaks()
    - RsdkSptDbfDoa64Beams4Ch128PeaksCp4d()
    - RsdkSptDbfDoa64Beams8Ch128Peaks()
    - RsdkSptDbfDoa64Beams8Ch128PeaksCp4d()
    - RsdkSptDbfDoa64Beams12Ch128Peaks()
    - RsdkSptDbfDoa64Beams12Ch128PeaksCp4d()
    - RsdkSptDbfDoa64Beams16Ch128PeaksCp4d()
    - RsdkSptDbfFFT128Bins12Ch128PeaksCp4d()
    - RsdkSptDbfFFT128Bins48Ch128PeaksCp4d()
  - TRAM persistent memory check:
    - RsdkSptCheckTram512smp128crp4ch()

        - RsdkSptCheckTram256smp256crp4ch()
        - RsdkSptCheckTram1024smp128crp4ch()
        - RsdkSptCheckTram256smp256crp4ch2TxTdMimo()
        - RsdkSptCheckTram256smp128crp4ch3TxTdMimo()
        - RsdkSptCheckTram1024smp64crp4ch4TxTdMimo()
        - RsdkSptCheckTram1024smp64crp4ch()
        - RsdkSptCheckTram2048smp256crp4ch()
        - RsdkSptCheckTram1024smp256crp4ch3TxDdMimo()
        - RsdkSptCheckTram512smp128crp8ch()
        - RsdkSptCheckTram256smp256crp8ch()
        - RsdkSptCheckTram1024smp128crp8ch()
        - RsdkSptCheckTram256smp256crp8ch2TxTdMimo()
        - RsdkSptCheckTram256smp128crp8ch6TxTdMimo()
    - RF Build-in-self-test:
        - RsdkSptRfBist128smp1crp4ch()
        - RsdkSptRfBist2048smp1crp4ch()

For a detailed **Data Flow** please see Kernels_data_flow.

3.7 SPT Kernels Constants

SPT Kernels Constants.

Macros

- #define RSDK_SPT_GET_KERNEL_SIZE(kernelName) ((size_t)&kernelName##_size)

  Return SPT program code sizes in BYTES.
- #define RSDK_SPT_DOUBLE_BUFFER (2)

  Number of chirp buffers to be allocated in SRAM.
- #define RSDK_SPT_PEAK_BITMAP_PACK_SIZE(x) (x>>3)

  Packing of peak detects.
- #define RSDK_SPT_HIST_BINS (64)

  Number of SPT histogram bins.
- #define RSDK_SPT_DOA_TAG (0xFF80)

  Tag value for DOA detect:
- #define RSDK_SPT_DBFDOA_BEAMS_NUM (64)

  Number of beams used for DBF and DOA.
- #define RSDK_SPT_DBFDOA_PEAKS_NUM (128)

  Number of peaks used for DBF and DOA.
- #define RSDK_SPT_DBFFFT_PEAKS_NUM (128)

  Number of peaks used for DBF with FFT:
- #define RSDK_BYTES_CSI2_STATS (80)

  A data layout gap in ADC chirp data to accomodate the MIPICSI2 statistics (measured in bytes).
- #define RSDK_CSI2_STATS_DOUBLE_BUFFER ((RSDK_BYTES_CSI2_STATS∗RSDK_SPT_DOUBLE_BUFFER)>>1)

Number of points for the range FFT:

The SPT modules can be used only when The number of samples per chirp used in the radar application is equal to one of these values.

- #define RSDK_SPT_NP_RANGE2048 (2048)
- #define RSDK_SPT_NP_RANGE1024 (1024)
- #define RSDK_SPT_NP_RANGE512 (512)
- #define RSDK_SPT_NP_RANGE256 (256)

- #define RSDK_SPT_NP_DOPPLER512 (512)

Number of points for the Doppler FFT:

- #define RSDK_SPT_NP_DOPPLER256 (256)
- #define RSDK_SPT_NP_DOPPLER128 (128)
- #define RSDK_SPT_NP_DOPPLER64 (64)

- #define RSDK_SPT_NP_RFBIST128 (128)

Number of points for the RFBist FFT:

- #define RSDK_SPT_NP_RFBIST2048 (2048)

- #define RSDK_SPT_NP_DBF128 (128)

Number of points for the DBF FFT:

- #define RSDK_SPT_NP_DBF512 (512)

- #define RSDK_SPT_NUM_OF_OPERANDS_PER_BANK (8192)

Compute the number of loops for the RsdkSptOsCfar kernel

- #define RSDK_SPT_OSCFAR_NUM_OF_LOOPS(rangeBins, dopplerBins) ((rangeBins $*$ dopplerBins) / RSDK_SPT_NUM_OF_OPERANDS_PER_BANK)

- #define RSDK_SPT_OSCFAR_INPUT_BUF_SIZE(rangeBins, dopplerBins) (rangeBins $*$ dopplerBins)

Size of the OS-CFAR input buffer in uint16 values:

- #define RSDK_SPT_OSCFAR_OUTPUT_BUF_SIZE(rangeBins, dopplerBins) ((rangeBins $*$ dopplerBins) $>>$ 3)

Size of the OS-CFAR output buffer in uint8 values:

3.7.1 Detailed Description

SPT Kernels Constants.

3.7.2 Macro Definition Documentation

3.7.2.1 RSDK_BYTES_CSI2_STATS

```
#define RSDK_BYTES_CSI2_STATS (80)
```

A data layout gap in ADC chirp data to accomodate the MIPICSI2 statistics (measured in bytes).

Each chirp is followed by 80B of MIPI/CSI2 statistics. Buffer size considers double buffering and int16 values.

3.7.2.2 RSDK_CSI2_STATS_DOUBLE_BUFFER

```
#define RSDK_CSI2_STATS_DOUBLE_BUFFER ((RSDK_BYTES_CSI2_STATS*RSDK_SPT_DOUBLE_BUFFER)>>1)
```

3.7.2.3 RSDK_SPT_DBFDOA_BEAMS_NUM

```
#define RSDK_SPT_DBFDOA_BEAMS_NUM (64)
```

Number of beams used for DBF and DOA.

Number of beams used for Digital Beamforming and Direction of Arrival.

3.7.2.4 RSDK_SPT_DBFDOA_PEAKS_NUM

```
#define RSDK_SPT_DBFDOA_PEAKS_NUM (128)
```

Number of peaks used for DBF and DOA.

Number of peaks used for Digital Beamforming and Direction of Arrival.

3.7.2.5 RSDK_SPT_DBFFFT_PEAKS_NUM

```
#define RSDK_SPT_DBFFFT_PEAKS_NUM (128)
```

Number of peaks used for DBF with FFT:

Number of peaks used for Digital Beamforming with FFT.

3.7.2.6 RSDK_SPT_DOA_TAG

```
#define RSDK_SPT_DOA_TAG (0xFF80)
```

Tag value for DOA detect:

Beamforming output is tag/magnitude per each beam.

3.7.2.7 RSDK_SPT_DOUBLE_BUFFER

```
#define RSDK_SPT_DOUBLE_BUFFER (2)
```

Number of chirp buffers to be allocated in SRAM.

The SPT modules can be used only when The number of chirp buffers in SRAM used in the radar application is equal to this values.

3.7.2.8 RSDK_SPT_GET_KERNEL_SIZE

```
#define RSDK_SPT_GET_KERNEL_SIZE(
              kernelName ) ((size_t)&kernelName##_size)
```

Return SPT program code sizes in BYTES.

Used for relocation to SRAM. Use it in a .c file like: memcpy(gSptModuleCodeRelocBuf, (void∗)sptKernelLoadAddr, RSDK_SPT_GET_KERNEL_SIZE(RsdkSptInit512smp128crp4ch));

3.7.2.9 RSDK_SPT_HIST_BINS

```
#define RSDK_SPT_HIST_BINS (64)
```

Number of SPT histogram bins.

Number of SPT histogram bins.

3.7.2.10 RSDK_SPT_NP_DBF128

```
#define RSDK_SPT_NP_DBF128 (128)
```

Number of points for the DBF FFT:

FFT length for the Digital Beamforming.

3.7.2.11 RSDK_SPT_NP_DBF512

```
#define RSDK_SPT_NP_DBF512 (512)
```

3.7.2.12 RSDK_SPT_NP_DOPPLER128

```
#define RSDK_SPT_NP_DOPPLER128 (128)
```

3.7.2.13 RSDK_SPT_NP_DOPPLER256

```
#define RSDK_SPT_NP_DOPPLER256 (256)
```

3.7.2.14 RSDK_SPT_NP_DOPPLER512

```
#define RSDK_SPT_NP_DOPPLER512 (512)
```

Number of points for the Doppler FFT:

The SPT modules can be used only when the number of chirps per acquisition frame used in the radar application is equal to one of these values.

3.7.2.15 RSDK_SPT_NP_DOPPLER64

```
#define RSDK_SPT_NP_DOPPLER64 (64)
```

3.7.2.16 RSDK_SPT_NP_RANGE1024

```
#define RSDK_SPT_NP_RANGE1024 (1024)
```

3.7.2.17 RSDK_SPT_NP_RANGE2048

```
#define RSDK_SPT_NP_RANGE2048 (2048)
```

3.7.2.18 RSDK_SPT_NP_RANGE256

```
#define RSDK_SPT_NP_RANGE256 (256)
```

3.7.2.19 RSDK_SPT_NP_RANGE512

```
#define RSDK_SPT_NP_RANGE512 (512)
```

3.7.2.20 RSDK_SPT_NP_RFBIST128

```
#define RSDK_SPT_NP_RFBIST128 (128)
```

Number of points for the RFBist FFT:

The SPT modules can be used only when The number of samples per chirp used in the radar application is equal to one of these values.

3.7.2.21 RSDK_SPT_NP_RFBIST2048

```
#define RSDK_SPT_NP_RFBIST2048 (2048)
```

3.7.2.22 RSDK_SPT_NUM_OF_OPERANDS_PER_BANK

```
#define RSDK_SPT_NUM_OF_OPERANDS_PER_BANK (8192)
```

Compute the number of loops for the RsdkSptOsCfar kernel

1 ORAM bank (8192) operands processed each loop with a total number of rangeBins $*$ dopplerBins operands

3.7.2.23 RSDK_SPT_OSCFAR_INPUT_BUF_SIZE

```
#define RSDK_SPT_OSCFAR_INPUT_BUF_SIZE(
             rangeBins,
             dopplerBins ) (rangeBins * dopplerBins)
```

Size of the OS-CFAR input buffer in uint16 values:

Can be used by the upper layers at application level to allocate memory space for OS-CFAR input.

3.7.2.24 RSDK_SPT_OSCFAR_NUM_OF_LOOPS

```
#define RSDK_SPT_OSCFAR_NUM_OF_LOOPS(
              rangeBins,
              dopplerBins ) ((rangeBins * dopplerBins) / RSDK_SPT_NUM_OF_OPERANDS_PER_BANK)
```

3.7.2.25 RSDK_SPT_OSCFAR_OUTPUT_BUF_SIZE

```
#define RSDK_SPT_OSCFAR_OUTPUT_BUF_SIZE(
              rangeBins,
              dopplerBins ) ((rangeBins * dopplerBins) >> 3)
```

Size of the OS-CFAR output buffer in uint8 values:

Can be used by the upper layers at application level to allocate memory space for OS-CFAR output.

3.7.2.26 RSDK_SPT_PEAK_BITMAP_PACK_SIZE

```
#define RSDK_SPT_PEAK_BITMAP_PACK_SIZE(
              x ) (x>>3)
```

Packing of peak detects.

Peak search output is bitmap - 8 detect tags per byte.

3.8 SPT Kernels Data Types

SPT Kernels Data Types.

Variables

- char RSDK_SPT_CUBE_BASE_ADDR [ ]
- char RSDK_SPT_OTHER_BASE_ADDR [ ]
- char RsdkSptInit512smp128crp4ch_size
  Kernel size.
- char RsdkSptInit256smp256crp4ch_size
  Kernel size.
- char RsdkSptInit1024smp64crp4ch_size
  Kernel size.
- char RsdkSptInit1024smp128crp4ch_size
  Kernel size.
- char RsdkSptInit2048smp256crp4ch_size
  Kernel size.
- char RsdkSptInit256smp256crp4ch2TxTdMimo_size
  Kernel size.

- char RsdkSptInit256smp128crp4ch3TxTdMimo_size
  Kernel size.
- char RsdkSptInit1024smp64crp4ch4TxTdMimo_size
  Kernel size.
- char RsdkSptInit1024smp256crp4ch3TxDdMimo_size
  Kernel size.
- char RsdkSptInit512smp128crp8ch_size
  Kernel size.
- char RsdkSptInit256smp256crp8ch_size
  Kernel size.
- char RsdkSptInit1024smp128crp8ch_size
  Kernel size.
- char RsdkSptInit256smp256crp8ch2TxTdMimo_size
  Kernel size.
- char RsdkSptInit256smp128crp8ch6TxTdMimo_size
  Kernel size.
- char RsdkSptRange512smp128crp4ch_size
  Kernel size.
- char RsdkSptRange512smp128crp4chAdptv_size
  Kernel size.
- char RsdkSptRange256smp256crp4ch_size
  Kernel size.
- char RsdkSptRange256smp256crp4chAdptv_size
  Kernel size.
- char RsdkSptRange1024smp64crp4chCp4d_size
  Kernel size.
- char RsdkSptRange1024smp128crp4chCp4d_size
  Kernel size.
- char RsdkSptRange2048smp256crp4chCp4d_size
  Kernel size.
- char RsdkSptRange256smp256crp4ch2TxTdMimoCp4d_size
  Kernel size.
- char RsdkSptRange256smp128crp4ch3TxTdMimo_size
  Kernel size.
- char RsdkSptRange256smp128crp4ch3TxTdMimoAdptv_size
  Kernel size.
- char RsdkSptRange1024smp64crp4ch4TxTdMimoCp4d_size
  Kernel size.
- char RsdkSptRange1024smp256crp4ch3TxDdMimoCp4d_size
  Kernel size.
- char RsdkSptRange512smp128crp8ch_size
  Kernel size.
- char RsdkSptRange512smp128crp8chAdptv_size
  Kernel size.
- char RsdkSptRange256smp256crp8ch_size
  Kernel size.
- char RsdkSptRange256smp256crp8chAdptv_size
  Kernel size.
- char RsdkSptRange1024smp128crp8chCp4d_size
  Kernel size.
- char RsdkSptRange256smp256crp8ch2TxTdMimoCp4d_size

Kernel size.

- char RsdkSptRange256smp128crp8ch6TxTdMimoCp4d_size

  Kernel size.
- char RsdkSptDoppler512smp128crp4ch_size

  Kernel size.
- char RsdkSptDoppler512smp128crp4chRcomp_size

  Kernel size.
- char RsdkSptDoppler256smp256crp4ch_size

  Kernel size.
- char RsdkSptDoppler256smp256crp4chRcomp_size

  Kernel size.
- char RsdkSptDoppler1024smp64crp4chCp4d_size

  Kernel size.
- char RsdkSptDoppler1024smp128crp4chCp4d_size

  Kernel size.
- char RsdkSptDoppler2048smp256crp4chCp4d_size

  Kernel size.
- char RsdkSptDoppler256smp256crp4ch2TxTdMimoCp4d_size

  Kernel size.
- char RsdkSptDoppler256smp128crp4ch3TxTdMimo_size

  Kernel size.
- char RsdkSptDoppler256smp128crp4ch3TxTdMimoRcomp_size

  Kernel size.
- char RsdkSptDoppler1024smp256crp4ch3TxDdMimoCp4d_size

  Kernel size.
- char RsdkSptDoppler1024smp64crp4ch4TxTdMimoCp4d_size

  Kernel size.
- char RsdkSptDoppler512smp128crp8ch_size

  Kernel size.
- char RsdkSptDoppler512smp128crp8chRcomp_size

  Kernel size.
- char RsdkSptDoppler256smp256crp8ch_size

  Kernel size.
- char RsdkSptDoppler256smp256crp8chRcomp_size

  Kernel size.
- char RsdkSptDoppler1024smp128crp8chCp4d_size

  Kernel size.
- char RsdkSptDoppler256smp256crp8ch2TxTdMimoCp4d_size

  Kernel size.
- char RsdkSptDoppler256smp128crp8ch6TxTdMimoCp4d_size

  Kernel size.
- char RsdkSptNcc512smp128crp4ch_size

  Kernel size.
- char RsdkSptNcc256smp256crp4ch_size

  Kernel size.
- char RsdkSptNcc1024smp64crp4chCp4d_size

  Kernel size.
- char RsdkSptNcc1024smp128crp4chCp4d_size

  Kernel size.
- char RsdkSptNcc2048smp256crp4chCp4d_size

  Kernel size.

- char RsdkSptNcc256smp256crp4ch2TxTdMimoCp4d_size
  Kernel size.
- char RsdkSptNcc256smp128crp4ch3TxTdMimo_size
  Kernel size.
- char RsdkSptNcc1024smp64crp4ch4TxTdMimoCp4d_size
  Kernel size.
- char RsdkSptNcc1024smp64crp4ch3TxDdMimoCp4d_size
  Kernel size.
- char RsdkSptNcc512smp128crp8ch_size
  Kernel size.
- char RsdkSptNcc256smp256crp8ch_size
  Kernel size.
- char RsdkSptNcc1024smp128crp8chCp4d_size
  Kernel size.
- char RsdkSptNcc256smp256crp8ch2TxTdMimoCp4d_size
  Kernel size.
- char RsdkSptNcc256smp128crp8ch6TxTdMimoCp4d_size
  Kernel size.
- char RsdkSpt3Dfft512smp128crp4ch_size
  Kernel size.
- char RsdkSpt3Dfft256smp256crp4ch_size
  Kernel size.
- char RsdkSpt3Dfft1024smp128crp4chCp4d_size
  Kernel size.
- char RsdkSpt3Dfft1024smp64crp4chCp4d_size
  Kernel size.
- char RsdkSpt3Dfft2048smp256crp4chCp4d_size
  Kernel size.
- char RsdkSpt3Dfft256smp256crp4ch2TxTdMimoCp4d_size
  Kernel size.
- char RsdkSpt3Dfft256smp128crp4ch3TxTdMimo_size
  Kernel size.
- char RsdkSpt3Dfft1024smp64crp4ch4TxTdMimoCp4d_size
  Kernel size.
- char RsdkSpt3Dfft512smp128crp8ch_size
  Kernel size.
- char RsdkSpt3Dfft256smp256crp8ch_size
  Kernel size.
- char RsdkSpt3Dfft1024smp128crp8chCp4d_size
  Kernel size.
- char RsdkSpt3Dfft256smp256crp8ch2TxTdMimoCp4d_size
  Kernel size.
- char RsdkSpt3Dfft256smp128crp8ch6TxTdMimoCp4d_size
  Kernel size.
- char RsdkSptPeakSearch512smp128crp_size
  Kernel size.
- char RsdkSptPeakSearch256smp256crp_size
  Kernel size.
- char RsdkSptPeakSearch1024smp128crp_size
  Kernel size.
- char RsdkSptPeakSearch256smp128crp_size

Kernel size.

- char RsdkSptPeakSearch1024smp64crp_size

  Kernel size.
- char RsdkSptPeakSearch1024smp64crp1D_size

  Kernel size.
- char RsdkSptPeakSearch2048smp256crp_size

  Kernel size.
- char RsdkSptPeakSearch2048smp256crp1D_size

  Kernel size.
- char RsdkSptDbfDoa64Beams4Ch128Peaks_size

  Kernel size.
- char RsdkSptDbfDoa64Beams4Ch128PeaksCp4d_size

  Kernel size.
- char RsdkSptDbfDoa64Beams8Ch128Peaks_size

  Kernel size.
- char RsdkSptDbfDoa64Beams8Ch128PeaksCp4d_size

  Kernel size.
- char RsdkSptDbfDoa64Beams12Ch128Peaks_size

  Kernel size.
- char RsdkSptDbfDoa64Beams12Ch128PeaksCp4d_size

  Kernel size.
- char RsdkSptDbfDoa64Beams16Ch128PeaksCp4d_size

  Kernel size.
- char RsdkSptDbfFFT128Bins12Ch128PeaksCp4d_size

  Kernel size.
- char RsdkSptDbfFFT128Bins48Ch128PeaksCp4d_size

  Kernel size.
- char RsdkSptCheckTram512smp128crp4ch_size

  Kernel size.
- char RsdkSptCheckTram256smp256crp4ch_size

  Kernel size.
- char RsdkSptCheckTram1024smp128crp4ch_size

  Kernel size.
- char RsdkSptCheckTram1024smp64crp4ch_size

  Kernel size.
- char RsdkSptCheckTram2048smp256crp4ch_size

  Kernel size.
- char RsdkSptCheckTram256smp256crp4ch2TxTdMimo_size

  Kernel size.
- char RsdkSptCheckTram256smp128crp4ch3TxTdMimo_size

  Kernel size.
- char RsdkSptCheckTram1024smp64crp4ch4TxTdMimo_size

  Kernel size.
- char RsdkSptCheckTram1024smp256crp4ch3TxDdMimo_size

  Kernel size.
- char RsdkSptCheckTram512smp128crp8ch_size

  Kernel size.
- char RsdkSptCheckTram256smp256crp8ch_size

  Kernel size.
- char RsdkSptCheckTram1024smp128crp8ch_size

  Kernel size.

- char RsdkSptCheckTram256smp256crp8ch2TxTdMimo_size
  Kernel size.
- char RsdkSptCheckTram256smp128crp8ch6TxTdMimo_size
  Kernel size.
- char RsdkSptRfBist128smp1crp4ch_size
  Kernel size.
- char RsdkSptRfBist2048smp1crp4ch_size
  Kernel size.
- char RsdkSptMemRwErrInject_size
  Kernel size.
- char RsdkSptDspExampleDirectBlocking_size
  Kernel size.
- char RsdkSptDspExampleIndirectBlocking_size
  Kernel size.
- char RsdkSptOsCfar_size
  Kernel size.

3.8.1 Detailed Description

SPT Kernels Data Types.

3.8.2 Variable Documentation

3.8.2.1 RSDK_SPT_CUBE_BASE_ADDR

```
char RSDK_SPT_CUBE_BASE_ADDR[]  [extern]
```

"RSDK_SPT_CUBE_BASE_ADDR"

3.8.2.2 RSDK_SPT_OTHER_BASE_ADDR

```
char RSDK_SPT_OTHER_BASE_ADDR[]  [extern]
```

"RSDK_SPT_OTHER_BASE_ADDR"

3.8.2.3 RsdkSpt3Dfft1024smp128crp4chCp4d_size

```
char RsdkSpt3Dfft1024smp128crp4chCp4d_size  [extern]
```

Kernel size.

3.8.2.4 RsdkSpt3Dfft1024smp128crp8chCp4d_size

```
char RsdkSpt3Dfft1024smp128crp8chCp4d_size  [extern]
```

Kernel size.

3.8.2.5 RsdkSpt3Dfft1024smp64crp4ch4TxTdMimoCp4d_size

```
char RsdkSpt3Dfft1024smp64crp4ch4TxTdMimoCp4d_size  [extern]
```

Kernel size.

3.8.2.6 RsdkSpt3Dfft1024smp64crp4chCp4d_size

```
char RsdkSpt3Dfft1024smp64crp4chCp4d_size  [extern]
```

Kernel size.

3.8.2.7 RsdkSpt3Dfft2048smp256crp4chCp4d_size

```
char RsdkSpt3Dfft2048smp256crp4chCp4d_size  [extern]
```

Kernel size.

3.8.2.8 RsdkSpt3Dfft256smp128crp4ch3TxTdMimo_size

```
char RsdkSpt3Dfft256smp128crp4ch3TxTdMimo_size  [extern]
```

Kernel size.

3.8.2.9 RsdkSpt3Dfft256smp128crp8ch6TxTdMimoCp4d_size

```
char RsdkSpt3Dfft256smp128crp8ch6TxTdMimoCp4d_size  [extern]
```

Kernel size.

3.8.2.10 RsdkSpt3Dfft256smp256crp4ch2TxTdMimoCp4d_size

```
char RsdkSpt3Dfft256smp256crp4ch2TxTdMimoCp4d_size [extern]
```

Kernel size.

3.8.2.11 RsdkSpt3Dfft256smp256crp4ch_size

```
char RsdkSpt3Dfft256smp256crp4ch_size [extern]
```

Kernel size.

3.8.2.12 RsdkSpt3Dfft256smp256crp8ch2TxTdMimoCp4d_size

```
char RsdkSpt3Dfft256smp256crp8ch2TxTdMimoCp4d_size [extern]
```

Kernel size.

3.8.2.13 RsdkSpt3Dfft256smp256crp8ch_size

```
char RsdkSpt3Dfft256smp256crp8ch_size [extern]
```

Kernel size.

3.8.2.14 RsdkSpt3Dfft512smp128crp4ch_size

```
char RsdkSpt3Dfft512smp128crp4ch_size [extern]
```

Kernel size.

3.8.2.15 RsdkSpt3Dfft512smp128crp8ch_size

```
char RsdkSpt3Dfft512smp128crp8ch_size [extern]
```

Kernel size.

3.8.2.16 RsdkSptCheckTram1024smp128crp4ch_size

```
char RsdkSptCheckTram1024smp128crp4ch_size  [extern]
```

Kernel size.

3.8.2.17 RsdkSptCheckTram1024smp128crp8ch_size

```
char RsdkSptCheckTram1024smp128crp8ch_size  [extern]
```

Kernel size.

3.8.2.18 RsdkSptCheckTram1024smp256crp4ch3TxDdMimo_size

```
char RsdkSptCheckTram1024smp256crp4ch3TxDdMimo_size  [extern]
```

Kernel size.

3.8.2.19 RsdkSptCheckTram1024smp64crp4ch4TxTdMimo_size

```
char RsdkSptCheckTram1024smp64crp4ch4TxTdMimo_size  [extern]
```

Kernel size.

3.8.2.20 RsdkSptCheckTram1024smp64crp4ch_size

```
char RsdkSptCheckTram1024smp64crp4ch_size  [extern]
```

Kernel size.

3.8.2.21 RsdkSptCheckTram2048smp256crp4ch_size

```
char RsdkSptCheckTram2048smp256crp4ch_size  [extern]
```

Kernel size.

3.8.2.22 RsdkSptCheckTram256smp128crp4ch3TxTdMimo_size

```
char RsdkSptCheckTram256smp128crp4ch3TxTdMimo_size  [extern]
```

Kernel size.

3.8.2.23 RsdkSptCheckTram256smp128crp8ch6TxTdMimo_size

```
char RsdkSptCheckTram256smp128crp8ch6TxTdMimo_size  [extern]
```

Kernel size.

3.8.2.24 RsdkSptCheckTram256smp256crp4ch2TxTdMimo_size

```
char RsdkSptCheckTram256smp256crp4ch2TxTdMimo_size  [extern]
```

Kernel size.

3.8.2.25 RsdkSptCheckTram256smp256crp4ch_size

```
char RsdkSptCheckTram256smp256crp4ch_size  [extern]
```

Kernel size.

3.8.2.26 RsdkSptCheckTram256smp256crp8ch2TxTdMimo_size

```
char RsdkSptCheckTram256smp256crp8ch2TxTdMimo_size  [extern]
```

Kernel size.

3.8.2.27 RsdkSptCheckTram256smp256crp8ch_size

```
char RsdkSptCheckTram256smp256crp8ch_size  [extern]
```

Kernel size.

3.8.2.28 RsdkSptCheckTram512smp128crp4ch_size

```
char RsdkSptCheckTram512smp128crp4ch_size  [extern]
```

Kernel size.

3.8.2.29 RsdkSptCheckTram512smp128crp8ch_size

```
char RsdkSptCheckTram512smp128crp8ch_size  [extern]
```

Kernel size.

3.8.2.30 RsdkSptDbfDoa64Beams12Ch128Peaks_size

```
char RsdkSptDbfDoa64Beams12Ch128Peaks_size  [extern]
```

Kernel size.

3.8.2.31 RsdkSptDbfDoa64Beams12Ch128PeaksCp4d_size

```
char RsdkSptDbfDoa64Beams12Ch128PeaksCp4d_size  [extern]
```

Kernel size.

3.8.2.32 RsdkSptDbfDoa64Beams16Ch128PeaksCp4d_size

```
char RsdkSptDbfDoa64Beams16Ch128PeaksCp4d_size  [extern]
```

Kernel size.

3.8.2.33 RsdkSptDbfDoa64Beams4Ch128Peaks_size

```
char RsdkSptDbfDoa64Beams4Ch128Peaks_size  [extern]
```

Kernel size.

3.8.2.34 RsdkSptDbfDoa64Beams4Ch128PeaksCp4d_size

```
char RsdkSptDbfDoa64Beams4Ch128PeaksCp4d_size  [extern]
```

Kernel size.

3.8.2.35 RsdkSptDbfDoa64Beams8Ch128Peaks_size

```
char RsdkSptDbfDoa64Beams8Ch128Peaks_size  [extern]
```

Kernel size.

3.8.2.36 RsdkSptDbfDoa64Beams8Ch128PeaksCp4d_size

```
char RsdkSptDbfDoa64Beams8Ch128PeaksCp4d_size  [extern]
```

Kernel size.

3.8.2.37 RsdkSptDbfFFT128Bins12Ch128PeaksCp4d_size

```
char RsdkSptDbfFFT128Bins12Ch128PeaksCp4d_size  [extern]
```

Kernel size.

3.8.2.38 RsdkSptDbfFFT128Bins48Ch128PeaksCp4d_size

```
char RsdkSptDbfFFT128Bins48Ch128PeaksCp4d_size  [extern]
```

Kernel size.

3.8.2.39 RsdkSptDoppler1024smp128crp4chCp4d_size

```
char RsdkSptDoppler1024smp128crp4chCp4d_size  [extern]
```

Kernel size.

3.8.2.40 RsdkSptDoppler1024smp128crp8chCp4d_size

```
char RsdkSptDoppler1024smp128crp8chCp4d_size  [extern]
```

Kernel size.

3.8.2.41 RsdkSptDoppler1024smp256crp4ch3TxDdMimoCp4d_size

```
char RsdkSptDoppler1024smp256crp4ch3TxDdMimoCp4d_size  [extern]
```

Kernel size.

3.8.2.42 RsdkSptDoppler1024smp64crp4ch4TxTdMimoCp4d_size

```
char RsdkSptDoppler1024smp64crp4ch4TxTdMimoCp4d_size  [extern]
```

Kernel size.

3.8.2.43 RsdkSptDoppler1024smp64crp4chCp4d_size

```
char RsdkSptDoppler1024smp64crp4chCp4d_size  [extern]
```

Kernel size.

3.8.2.44 RsdkSptDoppler2048smp256crp4chCp4d_size

```
char RsdkSptDoppler2048smp256crp4chCp4d_size  [extern]
```

Kernel size.

3.8.2.45 RsdkSptDoppler256smp128crp4ch3TxTdMimo_size

```
char RsdkSptDoppler256smp128crp4ch3TxTdMimo_size  [extern]
```

Kernel size.

3.8.2.46 RsdkSptDoppler256smp128crp4ch3TxTdMimoRcomp_size

```
char RsdkSptDoppler256smp128crp4ch3TxTdMimoRcomp_size  [extern]
```

Kernel size.

3.8.2.47 RsdkSptDoppler256smp128crp8ch6TxTdMimoCp4d_size

```
char RsdkSptDoppler256smp128crp8ch6TxTdMimoCp4d_size  [extern]
```

Kernel size.

3.8.2.48 RsdkSptDoppler256smp256crp4ch2TxTdMimoCp4d_size

```
char RsdkSptDoppler256smp256crp4ch2TxTdMimoCp4d_size  [extern]
```

Kernel size.

3.8.2.49 RsdkSptDoppler256smp256crp4ch_size

```
char RsdkSptDoppler256smp256crp4ch_size  [extern]
```

Kernel size.

3.8.2.50 RsdkSptDoppler256smp256crp4chRcomp_size

```
char RsdkSptDoppler256smp256crp4chRcomp_size  [extern]
```

Kernel size.

3.8.2.51 RsdkSptDoppler256smp256crp8ch2TxTdMimoCp4d_size

```
char RsdkSptDoppler256smp256crp8ch2TxTdMimoCp4d_size  [extern]
```

Kernel size.

3.8.2.52 RsdkSptDoppler256smp256crp8ch_size

```
char RsdkSptDoppler256smp256crp8ch_size  [extern]
```

Kernel size.

3.8.2.53 RsdkSptDoppler256smp256crp8chRcomp_size

```
char RsdkSptDoppler256smp256crp8chRcomp_size  [extern]
```

Kernel size.

3.8.2.54 RsdkSptDoppler512smp128crp4ch_size

```
char RsdkSptDoppler512smp128crp4ch_size  [extern]
```

Kernel size.

3.8.2.55 RsdkSptDoppler512smp128crp4chRcomp_size

```
char RsdkSptDoppler512smp128crp4chRcomp_size  [extern]
```

Kernel size.

3.8.2.56 RsdkSptDoppler512smp128crp8ch_size

```
char RsdkSptDoppler512smp128crp8ch_size  [extern]
```

Kernel size.

3.8.2.57 RsdkSptDoppler512smp128crp8chRcomp_size

```
char RsdkSptDoppler512smp128crp8chRcomp_size  [extern]
```

Kernel size.

3.8.2.58 RsdkSptDspExampleDirectBlocking_size

```
char RsdkSptDspExampleDirectBlocking_size  [extern]
```

Kernel size.

3.8.2.59 RsdkSptDspExampleIndirectBlocking_size

```
char RsdkSptDspExampleIndirectBlocking_size  [extern]
```

Kernel size.

3.8.2.60 RsdkSptInit1024smp128crp4ch_size

```
char RsdkSptInit1024smp128crp4ch_size  [extern]
```

Kernel size.

3.8.2.61 RsdkSptInit1024smp128crp8ch_size

```
char RsdkSptInit1024smp128crp8ch_size  [extern]
```

Kernel size.

3.8.2.62 RsdkSptInit1024smp256crp4ch3TxDdMimo_size

```
char RsdkSptInit1024smp256crp4ch3TxDdMimo_size  [extern]
```

Kernel size.

3.8.2.63 RsdkSptInit1024smp64crp4ch4TxTdMimo_size

```
char RsdkSptInit1024smp64crp4ch4TxTdMimo_size  [extern]
```

Kernel size.

3.8.2.64 RsdkSptInit1024smp64crp4ch_size

```
char RsdkSptInit1024smp64crp4ch_size  [extern]
```

Kernel size.

3.8.2.65 RsdkSptInit2048smp256crp4ch_size

```
char RsdkSptInit2048smp256crp4ch_size  [extern]
```

Kernel size.

3.8.2.66 RsdkSptInit256smp128crp4ch3TxTdMimo_size

```
char RsdkSptInit256smp128crp4ch3TxTdMimo_size  [extern]
```

Kernel size.

3.8.2.67 RsdkSptInit256smp128crp8ch6TxTdMimo_size

```
char RsdkSptInit256smp128crp8ch6TxTdMimo_size  [extern]
```

Kernel size.

3.8.2.68 RsdkSptInit256smp256crp4ch2TxTdMimo_size

```
char RsdkSptInit256smp256crp4ch2TxTdMimo_size  [extern]
```

Kernel size.

3.8.2.69 RsdkSptInit256smp256crp4ch_size

```
char RsdkSptInit256smp256crp4ch_size  [extern]
```

Kernel size.

3.8.2.70 RsdkSptInit256smp256crp8ch2TxTdMimo_size

```
char RsdkSptInit256smp256crp8ch2TxTdMimo_size  [extern]
```

Kernel size.

3.8.2.71 RsdkSptInit256smp256crp8ch_size

```
char RsdkSptInit256smp256crp8ch_size  [extern]
```

Kernel size.

3.8.2.72 RsdkSptInit512smp128crp4ch_size

```
char RsdkSptInit512smp128crp4ch_size  [extern]
```

Kernel size.

3.8.2.73 RsdkSptInit512smp128crp8ch_size

```
char RsdkSptInit512smp128crp8ch_size  [extern]
```

Kernel size.

3.8.2.74 RsdkSptMemRwErrInject_size

```
char RsdkSptMemRwErrInject_size  [extern]
```

Kernel size.

3.8.2.75 RsdkSptNcc1024smp128crp4chCp4d_size

```
char RsdkSptNcc1024smp128crp4chCp4d_size  [extern]
```

Kernel size.

3.8.2.76 RsdkSptNcc1024smp128crp8chCp4d_size

```
char RsdkSptNcc1024smp128crp8chCp4d_size  [extern]
```

Kernel size.

3.8.2.77 RsdkSptNcc1024smp64crp4ch3TxDdMimoCp4d_size

```
char RsdkSptNcc1024smp64crp4ch3TxDdMimoCp4d_size  [extern]
```

Kernel size.

3.8.2.78 RsdkSptNcc1024smp64crp4ch4TxTdMimoCp4d_size

```
char RsdkSptNcc1024smp64crp4ch4TxTdMimoCp4d_size  [extern]
```

Kernel size.

3.8.2.79 RsdkSptNcc1024smp64crp4chCp4d_size

```
char RsdkSptNcc1024smp64crp4chCp4d_size  [extern]
```

Kernel size.

3.8.2.80 RsdkSptNcc2048smp256crp4chCp4d_size

```
char RsdkSptNcc2048smp256crp4chCp4d_size  [extern]
```

Kernel size.

3.8.2.81 RsdkSptNcc256smp128crp4ch3TxTdMimo_size

```
char RsdkSptNcc256smp128crp4ch3TxTdMimo_size  [extern]
```

Kernel size.

3.8.2.82 RsdkSptNcc256smp128crp8ch6TxTdMimoCp4d_size

```
char RsdkSptNcc256smp128crp8ch6TxTdMimoCp4d_size  [extern]
```

Kernel size.

3.8.2.83 RsdkSptNcc256smp256crp4ch2TxTdMimoCp4d_size

```
char RsdkSptNcc256smp256crp4ch2TxTdMimoCp4d_size  [extern]
```

Kernel size.

3.8.2.84 RsdkSptNcc256smp256crp4ch_size

```
char RsdkSptNcc256smp256crp4ch_size  [extern]
```

Kernel size.

3.8.2.85 RsdkSptNcc256smp256crp8ch2TxTdMimoCp4d_size

```
char RsdkSptNcc256smp256crp8ch2TxTdMimoCp4d_size  [extern]
```

Kernel size.

3.8.2.86 RsdkSptNcc256smp256crp8ch_size

```
char RsdkSptNcc256smp256crp8ch_size  [extern]
```

Kernel size.

3.8.2.87 RsdkSptNcc512smp128crp4ch_size

```
char RsdkSptNcc512smp128crp4ch_size  [extern]
```

Kernel size.

3.8.2.88 RsdkSptNcc512smp128crp8ch_size

```
char RsdkSptNcc512smp128crp8ch_size  [extern]
```

Kernel size.

3.8.2.89 RsdkSptOsCfar_size

```
char RsdkSptOsCfar_size  [extern]
```

Kernel size.

3.8.2.90 RsdkSptPeakSearch1024smp128crp_size

```
char RsdkSptPeakSearch1024smp128crp_size  [extern]
```

Kernel size.

3.8.2.91 RsdkSptPeakSearch1024smp64crp1D_size

```
char RsdkSptPeakSearch1024smp64crp1D_size  [extern]
```

Kernel size.

3.8.2.92 RsdkSptPeakSearch1024smp64crp_size

```
char RsdkSptPeakSearch1024smp64crp_size  [extern]
```

Kernel size.

3.8.2.93 RsdkSptPeakSearch2048smp256crp1D_size

```
char RsdkSptPeakSearch2048smp256crp1D_size  [extern]
```

Kernel size.

3.8.2.94 RsdkSptPeakSearch2048smp256crp_size

```
char RsdkSptPeakSearch2048smp256crp_size  [extern]
```

Kernel size.

3.8.2.95 RsdkSptPeakSearch256smp128crp_size

```
char RsdkSptPeakSearch256smp128crp_size  [extern]
```

Kernel size.

3.8.2.96 RsdkSptPeakSearch256smp256crp_size

```
char RsdkSptPeakSearch256smp256crp_size  [extern]
```

Kernel size.

3.8.2.97 RsdkSptPeakSearch512smp128crp_size

```
char RsdkSptPeakSearch512smp128crp_size  [extern]
```

Kernel size.

3.8.2.98 RsdkSptRange1024smp128crp4chCp4d_size

```
char RsdkSptRange1024smp128crp4chCp4d_size  [extern]
```

Kernel size.

3.8.2.99 RsdkSptRange1024smp128crp8chCp4d_size

```
char RsdkSptRange1024smp128crp8chCp4d_size  [extern]
```

Kernel size.

3.8.2.100 RsdkSptRange1024smp256crp4ch3TxDdMimoCp4d_size

```
char RsdkSptRange1024smp256crp4ch3TxDdMimoCp4d_size  [extern]
```

Kernel size.

3.8.2.101 RsdkSptRange1024smp64crp4ch4TxTdMimoCp4d_size

```
char RsdkSptRange1024smp64crp4ch4TxTdMimoCp4d_size  [extern]
```

Kernel size.

3.8.2.102 RsdkSptRange1024smp64crp4chCp4d_size

```
char RsdkSptRange1024smp64crp4chCp4d_size  [extern]
```

Kernel size.

3.8.2.103 RsdkSptRange2048smp256crp4chCp4d_size

```
char RsdkSptRange2048smp256crp4chCp4d_size  [extern]
```

Kernel size.

3.8.2.104 RsdkSptRange256smp128crp4ch3TxTdMimo_size

```
char RsdkSptRange256smp128crp4ch3TxTdMimo_size  [extern]
```

Kernel size.

3.8.2.105 RsdkSptRange256smp128crp4ch3TxTdMimoAdptv_size

```
char RsdkSptRange256smp128crp4ch3TxTdMimoAdptv_size  [extern]
```

Kernel size.

3.8.2.106 RsdkSptRange256smp128crp8ch6TxTdMimoCp4d_size

```
char RsdkSptRange256smp128crp8ch6TxTdMimoCp4d_size [extern]
```

Kernel size.

3.8.2.107 RsdkSptRange256smp256crp4ch2TxTdMimoCp4d_size

```
char RsdkSptRange256smp256crp4ch2TxTdMimoCp4d_size [extern]
```

Kernel size.

3.8.2.108 RsdkSptRange256smp256crp4ch_size

```
char RsdkSptRange256smp256crp4ch_size [extern]
```

Kernel size.

3.8.2.109 RsdkSptRange256smp256crp4chAdptv_size

```
char RsdkSptRange256smp256crp4chAdptv_size [extern]
```

Kernel size.

3.8.2.110 RsdkSptRange256smp256crp8ch2TxTdMimoCp4d_size

```
char RsdkSptRange256smp256crp8ch2TxTdMimoCp4d_size [extern]
```

Kernel size.

3.8.2.111 RsdkSptRange256smp256crp8ch_size

```
char RsdkSptRange256smp256crp8ch_size [extern]
```

Kernel size.

3.8.2.112 RsdkSptRange256smp256crp8chAdptv_size

```
char RsdkSptRange256smp256crp8chAdptv_size  [extern]
```

Kernel size.

3.8.2.113 RsdkSptRange512smp128crp4ch_size

```
char RsdkSptRange512smp128crp4ch_size  [extern]
```

Kernel size.

3.8.2.114 RsdkSptRange512smp128crp4chAdptv_size

```
char RsdkSptRange512smp128crp4chAdptv_size  [extern]
```

Kernel size.

3.8.2.115 RsdkSptRange512smp128crp8ch_size

```
char RsdkSptRange512smp128crp8ch_size  [extern]
```

Kernel size.

3.8.2.116 RsdkSptRange512smp128crp8chAdptv_size

```
char RsdkSptRange512smp128crp8chAdptv_size  [extern]
```

Kernel size.

3.8.2.117 RsdkSptRfBist128smp1crp4ch_size

```
char RsdkSptRfBist128smp1crp4ch_size  [extern]
```

Kernel size.

3.8.2.118 RsdkSptRfBist2048smp1crp4ch_size

```
char RsdkSptRfBist2048smp1crp4ch_size  [extern]
```

Kernel size.

3.9 SPT Kernels Functions

SPT Kernels Functions.

Functions

- void * RsdkSptInit512smp128crp4ch (void)

  Initialization function for 512 samples and 128 chirps, 4 physical channels.
- void * RsdkSptInit256smp256crp4ch (void)

  Initialization function for 256 samples and 256 chirps, 4 physical channels.
- void * RsdkSptInit1024smp64crp4ch (void)

  Initialization function for 1024 samples and 64 chirps, 4 physical channels.
- void * RsdkSptInit1024smp128crp4ch (void)

  Initialization function for 1024 samples and 128 chirps, 4 physical channels.
- void * RsdkSptInit2048smp256crp4ch (void)

  Initialization function for 2048 samples and 256 chirps, 4 physical channels.
- void * RsdkSptInit256smp256crp4ch2TxTdMimo (void)

  Initialization function for 256 samples and 256 chirps, 4 physical channels/TX slot, 2TX TD-Mimo.
- void * RsdkSptInit256smp128crp4ch3TxTdMimo (void)

  Initialization function for 256 samples and 128 chirps, 4 physical channels/TX slot, 3TX TD-Mimo.
- void * RsdkSptInit1024smp256crp4ch3TxDdMimo (void)

  Initialization function for 1024 samples and 256 chirps, 4 physical channels, 3TX DD-Mimo.
- void * RsdkSptInit1024smp64crp4ch4TxTdMimo (void)

  Initialization function for 1024 samples and 64 chirps, 4 physical channels/TX slot, 4TX TD-Mimo.
- void * RsdkSptInit512smp128crp8ch (void)
- void * RsdkSptInit256smp256crp8ch (void)
- void * RsdkSptInit1024smp128crp8ch (void)
- void * RsdkSptInit256smp256crp8ch2TxTdMimo (void)
- void * RsdkSptInit256smp128crp8ch6TxTdMimo (void)
- void * RsdkSptRange512smp128crp4ch (void)

  Function for Range FFT 512, 4 channels(CH#0..CH#3)
- void * RsdkSptRange512smp128crp4chAdptv (void)

  Function for Range FFT 512, 4 channels(CH#0..CH#3), Adaptive scaling.
- void * RsdkSptRange256smp256crp4ch (void)

  Function for Range FFT 256, 4 channels(CH#0..CH#3)
- void * RsdkSptRange256smp256crp4chAdptv (void)

  Function for Range FFT 256, 4 channels(CH#0..CH#3), Adaptive scaling.
- void * RsdkSptRange1024smp64crp4chCp4d (void)

  Function for Range FFT 1024, 4 physical channels(CH#0..CH#3)
- void * RsdkSptRange1024smp128crp4chCp4d (void)

  Function for Range FFT 1024, 4 channels(CH#0..CH#3)
- void * RsdkSptRange2048smp256crp4chCp4d (void)

  Function for Range FFT 2048, 4 channels(CH#0..CH#3)

- void * RsdkSptRange256smp256crp4ch2TxTdMimoCp4d (void)

  Function for Range FFT 256, 4 physical channels/TX slot, 2TX TD-MIMO.
- void * RsdkSptRange256smp128crp4ch3TxTdMimo (void)

  Function for Range FFT 256, 4 physical channels/TX slot, 3TX TD-MIMO.
- void * RsdkSptRange256smp128crp4ch3TxTdMimoAdptv (void)

  Function for Range FFT 256, 4 physical channels/TX slot, 3TX TD-MIMO, Adaptive scaling.
- void * RsdkSptRange1024smp256crp4ch3TxDdMimoCp4d (void)

  Function for Range FFT 1024, 4 channels(CH#0..CH#3), DD-Mimo use case.
- void * RsdkSptRange1024smp64crp4ch4TxTdMimoCp4d (void)

  Function for Range FFT 1024, 4 physical channels/TX slot, 4TX TD-MIMO.
- void * RsdkSptRange512smp128crp8ch (void)
- void * RsdkSptRange512smp128crp8chAdptv (void)
- void * RsdkSptRange256smp256crp8ch (void)
- void * RsdkSptRange256smp256crp8chAdptv (void)
- void * RsdkSptRange1024smp128crp8chCp4d (void)
- void * RsdkSptRange256smp256crp8ch2TxTdMimoCp4d (void)
- void * RsdkSptRange256smp128crp8ch6TxTdMimoCp4d (void)
- void * RsdkSptDoppler512smp128crp4ch (void)

  Function for Doppler FFT 128, 4 channels(CH#0..CH#3)
- void * RsdkSptDoppler512smp128crp4chRcomp (void)

  Function for Doppler FFT 128, 4 channels(CH#0..CH#3).
- void * RsdkSptDoppler256smp256crp4ch (void)

  Function for Doppler FFT 256, 4 channels(CH#0..CH#3)
- void * RsdkSptDoppler256smp256crp4chRcomp (void)

  Function for Doppler FFT 256, 4 channels(CH#0..CH#3)
- void * RsdkSptDoppler1024smp64crp4chCp4d (void)

  Function for Doppler FFT 64, 4 channels(CH#0..CH#3)
- void * RsdkSptDoppler1024smp128crp4chCp4d (void)

  Function for Doppler FFT 128, 4 channels(CH#0..CH#3)
- void * RsdkSptDoppler2048smp256crp4chCp4d (void)

  Function for Doppler FFT 256, 4 channels(CH#0..CH#3)
- void * RsdkSptDoppler256smp256crp4ch2TxTdMimoCp4d (void)

  Function for Doppler FFT 256, 4 physical channels/TX slot, 2TX TD-MIMO.
- void * RsdkSptDoppler256smp128crp4ch3TxTdMimo (void)

  Function for Doppler FFT 128, 4 physical channels/TX slot, 3TX TD-MIMO.
- void * RsdkSptDoppler256smp128crp4ch3TxTdMimoRcomp (void)

  Function for Doppler FFT 128, 4 physical channels/TX slot, 3TX TD-MIMO.
- void * RsdkSptDoppler1024smp256crp4ch3TxDdMimoCp4d (void)

  Function for Doppler FFT256, 4 channels(CH#0..CH#3), DD-MIMO use case.
- void * RsdkSptDoppler1024smp64crp4ch4TxTdMimoCp4d (void)

  Function for Doppler FFT 1024, 4 physical channels/TX slot, 4TX TD-MIMO.
- void * RsdkSptDoppler512smp128crp8ch (void)
- void * RsdkSptDoppler512smp128crp8chRcomp (void)
- void * RsdkSptDoppler256smp256crp8ch (void)
- void * RsdkSptDoppler256smp256crp8chRcomp (void)
- void * RsdkSptDoppler1024smp128crp8chCp4d (void)
- void * RsdkSptDoppler256smp256crp8ch2TxTdMimoCp4d (void)
- void * RsdkSptDoppler256smp128crp8ch6TxTdMimoCp4d (void)
- void * RsdkSptNcc512smp128crp4ch (void)

  Function for Non coherent combining, 128 chirps, 512 sample/chirp, 4 channels(CH#0..CH#3).
- void * RsdkSptNcc256smp256crp4ch (void)

  Function for Non coherent combining, 256 chirps, 256 sample/chirp, 4 channels(CH#0..CH#3)

- void * RsdkSptNcc1024smp64crp4chCp4d (void)

  Function for Non coherent combining, 64 chirps, 1024 sample/chirp, 4 channels(CH#0..CH#3)
- void * RsdkSptNcc1024smp128crp4chCp4d (void)

  Function for Non coherent combining, 128 chirps, 1024 sample/chirp, 4 channels(CH#0..CH#3)
- void * RsdkSptNcc2048smp256crp4chCp4d (void)

  Function for Non coherent combining, 256 chirps, 2048 sample/chirp, 4 channels(CH#0..CH#3)
- void * RsdkSptNcc256smp256crp4ch2TxTdMimoCp4d (void)

  Function for Non coherent combining, 256 chirps, 256 sample/chirp, 4 physical channels/TX slot, 2TX TD-MIMO.
- void * RsdkSptNcc256smp128crp4ch3TxTdMimo (void)

  Function for Non coherent combining, 128 chirps, 256 sample/chirp, 4 physical channels/TX slot, 3TX TD-MIMO.
- void * RsdkSptNcc1024smp64crp4ch4TxTdMimoCp4d (void)

  Function for Non coherent combining, 64 chirps, 1024 sample/chirp, 4 physical channels/TX slot, 4TX TD-MIMO.
- void * RsdkSptNcc1024smp64crp4ch3TxDdMimoCp4d (void)

  Function for Non coherent combining, 64 chirps/TX DD-MIMO slot(from 256chirps/frame), 1024 samples, 4 channels(CH#0..CH#3).
- void * RsdkSptNcc512smp128crp8ch (void)
- void * RsdkSptNcc256smp256crp8ch (void)
- void * RsdkSptNcc1024smp128crp8chCp4d (void)
- void * RsdkSptNcc256smp256crp8ch2TxTdMimoCp4d (void)
- void * RsdkSptNcc256smp128crp8ch6TxTdMimoCp4d (void)
- void * RsdkSpt3Dfft512smp128crp4ch (void)

  Function for 3D FFT, 128 chirps, 512 sample/chirp, 4 channels(CH#0..CH#3)
- void * RsdkSpt3Dfft256smp256crp4ch (void)

  Function for 3D FFT, 256 chirps, 256 sample/chirp, 4 channels(CH#0..CH#3).
- void * RsdkSpt3Dfft1024smp128crp4chCp4d (void)

  Function for 3D FFT, 128 chirps, 1024 sample/chirp, 4 channels(CH#0..CH#3)
- void * RsdkSpt3Dfft1024smp64crp4chCp4d (void)

  Function for 3D FFT, 64 chirps, 1024 sample/chirp, 4 channels(CH#0..CH#3)
- void * RsdkSpt3Dfft2048smp256crp4chCp4d (void)

  Function for 3D FFT, 256 chirps, 2048 sample/chirp, 4 channels(CH#0..CH#3).
- void * RsdkSpt3Dfft256smp256crp4ch2TxTdMimoCp4d (void)

  Function for 3D FFT, 256 chirps, 256 sample/chirp, 4 physical channels/TX slot, 2TX TD-MIMO.
- void * RsdkSpt3Dfft256smp128crp4ch3TxTdMimo (void)

  Function for 3D FFT(Coherent combining), 128 chirps, 256 sample/chirp, 4 physical channels/TX slot, 3TX TD-MIMO.
- void * RsdkSpt3Dfft1024smp64crp4ch4TxTdMimoCp4d (void)

  Function for 3D FFT, 64 chirps, 1024 sample/chirp, 4 physical channels/TX slot, 4TX TD-MIMO.
- void * RsdkSpt3Dfft512smp128crp8ch (void)
- void * RsdkSpt3Dfft256smp256crp8ch (void)
- void * RsdkSpt3Dfft1024smp128crp8chCp4d (void)
- void * RsdkSpt3Dfft256smp256crp8ch2TxTdMimoCp4d (void)
- void * RsdkSpt3Dfft256smp128crp8ch6TxTdMimoCp4d (void)
- void * RsdkSptPeakSearch512smp128crp (void)

  Function for Peak Search, 128 chirps (doppler bins), 512 sample/chirp (256 range bins).
- void * RsdkSptPeakSearch256smp256crp (void)

  Function for Peak Search, 256 chirps (doppler bins), 256 sample/chirp (128 range bins).
- void * RsdkSptPeakSearch1024smp128crp (void)

  Function for Peak Search, 128 chirps (doppler bins), 1024 sample/chirp (512 range bins).
- void * RsdkSptPeakSearch256smp128crp (void)

  Function for Peak Search, 128 chirps (doppler bins), 256 sample/chirp (128 range bins).
- void * RsdkSptPeakSearch1024smp64crp (void)

  Function for Peak Search, 64 chirps (doppler bins), 1024 sample/chirp (512 range bins).
- void * RsdkSptPeakSearch2048smp256crp (void)

Function for Peak Search, 256 chirps (doppler bins), 2048 sample/chirp (1024 range bins).

- void * RsdkSptPeakSearch2048smp256crp1D (void)

  Function for Peak Search 1D, 256 chirps (doppler bins), 2048 sample/chirp (1024 range bins).
- void * RsdkSptPeakSearch1024smp64crp1D (void)

  Function for Peak Search, 64 chirps (doppler bins), 1024 sample/chirp (512 range bins).
- void * RsdkSptDbfDoa64Beams4Ch128Peaks (void)

  Function for Digital BeamForming - DBF for 64 Steering Vector, 4 channels and 128 peaks and Direction of Arrival.
- void * RsdkSptDbfDoa64Beams4Ch128PeaksCp4d (void)

  Function for Digital BeamForming - DBF for 64 Steering Vector, 4 channels and 128 peaks and Direction of Arrival, CP4D PDMA decompression.
- void * RsdkSptDbfDoa64Beams8Ch128Peaks (void)
- void * RsdkSptDbfDoa64Beams8Ch128PeaksCp4d (void)

  Function for Digital BeamForming - DBF for 64 Steering Vector, 8 channels and 128 peaks and Direction of Arrival, CP4D PDMA decompression.
- void * RsdkSptDbfDoa64Beams12Ch128Peaks (void)

  Function for Digital BeamForming - DBF for 64 Steering Vector, 12 channels and 128 peaks and Direction of Arrival.
- void * RsdkSptDbfDoa64Beams12Ch128PeaksCp4d (void)

  Function for Digital BeamForming - DBF for 64 Steering Vector, 12 channels and 128 peaks and Direction of Arrival, CP4D PDMA decompression.
- void * RsdkSptDbfDoa64Beams16Ch128PeaksCp4d (void)

  Function for Digital BeamForming - DBF for 64 Steering Vector, 16 channels and 128 peaks and Direction of Arrival, CP4D PDMA decompression.
- void * RsdkSptDbfFFT128Bins12Ch128PeaksCp4d (void)

  Function for Digital BeamForming - DBF for 128 bins, 12 channels and 128 peaks.
- void * RsdkSptDbfFFT128Bins48Ch128PeaksCp4d (void)
- void * RsdkSptCheckTram512smp128crp4ch (void)

  Function for checking sanity of persistent TRAM content. 512 samples, 128 chirps and 4 physical channels.
- void * RsdkSptCheckTram256smp256crp4ch (void)

  Function for checking sanity of persistent TRAM content. 256 samples, 256 chirps and 4 physical channels.
- void * RsdkSptCheckTram1024smp128crp4ch (void)

  Function for checking sanity of persistent TRAM content. 1024 samples, 128 chirps and 4 physical channels.
- void * RsdkSptCheckTram1024smp64crp4ch (void)

  Function for checking sanity of persistent TRAM content. 1024 samples, 64 chirps and 4 physical channels.
- void * RsdkSptCheckTram2048smp256crp4ch (void)

  Function for checking sanity of persistent TRAM content. 2048 samples, 256 chirps and 4 physical channels.
- void * RsdkSptCheckTram256smp256crp4ch2TxTdMimo (void)

  Function for checking sanity of persistent TRAM content. 256 samples, 256 chirps, 4 physical channels/TX slot, 2TX TD-Mimo.
- void * RsdkSptCheckTram256smp128crp4ch3TxTdMimo (void)

  Function for checking sanity of persistent TRAM content. 256 samples, 128 chirps, 4 physical channels/TX slot, 3TX TD-Mimo.
- void * RsdkSptCheckTram1024smp64crp4ch4TxTdMimo (void)

  Function for checking sanity of persistent TRAM content. 1024 samples, 64 chirps, 4 physical channels/TX slot, 4TX TD-Mimo.
- void * RsdkSptCheckTram1024smp256crp4ch3TxDdMimo (void)

  Function for checking sanity of persistent TRAM content. 1024 samples, 256 chirps, 4 physical channels, 3TX DD-↩ Mimo.
- void * RsdkSptCheckTram512smp128crp8ch (void)
- void * RsdkSptCheckTram256smp256crp8ch (void)
- void * RsdkSptCheckTram1024smp128crp8ch (void)
- void * RsdkSptCheckTram256smp256crp8ch2TxTdMimo (void)
- void * RsdkSptCheckTram256smp128crp8ch6TxTdMimo (void)
- void * RsdkSptRfBist128smp1crp4ch (void)

Function for RF Build-in Self Test 128 samples, 1 chirp, 4 channels(CH#0..CH#3). Offline version only.

- void * RsdkSptRfBist2048smp1crp4ch (void)

  Function for RF Build-in Self Test 2048 samples, 1 chirp, 4 channels(CH_0..CH_3). Offline version only.

- void * RsdkSptMemRwErrInject (void)

  This kernel does a useless read-after-write on one OPRAM bank, to generate parity errors, then trigger a memory error interrupt on the SPT ECS interrupt line, when MEM_ERR_INJECT_CTRL register is set.

- void * RsdkSptDspExampleDirectBlocking (void)
- void * RsdkSptDspExampleIndirectBlocking (void)
- void * RsdkSptOsCfar (void)

  Function for Sliding OS-CFAR, any number of Range Bins and maximum 4096 Doppler Bins.

3.9.1 Detailed Description

SPT Kernels Functions.

3.9.2 Function Documentation

3.9.2.1 RsdkSpt3Dfft1024smp128crp4chCp4d()

```
void * RsdkSpt3Dfft1024smp128crp4chCp4d (
            void  )
```

Function for 3D FFT, 128 chirps, 1024 sample/chirp, 4 channels(CH#0..CH#3)

Compute 3D FFT, storing results in SRAM.

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Doppler FFT compressed results), relative to SRAM start address, RSDK_SPT_CUBE_BASE_ADDR. * INPUT Memory layout: * SRAM, 64bit compressed data * 63:0 * CH0_0-CH1_0-CH2_0-CH3_0_Crp0 * | * CH0_0-CH1_0-CH2_0-CH3_0-Crp127 ---> RangeBin ##0 * ---------- * | * | * | * ---------- * CH0_511-CH1_511-CH2_511-CH3_511-Crp0 * | * CH0_511-CH1_511-CH2_511-CH3_511-Crp127 ---> RangeBin ##511 * ---------- *

Parameters

out	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the RDM OUTPUT buffer(3D FFT matrix), relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 16 bit real Log2 Mag2 format * 63:48 47:32 31:16 15:0 * R0_D0_MAG R0_D1_MAG R0_D2_MAG R0_D3_MAG * | | | | * R0_D124_MAG R0_D125_MAG R0_D126_MAG R0_D127_MAG * R1_D0_MAG R1_D1_MAG R1_D2_MAG R1_D3_MAG * | | | | * R1_D124_MAG R1_D125_MAG R1_D126_MAG R1_D127_MAG * | | | | * | | | | * | | | | * | | | | * | | | | * R511_D0_MAG R511_D1_MAG R511_D2_MAG R511_D3_MAG * | | | | * R511_D124_MAG R511_D125_MAG R511_D126_MAG R511_D127_MAG *
out	*WR↩ _3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the Histogram OUTPUT buffer, relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 16 bit values histogram counters * 0x0 bin1 bin0 * 0x4 bin3 bin2 * | | * bin61 bin60 * bin63 bin62 -> Range Bin 0 * ---------- ---------- * bin1 bin0 * bin3 bin2 * | | * bin61 bin60 * bin63 bin62 -> Range Bin 1 * ---------- ---------- * | | * | | * | | * ---------- ---------- * bin1 bin0 * bin3 bin2 * | | * bin61 bin60 * bin63 bin62 -> Range Bin 511 * ---------- ---------- *

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

Twiddle coeficients are placed in TRAM at TR_0_288_0 by the Init function.

Note

The current implementation is ONLY to be used in combination with 1024 Range FFT and 128 Doppler FFT.
The input buffer MUST NOT be reused for the histogram or RDM output.
All buffers MUST be 8 bytes aligned.

3.9.2.2 RsdkSpt3Dfft1024smp128crp8chCp4d()

```
void * RsdkSpt3Dfft1024smp128crp8chCp4d (
            void  )
```

3.9.2.3 RsdkSpt3Dfft1024smp64crp4ch4TxTdMimoCp4d()

```
void * RsdkSpt3Dfft1024smp64crp4ch4TxTdMimoCp4d (
            void  )
```

Function for 3D FFT, 64 chirps, 1024 sample/chirp, 4 physical channels/TX slot, 4TX TD-MIMO.

Compute 3D FFT, storing results in SRAM.

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Doppler FFT compressed results), relative to SRAM start address, RSDK_SPT_CUBE_BASE_ADDR. * INPUT Memory layout: * SRAM, 64bit compressed data * 63:0 * CH0_0-CH1_0-CH2_0-CH3_0-TX0_Crp0 * CH0_0-CH1_0-CH2_0-CH3_0-TX1_Crp0 * CH0_0-CH1_0-CH2_0-CH3_0-TX2_Crp0 * CH0_0-CH1_0-CH2_0-CH3_0-TX3_Crp0 * | * CH0_0-CH1_0-CH2_0-CH3_0-TX0_Crp63 * CH0_0-CH1_0-CH2_0-CH3_0-TX1_Crp63 * CH0_0-CH1_0-CH2_0-CH3_0-TX2_Crp63 * CH0_0-CH1_0-CH2_0-CH3_0-TX3_Crp63 ---> RangeBin ##0 * ---------- * | * | * | * ---------- * CH0_511-CH1_511-CH2_511-CH3_511-TX0_Crp0 * CH0_511-CH1_511-CH2_511-CH3_511-TX1_Crp0 * CH0_511-CH1_511-CH2_511-CH3_511-TX2_Crp0 * CH0_511-CH1_511-CH2_511-CH3_511-TX3_Crp0 * | * CH0_511-CH1_511-CH2_511-CH3_511-TX0_Crp63 * CH0_511-CH1_511-CH2_511-CH3_511-TX1_Crp63 * CH0_511-CH1_511-CH2_511-CH3_511-TX2_Crp63 * CH0_511-CH1_511-CH2_511-CH3_511-TX3_Crp63 ---> RangeBin ##511 * ---------- *
out	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the RDM OUTPUT buffer(3D FFT matrix), relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 16 bit real Log2 Mag2 format * 63:48 47:32 31:16 15:0 * R0_D0_MAG R0_D1_MAG R0_D2_MAG R0_D3_MAG * | | | | * R0_D60_MAG R0_D61_MAG R0_D62_MAG R0_D63_MAG * R1_D0_MAG R1_D1_MAG R1_D2_MAG R1_D3_MAG * | | | | * R1_D60_MAG R1_D61_MAG R1_D62_MAG R1_D63_MAG * | | | | * | | | | * | | | | * | | | | * | | | | * R511_D0_MAG R511_D1_MAG R511_D2_MAG R511_D3_MAG * | | | | * R511_D60_MAG R511_D61_MAG R511_D62_MAG R511_D63_MAG *

Parameters

out	*WR↩ _3*	24bit(RE): 0
		24bit(IM): Offset(restricted to 24bit value!) of the Histogram OUTPUT buffer, relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR.
		
		`*  OUTPUT Memory layout:`
		`*  SRAM, 16 bit values histogram counters`
		`*  0x0  bin1  bin0`
		`*  0x4  bin3  bin2`
		`*  |  |`
		`*  bin61 bin60`
		`*  bin63 bin62 -> Range Bin 0`
		`*  ----------`
		`*  bin1  bin0`
		`*  bin3  bin2`
		`*  |  |`
		`*  bin61 bin60`
		`*  bin63 bin62 -> Range Bin 1`
		`*  ----------`
		`*  |  |`
		`*  |  |`
		`*  |  |`
		`*  ----------`
		`*  bin1  bin0`
		`*  bin3  bin2`
		`*  |  |`
		`*  bin61 bin60`
		`*  bin63 bin62 -> Range Bin 511`
		`*  ----------`
		`*`

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

Twiddle coeficients are placed in TRAM at TR_0_272_0 by the Init function.

Note

The current implementation is ONLY to be used in combination with 1024 Range FFT and 64 Doppler FFT, 4TX TD-MIMO.
The input buffer MUST NOT be reused for the histogram or RDM output.
All buffers MUST be 8 bytes aligned.

3.9.2.4 RsdkSpt3Dfft1024smp64crp4chCp4d()

```
void * RsdkSpt3Dfft1024smp64crp4chCp4d (
            void  )
```

Function for 3D FFT, 64 chirps, 1024 sample/chirp, 4 channels(CH#0..CH#3)

Compute 3D FFT, storing results in SRAM.

Parameters

in	*WR↩ _1*	24bit(RE): 0
		24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Doppler FFT compressed results), relative to SRAM start address, RSDK_SPT_CUBE_BASE_ADDR.
		
		`*  INPUT Memory layout:`
		`*  SRAM, 64bit compressed data`
		`*  63:0`
		`*  CH0_0-CH1_0-CH2_0-CH3_0_Crp0`
		`*  |`
		`*  CH0_0-CH1_0-CH2_0-CH3_0-Crp63 ---> RangeBin ##0`
		`*  ----------`
		`*  |`
		`*  |`
		`*  |`
		`*  ----------`
		`*  CH0_511-CH1_511-CH2_511-CH3_511-Crp0`
		`*  |`
		`*  CH0_511-CH1_511-CH2_511-CH3_511-Crp63 ---> RangeBin ##511`
		`*  ----------`
		`*`

out	*WR↩ _2*	24bit(RE): 0
		24bit(IM): Offset(restricted to 24bit value!) of the RDM OUTPUT buffer(3D FFT matrix), relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR.
		
		`*  OUTPUT Memory layout:`
		`*  SRAM, 16 bit real Log2 Mag2 format`
		`*  63:48 47:32 31:16 15:0`
		`*  R0_D0_MAG R0_D1_MAG R0_D2_MAG R0_D3_MAG`
		`*  | | | |`
		`*  R0_D60_MAG R0_D61_MAG R0_D62_MAG R0_D63_MAG`
		`*  R1_D0_MAG R1_D1_MAG R1_D2_MAG R1_D3_MAG`
		`*  | | | |`
		`*  R1_D60_MAG R1_D61_MAG R1_D62_MAG R1_D63_MAG`
		`*  | | | |`
		`*  | | | |`
		`*  | | | |`
		`*  | | | |`
		`*  | | | |`
		`*  R511_D0_MAG R511_D1_MAG R511_D2_MAG R511_D3_MAG`
		`*  | | | |`
		`*  R511_D60_MAG R511_D61_MAG R511_D62_MAG R511_D63_MAG`
		`*`

Parameters

out	*WR↩ _3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the Histogram OUTPUT buffer, relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 16 bit values histogram counters * 0x0 bin1 bin0 * 0x4 bin3 bin2 * | | * bin61 bin60 * bin63 bin62 -> Range Bin 0 * ---------- ---------- * bin1 bin0 * bin3 bin2 * | | * bin61 bin60 * bin63 bin62 -> Range Bin 1 * ---------- ---------- * | | * | | * | | * ---------- ---------- * bin1 bin0 * bin3 bin2 * | | * bin61 bin60 * bin63 bin62 -> Range Bin 511 * ---------- ---------- *

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

Twiddle coeficients are placed in TRAM at TR_0_272_0 by the Init function.

Note

The current implementation is ONLY to be used in combination with 1024 Range FFT and 64 Doppler FFT CP4D.
The input buffer MUST NOT be reused for the histogram or RDM output.
All buffers MUST be 8 bytes aligned.

3.9.2.5 RsdkSpt3Dfft2048smp256crp4chCp4d()

```
void * RsdkSpt3Dfft2048smp256crp4chCp4d (
            void  )
```

Function for 3D FFT, 256 chirps, 2048 sample/chirp, 4 channels(CH#0..CH#3).

Compute 3D FFT, storing results in SRAM.

Parameters

in	*WR↩ _1*	24bit(RE): 0
		24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Doppler FFT results), relative to SRAM start address, RSDK_SPT_CUBE_BASE_ADDR.
		
		`*  INPUT Memory layout:`
		`*  SRAM, 64bit compressed data`
		`*  63:0`
		`*  CH0_0-CH1_0-CH2_0-CH3_0_Crp0`
		`*  |`
		`*  CH0_0-CH1_0-CH2_0-CH3_0-Crp255 ---> RangeBin ##0`
		`*  ----------`
		`*  |`
		`*  |`
		`*  |`
		`*  ----------`
		`*  CH0_511-CH1_511-CH2_511-CH3_511-Crp0`
		`*  |`
		`*  CH0_511-CH1_511-CH2_511-CH3_511-Crp255 ---> RangeBin ##1023`
		`*  ----------`
		`*`

out	*WR↩ _2*	24bit(RE): 0
		24bit(IM): Offset(restricted to 24bit value!) of the RDM OUTPUT buffer(3D FFT matrix), relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR.
		
		`*  OUTPUT Memory layout:`
		`*  SRAM, 16 bit real Log2 Mag2 format`
		`*  63:48 47:32 31:16 15:0`
		`*  R0_D0_MAG R0_D1_MAG R0_D2_MAG R0_D3_MAG`
		`*  | | | |`
		`*  R0_D252_MAG R0_D253_MAG R0_D254_MAG R0_D255_MAG`
		`*  R1_D0_MAG R1_D1_MAG R1_D2_MAG R1_D3_MAG`
		`*  | | | |`
		`*  R1_D252_MAG R1_D253_MAG R1_D254_MAG R1_D255_MAG`
		`*  | | | |`
		`*  | | | |`
		`*  | | | |`
		`*  | | | |`
		`*  | | | |`
		`*  R1023_D0_MAG R1023_D1_MAG R1023_D2_MAG R1023_D3_MAG`
		`*  | | | |`
		`*  R1023_D252_MAG R1023_D253_MAG R1023_D254_MAG R1023_D255_MAG`
		`*`

Parameters

out	*WR↩ _3*	24bit(RE): 0
		24bit(IM): Offset(restricted to 24bit value!) of the Histogram OUTPUT buffer, relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR.
		
		`*  OUTPUT Memory layout:`
		`*  SRAM, 16 bit values histogram counters`
		`*  0x0  bin1  bin0`
		`*  0x4  bin3  bin2`
		`*  |  |`
		`*  bin61 bin60`
		`*  bin63 bin62 -> Range Bin 0`
		`*  ----------`
		`*  bin1  bin0`
		`*  bin3  bin2`
		`*  |  |`
		`*  bin61 bin60`
		`*  bin63 bin62 -> Range Bin 1`
		`*  ----------`
		`*  |  |`
		`*  |  |`
		`*  |  |`
		`*  ----------`
		`*  bin1  bin0`
		`*  bin3  bin2`
		`*  |  |`
		`*  bin61 bin60`
		`*  bin63 bin62 -> Range Bin 1023`
		`*  ----------`
		`*`

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

Twiddle coeficients are placed in TRAM at TR_0_320_0 by the Init function.

Note

The current implementation is ONLY to be used in combination with 2048 Range FFT and 256 Doppler FFT CP4D.
The input buffer MUST NOT be reused for the histogram or RDM output.
All buffers MUST be 8 bytes aligned.

3.9.2.6 RsdkSpt3Dfft256smp128crp4ch3TxTdMimo()

```
void * RsdkSpt3Dfft256smp128crp4ch3TxTdMimo (
            void  )
```

Function for 3D FFT(Coherent combining), 128 chirps, 256 sample/chirp, 4 physical channels/TX slot, 3TX TD-↩ MIMO.

Compute 3D FFT, storing results in SRAM.

Parameters

in	*WR_1*	1st 24bit(RE): 0 2nd 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer (Doppler FFT results), relative to SRAM start address, RSDK_SPT_CUBE_BASE_ADDR.

```
*                 INPUT Memory layout:
*                 SRAM, 32bit complex(16b_Im + 16b_Re)
*                 63:48              47:32              31:16              15:0
*                 CH0_0_Im-Crp0      CH0_0_Re-Crp0      CH1_0_Im-Crp0      CH1_0_Re-Crp0
*                 CH2_0_Im-Crp0      CH2_0_Re-Crp0      CH3_0_Im-Crp0      CH3_0_Re-Crp0
*                 ----------------------------------------------------------------------------
*                 CH0_0_Im-Crp0      CH0_0_Re-Crp0      CH1_0_Im-Crp0      CH1_0_Re-Crp0
*                 CH2_0_Im-Crp0      CH2_0_Re-Crp0      CH3_0_Im-Crp0      CH3_0_Re-Crp0
*                 ----------------------------------------------------------------------------
*                 CH0_0_Im-Crp0      CH0_0_Re-Crp0      CH1_0_Im-Crp0      CH1_0_Re-Crp0
*                 CH2_0_Im-Crp0      CH2_0_Re-Crp0      CH3_0_Im-Crp0      CH3_0_Re-Crp0
*                 ----------------------------------------------------------------------------
*                 |                  |                  |                  |
*                 |                  |                  |                  |
*                 |                  |                  |                  |
*                 CH0_0_Im-Crp127    CH0_0_Re-Crp127    CH1_0_Im-Crp127    CH1_0_Re-Crp127
*                 CH2_0_Im-Crp127    CH2_0_Re-Crp127    CH3_0_Im-Crp127    CH3_0_Re-Crp127
*                 ----------------------------------------------------------------------------
*                 CH0_0_Im-Crp127    CH0_0_Re-Crp127    CH1_0_Im-Crp127    CH1_0_Re-Crp127
*                 CH2_0_Im-Crp127    CH2_0_Re-Crp127    CH3_0_Im-Crp127    CH3_0_Re-Crp127
*                 ----------------------------------------------------------------------------
*                 CH0_0_Im-Crp127    CH0_0_Re-Crp127    CH1_0_Im-Crp127    CH1_0_Re-Crp127
*                 CH2_0_Im-Crp127    CH2_0_Re-Crp127    CH3_0_Im-Crp127    CH3_0_Re-Crp127
*                 ----------------------------------------------------------------------------
*                 |                  |                  |                  |
*                 |                  |                  |                  |
*                 |                  |                  |                  |
*                 |                  |                  |                  |
*                 |                  |                  |                  |
*                 |                  |                  |                  |
*                 CH0_127_Im-Crp0    CH0_127_Re-Crp0    CH1_127_Im-Crp0    CH1_127_Re-Crp0
*                 CH2_127_Im-Crp0    CH2_127_Re-Crp0    CH3_127_Im-Crp0    CH3_127_Re-Crp0
*                 ----------------------------------------------------------------------------
*                 CH0_127_Im-Crp0    CH0_127_Re-Crp0    CH1_127_Im-Crp0    CH1_127_Re-Crp0
*                 CH2_127_Im-Crp0    CH2_127_Re-Crp0    CH3_127_Im-Crp0    CH3_127_Re-Crp0
*                 ----------------------------------------------------------------------------
*                 CH0_127_Im-Crp0    CH0_127_Re-Crp0    CH1_127_Im-Crp0    CH1_127_Re-Crp0
*                 CH2_127_Im-Crp0    CH2_127_Re-Crp0    CH3_127_Im-Crp0    CH3_127_Re-Crp0
*                 ----------------------------------------------------------------------------
*                 |                  |                  |                  |
*                 |                  |                  |                  |
*                 |                  |                  |                  |
*                 CH0_127_Im-Crp127  CH0_127_Re-Crp127  CH1_127_Im-Crp127  CH1_127_Re-Crp127
*                 CH2_127_Im-Crp127  CH2_127_Re-Crp127  CH3_127_Im-Crp127  CH3_127_Re-Crp127
*                 ----------------------------------------------------------------------------
*                 CH0_127_Im-Crp127  CH0_127_Re-Crp127  CH1_127_Im-Crp127  CH1_127_Re-Crp127
*                 CH2_127_Im-Crp127  CH2_127_Re-Crp127  CH3_127_Im-Crp127  CH3_127_Re-Crp127
*                 ----------------------------------------------------------------------------
*                 CH0_127_Im-Crp127  CH0_127_Re-Crp127  CH1_127_Im-Crp127  CH1_127_Re-Crp127
*                 CH2_127_Im-Crp127  CH2_127_Re-Crp127  CH3_127_Im-Crp127  CH3_127_Re-Crp127
*                 ----------------------------------------------------------------------------
*
```

Parameters

out	*WR↩ _2*	1st 24bit(RE): 0 2nd 24bit(IM): Offset(restricted to 24bit value!) of the RDM OUTPUT buffer (3D FFT matrix), relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 16 bit real Log2 Mag2 format * 63:48 47:32 31:16 15:0 * R0_D0_MAG R0_D1_MAG R0_D2_MAG R0_D3_MAG * | | | | * R0_D124_MAG R0_D125_MAG R0_D126_MAG R0_D127_MAG * R1_D0_MAG R1_D1_MAG R1_D2_MAG R1_D3_MAG * | | | | * R1_D124_MAG R1_D125_MAG R1_D126_MAG R1_D127_MAG * | | | | * | | | | * | | | | * | | | | * | | | | * R127_D0_MAG R127_D1_MAG R127_D2_MAG R127_D3_MAG * | | | | * R127_D124_MAG R127_D125_MAG R127_D126_MAG R127_D127_MAG *
out	*WR↩ _3*	1st 24bit(RE): 0 2nd 24bit(IM): Offset(restricted to 24bit value!) of the Histogram OUTPUT buffer, relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 16 bit values histogram counters * 0x0 bin1 bin0 * 0x4 bin3 bin2 * | | * bin61 bin60 * bin63 bin62 -> Range Bin 0 * ---------- ---------- * bin1 bin0 * bin3 bin2 * | | * bin61 bin60 * bin63 bin62 -> Range Bin 1 * ---------- ---------- * | | * | | * | | * ---------- ---------- * bin1 bin0 * bin3 bin2 * | | * bin61 bin60 * bin63 bin62 -> Range Bin 127 * ---------- ---------- *

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

Twiddle coeficients are placed in TRAM at TR_0_96_0 by the Init function.

Note

The current implementation is ONLY to be used in combination with 256 Range FFT and 128 Doppler FFT, 3TX TD-MIMO.
The input buffer MUST NOT be reused for the histogram or RDM output.
All buffers MUST be 8 bytes aligned.

3.9.2.7 RsdkSpt3Dfft256smp128crp8ch6TxTdMimoCp4d()

```
void * RsdkSpt3Dfft256smp128crp8ch6TxTdMimoCp4d (
            void  )
```

3.9.2.8 RsdkSpt3Dfft256smp256crp4ch()

```
void * RsdkSpt3Dfft256smp256crp4ch (
            void  )
```

Function for 3D FFT, 256 chirps, 256 sample/chirp, 4 channels(CH#0..CH#3).

Compute 3D FFT, storing results in SRAM.

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Doppler FFT results), relative to SRAM start address, RSDK_SPT_CUBE_BASE_ADDR.

```
*                 INPUT Memory layout:
*                 SRAM, 32bit complex(16b_Im + 16b_Re)
*                 63:48               47:32               31:16               15:0
*                 CH0_0-Im-Crp0       CH0_0-Re-Crp0       CH1_0-Im-Crp0       CH1_0-Re-Crp0
*                 CH2_0-Im-Crp0       CH2_0-Re-Crp0       CH3_0-Im-Crp0       CH3_0-Re-Crp0
*                 |                   |                   |                   |
*                 CH0_0-Im-Crp255     CH0_0-Re-Crp255     CH1_0-Im-Crp255     CH1_0-Re-Crp255
*                 CH2_0-Im-Crp255     CH2_0-Re-Crp255     CH3_0-Im-Crp255     CH3_0-Re-Crp255
*                 ------------------------------------------------------------------------------
*                 |                   |                   |                   |
*                 |                   |                   |                   |
*                 |                   |                   |                   |
*                 ------------------------------------------------------------------------------
*                 CH0_127-Im-Crp0     CH0_127-Re-Crp0     CH1_127-Im-Crp0     CH1_127-Re-Crp0
*                 CH2_127-Im-Crp0     CH2_127-Re-Crp0     CH3_127_Im-Crp0     CH3_127-Re-Crp0
*                 |                   |                   |                   |
*                 CH0_127-Im-Crp255   CH0_127-Re-Crp255   CH1_127-Im-Crp255   CH1_127-Re-Crp2
*                 CH2_127-Im-Crp255   CH2_127-Re-Crp255   CH3_127_Im-Crp255   CH3_127-Re-Crp2
*                 ------------------------------------------------------------------------------
*
```

Parameters

out	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the RDM OUTPUT buffer(3D FFT matrix), relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 16 bit real Log2 Mag2 format * 63:48 47:32 31:16 15:0 * R0_D0_MAG R0_D1_MAG R0_D2_MAG R0_D3_MAG * | | | | * R0_D252_MAG R0_D253_MAG R0_D254_MAG R0_D255_MAG * R1_D0_MAG R1_D1_MAG R1_D2_MAG R1_D3_MAG * | | | | * R1_D252_MAG R1_D253_MAG R1_D254_MAG R1_D255_MAG * | | | | * | | | | * | | | | * | | | | * | | | | * R127_D0_MAG R127_D1_MAG R127_D2_MAG R127_D3_MAG * | | | | * R127_D252_MAG R127_D253_MAG R127_D254_MAG R127_D255_MAG *
out	*WR↩ _3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the Histogram OUTPUT buffer, relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 16 bit values histogram counters * 0x0 bin1 bin0 * 0x4 bin3 bin2 * | | * bin61 bin60 * bin63 bin62 -> Range Bin 0 * --------------------------- * bin1 bin0 * bin3 bin2 * | | * bin61 bin60 * bin63 bin62 -> Range Bin 1 * --------------------------- * | | * | | * | | * --------------------------- * bin1 bin0 * bin3 bin2 * | | * bin61 bin60 * bin63 bin62 -> Range Bin 127 * --------------------------- *

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

Twiddle coeficients are placed in TRAM at TR_0_96_0 by the Init function.

Note

The current implementation is ONLY to be used in combination with 256 Range FFT and 256 Doppler FFT.
The input buffer MUST NOT be reused for the histogram or RDM output.
All buffers MUST be 8 bytes aligned.

3.9.2.9 RsdkSpt3Dfft256smp256crp4ch2TxTdMimoCp4d()

```
void * RsdkSpt3Dfft256smp256crp4ch2TxTdMimoCp4d (
            void  )
```

Function for 3D FFT, 256 chirps, 256 sample/chirp, 4 physical channels/TX slot, 2TX TD-MIMO.

Compute 3D FFT, storing results in SRAM.

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Doppler FFT compressed results), relative to SRAM start address, RSDK_SPT_CUBE_BASE_ADDR. * INPUT Memory layout: * SRAM, 64bit compressed data * 63:0 * CH0_0-CH1_0-CH2_0-CH3_0-TX0_Crp0 * CH0_0-CH1_0-CH2_0-CH3_0-TX1_Crp0 * | * CH0_0-CH1_0-CH2_0-CH3_0-TX0_Crp255 * CH0_0-CH1_0-CH2_0-CH3_0-TX1_Crp255 ---> RangeBin ##0 * ---------- * | * | * | * ---------- * CH0_127-CH1_127-CH2_127-CH3_127-TX0_Crp0 * CH0_127-CH1_127-CH2_127-CH3_127-TX1_Crp0 * | * CH0_127-CH1_127-CH2_127-CH3_127-TX0_Crp255 * CH0_127-CH1_127-CH2_127-CH3_127-TX1_Crp255 ---> RangeBin ##127 * ---------- *

Parameters

out	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the RDM OUTPUT buffer(3D FFT matrix), relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 16 bit real Log2 Mag2 format * 63:48 47:32 31:16 15:0 * R0_D0_MAG R0_D1_MAG R0_D2_MAG R0_D3_MAG * | | | | * R0_D252_MAG R0_D253_MAG R0_D254_MAG R0_D255_MAG * R1_D0_MAG R1_D1_MAG R1_D2_MAG R1_D3_MAG * | | | | * R1_D252_MAG R1_D253_MAG R1_D254_MAG R1_D255_MAG * | | | | * | | | | * | | | | * | | | | * | | | | * R127_D0_MAG R127_D1_MAG R127_D2_MAG R127_D3_MAG * | | | | * R127_D252_MAG R127_D253_MAG R127_D254_MAG R127_D255_MAG *
out	*WR↩ _3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the Histogram OUTPUT buffer, relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 16 bit values histogram counters * 0x0 bin1 bin0 * 0x4 bin3 bin2 * | | * bin61 bin60 * bin63 bin62 -> Range Bin 0 * --------------------------- * bin1 bin0 * bin3 bin2 * | | * bin61 bin60 * bin63 bin62 -> Range Bin 1 * --------------------------- * | | * | | * | | * --------------------------- * bin1 bin0 * bin3 bin2 * | | * bin61 bin60 * bin63 bin62 -> Range Bin 127 * --------------------------- *

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

Twiddle coeficients are placed in TRAM at TR_0_96_0 by the Init function.

Note

The current implementation is ONLY to be used in combination with 256 Range FFT and 256 Doppler FFT, 2TX TD-MIMO.
The input buffer MUST NOT be reused for the histogram or RDM output.
All buffers MUST be 8 bytes aligned.

3.9.2.10 RsdkSpt3Dfft256smp256crp8ch()

```
void * RsdkSpt3Dfft256smp256crp8ch (
            void  )
```

3.9.2.11 RsdkSpt3Dfft256smp256crp8ch2TxTdMimoCp4d()

```
void * RsdkSpt3Dfft256smp256crp8ch2TxTdMimoCp4d (
            void  )
```

3.9.2.12 RsdkSpt3Dfft512smp128crp4ch()

```
void * RsdkSpt3Dfft512smp128crp4ch (
            void  )
```

Function for 3D FFT, 128 chirps, 512 sample/chirp, 4 channels(CH#0..CH#3)

Compute 3D FFT, storing results in SRAM.

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Doppler FFT results), relative to SRAM start address, RSDK_SPT_CUBE_BASE_ADDR. * INPUT Memory layout: * SRAM, 32 bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * CH0_0-Im-Crp0 CH0_0-Re-Crp0 CH1_0-Im-Crp0 CH1_0-Re-Crp0 * CH2_0-Im-Crp0 CH2_0-Re-Crp0 CH3_0-Im-Crp0 CH3_0-Re-Crp0 * | | | | * CH0_0-Im-Crp127 CH0_0-Re-Crp127 CH1_0-Im-Crp127 CH1_0-Re-Crp127 * CH2_0-Im-Crp127 CH2_0-Re-Crp127 CH3_0-Im-Crp127 CH3_0-Re-Crp127 * ---------- * | | | | * | | | | * | | | | * ---------- * CH0_255-Im-Crp0 CH0_255-Re-Crp0 CH1_255-Im-Crp0 CH1_255-Re-Crp0 * CH2_255-Im-Crp0 CH2_255-Re-Crp0 CH3_255-Im-Crp0 CH3_255-Re-Crp0 * | | | | * CH0_255-Im-Crp127 CH0_255-Re-Crp127 CH1_255-Im-Crp127 CH1_255-Re-Crp1 * CH2_255-Im-Crp127 CH2_255-Re-Crp127 CH3_255-Im-Crp127 CH3_255-Re-Crp1 * ---------- *
out	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the RDM OUTPUT buffer(3D FFT matrix), relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 16 bit real Log2 Mag2 format * 63:48 47:32 31:16 15:0 * R0_D0_MAG R0_D1_MAG R0_D2_MAG R0_D3_MAG * | | | | * R0_D124_MAG R0_D125_MAG R0_D126_MAG R0_D127_MAG * R1_D0_MAG R1_D1_MAG R1_D2_MAG R1_D3_MAG * | | | | * R1_D124_MAG R1_D125_MAG R1_D126_MAG R1_D127_MAG * | | | | * | | | | * | | | | * | | | | * | | | | * R255_D0_MAG R255_D1_MAG R255_D2_MAG R255_D3_MAG * | | | | * R255_D124_MAG R255_D125_MAG R255_D126_MAG R255_D127_MAG *

Parameters

out	*WR↩ _3*	24bit(RE): 0
		24bit(IM): Offset(restricted to 24bit value!) of the Histogram OUTPUT buffer, relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR.
		
		`*  OUTPUT Memory layout:`
		`*  SRAM, 16 bit values histogram counters`
		`*  0x0  bin1  bin0`
		`*  0x4  bin3  bin2`
		`*  |  |`
		`*  bin61 bin60`
		`*  bin63 bin62 -> Range Bin 0`
		`*  ----------`
		`*  bin1  bin0`
		`*  bin3  bin2`
		`*  |  |`
		`*  bin61 bin60`
		`*  bin63 bin62 -> Range Bin 1`
		`*  ----------`
		`*  |  |`
		`*  |  |`
		`*  |  |`
		`*  ----------`
		`*  bin1  bin0`
		`*  bin3  bin2`
		`*  |  |`
		`*  bin61 bin60`
		`*  bin63 bin62 -> Range Bin 255`
		`*  ----------`
		`*`

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

Twiddle coeficients are placed in TRAM at TR_0_160_0 by the Init function.

Note

The current implementation is ONLY to be used in combination with 512 Range FFT and 128 Doppler FFT.
The input buffer MUST NOT be reused for the histogram or RDM output.
All buffers MUST be 8 bytes aligned.

3.9.2.13 RsdkSpt3Dfft512smp128crp8ch()

```
void * RsdkSpt3Dfft512smp128crp8ch (
            void  )
```

3.9.2.14 RsdkSptCheckTram1024smp128crp4ch()

```
void * RsdkSptCheckTram1024smp128crp4ch (
            void  )
```

Function for checking sanity of persistent TRAM content. 1024 samples, 128 chirps and 4 physical channels.

Compare persistent TRAM content with the SRAM/flash reference. The kernell will reload the twiddles, window, steering vector coefficients into SPT memory and compare them with the data already stored in TRAM at the initialization time.

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT1024 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * | | | | * | | | | * | | | | * tw128_Im tw128_Re tw128_Im tw128_Re * tw128_Im tw128_Re tw128_Im tw128_Re * tw128_Im tw128_Re tw128_Im tw128_Re * tw128_Im tw128_Re tw128_Im tw128_Re *
in	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT128 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * | | | | * | | | | * | | | | * tw16_Im tw16_Re tw16_Im tw16_Re * tw16_Im tw16_Re tw16_Im tw16_Re * tw16_Im tw16_Re tw16_Im tw16_Re * tw16_Im tw16_Re tw16_Im tw16_Re *

Parameters

in	*WR_3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Reordered Window coef for Range FFT1024), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 16bit Real Window Coef * 63:48 47:32 31:16 15:0 * win0 win256 win512 win768 * win64 win320 win576 win832 * | | | | * | | | | * win63 win319 win575 win831 * win127 win383 win639 win895 * win128 win384 win640 win896 * win192 win448 win704 win960 * | | | | * | | | | * win191 win447 win703 win959 * win255 win511 win767 win1023 *
in	*WR_4*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Reordered Window coef for Doppler FFT128), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 16bit Real Window Coef * 63:48 47:32 31:16 15:0 * win0 win32 win64 win96 * win8 win40 win72 win104 * | | | | * | | | | * win7 win39 win71 win103 * win15 win47 win79 win111 * win16 win48 win80 win112 * win24 win56 win88 win120 * | | | | * | | | | * win23 win55 win87 win119 * win31 win63 win95 win127 *
in	*WR_5*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT16 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re *

Parameters

in	*WR↩ _6*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Beamforming Steering Vector), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 32 bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * SV0_Im-CH0 SV0_Re-CH0 SV0_Im-CH1 SV0_Re-CH1 * SV0_Im-CH2 SV0_Re-CH2 SV0_Im-CH3 SV0_Re-CH3 * SV1_Im-CH0 SV1_Re-CH0 SV1_Im-CH1 SV1_Re-CH1 * SV1_Im-CH2 SV1_Re-CH2 SV1_Im-CH3 SV1_Re-CH3 * | | | | * | | | | * | | | | * SV63_Im-CH0 SV63_Re-CH0 SV63_Im-CH1 SV63_Re-CH1 * SV63_Im-CH2 SV63_Re-CH2 SV63_Im-CH3 SV63_Re-CH3 *

Returns

WR_0: will indicate if persistent TRAM content is corrupted or not.

Return values

0,if	persistent TRAM content is not corrupted. >0, if persistent TRAM content is corrupted.

Note

All buffers MUST be 8 bytes aligned.

3.9.2.15 RsdkSptCheckTram1024smp128crp8ch()

```
void * RsdkSptCheckTram1024smp128crp8ch (
            void  )
```

3.9.2.16 RsdkSptCheckTram1024smp256crp4ch3TxDdMimo()

```
void * RsdkSptCheckTram1024smp256crp4ch3TxDdMimo (
            void  )
```

Function for checking sanity of persistent TRAM content. 1024 samples, 256 chirps, 4 physical channels, 3TX DD-Mimo.

Compare persistent TRAM content with the SRAM/flash reference. The kernell will reload the twiddles, window, steering vector coefficients into SPT memory and compare them with the data already stored in TRAM at the initialization time.

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT1024 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * | | | | * | | | | * | | | | * tw128_Im tw128_Re tw128_Im tw128_Re * tw128_Im tw128_Re tw128_Im tw128_Re * tw128_Im tw128_Re tw128_Im tw128_Re * tw128_Im tw128_Re tw128_Im tw128_Re *
in	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT256 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * | | | | * | | | | * | | | | * tw32_Im tw32_Re tw32_Im tw32_Re * tw32_Im tw32_Re tw32_Im tw32_Re * tw32_Im tw32_Re tw32_Im tw32_Re * tw32_Im tw32_Re tw32_Im tw32_Re *
in	*WR↩ _3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Reordered Window coef for Range FFT1024), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 16bit Real Window Coef * 63:48 47:32 31:16 15:0 * win0 win256 win512 win768 * win64 win320 win576 win832 * | | | | * | | | | * win63 win319 win575 win831 * win127 win383 win639 win895 * win128 win384 win640 win896 * win192 win448 win704 win960 * | | | | * | | | | * win191 win447 win703 win959 * win255 win511 win767 win1023 *

Parameters

in	*WR_4*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Reordered Window coef for Doppler FFT256), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 16bit Real Window Coef * 63:48 47:32 31:16 15:0 * win0 win64 win128 win192 * win16 win80 win144 win208 * | | | | * | | | | * win15 win79 win143 win207 * win31 win95 win159 win223 * win32 win96 win160 win224 * win48 win112 win176 win240 * | | | | * | | | | * win47 win111 win175 win239 * win63 win127 win191 win255 *
in	*WR_5*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Beamforming Steering Vector), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 32 bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * SV0_Im-CH0 SV0_Re-CH0 SV0_Im-CH1 SV0_Re-CH1 * SV0_Im-CH2 SV0_Re-CH2 SV0_Im-CH3 SV0_Re-CH3 * SV0_Im-CH4 SV0_Re-CH4 SV0_Im-CH5 SV0_Re-CH5 * SV0_Im-CH6 SV0_Re-CH6 SV0_Im-CH7 SV0_Re-CH7 * SV0_Im-CH8 SV0_Re-CH8 SV0_Im-CH9 SV0_Re-CH9 * SV0_Im-CH10 SV0_Re-CH10 SV0_Im-CH11 SV0_Re-CH11 * | | | | * | | | | * | | | | * SV63_Im-CH0 SV63_Re-CH0 SV63_Im-CH1 SV63_Re-CH1 * SV63_Im-CH2 SV63_Re-CH2 SV63_Im-CH3 SV63_Re-CH3 * SV63_Im-CH4 SV63_Re-CH4 SV63_Im-CH5 SV63_Re-CH5 * SV63_Im-CH6 SV63_Re-CH6 SV63_Im-CH7 SV63_Re-CH7 * SV63_Im-CH8 SV63_Re-CH8 SV63_Im-CH9 SV63_Re-CH9 * SV63_Im-CH10 SV63_Re-CH10 SV63_Im-CH11 SV63_Re-CH11 *

Returns

WR_0: will indicate if persistent TRAM content is corrupted or not.

Return values

0,if	persistent TRAM content is not corrupted. >0, if persistent TRAM content is corrupted.

Note

All buffers MUST be 8 bytes aligned.

3.9.2.17 RsdkSptCheckTram1024smp64crp4ch()

```
void * RsdkSptCheckTram1024smp64crp4ch (
            void  )
```

Function for checking sanity of persistent TRAM content. 1024 samples, 64 chirps and 4 physical channels.

Compare persistent TRAM content with the SRAM/flash reference. The kernell will reload the twiddles, window, steering vector coefficients into SPT memory and compare them with the data already stored in TRAM at the initialization time.

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT1024 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * | | | | * | | | | * | | | | * tw128_Im tw128_Re tw128_Im tw128_Re * tw128_Im tw128_Re tw128_Im tw128_Re * tw128_Im tw128_Re tw128_Im tw128_Re * tw128_Im tw128_Re tw128_Im tw128_Re *
in	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT64 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * | | | | * | | | | * | | | | * tw8_Im tw8_Re tw8_Im tw8_Re * tw8_Im tw8_Re tw8_Im tw8_Re * tw8_Im tw8_Re tw8_Im tw8_Re * tw8_Im tw8_Re tw8_Im tw8_Re *

Parameters

in	*WR↩ _3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Reordered Window coef for Range FFT1024), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 16bit Real Window Coef * 63:48 47:32 31:16 15:0 * win0 win256 win512 win768 * win64 win320 win576 win832 * | | | | * | | | | * win63 win319 win575 win831 * win127 win383 win639 win895 * win128 win384 win640 win896 * win192 win448 win704 win960 * | | | | * | | | | * win191 win447 win703 win959 * win255 win511 win767 win1023 *
in	*WR↩ _4*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Reordered Window coef for Doppler FFT64), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 16bit Real Window Coef * 63:48 47:32 31:16 15:0 * win0 win16 win32 win48 * win4 win20 win36 win52 * | | | | * | | | | * win3 win19 win35 win51 * win7 win23 win39 win55 * win8 win24 win40 win56 * win12 win28 win44 win60 * | | | | * | | | | * win11 win27 win43 win59 * win15 win31 win47 win63 *
in	*WR↩ _5*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT16 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re *

Parameters

in	*WR_6*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Beamforming Steering Vector), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 32 bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * SV0_Im-CH0 SV0_Re-CH0 SV0_Im-CH1 SV0_Re-CH1 * SV0_Im-CH2 SV0_Re-CH2 SV0_Im-CH3 SV0_Re-CH3 * SV1_Im-CH0 SV1_Re-CH0 SV1_Im-CH1 SV1_Re-CH1 * SV1_Im-CH2 SV1_Re-CH2 SV1_Im-CH3 SV1_Re-CH3 * | | | | * | | | | * | | | | * SV63_Im-CH0 SV63_Re-CH0 SV63_Im-CH1 SV63_Re-CH1 * SV63_Im-CH2 SV63_Re-CH2 SV63_Im-CH3 SV63_Re-CH3 *

Returns

WR_0: will indicate if persistent TRAM content is corrupted or not.

Return values

0,if	persistent TRAM content is not corrupted. >0, if persistent TRAM content is corrupted.

Note

All buffers MUST be 8 bytes aligned.

3.9.2.18 RsdkSptCheckTram1024smp64crp4ch4TxTdMimo()

```
void * RsdkSptCheckTram1024smp64crp4ch4TxTdMimo (
            void  )
```

Function for checking sanity of persistent TRAM content. 1024 samples, 64 chirps, 4 physical channels/TX slot, 4TX TD-Mimo.

Compare persistent TRAM content with the SRAM/flash reference. The kernell will reload the twiddles, window, derotation and steering vector coefficients into SPT memory and compare them with the data already stored in TRAM at the initialization time.

Parameters

in	*WR_1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT1024 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For twiddle generation algorithm, see S32R41RM_Rev1 68.9.5.3. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * | | | | * | | | | * | | | | * tw128_Im tw128_Re tw128_Im tw128_Re * tw128_Im tw128_Re tw128_Im tw128_Re * tw128_Im tw128_Re tw128_Im tw128_Re * tw128_Im tw128_Re tw128_Im tw128_Re *
in	*WR_2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT64 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For twiddle generation algorithm, see S32R41RM_Rev1 68.9.5.3. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * | | | | * | | | | * | | | | * tw8_Im tw8_Re tw8_Im tw8_Re * tw8_Im tw8_Re tw8_Im tw8_Re * tw8_Im tw8_Re tw8_Im tw8_Re * tw8_Im tw8_Re tw8_Im tw8_Re *

Parameters

in	*WR_3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Reordered Window coef for Range FFT 1024), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For reorder algorithm, see S32R41RM_Rev1 68.9.9.7 (Table 508). * INPUT Memory layout: * SRAM, 16bit Real Window Coef * 63:48 47:32 31:16 15:0 * win0 win256 win512 win768 * win64 win320 win576 win832 * | | | | * | | | | * win63 win319 win575 win831 * win127 win383 win639 win895 * win128 win384 win640 win896 * win192 win448 win704 win960 * | | | | * | | | | * win191 win447 win703 win959 * win255 win511 win767 win1023 *
in	*WR_4*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Reordered Window coef for Doppler FFT 64), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For reorder algorithm, see see S32R41RM_Rev1 68.9.9.7. * INPUT Memory layout: * SRAM, 16bit Real Window Coef * 63:48 47:32 31:16 15:0 * win0 win16 win32 win48 * win4 win20 win36 win52 * | | | | * | | | | * win3 win19 win35 win51 * win7 win23 win39 win55 * win8 win24 win40 win56 * win12 win28 win44 win60 * | | | | * | | | | * win11 win27 win43 win59 * win15 win31 win47 win63 *

Parameters

in	*WR↩ _5*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT32 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For twiddle generation algorithm, see S32R41RM_Rev1 68.9.5.3. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Im tw1_Im tw1_Im * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw3_Im tw3_Re tw3_Im tw3_Re * tw3_Im tw3_Re tw3_Im tw3_Re * tw3_Im tw3_Re tw3_Im tw3_Re * tw3_Im tw3_Re tw3_Im tw3_Re * tw4_Im tw4_Re tw4_Im tw4_Re * tw4_Im tw4_Re tw4_Im tw4_Re * tw4_Im tw4_Re tw4_Im tw4_Re * tw4_Im tw4_Re tw4_Im tw4_Re *
in	*WR↩ _6*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Derotation Phased Vector), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For derotation coeficients generation algorithm, see rSDK_Auxiliary_vector_generation.docx. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * DeRotPh0_Im-TX1 DeRotPh0_Re-TX1 DeRotPh1_Im-TX1 DeRotPh1_Re-TX1 * DeRotPh2_Im-TX1 DeRotPh2_Re-TX1 DeRotPh3_Im-TX1 DeRotPh3_Re-TX1 * | | | | * | | | | * | | | | * DeRotPh60_Im-TX1 DeRotPh60_Re-TX1 DeRotPh61_Im-TX1 DeRotPh61_Re-TX1 * DeRotPh62_Im-TX1 DeRotPh62_Re-TX1 DeRotPh63_Im-TX1 DeRotPh63_Re-TX1 * DeRotPh0_Im-TX2 DeRotPh0_Re-TX2 DeRotPh1_Im-TX2 DeRotPh1_Re-TX2 * DeRotPh2_Im-TX2 DeRotPh2_Re-TX2 DeRotPh3_Im-TX2 DeRotPh3_Re-TX2 * | | | | * | | | | * | | | | * DeRotPh60_Im-TX2 DeRotPh60_Re-TX2 DeRotPh61_Im-TX2 DeRotPh61_Re-TX2 * DeRotPh62_Im-TX2 DeRotPh62_Re-TX2 DeRotPh63_Im-TX2 DeRotPh63_Re-TX2 * DeRotPh0_Im-TX3 DeRotPh0_Re-TX3 DeRotPh1_Im-TX3 DeRotPh1_Re-TX3 * DeRotPh2_Im-TX3 DeRotPh2_Re-TX3 DeRotPh3_Im-TX3 DeRotPh3_Re-TX3 * | | | | * | | | | * | | | | * DeRotPh60_Im-TX3 DeRotPh60_Re-TX3 DeRotPh61_Im-TX3 DeRotPh61_Re-TX3 * DeRotPh62_Im-TX3 DeRotPh62_Re-TX3 DeRotPh63_Im-TX3 DeRotPh63_Re-TX3 *

Parameters

in	*WR_7*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Beamforming Steering Vector), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For steering coeficients generation algorithm, see rSDK_Auxiliary_vector_generation.docx.

```
*                  INPUT Memory layout:
*                  SRAM, 32 bit complex(16b_Im + 16b_Re)
*                  63:48          47:32           31:16           15:0
*                  SV0_Im-CH0     SV0_Re-CH0      SV0_Im-CH1      SV0_Re-CH1
*                  SV0_Im-CH2     SV0_Re-CH2      SV0_Im-CH3      SV0_Re-CH3
*                  SV0_Im-CH4     SV0_Re-CH4      SV0_Im-CH5      SV0_Re-CH5
*                  SV0_Im-CH6     SV0_Re-CH6      SV0_Im-CH7      SV0_Re-CH7
*                  SV0_Im-CH8     SV0_Re-CH8      SV0_Im-CH9      SV0_Re-CH9
*                  SV0_Im-CH10    SV0_Re-CH10     SV0_Im-CH11     SV0_Re-CH11
*                  SV0_Im-CH12    SV0_Re-CH12     SV0_Im-CH13     SV0_Re-CH13
*                  SV0_Im-CH14    SV0_Re-CH14     SV0_Im-CH15     SV0_Re-CH15
*                  SV1_Im-CH0     SV1_Re-CH0      SV1_Im-CH1      SV1_Re-CH1
*                  SV1_Im-CH2     SV1_Re-CH2      SV1_Im-CH3      SV1_Re-CH3
*                  SV1_Im-CH4     SV1_Re-CH4      SV1_Im-CH5      SV1_Re-CH5
*                  SV1_Im-CH6     SV1_Re-CH6      SV1_Im-CH7      SV1_Re-CH7
*                  SV1_Im-CH8     SV1_Re-CH8      SV1_Im-CH9      SV1_Re-CH9
*                  SV1_Im-CH10    SV1_Re-CH10     SV1_Im-CH11     SV1_Re-CH11
*                  SV1_Im-CH12    SV1_Re-CH12     SV1_Im-CH13     SV1_Re-CH13
*                  SV1_Im-CH14    SV1_Re-CH14     SV1_Im-CH15     SV1_Re-CH15
*                  |              |               |               |
*                  |              |               |               |
*                  |              |               |               |
*                  SV63_Im-CH0    SV63_Re-CH0     SV63_Im-CH1     SV63_Re-CH1
*                  SV63_Im-CH2    SV63_Re-CH2     SV63_Im-CH3     SV63_Re-CH3
*                  SV63_Im-CH4    SV63_Re-CH4     SV63_Im-CH5     SV63_Re-CH5
*                  SV63_Im-CH6    SV63_Re-CH6     SV63_Im-CH7     SV63_Re-CH7
*                  SV63_Im-CH8    SV63_Re-CH8     SV63_Im-CH9     SV63_Re-CH9
*                  SV63_Im-CH10   SV63_Re-CH10    SV63_Im-CH11    SV63_Re-CH11
*                  SV63_Im-CH12   SV63_Re-CH12    SV63_Im-CH13    SV63_Re-CH13
*                  SV63_Im-CH14   SV63_Re-CH14    SV63_Im-CH15    SV63_Re-CH15
*
```

Returns

WR_0: will indicate if persistent TRAM content is corrupted or not.

Return values

0,if	persistent TRAM content is not corrupted. >0, if persistent TRAM content is corrupted.

Note

All buffers MUST be 8 bytes aligned.

3.9.2.19 RsdkSptCheckTram2048smp256crp4ch()

```
void * RsdkSptCheckTram2048smp256crp4ch (
            void  )
```

Function for checking sanity of persistent TRAM content. 2048 samples, 256 chirps and 4 physical channels.

Compare persistent TRAM content with the SRAM/flash reference. The kernell will reload the twiddles, window, steering vector coefficients into SPT memory and compare them with the data already stored in TRAM at the initialization time.

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT2048 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For twiddle generation algorithm, see S32R41_RM 68.9.5.3 / SAF85xx_RM 68.9.5.3. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * | | | | * | | | | * | | | | * tw256_Im tw256_Re tw256_Im tw256_Re * tw256_Im tw256_Re tw256_Im tw256_Re * tw256_Im tw256_Re tw256_Im tw256_Re * tw256_Im tw256_Re tw256_Im tw256_Re *
in	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT256 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For twiddle generation algorithm, see S32R41_RM 68.9.5.3 / SAF85xx_RM 68.9.5.3. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * | | | | * | | | | * | | | | * tw32_Im tw32_Re tw32_Im tw32_Re * tw32_Im tw32_Re tw32_Im tw32_Re * tw32_Im tw32_Re tw32_Im tw32_Re * tw32_Im tw32_Re tw32_Im tw32_Re *

Parameters

in	*WR↩ _3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Reordered Window coef for Range FFT2048), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For reorder algorithm, see S32R41_RM 68.9.9.7 / SAF85xx_RM 68.9.9.7. * INPUT Memory layout: * SRAM, 16bit Real Window Coef * 63:48 47:32 31:16 15:0 * win0 win512 win1024 win1536 * win128 win640 win1152 win1664 * | | | | * | | | | * win127 win639 win1151 win1663 * win255 win767 win1279 win1791 * win256 win768 win1280 win1792 * win384 win896 win1408 win1920 * | | | | * | | | | * win383 win895 win1407 win1919 * win511 win1023 win1535 win2047 *
in	*WR↩ _4*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Reordered Window coef for Doppler FFT 256), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For reorder algorithm, S32R41_RM 68.9.9.7 / SAF85xx_RM 68.9.9.7. * INPUT Memory layout: * SRAM, 16bit Real Window Coef * 63:48 47:32 31:16 15:0 * win0 win64 win128 win192 * win16 win80 win144 win208 * | | | | * | | | | * win15 win79 win143 win207 * win31 win95 win159 win223 * win32 win96 win160 win224 * win48 win112 win176 win240 * | | | | * | | | | * win47 win111 win175 win239 * win63 win127 win191 win255 *
in	*WR↩ _5*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT16 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For twiddle generation algorithm, see S32R41_RM 68.9.5.3 / SAF85xx_RM 68.9.5.3. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Im tw1_Im tw1_Im * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re *

Parameters

in	*WR↩ _6*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Beamforming Steering Vector), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For steering coeficients generation algorithm, see rSDK_Auxiliary_vector_generation.docx. * INPUT Memory layout: * SRAM, 32 bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * SV0_Im-CH0 SV0_Re-CH0 SV0_Im-CH1 SV0_Re-CH1 * SV0_Im-CH2 SV0_Re-CH2 SV0_Im-CH3 SV0_Re-CH3 * SV1_Im-CH0 SV1_Re-CH0 SV1_Im-CH1 SV1_Re-CH1 * SV1_Im-CH2 SV1_Re-CH2 SV1_Im-CH3 SV1_Re-CH3 * | | | | * | | | | * | | | | * SV63_Im-CH0 SV63_Re-CH0 SV63_Im-CH1 SV63_Re-CH1 * SV63_Im-CH2 SV63_Re-CH2 SV63_Im-CH3 SV63_Re-CH3 *

Returns

WR_0: will indicate if persistent TRAM content is corrupted or not.

Return values

0,if	persistent TRAM content is not corrupted. >0, if persistent TRAM content is corrupted.

Note

All buffers MUST be 8 bytes aligned.

3.9.2.20 RsdkSptCheckTram256smp128crp4ch3TxTdMimo()

```
void * RsdkSptCheckTram256smp128crp4ch3TxTdMimo (
            void  )
```

Function for checking sanity of persistent TRAM content. 256 samples, 128 chirps, 4 physical channels/TX slot, 3TX TD-Mimo.

Compare persistent TRAM content with the SRAM/flash reference. The kernell will reload the twiddles, window, derotation and steering vector coefficients into SPT memory and compare them with the data already stored in TRAM at the initialization time.

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT256 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * | | | | * | | | | * | | | | * tw32_Im tw32_Re tw32_Im tw32_Re * tw32_Im tw32_Re tw32_Im tw32_Re * tw32_Im tw32_Re tw32_Im tw32_Re * tw32_Im tw32_Re tw32_Im tw32_Re *
in	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT128 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * | | | | * | | | | * | | | | * tw16_Im tw16_Re tw16_Im tw16_Re * tw16_Im tw16_Re tw16_Im tw16_Re * tw16_Im tw16_Re tw16_Im tw16_Re * tw16_Im tw16_Re tw16_Im tw16_Re *
in	*WR↩ _3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Reordered Window coef for Range FFT256), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 16bit Real Window Coef * 63:48 47:32 31:16 15:0 * win0 win64 win128 win192 * win16 win80 win144 win208 * | | | | * | | | | * win15 win79 win143 win207 * win31 win95 win159 win223 * win32 win96 win160 win224 * win48 win112 win176 win240 * | | | | * | | | | * win47 win111 win175 win239 * win63 win127 win191 win255 *

Parameters

in	*WR↩ _4*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Reordered Window coef for Doppler FFT128), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 16bit Real Window Coef * 63:48 47:32 31:16 15:0 * win0 win32 win64 win96 * win8 win40 win72 win104 * | | | | * | | | | * win7 win39 win71 win103 * win15 win47 win79 win111 * win16 win48 win80 win112 * win24 win56 win88 win120 * | | | | * | | | | * win23 win55 win87 win119 * win31 win63 win95 win127 *
in	*WR↩ _5*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT32 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw3_Im tw3_Re tw3_Im tw3_Re * tw3_Im tw3_Re tw3_Im tw3_Re * tw3_Im tw3_Re tw3_Im tw3_Re * tw3_Im tw3_Re tw3_Im tw3_Re * tw4_Im tw4_Re tw4_Im tw4_Re * tw4_Im tw4_Re tw4_Im tw4_Re * tw4_Im tw4_Re tw4_Im tw4_Re *
in	*WR↩ _6*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Derotation Phased Vector), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * DeRotPh0_Im-TX1 DeRotPh0_Re-TX1 DeRotPh1_Im-TX1 DeRotPh1_Re-TX1 * DeRotPh2_Im-TX1 DeRotPh2_Re-TX1 DeRotPh3_Im-TX1 DeRotPh3_Re-TX1 * | | | | * | | | | * | | | | * DeRotPh124_Im-TX1 DeRotPh124_Re-TX1 DeRotPh125_Im-TX1 DeRotPh125_Re-TX1 * DeRotPh126_Im-TX1 DeRotPh126_Re-TX1 DeRotPh127_Im-TX1 DeRotPh127_Re-TX1 * DeRotPh0_Im-TX2 DeRotPh0_Re-TX2 DeRotPh1_Im-TX2 DeRotPh1_Re-TX2 * DeRotPh2_Im-TX2 DeRotPh2_Re-TX2 DeRotPh3_Im-TX2 DeRotPh3_Re-TX2 * | | | | * | | | | * | | | | * DeRotPh124_Im-TX2 DeRotPh124_Re-TX2 DeRotPh125_Im-TX2 DeRotPh125_Re-TX2 * DeRotPh126_Im-TX2 DeRotPh126_Re-TX2 DeRotPh127_Im-TX2 DeRotPh127_Re-TX2 *

Parameters

in	*WR_7*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Beamforming Steering Vector), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR.

```
*                  INPUT Memory layout:
*                  SRAM, 32 bit complex(16b_Im + 16b_Re)
*                  63:48          47:32           31:16           15:0
*                  SV0_Im-CH0     SV0_Re-CH0      SV0_Im-CH1      SV0_Re-CH1
*                  SV0_Im-CH2     SV0_Re-CH2      SV0_Im-CH3      SV0_Re-CH3
*                  SV0_Im-CH4     SV0_Re-CH4      SV0_Im-CH5      SV0_Re-CH5
*                  SV0_Im-CH6     SV0_Re-CH6      SV0_Im-CH7      SV0_Re-CH7
*                  SV0_Im-CH8     SV0_Re-CH8      SV0_Im-CH9      SV0_Re-CH9
*                  SV0_Im-CH10    SV0_Re-CH10     SV0_Im-CH11     SV0_Re-CH11
*                  |              |               |               |
*                  |              |               |               |
*                  |              |               |               |
*                  SV63_Im-CH0    SV63_Re-CH0     SV63_Im-CH1     SV63_Re-CH1
*                  SV63_Im-CH2    SV63_Re-CH2     SV63_Im-CH3     SV63_Re-CH3
*                  SV63_Im-CH4    SV63_Re-CH4     SV63_Im-CH5     SV63_Re-CH5
*                  SV63_Im-CH6    SV63_Re-CH6     SV63_Im-CH7     SV63_Re-CH7
*                  SV63_Im-CH8    SV63_Re-CH8     SV63_Im-CH9     SV63_Re-CH9
*                  SV63_Im-CH10   SV63_Re-CH10    SV63_Im-CH11    SV63_Re-CH11
*
```

Returns

WR_0: will indicate if persistent TRAM content is corrupted or not.

Return values

0,if	persistent TRAM content is not corrupted. $>$0, if persistent TRAM content is corrupted.

Note

All buffers MUST be 8 bytes aligned.

3.9.2.21 RsdkSptCheckTram256smp128crp8ch6TxTdMimo()

```
void * RsdkSptCheckTram256smp128crp8ch6TxTdMimo (
            void  )
```

3.9.2.22 RsdkSptCheckTram256smp256crp4ch()

```
void * RsdkSptCheckTram256smp256crp4ch (
            void  )
```

Function for checking sanity of persistent TRAM content. 256 samples, 256 chirps and 4 physical channels.

Compare persistent TRAM content with the SRAM/flash reference. The kernell will reload the twiddles, window, steering vector coefficients into SPT memory and compare them with the data already stored in TRAM at the initialization time.

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT256 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * | | | | * | | | | * | | | | * tw32_Im tw32_Re tw32_Im tw32_Re * tw32_Im tw32_Re tw32_Im tw32_Re * tw32_Im tw32_Re tw32_Im tw32_Re * tw32_Im tw32_Re tw32_Im tw32_Re *
in	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Reordered Window coef for Range FFT 256), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 16bit Real Window Coef * 63:48 47:32 31:16 15:0 * win0 win64 win128 win192 * win16 win80 win144 win208 * | | | | * | | | | * win15 win79 win143 win207 * win31 win95 win159 win223 * win32 win96 win160 win224 * win48 win112 win176 win240 * | | | | * | | | | * win47 win111 win175 win239 * win63 win127 win191 win255 *
in	*WR↩ _3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Reordered Window coef for Doppler FFT 256), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 16bit Real Window Coef * 63:48 47:32 31:16 15:0 * win0 win64 win128 win192 * win16 win80 win144 win208 * | | | | * | | | | * win15 win79 win143 win207 * win31 win95 win159 win223 * win32 win96 win160 win224 * win48 win112 win176 win240 * | | | | * | | | | * win47 win111 win175 win239 * win63 win127 win191 win255 *

Parameters

in	*WR_4*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT16 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Im tw1_Im tw1_Im * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re *
in	*WR_5*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Beamforming Steering Vector), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 32 bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * SV0_Im-CH0 SV0_Re-CH0 SV0_Im-CH1 SV0_Re-CH1 * SV0_Im-CH2 SV0_Re-CH2 SV0_Im-CH3 SV0_Re-CH3 * SV1_Im-CH0 SV1_Re-CH0 SV1_Im-CH1 SV1_Re-CH1 * SV1_Im-CH2 SV1_Re-CH2 SV1_Im-CH3 SV1_Re-CH3 * | | | | * | | | | * | | | | * SV63_Im-CH0 SV63_Re-CH0 SV63_Im-CH1 SV63_Re-CH1 * SV63_Im-CH2 SV63_Re-CH2 SV63_Im-CH3 SV63_Re-CH3 *

Returns

WR_0: will indicate if persistent TRAM content is corrupted or not.

Return values

0,if	persistent TRAM content is not corrupted. >0, if persistent TRAM content is corrupted.

Note

All buffers MUST be 8 bytes aligned.

3.9.2.23 RsdkSptCheckTram256smp256crp4ch2TxTdMimo()

```
void * RsdkSptCheckTram256smp256crp4ch2TxTdMimo (
            void  )
```

Function for checking sanity of persistent TRAM content. 256 samples, 256 chirps, 4 physical channels/TX slot, 2TX TD-Mimo.

Compare persistent TRAM content with the SRAM/flash reference. The kernell will reload the twiddles, window, derotation and steering vector coefficients into SPT memory and compare them with the data already stored in TRAM at the initialization time.

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT256 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * | | | | * | | | | * | | | | * tw32_Im tw32_Re tw32_Im tw32_Re * tw32_Im tw32_Re tw32_Im tw32_Re * tw32_Im tw32_Re tw32_Im tw32_Re * tw32_Im tw32_Re tw32_Im tw32_Re *
in	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Reordered Window coef for Range FFT 256), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 16bit Real Window Coef * 63:48 47:32 31:16 15:0 * win0 win64 win128 win192 * win16 win80 win144 win208 * | | | | * | | | | * win15 win79 win143 win207 * win31 win95 win159 win223 * win32 win96 win160 win224 * win48 win112 win176 win240 * | | | | * | | | | * win47 win111 win175 win239 * win63 win127 win191 win255 *

Parameters

in	*WR_3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Reordered Window coef for Doppler FFT 256), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 16bit Real Window Coef * 63:48 47:32 31:16 15:0 * win0 win64 win128 win192 * win16 win80 win144 win208 * | | | | * | | | | * win15 win79 win143 win207 * win31 win95 win159 win223 * win32 win96 win160 win224 * win48 win112 win176 win240 * | | | | * | | | | * win47 win111 win175 win239 * win63 win127 win191 win255 *
in	*WR_4*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT16 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Im tw1_Im tw1_Im * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re *
in	*WR_5*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Derotation Phased Vector), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * DeRotPh0_Im DeRotPh0_Re DeRotPh1_Im DeRotPh1_Re * DeRotPh2_Im DeRotPh2_Re DeRotPh3_Im DeRotPh3_Re * | | | | * | | | | * | | | | * DeRotPh252_Im DeRotPh252_Re DeRotPh253_Im DeRotPh253_Re * DeRotPh254_Im DeRotPh254_Re DeRotPh255_Im DeRotPh255_Re *

Parameters

in	*WR↩ _6*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Beamforming Steering Vector), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 32 bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * SV0_Im-CH0 SV0_Re-CH0 SV0_Im-CH1 SV0_Re-CH1 * SV0_Im-CH2 SV0_Re-CH2 SV0_Im-CH3 SV0_Re-CH3 * SV0_Im-CH4 SV0_Re-CH4 SV0_Im-CH5 SV0_Re-CH5 * SV0_Im-CH6 SV0_Re-CH6 SV0_Im-CH7 SV0_Re-CH7 * SV1_Im-CH0 SV1_Re-CH0 SV1_Im-CH1 SV1_Re-CH1 * SV1_Im-CH2 SV1_Re-CH2 SV1_Im-CH3 SV1_Re-CH3 * SV1_Im-CH4 SV1_Re-CH4 SV1_Im-CH5 SV1_Re-CH5 * SV1_Im-CH6 SV1_Re-CH6 SV1_Im-CH7 SV1_Re-CH7 *			| *			| *			| * SV63_Im-CH0 SV63_Re-CH0 SV63_Im-CH1 SV63_Re-CH1 * SV63_Im-CH2 SV63_Re-CH2 SV63_Im-CH3 SV63_Re-CH3 * SV63_Im-CH4 SV63_Re-CH4 SV63_Im-CH5 SV63_Re-CH5 * SV63_Im-CH6 SV63_Re-CH6 SV63_Im-CH7 SV63_Re-CH7 *

Returns

WR_0: will indicate if persistent TRAM content is corrupted or not.

Return values

0,if	persistent TRAM content is not corrupted. >0, if persistent TRAM content is corrupted.

Note

All buffers MUST be 8 bytes aligned.

3.9.2.24 RsdkSptCheckTram256smp256crp8ch()

```
void * RsdkSptCheckTram256smp256crp8ch (
            void  )
```

3.9.2.25 RsdkSptCheckTram256smp256crp8ch2TxTdMimo()

```
void * RsdkSptCheckTram256smp256crp8ch2TxTdMimo (
            void  )
```

3.9.2.26 RsdkSptCheckTram512smp128crp4ch()

```
void * RsdkSptCheckTram512smp128crp4ch (
            void  )
```

Function for checking sanity of persistent TRAM content. 512 samples, 128 chirps and 4 physical channels.

Compare persistent TRAM content with the SRAM/flash reference. The kernell will reload the twiddles, window, steering vector coefficients into SPT memory and compare them with the data already stored in TRAM at the initialization time.

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT512 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * | | | | * | | | | * | | | | * tw64_Im tw64_Re tw64_Im tw64_Re * tw64_Im tw64_Re tw64_Im tw64_Re * tw64_Im tw64_Re tw64_Im tw64_Re * tw64_Im tw64_Re tw64_Im tw64_Re *
in	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT128 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * | | | | * | | | | * | | | | * tw16_Im tw16_Re tw16_Im tw16_Re * tw16_Im tw16_Re tw16_Im tw16_Re * tw16_Im tw16_Re tw16_Im tw16_Re * tw16_Im tw16_Re tw16_Im tw16_Re *

Parameters

in	*WR↩ _3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Reordered Window coef for Range FFT512), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 16bit Real Window Coef * 63:48 47:32 31:16 15:0 * win0 win128 win256 win384 * win32 win160 win288 win416 * | | | | * | | | | * win31 win159 win287 win415 * win63 win191 win319 win447 * win64 win192 win320 win448 * win96 win224 win352 win480 * | | | | * | | | | * win95 win223 win351 win479 * win127 win255 win383 win511 *
in	*WR↩ _4*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Reordered Window coef for Doppler FFT128), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 16bit Real Window Coef * 63:48 47:32 31:16 15:0 * win0 win32 win64 win96 * win8 win40 win72 win104 * | | | | * | | | | * win7 win39 win71 win103 * win15 win47 win79 win111 * win16 win48 win80 win112 * win24 win56 win88 win120 * | | | | * | | | | * win23 win55 win87 win119 * win31 win63 win95 win127 *
in	*WR↩ _5*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT16 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re *

Parameters

in	*WR_6*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Beamforming Steering Vector), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 32 bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * SV0_Im-CH0 SV0_Re-CH0 SV0_Im-CH1 SV0_Re-CH1 * SV0_Im-CH2 SV0_Re-CH2 SV0_Im-CH3 SV0_Re-CH3 * SV1_Im-CH0 SV1_Re-CH0 SV1_Im-CH1 SV1_Re-CH1 * SV1_Im-CH2 SV1_Re-CH2 SV1_Im-CH3 SV1_Re-CH3 * | | | | * | | | | * | | | | * SV63_Im-CH0 SV63_Re-CH0 SV63_Im-CH1 SV63_Re-CH1 * SV63_Im-CH2 SV63_Re-CH2 SV63_Im-CH3 SV63_Re-CH3 *

Returns

WR_0: will indicate if persistent TRAM content is corrupted or not.

Return values

0,if	persistent TRAM content is not corrupted. >0, if persistent TRAM content is corrupted.

Note

All buffers MUST be 8 bytes aligned.

3.9.2.27 RsdkSptCheckTram512smp128crp8ch()

```
void * RsdkSptCheckTram512smp128crp8ch (
            void  )
```

3.9.2.28 RsdkSptDbfDoa64Beams12Ch128Peaks()

```
void * RsdkSptDbfDoa64Beams12Ch128Peaks (
            void  )
```

Function for Digital BeamForming - DBF for 64 Steering Vector, 12 channels and 128 peaks and Direction of Arrival.

Compute Magnitudes(log2 format) for Beamscans (for each peak). Compute Global Max, calculate thresold and compute the local maxima, per Beamscan. Results stored in SRAM (log2 format + tagging for local peaks)

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Peak list), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * P0_Im-CH0 P0_Re-CH0 P0_Im-CH1 P0_Re-CH1 * P0_Im-CH2 P0_Re-CH2 P0_Im-CH3 P0_Re-CH3 * P0_Im-CH4 P0_Re-CH4 P0_Im-CH5 P0_Re-CH5 * P0_Im-CH6 P0_Re-CH6 P0_Im-CH7 P0_Re-CH7 * P0_Im-CH8 P0_Re-CH8 P0_Im-CH9 P0_Re-CH9 * P0_Im-CH10 P0_Re-CH10 P0_Im-CH11 P0_Re-CH11 * | | | | * | | | | * | | | | * P127_Im-CH0 P127_Re-CH0 P127_Im-CH1 P127_Re-CH1 * P127_Im-CH2 P127_Re-CH2 P127_Im-CH3 P127_Re-CH3 * P127_Im-CH4 P127_Re-CH4 P127_Im-CH5 P127_Re-CH5 * P127_Im-CH6 P127_Re-CH6 P127_Im-CH7 P127_Re-CH7 * P127_Im-CH8 P127_Re-CH8 P127_Im-CH9 P127_Re-CH9 * P127_Im-CH10 P127_Re-CH10 P127_Im-CH11 P127_Re-CH11 *
out	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(128 beamscans of 64 beams), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 32 bit real. First 16 bit reserved for tagging, next 16bit for log2 m * 63:48 47:32 31:16 15:0 * P0-TAG-Beam0 P0-MAG-Beam0 P0-TAG-Beam1 P0-MAG-Beam1 * P0-TAG-Beam2 P0-MAG-Beam2 P0-TAG-Beam3 P0-MAG-Beam3 * | | | | * P0-TAG-Beam62 P0-MAG-Beam62 P0-TAG-Beam63 P0-MAG-Beam63 -> Beamscan 0 * ---------- * P1-TAG-Beam0 P1-MAG-Beam0 P1-TAG-Beam1 P1-MAG-Beam1 * P1-TAG-Beam2 P1-MAG-Beam2 P1-TAG-Beam3 P1-MAG-Beam3 * | | | | * P1-TAG-Beam62 P1-MAG-Beam62 P1-TAG-Beam63 P1-MAG-Beam63 -> Beamscan 1 * ---------- * | | | | * | | | | * | | | | * ---------- * P127-TAG-Beam0 P127-MAG-Beam0 P127-TAG-Beam1 P127-MAG-Beam1 * P127-TAG-Beam2 P127-MAG-Beam2 P127-TAG-Beam3 P127-MAG-Beam3 * | | | | * P127-TAG-Beam62 P127-MAG-Beam62 P127-TAG-Beam63 P127-MAG-Beam63 -> Beamscan * ---------- *
in	*WR↩ _3*	24bit(RE): Threshold Factor, used for Threshold calculation (14bit log2 format). 24bit(IM): 0.

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

Steering vector coeficients are placed in TRAM at TR_0_384_0 by the Init function.
The kernel rely on the assumption that TR_381..3_∗ can be used as temp location.

Note

All buffers MUST be 8 bytes aligned.
The input buffer CANNOT be reused for the output buffers.

3.9.2.29 RsdkSptDbfDoa64Beams12Ch128PeaksCp4d()

```
void * RsdkSptDbfDoa64Beams12Ch128PeaksCp4d (
            void  )
```

Function for Digital BeamForming - DBF for 64 Steering Vector, 12 channels and 128 peaks and Direction of Arrival, CP4D PDMA decompression.

Compute Magnitudes(log2 format) for Beamscans (for each peak). Compute Global Max, calculate thresold and compute the local maxima, per Beamscan. Results stored in SRAM (log2 format + tagging for local peaks)

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Peak list), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 64bit compressed data * 63:0 * CH0_CH1_CH2_CH3-P0 * CH4_CH5_CH6_CH7-P0 * CH8_CH9_CH10_CH11-P0 * | * | * | * CH0_CH1_CH2_CH3-P127 * CH4_CH5_CH6_CH7-P127 * CH8_CH9_CH10_CH11-P127 *

Parameters

out	*WR↩_2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(128 beamscans of 64 beams), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 32 bit real. First 16 bit reserved for tagging, next 16bit for log2 m * 63:48 47:32 31:16 15:0 * P0-TAG-Beam0 P0-MAG-Beam0 P0-TAG-Beam1 P0-MAG-Beam1 * P0-TAG-Beam2 P0-MAG-Beam2 P0-TAG-Beam3 P0-MAG-Beam3 * | | | | * P0-TAG-Beam62 P0-MAG-Beam62 P0-TAG-Beam63 P0-MAG-Beam63 -> Beamscan 0 * ---------- * P1-TAG-Beam0 P1-MAG-Beam0 P1-TAG-Beam1 P1-MAG-Beam1 * P1-TAG-Beam2 P1-MAG-Beam2 P1-TAG-Beam3 P1-MAG-Beam3 * | | | | * P1-TAG-Beam62 P1-MAG-Beam62 P1-TAG-Beam63 P1-MAG-Beam63 -> Beamscan 1 * ---------- * | | | | * | | | | * | | | | * ---------- * P127-TAG-Beam0 P127-MAG-Beam0 P127-TAG-Beam1 P127-MAG-Beam1 * P127-TAG-Beam2 P127-MAG-Beam2 P127-TAG-Beam3 P127-MAG-Beam3 * | | | | * P127-TAG-Beam62 P127-MAG-Beam62 P127-TAG-Beam63 P127-MAG-Beam63 -> Beamscan * ---------- *
in	*WR↩_3*	24bit(RE): Threshold Factor, used for Threshold calculation (14bit log2 format). 24bit(IM): 0.

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

Steering vector coeficients are placed in TRAM at TR_0_384_0 by the Init function.
The kernel rely on the assumption that TR_0_381_0 to TR_0_383_7 can be used as temp location.

Note

All buffers MUST be 8 bytes aligned.
The input buffer CANNOT be reused for the output buffers.

3.9.2.30 RsdkSptDbfDoa64Beams16Ch128PeaksCp4d()

```
void * RsdkSptDbfDoa64Beams16Ch128PeaksCp4d (
            void  )
```

Function for Digital BeamForming - DBF for 64 Steering Vector, 16 channels and 128 peaks and Direction of Arrival, CP4D PDMA decompression.

Compute Magnitudes(log2 format) for Beamscans (for each peak). Compute Global Max, calculate thresold and compute the local maxima, per Beamscan. Results stored in SRAM (log2 format + tagging for local peaks)

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Peak list), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 64bit compressed data * 63:0 * CH##0_CH##1_CH##2_CH##3-P0 * CH##4_CH##5_CH##6_CH##7-P0 * CH##8_CH##9_CH##10_CH##11-P0 * CH##12_CH##13_CH##14_CH##15-P0 * | * | * | * CH##0_CH##1_CH##2_CH##3-P127 * CH##4_CH##5_CH##6_CH##7-P127 * CH##8_CH##9_CH##10_CH##11-P127 * CH##12_CH##13_CH##14_CH##15-P127 *
out	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(128 beamscans of 64 beams), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 32 bit real. First 16 bit reserved for tagging, next 16bit for log2 m * 63:48 47:32 31:16 15:0 * P0-TAG-Beam0 P0-MAG-Beam0 P0-TAG-Beam1 P0-MAG-Beam1 * P0-TAG-Beam2 P0-MAG-Beam2 P0-TAG-Beam3 P0-MAG-Beam3 * | | | | * P0-TAG-Beam62 P0-MAG-Beam62 P0-TAG-Beam63 P0-MAG-Beam63 -> Beamscan 0 * ---------- * P1-TAG-Beam0 P1-MAG-Beam0 P1-TAG-Beam1 P1-MAG-Beam1 * P1-TAG-Beam2 P1-MAG-Beam2 P1-TAG-Beam3 P1-MAG-Beam3 * | | | | * P1-TAG-Beam62 P1-MAG-Beam62 P1-TAG-Beam63 P1-MAG-Beam63 -> Beamscan 1 * ---------- * | | | | * | | | | * | | | | * ---------- * P127-TAG-Beam0 P127-MAG-Beam0 P127-TAG-Beam1 P127-MAG-Beam1 * P127-TAG-Beam2 P127-MAG-Beam2 P127-TAG-Beam3 P127-MAG-Beam3 * | | | | * P127-TAG-Beam62 P127-MAG-Beam62 P127-TAG-Beam63 P127-MAG-Beam63 -> Beamscan * ---------- *
in	*WR↩ _3*	24bit(RE): Threshold Factor, used for Threshold calculation (14bit log2 format). 24bit(IM): 0.

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

Steering vector coeficients are placed in TRAM at TR_0_384_0 by the Init function.
The kernel rely on the assumption that TR_381..3 can be used as temp location.

Note

All buffers MUST be 8 bytes aligned.
The input buffer CANNOT be reused for the output buffers.

3.9.2.31 RsdkSptDbfDoa64Beams4Ch128Peaks()

```
void * RsdkSptDbfDoa64Beams4Ch128Peaks (
            void  )
```

Function for Digital BeamForming - DBF for 64 Steering Vector, 4 channels and 128 peaks and Direction of Arrival.

Compute Magnitudes(log2 format) for Beamscans (for each peak). Compute Global Max, calculate thresold and compute the local maxima, per Beamscan. Results stored in SRAM (log2 format + tagging for local peaks)

Parameters

in	*WR_1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Peak list), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR.

```
*                INPUT Memory layout:
*                SRAM, 32bit complex(16b_Im + 16b_Re)
*                63:48          47:32           31:16           15:0
*                P0_Im-CH0      P0_Re-CH0       P0_Im-CH1       P0_Re-CH1
*                P0_Im-CH2      P0_Re-CH2       P0_Im-CH3       P0_Re-CH3
*                P1_Im-CH0      P1_Re-CH0       P1_Im-CH1       P1_Re-CH1
*                P1_Im-CH2      P1_Re-CH2       P1_Im-CH3       P1_Re-CH3
*                |              |               |               |
*                |              |               |               |
*                |              |               |               |
*                P127_Im-CH0    P127_Re-CH0     P127_Im-CH1     P127_Re-CH1
*                P127_Im-CH2    P127_Re-CH2     P127_Im-CH3     P127_Re-CH3
*
```

Parameters

out	*WR↩_2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(128 beamscans of 64 beams), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 32 bit real. First 16 bit reserved for tagging, next 16bit for log2 m * 63:48 47:32 31:16 15:0 * P0-TAG-Beam0 P0-MAG-Beam0 P0-TAG-Beam1 P0-MAG-Beam1 * P0-TAG-Beam2 P0-MAG-Beam2 P0-TAG-Beam3 P0-MAG-Beam3 * | | | | * P0-TAG-Beam62 P0-MAG-Beam62 P0-TAG-Beam63 P0-MAG-Beam63 -> Beamscan 0 * ---------- * P1-TAG-Beam0 P1-MAG-Beam0 P1-TAG-Beam1 P1-MAG-Beam1 * P1-TAG-Beam2 P1-MAG-Beam2 P1-TAG-Beam3 P1-MAG-Beam3 * | | | | * P1-TAG-Beam62 P1-MAG-Beam62 P1-TAG-Beam63 P1-MAG-Beam63 -> Beamscan 1 * ---------- * | | | | * | | | | * | | | | * ---------- * P127-TAG-Beam0 P127-MAG-Beam0 P127-TAG-Beam1 P127-MAG-Beam1 * P127-TAG-Beam2 P127-MAG-Beam2 P127-TAG-Beam3 P127-MAG-Beam3 * | | | | * P127-TAG-Beam62 P127-MAG-Beam62 P127-TAG-Beam63 P127-MAG-Beam63 -> Beamscan * ---------- *
in	*WR↩_3*	24bit(RE): Threshold Factor, used for Threshold calculation (14bit log2 format). 24bit(IM): 0.

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

Steering vector coeficients are placed in TRAM at TR_0_448_0 by the Init function.
The kernel rely on the assumption that TR_0_447_0 can be used as temp location.

Note

All buffers MUST be 8 bytes aligned.
The input buffer CANNOT be reused for the output buffers.

3.9.2.32 RsdkSptDbfDoa64Beams4Ch128PeaksCp4d()

```
void * RsdkSptDbfDoa64Beams4Ch128PeaksCp4d (
            void  )
```

Function for Digital BeamForming - DBF for 64 Steering Vector, 4 channels and 128 peaks and Direction of Arrival, CP4D PDMA decompression.

Compute Magnitudes(log2 format) for Beamscans (for each peak). Compute Global Max, calculate thresold and compute the local maxima, per Beamscan. Results stored in SRAM (log2 format + tagging for local peaks)

Parameters

in	*WR↩_1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Peak list, CP4D compressed), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 64bit compressed data * 63:0 * CH0_CH1_CH2_CH3-P0 * CH0_CH1_CH2_CH3-P1 * | * | * | * CH0_CH1_CH2_CH3-P127 *
out	*WR↩_2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(128 beamscans of 64 beams), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 32 bit real. First 16 bit reserved for tagging, next 16bit for log2 m * 63:48 47:32 31:16 15:0 * P0-TAG-Beam0 P0-MAG-Beam0 P0-TAG-Beam1 P0-MAG-Beam1 * P0-TAG-Beam2 P0-MAG-Beam2 P0-TAG-Beam3 P0-MAG-Beam3 * | | | | * P0-TAG-Beam62 P0-MAG-Beam62 P0-TAG-Beam63 P0-MAG-Beam63 -> Beamscan 0 * ---------- * P1-TAG-Beam0 P1-MAG-Beam0 P1-TAG-Beam1 P1-MAG-Beam1 * P1-TAG-Beam2 P1-MAG-Beam2 P1-TAG-Beam3 P1-MAG-Beam3 * | | | | * P1-TAG-Beam62 P1-MAG-Beam62 P1-TAG-Beam63 P1-MAG-Beam63 -> Beamscan 1 * ---------- * | | | | * | | | | * | | | | * ---------- * P127-TAG-Beam0 P127-MAG-Beam0 P127-TAG-Beam1 P127-MAG-Beam1 * P127-TAG-Beam2 P127-MAG-Beam2 P127-TAG-Beam3 P127-MAG-Beam3 * | | | | * P127-TAG-Beam62 P127-MAG-Beam62 P127-TAG-Beam63 P127-MAG-Beam63 -> Beamscan * ---------- *
in	*WR↩_3*	24bit(RE): Threshold Factor, used for Threshold calculation (14bit log2 format). 24bit(IM): 0.

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

Steering vector coeficients are placed in TRAM at TR_0_448_0 by the Init function.
The kernel rely on the assumption that TR_0_447_0 can be used as temp location.

Note

All buffers MUST be 8 bytes aligned.
The input buffer CANNOT be reused for the output buffers.

3.9.2.33 RsdkSptDbfDoa64Beams8Ch128Peaks()

```
void * RsdkSptDbfDoa64Beams8Ch128Peaks (
            void  )
```

3.9.2.34 RsdkSptDbfDoa64Beams8Ch128PeaksCp4d()

```
void * RsdkSptDbfDoa64Beams8Ch128PeaksCp4d (
            void  )
```

Function for Digital BeamForming - DBF for 64 Steering Vector, 8 channels and 128 peaks and Direction of Arrival, CP4D PDMA decompression.

Compute Magnitudes(log2 format) for Beamscans (for each peak). Compute Global Max, calculate thresold and compute the local maxima, per Beamscan. Results stored in SRAM (log2 format + tagging for local peaks)

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Peak list, CP4D compressed), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 64bit compressed data * 63:0 * CH0_CH1_CH2_CH3-P0 * CH4_CH5_CH6_CH7-P0 * CH0_CH1_CH2_CH3-P1 * CH4_CH5_CH6_CH7-P1 * | * | * | * CH0_CH1_CH2_CH3-P127 * CH4_CH5_CH6_CH7-P127 *

Parameters

out	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(128 beamscans of 64 beams), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 32 bit real. First 16 bit reserved for tagging, next 16bit for log2 m * 63:48 47:32 31:16 15:0 * P0-TAG-Beam0 P0-MAG-Beam0 P0-TAG-Beam1 P0-MAG-Beam1 * P0-TAG-Beam2 P0-MAG-Beam2 P0-TAG-Beam3 P0-MAG-Beam3 * | | | | * P0-TAG-Beam62 P0-MAG-Beam62 P0-TAG-Beam63 P0-MAG-Beam63 -> Beamscan 0 * ---------- * P1-TAG-Beam0 P1-MAG-Beam0 P1-TAG-Beam1 P1-MAG-Beam1 * P1-TAG-Beam2 P1-MAG-Beam2 P1-TAG-Beam3 P1-MAG-Beam3 * | | | | * P1-TAG-Beam62 P1-MAG-Beam62 P1-TAG-Beam63 P1-MAG-Beam63 -> Beamscan 1 * ---------- * | | | | * | | | | * | | | | * ---------- * P127-TAG-Beam0 P127-MAG-Beam0 P127-TAG-Beam1 P127-MAG-Beam1 * P127-TAG-Beam2 P127-MAG-Beam2 P127-TAG-Beam3 P127-MAG-Beam3 * | | | | * P127-TAG-Beam62 P127-MAG-Beam62 P127-TAG-Beam63 P127-MAG-Beam63 -> Beamscan * ---------- *
in	*WR↩ _3*	24bit(RE): Threshold Factor, used for Threshold calculation (14bit log2 format). 24bit(IM): 0.

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

Steering vector coeficients are placed in TRAM at TR_0_448_0 by the Init function.
The kernel rely on the assumption that TR_0_447_0 can be used as temp location.

Note

All buffers MUST be 8 bytes aligned.
The input buffer CANNOT be reused for the output buffers.

3.9.2.35 RsdkSptDbfFFT128Bins12Ch128PeaksCp4d()

```
void * RsdkSptDbfFFT128Bins12Ch128PeaksCp4d (
            void  )
```

Function for Digital BeamForming - DBF for 128 bins, 12 channels and 128 peaks.

Compute FFT 128 (16 bit complex format) for each peak.

Parameters

<table>
<tr><td>in</td><td>WR_1</td><td>24bit(RE): 0
24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Compressed Peak list), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR.<pre>
*   INPUT Memory layout:
*   SRAM, 64bit compressed data
*   63:0
*   CH0_0-CH1_0-CH2_0-CH3_0   -> P0
*   CH4_0-CH5_0-CH6_0-CH7_0   -> P0
*   CH8_0-CH9_0-CH10_0-CH11_0   -> P0
* |
* |
* |
*   CH0_0-CH1_0-CH2_0-CH3_0   -> P127
*   CH4_0-CH5_0-CH6_0-CH7_0   -> P127
*   CH8_0-CH9_0-CH10_0-CH11_0   -> P127
*
</pre></td></tr>
<tr><td>out</td><td>WR_2</td><td>24bit(RE): 0
24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(128 Azimut bins for 128 peaks), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR.<pre>
*   OUTPUT Memory layout:
*   SRAM, 16 bit real Log2 Mag2 format
*   63:48   47:32   31:16   15:0
*   AzBin0-Im   AzBin0-Re   AzBin1-Im   AzBin1-Re
*   AzBin2-Im   AzBin2-Re   AzBin3-Im   AzBin3-Re
* | | | |
*   AzBin124-Im   AzBin124-Re   AzBin125-Im   AzBin125-Re
*   AzBin126-Im   AzBin126-Re   AzBin127-Im   AzBin127-Re ---> P0
*   --
* | | | |
* | | | |
* | | | |
*   --
*   AzBin0-Im   AzBin0-Re   AzBin1-Im   AzBin1-Re
*   AzBin2-Im   AzBin2-Re   AzBin3-Im   AzBin3-Re
* | | | |
*   AzBin124-Im   AzBin124-Re   AzBin125-Im   AzBin125-Re
*   AzBin126-Im   AzBin126-Re   AzBin127-Im   AzBin127-Re ---> P127
*   ---
*
</pre></td></tr>
<tr><td>in</td><td>WR_3</td><td>24bit(RE): 0
24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT128 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR.
For twiddle generation algorithm, see S32R41_RM 68.9.5.3 / SAF85xx_RM 68.9.5.3.<pre>
*   INPUT Memory layout:
*   SRAM, 32bit complex(16b_Im + 16b_Re)
*   63:48   47:32   31:16   15:0
*   tw1_Im   tw1_Re   tw1_Im   tw1_Re
*   tw1_Im   tw1_Re   tw1_Im   tw1_Re
*   tw1_Im   tw1_Re   tw1_Im   tw1_Re
*   tw1_Im   tw1_Re   tw1_Im   tw1_Re
*   tw2_Im   tw2_Re   tw2_Im   tw2_Re
*   tw2_Im   tw2_Re   tw2_Im   tw2_Re
*   tw2_Im   tw2_Re   tw2_Im   tw2_Re
*   tw2_Im   tw2_Re   tw2_Im   tw2_Re
* | | | |
* | | | |
* | | | |
*   tw16_Im tw16_Re tw16_Im tw16_Re
*   tw16_Im tw16_Re tw16_Im tw16_Re
*   tw16_Im tw16_Re tw16_Im tw16_Re
*   tw16_Im tw16_Re tw16_Im tw16_Re
*
</pre></td></tr>
</table>

Parameters

in	*WR_4*	24bit(RE): 0
		24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Reordered Window coef for DBF FFT128), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR.
		For reorder algorithm, see S32R41_RM 68.9.9.7 / SAF85xx_RM 68.9.9.7.
		
		`* INPUT Memory layout:`
		`* SRAM, 16bit Real Window Coef`
		`* 63:48 47:32 31:16 15:0`
		`* win0 win32 win64 win96`
		`* win8 win40 win72 win104`
		`* | | | |`
		`* | | | |`
		`* win7 win39 win71 win103`
		`* win15 win47 win79 win111`
		`* win16 win48 win80 win112`
		`* win24 win56 win88 win120`
		`* | | | |`
		`* | | | |`
		`* win23 win55 win87 win119`
		`* win31 win63 win95 win127`
		`*`

in	*WR_5*	24bit(RE): Scaling Factor, e.g. 1 for scale left with 1. 24bit(IM): 0.

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

The kernel rely on the assumption that TRAM can be used as temp buffer from TR_0_320_0 to TR_0_351_7.

Note

The kernel implementation guarantees no saturation, for 0 output scaling factor. If other scaling is used, it's the user responsibility to prevent saturation.
The input buffer CANNOT be reused for the output buffer.
All buffers MUST be 8 bytes aligned.

3.9.2.36 RsdkSptDbfFFT128Bins48Ch128PeaksCp4d()

```
void * RsdkSptDbfFFT128Bins48Ch128PeaksCp4d (
            void  )
```

3.9.2.37 RsdkSptDoppler1024smp128crp4chCp4d()

```
void * RsdkSptDoppler1024smp128crp4chCp4d (
            void  )
```

Function for Doppler FFT 128, 4 channels(CH#0..CH#3)

Compute Doppler FFT storing compressed results(CP4D) in SRAM.

Parameters

in	*WR↩_1*	24bit(RE): 0
		24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Range FFT compressed results), relative to SRAM start address RSDK_SPT_CUBE_BASE_ADDR.
		
		`*  INPUT Memory layout:`
		`*  SRAM, 64bit compressed data`
		`*  63:0`
		`*  CH0_0-CH1_0-CH2_0-CH3_0_Crp0`
		`*  |`
		`*  CH0_0-CH1_0-CH2_0-CH3_0-Crp127  ---> RangeBin ##0`
		`*  ----------`
		`*  |`
		`*  |`
		`*  |`
		`*  ----------`
		`*  CH0_511-CH1_511-CH2_511-CH3_511-Crp0`
		`*  |`
		`*  CH0_511-CH1_511-CH2_511-CH3_511-Crp127  ---> RangeBin ##511`
		`*  ----------`
		`*`

out	*WR↩_2*	24bit(RE): 0
		24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(Doppler FFT compressed results), relative to SRAM start address RSDK_SPT_CUBE_BASE_ADDR.
		
		`*  OUTPUT Memory layout:`
		`*  SRAM, 64bit compressed data`
		`*  63:0`
		`*  CH0_0-CH1_0-CH2_0-CH3_0_Crp0`
		`*  |`
		`*  CH0_0-CH1_0-CH2_0-CH3_0-Crp127  ---> RangeBin ##0`
		`*  ----------`
		`*  |`
		`*  |`
		`*  |`
		`*  ----------`
		`*  CH0_511-CH1_511-CH2_511-CH3_511-Crp0`
		`*  |`
		`*  CH0_511-CH1_511-CH2_511-CH3_511-Crp127  ---> RangeBin ##511`
		`*  ----------`
		`*`

in	*WR↩_3*	24bit(RE): Pre-Scaling Factor, e.g. 1 for left shift with 1. 24bit(IM): 0. Value given from the application level, based on global max provided by the range kernel. Shall be computed on the CORE as: pre_scaling value = floor(log2(0x7FFFFF / 1Q23 max_glb)).
in	*WR↩_4*	24bit(RE): Scaling Factor on output data, e.g. 1 for left shift with 1. 24bit(IM): 0.

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

Twiddle coeficients are placed in TRAM at TR_0_128_0 by the Init function.
Window coeficients are placed in TRAM at TR_0_272_0 by the Init function.

Note

The current implementation is ONLY to be used in combination with 1024 CP4D Range FFT.
The input buffer CAN be reused for the output buffer.
All buffers MUST be 8 bytes aligned.

3.9.2.38 RsdkSptDoppler1024smp128crp8chCp4d()

```
void * RsdkSptDoppler1024smp128crp8chCp4d (
            void  )
```

3.9.2.39 RsdkSptDoppler1024smp256crp4ch3TxDdMimoCp4d()

```
void * RsdkSptDoppler1024smp256crp4ch3TxDdMimoCp4d (
            void  )
```

Function for Doppler FFT256, 4 channels(CH#0..CH#3), DD-MIMO use case.

Compute Doppler FFT, decodes DD-MIMO, storing compressed(CP4D) results in SRAM.
Processing valid for 3Tx antennas over 4Tx slots having the following coding scheme:
TX1 - 1 1 1 1
TX2 - 1 1 0 0
TX3 - 1 0 1 0

Parameters

in	*WR_1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Range FFT compressed results), relative to SRAM start address RSDK_SPT_CUBE_BASE_ADDR. * INPUT Memory layout: * SRAM, 64bit compressed data * 63:0 * CH0_0-CH1_0-CH2_0-CH3_0_Crp0 * | * CH0_0-CH1_0-CH2_0-CH3_0-Crp255 ---> RangeBin ##0 * ---------- * | * | * | * ---------- * CH0_511-CH1_511-CH2_511-CH3_511-Crp0 * | * CH0_511-CH1_511-CH2_511-CH3_511-Crp255 ---> RangeBin ##511 * ---------- *
out	*WR_2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(Doppler FFT compressed results), relative to SRAM start address RSDK_SPT_CUBE_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 64bit compressed data * 63:0 * CH0_0-CH1_0-CH2_0-CH3_0-Crp0 -> TX1 * CH4_0-CH5_0-CH6_0-CH7_0-Crp0 -> TX2 * CH8_0-CH9_0-CH10_0-CH11_0-Crp0 -> TX3 * | * CH0_0-CH1_0-CH2_0-CH3_0-Crp63 -> TX1 * CH4_0-CH5_0-CH6_0-CH7_0-Crp63 -> TX2 * CH8_0-CH9_0-CH10_0-CH11_0-Crp63 -> TX3 ---> RangeBin ##0 * ---------- * | * | * | * ---------- * CH0_511-CH1_511-CH2_511-CH3_511-Crp0 -> TX1 * CH4_511-CH5_511-CH6_511-CH7_511-Crp0 -> TX2 * CH8_511-CH9_511-CH10_511-CH11_511-Crp0 -> TX3 * | * CH0_511-CH1_511-CH2_511-CH3_511-Crp63 -> TX1 * CH4_511-CH5_511-CH6_511-CH7_511-Crp63 -> TX2 * CH8_511-CH9_511-CH10_511-CH11_511-Crp63 -> TX3 ---> RangeBin ##511 * ---------- *
in	*WR_3*	24bit(RE): Pre-Scaling Factor, e.g. 1 for left shift with 1. 24bit(IM): 0. Value given from the application level, based on global max provided by the range kernel. Shall be computed on the CORE as: pre_scaling value = floor(log2(0x7FFFFF / 1Q23 max_glb)).
in	*WR_4*	24bit(RE): Scaling Factor on output data, e.g. 1 for left shift with 1. 24bit(IM): 0.

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

Twiddle coeficients are placed in TRAM at TR_0_128_0 by the Init function.
Window coeficients are placed in TRAM at TR_0_288_0 by the Init function.
The kernel rely on the assumption that TR_0_320_0..3 can be used as temp location.

Note

The kernel implementation guarantees no saturation, for 0 output scaling factor. If other scaling is used, it's the user responsibility to prevent saturation.
The current implementation is ONLY to be used in combination with 1024 CP4D Range FFT.
The input buffer CAN be reused for the output buffer.
All buffers MUST be 8 bytes aligned.

3.9.2.40 RsdkSptDoppler1024smp64crp4ch4TxTdMimoCp4d()

```
void * RsdkSptDoppler1024smp64crp4ch4TxTdMimoCp4d (
            void  )
```

Function for Doppler FFT 1024, 4 physical channels/TX slot, 4TX TD-MIMO.

Compute Doppler FFT storing compressed results(CP4D) in SRAM.

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Range FFT compressed results), relative to SRAM start address RSDK_SPT_CUBE_BASE_ADDR. * INPUT Memory layout: * SRAM, 64bit compressed data * 63:0 * CH0_0-CH1_0-CH2_0-CH3_0-TX0_Crp0 * CH0_0-CH1_0-CH2_0-CH3_0-TX1_Crp0 * CH0_0-CH1_0-CH2_0-CH3_0-TX2_Crp0 * CH0_0-CH1_0-CH2_0-CH3_0-TX3_Crp0 * | * CH0_0-CH1_0-CH2_0-CH3_0-TX0_Crp63 * CH0_0-CH1_0-CH2_0-CH3_0-TX1_Crp63 * CH0_0-CH1_0-CH2_0-CH3_0-TX2_Crp63 * CH0_0-CH1_0-CH2_0-CH3_0-TX3_Crp63 ---> RangeBin ##0 * ---------- * | * | * | * ---------- * CH0_511-CH1_511-CH2_511-CH3_511-TX0_Crp0 * CH0_511-CH1_511-CH2_511-CH3_511-TX1_Crp0 * CH0_511-CH1_511-CH2_511-CH3_511-TX2_Crp0 * CH0_511-CH1_511-CH2_511-CH3_511-TX3_Crp0 * | * CH0_511-CH1_511-CH2_511-CH3_511-TX0_Crp63 * CH0_511-CH1_511-CH2_511-CH3_511-TX1_Crp63 * CH0_511-CH1_511-CH2_511-CH3_511-TX2_Crp63 * CH0_511-CH1_511-CH2_511-CH3_511-TX3_Crp63 ---> RangeBin ##511 * ---------- *
out	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(Doppler FFT compressed results), relative to SRAM start address RSDK_SPT_CUBE_BASE_ADDR * OUTPUT Memory layout: * SRAM, 64bit compressed data * 63:0 * CH0_0-CH1_0-CH2_0-CH3_0-TX0_Crp0 * CH0_0-CH1_0-CH2_0-CH3_0-TX1_Crp0 * CH0_0-CH1_0-CH2_0-CH3_0-TX2_Crp0 * CH0_0-CH1_0-CH2_0-CH3_0-TX3_Crp0 * | * CH0_0-CH1_0-CH2_0-CH3_0-TX0_Crp63 * CH0_0-CH1_0-CH2_0-CH3_0-TX1_Crp63 * CH0_0-CH1_0-CH2_0-CH3_0-TX2_Crp63 * CH0_0-CH1_0-CH2_0-CH3_0-TX3_Crp63 ---> RangeBin ##0 * ---------- * | * | * | * ---------- * CH0_511-CH1_511-CH2_511-CH3_511-TX0_Crp0 * CH0_511-CH1_511-CH2_511-CH3_511-TX1_Crp0 * CH0_511-CH1_511-CH2_511-CH3_511-TX2_Crp0 * CH0_511-CH1_511-CH2_511-CH3_511-TX3_Crp0 * | * CH0_511-CH1_511-CH2_511-CH3_511-TX0_Crp63 * CH0_511-CH1_511-CH2_511-CH3_511-TX1_Crp63 * CH0_511-CH1_511-CH2_511-CH3_511-TX2_Crp63 * CH0_511-CH1_511-CH2_511-CH3_511-TX3_Crp63 ---> RangeBin ##511 * ---------- *

Parameters

in	*WR_3*	24bit(RE): Pre-Scaling Factor, e.g. 1 for left shift with 1. 24bit(IM): 0. Value given from the application level, based on global max provided by the range kernel. Shall be computed on the CORE as: pre_scaling value = floor(log2(0x7FFFFF / 1Q23 max_glb)).
in	*WR_4*	24bit(RE): Scaling Factor on output data, e.g. 1 for left shift with 1. 24bit(IM): 0.

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

Twiddle coeficients are placed in TRAM at TR_0_128_0 by the Init function.
Window coeficients are placed in TRAM at TR_0_264_0 by the Init function.
Derotation phase coeficients are places in TRAM at TR_0_276_0 by the Init function.

Note

The current implementation is ONLY to be used in combination with 1024 Range FFT 4TX TD-MIMO CP4D.
The input buffer CAN be reused for the output buffer.
All buffers MUST be 8 bytes aligned.

3.9.2.41 RsdkSptDoppler1024smp64crp4chCp4d()

```
void * RsdkSptDoppler1024smp64crp4chCp4d (
            void  )
```

Function for Doppler FFT 64, 4 channels(CH#0..CH#3)

Compute Doppler FFT storing compressed results(CP4D) in SRAM.

Parameters

in	*WR_1*	24bit(RE): 0
		24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Range FFT compressed results), relative to SRAM start address RSDK_SPT_CUBE_BASE_ADDR.
		
		`* INPUT Memory layout:`
		`* SRAM, 64bit compressed data`
		`* 63:0`
		`* CH0_0-CH1_0-CH2_0-CH3_0_Crp0`
		`* |`
		`* CH0_0-CH1_0-CH2_0-CH3_0-Crp63 ---> RangeBin ##0`
		`* ----------`
		`* |`
		`* |`
		`* |`
		`* ----------`
		`* CH0_511-CH1_511-CH2_511-CH3_511-Crp0`
		`* |`
		`* CH0_511-CH1_511-CH2_511-CH3_511-Crp63 ---> RangeBin ##511`
		`* ----------`
		`*`

out	*WR_2*	24bit(RE): 0
		24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(Doppler FFT compressed results), relative to SRAM start address RSDK_SPT_CUBE_BASE_ADDR.
		
		`* OUTPUT Memory layout:`
		`* SRAM, 64bit compressed data`
		`* 63:0`
		`* CH0_0-CH1_0-CH2_0-CH3_0_Crp0`
		`* |`
		`* CH0_0-CH1_0-CH2_0-CH3_0-Crp63 ---> RangeBin ##0`
		`* ----------`
		`* |`
		`* |`
		`* |`
		`* ----------`
		`* CH0_511-CH1_511-CH2_511-CH3_511-Crp0`
		`* |`
		`* CH0_511-CH1_511-CH2_511-CH3_511-Crp63 ---> RangeBin ##511`
		`* ----------`
		`*`

in	*WR_3*	24bit(RE): Pre-Scaling Factor, e.g. 1 for left shift with 1. 24bit(IM): 0. Value given from the application level, based on global max provided by the range kernel. Shall be computed on the CORE as: pre_scaling value = floor(log2(0x7FFFFF / 1Q23 max_glb)).
in	*WR_4*	24bit(RE): Scaling Factor on output data, e.g. 1 for left shift with 1. 24bit(IM): 0.

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

Twiddle coeficients are placed in TRAM at TR_0_128_0 by the Init function.
Window coeficients are placed in TRAM at TR_0_264_0 by the Init function.

Note

The current implementation is ONLY to be used in combination with 1024 CP4D Range FFT.
The input buffer CAN be reused for the output buffer.
All buffers MUST be 8 bytes aligned.

3.9.2.42 RsdkSptDoppler2048smp256crp4chCp4d()

```
void * RsdkSptDoppler2048smp256crp4chCp4d (
            void  )
```

Function for Doppler FFT 256, 4 channels(CH#0..CH#3)

Compute Doppler FFT storing compressed results(CP4D) in SRAM.

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Range FFT compressed results), relative to SRAM start address RSDK_SPT_CUBE_BASE_ADDR. * INPUT Memory layout: * SRAM, 64bit compressed data * 63:0 * CH0_0-CH1_0-CH2_0-CH3_0_Crp0 * | * CH0_0-CH1_0-CH2_0-CH3_0-Crp255 ---> RangeBin ##0 * ---------- * | * | * | * ---------- * CH0_1023-CH1_1023-CH2_1023-CH3_1023-Crp0 * | * CH0_1023-CH1_1023-CH2_1023-CH3_1023-Crp255 ---> RangeBin ##1023 * ---------- *
out	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(Doppler FFT compressed results), relative to SRAM start address RSDK_SPT_CUBE_BASE_ADDR * INPUT Memory layout: * SRAM, 64bit compressed data * 63:0 * CH0_0-CH1_0-CH2_0-CH3_0_Crp0 * | * CH0_0-CH1_0-CH2_0-CH3_0-Crp255 ---> RangeBin ##0 * ---------- * | * | * | * ---------- * CH0_1023-CH1_1023-CH2_1023-CH3_1023-Crp0 * | * CH0_1023-CH1_1023-CH2_1023-CH3_1023-Crp255 ---> RangeBin ##1023 * ---------- *

Parameters

in	*WR_3*	24bit(RE): Pre-Scaling Factor, e.g. 1 for left shift with 1. 24bit(IM): 0. Value given from the application level, based on global max provided by the range kernel. Shall be computed on the CORE as: pre_scaling value = floor(log2(0x7FFFFF / 1Q23 max_glb)).
in	*WR_4*	24bit(RE): Scaling Factor on output data, e.g. 1 for left shift with 1. 24bit(IM): 0.

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

Twiddle coeficients are placed in TRAM at TR_0_256_0 by the Init function.
Window coeficients are placed in TRAM at TR_0_288_0 by the Init function.

Note

The current implementation is ONLY to be used in combination with 2048 CP4D Range FFT.
The input buffer CAN be reused for the output buffer.
All buffers MUST be 8 bytes aligned.

3.9.2.43 RsdkSptDoppler256smp128crp4ch3TxTdMimo()

```
void * RsdkSptDoppler256smp128crp4ch3TxTdMimo (
            void  )
```

Function for Doppler FFT 128, 4 physical channels/TX slot, 3TX TD-MIMO.

Compute Doppler FFT storing results in SRAM.

Parameters

in	*WR_1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Range FFT results), relative to SRAM start address RSDK_SPT_CUBE_BASE_ADDR.

```
*                 INPUT Memory layout:
*                 SRAM, 32bit complex(16b_Im + 16b_Re)
*                 63:48               47:32               31:16               15:0
*                 CH0_0_Im-Crp0       CH0_0_Re-Crp0       CH1_0_Im-Crp0       CH1_0_Re-Crp0
*                 CH2_0_Im-Crp0       CH2_0_Re-Crp0       CH3_0_Im-Crp0       CH3_0_Re-Crp0
*                 ------------------------------------------------------------------------------
*                 CH0_0_Im-Crp0       CH0_0_Re-Crp0       CH1_0_Im-Crp0       CH1_0_Re-Crp0
*                 CH2_0_Im-Crp0       CH2_0_Re-Crp0       CH3_0_Im-Crp0       CH3_0_Re-Crp0
*                 ------------------------------------------------------------------------------
*                 CH0_0_Im-Crp0       CH0_0_Re-Crp0       CH1_0_Im-Crp0       CH1_0_Re-Crp0
*                 CH2_0_Im-Crp0       CH2_0_Re-Crp0       CH3_0_Im-Crp0       CH3_0_Re-Crp0
*                 ------------------------------------------------------------------------------
*                 |                   |                   |                   |
*                 |                   |                   |                   |
*                 |                   |                   |                   |
*                 CH0_0_Im-Crp127     CH0_0_Re-Crp127     CH1_0_Im-Crp127     CH1_0_Re-Crp127
*                 CH2_0_Im-Crp127     CH2_0_Re-Crp127     CH3_0_Im-Crp127     CH3_0_Re-Crp127
*                 ------------------------------------------------------------------------------
*                 CH0_0_Im-Crp127     CH0_0_Re-Crp127     CH1_0_Im-Crp127     CH1_0_Re-Crp127
*                 CH2_0_Im-Crp127     CH2_0_Re-Crp127     CH3_0_Im-Crp127     CH3_0_Re-Crp127
*                 ------------------------------------------------------------------------------
*                 CH0_0_Im-Crp127     CH0_0_Re-Crp127     CH1_0_Im-Crp127     CH1_0_Re-Crp127
*                 CH2_0_Im-Crp127     CH2_0_Re-Crp127     CH3_0_Im-Crp127     CH3_0_Re-Crp127
*                 ------------------------------------------------------------------------------
*                 |                   |                   |                   |
*                 |                   |                   |                   |
*                 |                   |                   |                   |
*                 |                   |                   |                   |
*                 |                   |                   |                   |
*                 |                   |                   |                   |
*                 CH0_127_Im-Crp0     CH0_127_Re-Crp0     CH1_127_Im-Crp0     CH1_127_Re-Crp0
*                 CH2_127_Im-Crp0     CH2_127_Re-Crp0     CH3_127_Im-Crp0     CH3_127_Re-Crp0
*                 ------------------------------------------------------------------------------
*                 CH0_127_Im-Crp0     CH0_127_Re-Crp0     CH1_127_Im-Crp0     CH1_127_Re-Crp0
*                 CH2_127_Im-Crp0     CH2_127_Re-Crp0     CH3_127_Im-Crp0     CH3_127_Re-Crp0
*                 ------------------------------------------------------------------------------
*                 CH0_127_Im-Crp0     CH0_127_Re-Crp0     CH1_127_Im-Crp0     CH1_127_Re-Crp0
*                 CH2_127_Im-Crp0     CH2_127_Re-Crp0     CH3_127_Im-Crp0     CH3_127_Re-Crp0
*                 ------------------------------------------------------------------------------
*                 |                   |                   |                   |
*                 |                   |                   |                   |
*                 |                   |                   |                   |
*                 CH0_127_Im-Crp127   CH0_127_Re-Crp127   CH1_127_Im-Crp127   CH1_127_Re-Crp127
*                 CH2_127_Im-Crp127   CH2_127_Re-Crp127   CH3_127_Im-Crp127   CH3_127_Re-Crp127
*                 ------------------------------------------------------------------------------
*                 CH0_127_Im-Crp127   CH0_127_Re-Crp127   CH1_127_Im-Crp127   CH1_127_Re-Crp127
*                 CH2_127_Im-Crp127   CH2_127_Re-Crp127   CH3_127_Im-Crp127   CH3_127_Re-Crp127
*                 ------------------------------------------------------------------------------
*                 CH0_127_Im-Crp127   CH0_127_Re-Crp127   CH1_127_Im-Crp127   CH1_127_Re-Crp127
*                 CH2_127_Im-Crp127   CH2_127_Re-Crp127   CH3_127_Im-Crp127   CH3_127_Re-Crp127
*                 ------------------------------------------------------------------------------
*
```

Parameters

<table>
<tr><td>out</td><td>WR_2</td><td>24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(Doppler FFT results), relative to SRAM start address RSDK_SPT_CUBE_BASE_ADDR
<pre>
*   OUTPUT Memory layout:
*   SRAM, 32bit complex(16b_Im + 16b_Re)
*   63:48   47:32   31:16   15:0
*   CH0_0_Im-Crp0   CH0_0_Re-Crp0   CH1_0_Im-Crp0   CH1_0_Re-Crp0
*   CH2_0_Im-Crp0   CH2_0_Re-Crp0   CH3_0_Im-Crp0   CH3_0_Re-Crp0
*   ---
*   CH0_0_Im-Crp0   CH0_0_Re-Crp0   CH1_0_Im-Crp0   CH1_0_Re-Crp0
*   CH2_0_Im-Crp0   CH2_0_Re-Crp0   CH3_0_Im-Crp0   CH3_0_Re-Crp0
*   ---
*   CH0_0_Im-Crp0   CH0_0_Re-Crp0   CH1_0_Im-Crp0   CH1_0_Re-Crp0
*   CH2_0_Im-Crp0   CH2_0_Re-Crp0   CH3_0_Im-Crp0   CH3_0_Re-Crp0
*   ---
* | | | |
* | | | |
* | | | |
*   CH0_0_Im-Crp127   CH0_0_Re-Crp127   CH1_0_Im-Crp127   CH1_0_Re-Crp127
*   CH2_0_Im-Crp127   CH2_0_Re-Crp127   CH3_0_Im-Crp127   CH3_0_Re-Crp127
*   ---
*   CH0_0_Im-Crp127   CH0_0_Re-Crp127   CH1_0_Im-Crp127   CH1_0_Re-Crp127
*   CH2_0_Im-Crp127   CH2_0_Re-Crp127   CH3_0_Im-Crp127   CH3_0_Re-Crp127
*   ---
*   CH0_0_Im-Crp127   CH0_0_Re-Crp127   CH1_0_Im-Crp127   CH1_0_Re-Crp127
*   CH2_0_Im-Crp127   CH2_0_Re-Crp127   CH3_0_Im-Crp127   CH3_0_Re-Crp127
*   ---
* | | | |
* | | | |
* | | | |
* | | | |
* | | | |
* | | | |
*   CH0_127_Im-Crp0   CH0_127_Re-Crp0   CH1_127_Im-Crp0   CH1_127_Re-Crp0
*   CH2_127_Im-Crp0   CH2_127_Re-Crp0   CH3_127_Im-Crp0   CH3_127_Re-Crp0
*   ---
*   CH0_127_Im-Crp0   CH0_127_Re-Crp0   CH1_127_Im-Crp0   CH1_127_Re-Crp0
*   CH2_127_Im-Crp0   CH2_127_Re-Crp0   CH3_127_Im-Crp0   CH3_127_Re-Crp0
*   ---
*   CH0_127_Im-Crp0   CH0_127_Re-Crp0   CH1_127_Im-Crp0   CH1_127_Re-Crp0
*   CH2_127_Im-Crp0   CH2_127_Re-Crp0   CH3_127_Im-Crp0   CH3_127_Re-Crp0
*   ---
* | | | |
* | | | |
* | | | |
*   CH0_127_Im-Crp127 CH0_127_Re-Crp127 CH1_127_Im-Crp127 CH1_127_Re-Crp127
*   CH2_127_Im-Crp127 CH2_127_Re-Crp127 CH3_127_Im-Crp127 CH3_127_Re-Crp127
*   ---
*   CH0_127_Im-Crp127 CH0_127_Re-Crp127 CH1_127_Im-Crp127 CH1_127_Re-Crp127
*   CH2_127_Im-Crp127 CH2_127_Re-Crp127 CH3_127_Im-Crp127 CH3_127_Re-Crp127
*   ---
*   CH0_127_Im-Crp127 CH0_127_Re-Crp127 CH1_127_Im-Crp127 CH1_127_Re-Crp127
*   CH2_127_Im-Crp127 CH2_127_Re-Crp127 CH3_127_Im-Crp127 CH3_127_Re-Crp127
*   ---
*
</pre></td></tr>
<tr><td>in</td><td>WR_3</td><td>24bit(RE): Scaling Factor, e.g. 1 for scale left with 1.
24bit(IM): 0.</td></tr>
</table>

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

Twiddle coeficients are placed in TRAM at TR_0_32_0 by the Init function.
Window coeficients are placed in TRAM at TR_0_80_0 by the Init function.
Derotation phase coeficients, are places in TRAM at TR_0_100_0 by the Init function.

Note

The current implementation is ONLY to be used in combination with 256 Range FFT 3TX TD-MIMO.
The input buffer CAN be reused for the output buffer.
All buffers MUST be 8 bytes aligned.

3.9.2.44 RsdkSptDoppler256smp128crp4ch3TxTdMimoRcomp()

```
void * RsdkSptDoppler256smp128crp4ch3TxTdMimoRcomp (
            void  )
```

Function for Doppler FFT 128, 4 physical channels/TX slot, 3TX TD-MIMO.

Compute Doppler FFT, plus compensation for Range Adaptive scaling, storing the results in SRAM.

Parameters

in	*WR_1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Range FFT results), relative to SRAM start address RSDK_SPT_CUBE_BASE_ADDR.

```
*                 INPUT Memory layout:
*                 SRAM, 32bit complex(16b_Im + 16b_Re)
*                 63:48              47:32              31:16              15:0
*                 CH0_0_Im-Crp0      CH0_0_Re-Crp0      CH1_0_Im-Crp0      CH1_0_Re-Crp0
*                 CH2_0_Im-Crp0      CH2_0_Re-Crp0      CH3_0_Im-Crp0      CH3_0_Re-Crp0
*                 ----------------------------------------------------------------------
*                 CH0_0_Im-Crp0      CH0_0_Re-Crp0      CH1_0_Im-Crp0      CH1_0_Re-Crp0
*                 CH2_0_Im-Crp0      CH2_0_Re-Crp0      CH3_0_Im-Crp0      CH3_0_Re-Crp0
*                 ----------------------------------------------------------------------
*                 CH0_0_Im-Crp0      CH0_0_Re-Crp0      CH1_0_Im-Crp0      CH1_0_Re-Crp0
*                 CH2_0_Im-Crp0      CH2_0_Re-Crp0      CH3_0_Im-Crp0      CH3_0_Re-Crp0
*                 ----------------------------------------------------------------------
*                 |                  |                  |                  |
*                 |                  |                  |                  |
*                 |                  |                  |                  |
*                 CH0_0_Im-Crp127    CH0_0_Re-Crp127    CH1_0_Im-Crp127    CH1_0_Re-Crp127
*                 CH2_0_Im-Crp127    CH2_0_Re-Crp127    CH3_0_Im-Crp127    CH3_0_Re-Crp127
*                 ----------------------------------------------------------------------
*                 CH0_0_Im-Crp127    CH0_0_Re-Crp127    CH1_0_Im-Crp127    CH1_0_Re-Crp127
*                 CH2_0_Im-Crp127    CH2_0_Re-Crp127    CH3_0_Im-Crp127    CH3_0_Re-Crp127
*                 ----------------------------------------------------------------------
*                 CH0_0_Im-Crp127    CH0_0_Re-Crp127    CH1_0_Im-Crp127    CH1_0_Re-Crp127
*                 CH2_0_Im-Crp127    CH2_0_Re-Crp127    CH3_0_Im-Crp127    CH3_0_Re-Crp127
*                 ----------------------------------------------------------------------
*                 |                  |                  |                  |
*                 |                  |                  |                  |
*                 |                  |                  |                  |
*                 |                  |                  |                  |
*                 |                  |                  |                  |
*                 |                  |                  |                  |
*                 CH0_127_Im-Crp0    CH0_127_Re-Crp0    CH1_127_Im-Crp0    CH1_127_Re-Crp0
*                 CH2_127_Im-Crp0    CH2_127_Re-Crp0    CH3_127_Im-Crp0    CH3_127_Re-Crp0
*                 ----------------------------------------------------------------------
*                 CH0_127_Im-Crp0    CH0_127_Re-Crp0    CH1_127_Im-Crp0    CH1_127_Re-Crp0
*                 CH2_127_Im-Crp0    CH2_127_Re-Crp0    CH3_127_Im-Crp0    CH3_127_Re-Crp0
*                 ----------------------------------------------------------------------
*                 CH0_127_Im-Crp0    CH0_127_Re-Crp0    CH1_127_Im-Crp0    CH1_127_Re-Crp0
*                 CH2_127_Im-Crp0    CH2_127_Re-Crp0    CH3_127_Im-Crp0    CH3_127_Re-Crp0
*                 ----------------------------------------------------------------------
*                 |                  |                  |                  |
*                 |                  |                  |                  |
*                 |                  |                  |                  |
*                 CH0_127_Im-Crp127  CH0_127_Re-Crp127  CH1_127_Im-Crp127  CH1_127_Re-Crp127
*                 CH2_127_Im-Crp127  CH2_127_Re-Crp127  CH3_127_Im-Crp127  CH3_127_Re-Crp127
*                 ----------------------------------------------------------------------
*                 CH0_127_Im-Crp127  CH0_127_Re-Crp127  CH1_127_Im-Crp127  CH1_127_Re-Crp127
*                 CH2_127_Im-Crp127  CH2_127_Re-Crp127  CH3_127_Im-Crp127  CH3_127_Re-Crp127
*                 ----------------------------------------------------------------------
*                 CH0_127_Im-Crp127  CH0_127_Re-Crp127  CH1_127_Im-Crp127  CH1_127_Re-Crp127
*                 CH2_127_Im-Crp127  CH2_127_Re-Crp127  CH3_127_Im-Crp127  CH3_127_Re-Crp127
*                 ----------------------------------------------------------------------
*
```

Parameters

out	*WR_2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(Doppler FFT results), relative to SRAM start address RSDK_SPT_CUBE_BASE_ADDR

```
*                  OUTPUT Memory layout:
*                  SRAM, 32bit complex(16b_Im + 16b_Re)
*                  63:48              47:32              31:16              15:0
*                  CH0_0_Im-Crp0      CH0_0_Re-Crp0      CH1_0_Im-Crp0      CH1_0_Re-Crp0
*                  CH2_0_Im-Crp0      CH2_0_Re-Crp0      CH3_0_Im-Crp0      CH3_0_Re-Crp0
*                  ----------------------------------------------------------------------------
*                  CH0_0_Im-Crp0      CH0_0_Re-Crp0      CH1_0_Im-Crp0      CH1_0_Re-Crp0
*                  CH2_0_Im-Crp0      CH2_0_Re-Crp0      CH3_0_Im-Crp0      CH3_0_Re-Crp0
*                  ----------------------------------------------------------------------------
*                  CH0_0_Im-Crp0      CH0_0_Re-Crp0      CH1_0_Im-Crp0      CH1_0_Re-Crp0
*                  CH2_0_Im-Crp0      CH2_0_Re-Crp0      CH3_0_Im-Crp0      CH3_0_Re-Crp0
*                  ----------------------------------------------------------------------------
*                  |                  |                  |                  |
*                  |                  |                  |                  |
*                  |                  |                  |                  |
*                  CH0_0_Im-Crp127    CH0_0_Re-Crp127    CH1_0_Im-Crp127    CH1_0_Re-Crp127
*                  CH2_0_Im-Crp127    CH2_0_Re-Crp127    CH3_0_Im-Crp127    CH3_0_Re-Crp127
*                  ----------------------------------------------------------------------------
*                  CH0_0_Im-Crp127    CH0_0_Re-Crp127    CH1_0_Im-Crp127    CH1_0_Re-Crp127
*                  CH2_0_Im-Crp127    CH2_0_Re-Crp127    CH3_0_Im-Crp127    CH3_0_Re-Crp127
*                  ----------------------------------------------------------------------------
*                  CH0_0_Im-Crp127    CH0_0_Re-Crp127    CH1_0_Im-Crp127    CH1_0_Re-Crp127
*                  CH2_0_Im-Crp127    CH2_0_Re-Crp127    CH3_0_Im-Crp127    CH3_0_Re-Crp127
*                  ----------------------------------------------------------------------------
*                  |                  |                  |                  |
*                  |                  |                  |                  |
*                  |                  |                  |                  |
*                  |                  |                  |                  |
*                  |                  |                  |                  |
*                  |                  |                  |                  |
*                  CH0_127_Im-Crp0    CH0_127_Re-Crp0    CH1_127_Im-Crp0    CH1_127_Re-Crp0
*                  CH2_127_Im-Crp0    CH2_127_Re-Crp0    CH3_127_Im-Crp0    CH3_127_Re-Crp0
*                  ----------------------------------------------------------------------------
*                  CH0_127_Im-Crp0    CH0_127_Re-Crp0    CH1_127_Im-Crp0    CH1_127_Re-Crp0
*                  CH2_127_Im-Crp0    CH2_127_Re-Crp0    CH3_127_Im-Crp0    CH3_127_Re-Crp0
*                  ----------------------------------------------------------------------------
*                  CH0_127_Im-Crp0    CH0_127_Re-Crp0    CH1_127_Im-Crp0    CH1_127_Re-Crp0
*                  CH2_127_Im-Crp0    CH2_127_Re-Crp0    CH3_127_Im-Crp0    CH3_127_Re-Crp0
*                  ----------------------------------------------------------------------------
*                  |                  |                  |                  |
*                  |                  |                  |                  |
*                  |                  |                  |                  |
*                  CH0_127_Im-Crp127  CH0_127_Re-Crp127  CH1_127_Im-Crp127  CH1_127_Re-Crp127
*                  CH2_127_Im-Crp127  CH2_127_Re-Crp127  CH3_127_Im-Crp127  CH3_127_Re-Crp127
*                  ----------------------------------------------------------------------------
*                  CH0_127_Im-Crp127  CH0_127_Re-Crp127  CH1_127_Im-Crp127  CH1_127_Re-Crp127
*                  CH2_127_Im-Crp127  CH2_127_Re-Crp127  CH3_127_Im-Crp127  CH3_127_Re-Crp127
*                  ----------------------------------------------------------------------------
*                  CH0_127_Im-Crp127  CH0_127_Re-Crp127  CH1_127_Im-Crp127  CH1_127_Re-Crp127
*                  CH2_127_Im-Crp127  CH2_127_Re-Crp127  CH3_127_Im-Crp127  CH3_127_Re-Crp127
*                  ----------------------------------------------------------------------------
*
*
```

Parameters

in	*WR_3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Compensation factors list), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. These compensation factors are multiplication factors, not to be used directly from Range CF which are shift factors. Shall be computed on the CORE based on the Range CF: doppler_cf[i] = 0x7FFF >> range_cf[i]. * OUTPUT Memory layout: * SRAM, 16bit real * 63:48 47:32 31:16 15:0 * CF0_CH0CH1 CF0_CH2CH3 CF1_CH0CH1 CF1_CH2CH3 * CF2_CH0CH1 CF2_CH2CH3 CF3_CH0CH1 CF3_CH2CH3 * | | | | * CF380_CH0CH1 CF380_CH2CH3 CF381_CH0CH1 CF381_CH2CH3 * CF382_CH0CH1 CF382_CH2CH3 CF383_CH0CH1 CF383_CH2CH3 *
in	*WR_4*	24bit(RE): Scaling Factor, e.g. 1 for scale left with 1. 24bit(IM): 0.

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

Twiddle coeficients are placed in TRAM at TR_0_32_0 by the Init function.
Window coeficients are placed in TRAM at TR_0_80_0 by the Init function.
Derotation phase coeficients, are places in TRAM at TR_0_100_0 by the Init function.
The kernel rely on the assumption that TRAM can be used as temp buffer from TR_0_132_0 to TR_0_324_0.

Note

The current implementation is ONLY to be used in combination with 256 Range FFT 3TX TD-MIMO.
The input buffer CAN be reused for the output buffer.
All buffers MUST be 8 bytes aligned.

3.9.2.45 RsdkSptDoppler256smp128crp8ch6TxTdMimoCp4d()

```
void * RsdkSptDoppler256smp128crp8ch6TxTdMimoCp4d (
            void  )
```

3.9.2.46 RsdkSptDoppler256smp256crp4ch()

```
void * RsdkSptDoppler256smp256crp4ch (
            void  )
```

Function for Doppler FFT 256, 4 channels(CH#0..CH#3)

Compute Doppler FFT storing results in SRAM.

Parameters

in	*WR_1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Range FFT results), relative to SRAM start address RSDK_SPT_CUBE_BASE_ADDR. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * CH0_0-Im-Crp0 CH0_0-Re-Crp0 CH1_0-Im-Crp0 CH1_0-Re-Crp0 * CH2_0-Im-Crp0 CH2_0-Re-Crp0 CH3_0_Im-Crp0 CH3_0-Re-Crp0 * | | | | * CH0_0-Im-Crp255 CH0_0-Re-Crp255 CH1_0-Im-Crp255 CH1_0-Re-Crp255 * CH2_0-Im-Crp255 CH2_0-Re-Crp255 CH3_0_Im-Crp255 CH3_0-Re-Crp255 * ---------- * | | | | * | | | | * | | | | * ---------- * CH0_127-Im-Crp0 CH0_127-Re-Crp0 CH1_127-Im-Crp0 CH1_127-Re-Crp0 * CH2_127-Im-Crp0 CH2_127-Re-Crp0 CH3_127_Im-Crp0 CH3_127-Re-Crp0 * | | | | * CH0_127-Im-Crp255 CH0_127-Re-Crp255 CH1_127-Im-Crp255 CH1_127-Re-Crp2 * CH2_127-Im-Crp255 CH2_127-Re-Crp255 CH3_127_Im-Crp255 CH3_127-Re-Crp2 * ---------- *
out	*WR_2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(Doppler FFT results), relative to SRAM start address RSDK_SPT_CUBE_BASE_ADDR * OUTPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * CH0_0-Im-Crp0 CH0_0-Re-Crp0 CH1_0-Im-Crp0 CH1_0-Re-Crp0 * CH2_0-Im-Crp0 CH2_0-Re-Crp0 CH3_0_Im-Crp0 CH3_0-Re-Crp0 * | | | | * CH0_0-Im-Crp255 CH0_0-Re-Crp255 CH1_0-Im-Crp255 CH1_0-Re-Crp255 * CH2_0-Im-Crp255 CH2_0-Re-Crp255 CH3_0_Im-Crp255 CH3_0-Re-Crp255 * ---------- * | | | | * | | | | * | | | | * ---------- * CH0_127-Im-Crp0 CH0_127-Re-Crp0 CH1_127-Im-Crp0 CH1_127-Re-Crp0 * CH2_127-Im-Crp0 CH2_127-Re-Crp0 CH3_127_Im-Crp0 CH3_127-Re-Crp0 * | | | | * CH0_127-Im-Crp255 CH0_127-Re-Crp255 CH1_127-Im-Crp255 CH1_127-Re-Crp2 * CH2_127-Im-Crp255 CH2_127-Re-Crp255 CH3_127_Im-Crp255 CH3_127-Re-Crp2 * ---------- *
in	*WR_3*	24bit(RE): Scaling Factor, e.g. 1 for scale left with 1. 24bit(IM): 0.

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

Twiddle coeficients are placed in TRAM at TR_0_0_0 by the Init function.
Window coeficients are placed in TRAM at TR_0_64_0 by the Init function.

Note

The kernel implementation guarantees no saturation for 0 output scaling factor. If other scaling is used, it's the user responsibility to prevent saturation.
The current implementation is ONLY to be used in combination with 256 Range FFT.
The input buffer CAN be reused for the output buffer.
All buffers MUST be 8 bytes aligned.

3.9.2.47 RsdkSptDoppler256smp256crp4ch2TxTdMimoCp4d()

```
void * RsdkSptDoppler256smp256crp4ch2TxTdMimoCp4d (
            void  )
```

Function for Doppler FFT 256, 4 physical channels/TX slot, 2TX TD-MIMO.

Compute Doppler FFT storing compressed results(CP4D) in SRAM.

Parameters

in	*WR_1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Range FFT compressed results), relative to SRAM start address RSDK_SPT_CUBE_BASE_ADDR. * INPUT Memory layout: * SRAM, 64bit compressed data * 63:0 * CH0_0-CH1_0-CH2_0-CH3_0-TX0_Crp0 * CH0_0-CH1_0-CH2_0-CH3_0-TX1_Crp0 * | * CH0_0-CH1_0-CH2_0-CH3_0-TX0_Crp255 * CH0_0-CH1_0-CH2_0-CH3_0-TX1_Crp255 ---> RangeBin ##0 * ---------- * | * | * | * ---------- * CH0_127-CH1_127-CH2_127-CH3_127-TX0_Crp0 * CH0_127-CH1_127-CH2_127-CH3_127-TX1_Crp0 * | * CH0_127-CH1_127-CH2_127-CH3_127-TX0_Crp255 * CH0_127-CH1_127-CH2_127-CH3_127-TX1_Crp255 ---> RangeBin ##127 * ---------- *

Parameters

out	*WR↩_2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(Doppler FFT compressed results), relative to SRAM start address RSDK_SPT_CUBE_BASE_ADDR * OUTPUT Memory layout: * SRAM, 64bit compressed data * 63:0 * CH0_0-CH1_0-CH2_0-CH3_0-TX0_Crp0 * CH0_0-CH1_0-CH2_0-CH3_0-TX1_Crp0 * | * CH0_0-CH1_0-CH2_0-CH3_0-TX0_Crp255 * CH0_0-CH1_0-CH2_0-CH3_0-TX1_Crp255 ---> RangeBin ##0 * ---------- * | * | * | * ---------- * CH0_127-CH1_127-CH2_127-CH3_127-TX0_Crp0 * CH0_127-CH1_127-CH2_127-CH3_127-TX1_Crp0 * | * CH0_127-CH1_127-CH2_127-CH3_127-TX0_Crp255 * CH0_127-CH1_127-CH2_127-CH3_127-TX1_Crp255 ---> RangeBin ##127 * ---------- *
in	*WR↩_3*	24bit(RE): Pre-Scaling Factor, e.g. 1 for left shift with 1. 24bit(IM): 0. Value given from the application level, based on global max provided by the range kernel. Shall be computed on the CORE as: pre_scaling value = floor(log2(0x7FFFFF / 1Q23 max_glb)).
in	*WR↩_4*	24bit(RE): Scaling Factor on output data, e.g. 1 for left shift with 1. 24bit(IM): 0.

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

Twiddle coeficients are placed in TRAM at TR_0_0_0 by the Init function.
Window coeficients are placed in TRAM at TR_0_64_0 by the Init function.
Derotation phase coeficients are places in TRAM at TR_0_98_0 by the Init function.

Note

The current implementation is ONLY to be used in combination with 256 Range FFT 2TX TD-MIMO.
The input buffer CAN be reused for the output buffer.
All buffers MUST be 8 bytes aligned.

3.9.2.48 RsdkSptDoppler256smp256crp4chRcomp()

```
void * RsdkSptDoppler256smp256crp4chRcomp (
            void  )
```

Function for Doppler FFT 256, 4 channels(CH#0..CH#3)

Compute Doppler FFT, plus compensation for Range Adaptive scaling, storing the results in SRAM.

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Range FFT results), relative to SRAM start address RSDK_SPT_CUBE_BASE_ADDR. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * CH0_0-Im-Crp0 CH0_0-Re-Crp0 CH1_0-Im-Crp0 CH1_0-Re-Crp0 * CH2_0-Im-Crp0 CH2_0-Re-Crp0 CH3_0_Im-Crp0 CH3_0-Re-Crp0 * | | | | * CH0_0-Im-Crp255 CH0_0-Re-Crp255 CH1_0-Im-Crp255 CH1_0-Re-Crp255 * CH2_0-Im-Crp255 CH2_0-Re-Crp255 CH3_0_Im-Crp255 CH3_0-Re-Crp255 * ---------- * | | | | * | | | | * | | | | * ---------- * CH0_127-Im-Crp0 CH0_127-Re-Crp0 CH1_127-Im-Crp0 CH1_127-Re-Crp0 * CH2_127-Im-Crp0 CH2_127-Re-Crp0 CH3_127_Im-Crp0 CH3_127-Re-Crp0 * | | | | * CH0_127-Im-Crp255 CH0_127-Re-Crp255 CH1_127-Im-Crp255 CH1_127-Re-Crp2 * CH2_127-Im-Crp255 CH2_127-Re-Crp255 CH3_127_Im-Crp255 CH3_127-Re-Crp2 * ---------- *
out	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(Doppler FFT results), relative to SRAM start address RSDK_SPT_CUBE_BASE_ADDR * OUTPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * CH0_0-Im-Crp0 CH0_0-Re-Crp0 CH1_0-Im-Crp0 CH1_0-Re-Crp0 * CH2_0-Im-Crp0 CH2_0-Re-Crp0 CH3_0_Im-Crp0 CH3_0-Re-Crp0 * | | | | * CH0_0-Im-Crp255 CH0_0-Re-Crp255 CH1_0-Im-Crp255 CH1_0-Re-Crp255 * CH2_0-Im-Crp255 CH2_0-Re-Crp255 CH3_0_Im-Crp255 CH3_0-Re-Crp255 * ---------- * | | | | * | | | | * | | | | * ---------- * CH0_127-Im-Crp0 CH0_127-Re-Crp0 CH1_127-Im-Crp0 CH1_127-Re-Crp0 * CH2_127-Im-Crp0 CH2_127-Re-Crp0 CH3_127_Im-Crp0 CH3_127-Re-Crp0 * | | | | * CH0_127-Im-Crp255 CH0_127-Re-Crp255 CH1_127-Im-Crp255 CH1_127-Re-Crp2 * CH2_127-Im-Crp255 CH2_127-Re-Crp255 CH3_127_Im-Crp255 CH3_127-Re-Crp2 * ---------- *

Parameters

in	*WR_3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Compensation factors list), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. These compensation factors are multiplication factors, not to be used directly from Range CF which are shift factors. Shall be computed on the CORE based on the Range CF: doppler_cf[i] = 0x7FFF >> range_cf[i]. * OUTPUT Memory layout: * SRAM, 16bit real * 63:48 47:32 31:16 15:0 * CF0_CH0CH1 CF1_CH0CH1 CF2_CH0CH1 CF3_CH0CH1 * | | | | * CF252_CH0CH1 CF253_CH0CH1 CF254_CH0CH1 CF255_CH0CH1 * CF0_CH2CH3 CF1_CH2CH3 CF2_CH2CH3 CF3_CH2CH3 * | | | | * CF252_CH2CH3 CF253_CH2CH3 CF254_CH2CH3 CF255_CH2CH3 *
in	*WR_4*	24bit(RE): Scaling Factor, e.g. 1 for scale left with 1. 24bit(IM): 0.

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

Twiddle coeficients are placed in TRAM at TR_0_0_0 by the Init function.
Window coeficients are placed in TRAM at TR_0_64_0 by the Init function.
The kernel rely on the assumption that TRAM can be used as temp buffer from TR_0_98_0 to TR_0_162_0.

Note

The kernel implementation guarantees no saturation for 0 output scaling factor. If other scaling is used, it's the user responsibility to prevent saturation.
The current implementation is ONLY to be used in combination with 256 Range FFT.
The input buffer CAN be reused for the output buffer.
All buffers MUST be 8 bytes aligned.

3.9.2.49 RsdkSptDoppler256smp256crp8ch()

```
void * RsdkSptDoppler256smp256crp8ch (
            void  )
```

3.9.2.50 RsdkSptDoppler256smp256crp8ch2TxTdMimoCp4d()

```
void * RsdkSptDoppler256smp256crp8ch2TxTdMimoCp4d (
            void  )
```

3.9.2.51 RsdkSptDoppler256smp256crp8chRcomp()

```
void * RsdkSptDoppler256smp256crp8chRcomp (
            void  )
```

3.9.2.52 RsdkSptDoppler512smp128crp4ch()

```
void * RsdkSptDoppler512smp128crp4ch (
            void  )
```

Function for Doppler FFT 128, 4 channels(CH#0..CH#3)

Compute Doppler FFT storing results in SRAM.

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Range FFT results), relative to SRAM start address RSDK_SPT_CUBE_BASE_ADDR.

```
*                 INPUT Memory layout:
*                 SRAM, 32bit complex(16b_Im + 16b_Re)
*                 63:48               47:32               31:16               15:0
*                 CH0_0-Im-Crp0       CH0_0-Re-Crp0       CH1_0-Im-Crp0       CH1_0-Re-Crp0
*                 CH2_0-Im-Crp0       CH2_0-Re-Crp0       CH3_0_Im-Crp0       CH3_0-Re-Crp0
*                 |                   |                   |                   |
*                 CH0_0-Im-Crp127     CH0_0-Re-Crp127     CH1_0-Im-Crp127     CH1_0-Re-Crp127
*                 CH2_0-Im-Crp127     CH2_0-Re-Crp127     CH3_0_Im-Crp127     CH3_0-Re-Crp127
*                 ------------------------------------------------------------------------------
*                 |                   |                   |                   |
*                 |                   |                   |                   |
*                 |                   |                   |                   |
*                 ------------------------------------------------------------------------------
*                 CH0_255-Im-Crp0     CH0_255-Re-Crp0     CH1_255-Im-Crp0     CH1_255-Re-Crp0
*                 CH2_255-Im-Crp0     CH2_255-Re-Crp0     CH3_255_Im-Crp0     CH3_255-Re-Crp0
*                 |                   |                   |                   |
*                 CH0_255-Im-Crp127   CH0_255-Re-Crp127   CH1_255-Im-Crp127   CH1_255-Re-Crp1
*                 CH2_255-Im-Crp127   CH2_255-Re-Crp127   CH3_255_Im-Crp127   CH3_255-Re-Crp1
*                 ------------------------------------------------------------------------------
*
```

Parameters

out	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(Doppler FFT results), relative to SRAM start address RSDK_SPT_CUBE_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * CH0_0-Im-Crp0 CH0_0-Re-Crp0 CH1_0-Im-Crp0 CH1_0-Re-Crp0 * CH2_0-Im-Crp0 CH2_0-Re-Crp0 CH3_0-Im-Crp0 CH3_0-Re-Crp0 * | | | | * CH0_0-Im-Crp127 CH0_0-Re-Crp127 CH1_0-Im-Crp127 CH1_0-Re-Crp127 * CH2_0-Im-Crp127 CH2_0-Re-Crp127 CH3_0-Im-Crp127 CH3_0-Re-Crp127 * ---------- * | | | | * | | | | * | | | | * ---------- * CH0_255-Im-Crp0 CH0_255-Re-Crp0 CH1_255-Im-Crp0 CH1_255-Re-Crp0 * CH2_255-Im-Crp0 CH2_255-Re-Crp0 CH3_255_Im-Crp0 CH3_255-Re-Crp0 * | | | | * CH0_255-Im-Crp127 CH0_255-Re-Crp127 CH1_255-Im-Crp127 CH1_255-Re-Crp1 * CH2_255-Im-Crp127 CH2_255-Re-Crp127 CH3_255_Im-Crp127 CH3_255-Re-Crp1 * ---------- *
in	*WR↩ _3*	24bit(RE): Scaling Factor, e.g. 1 for scale left with 1. 24bit(IM): 0.

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

Twiddle coeficients are placed in TRAM at TR_0_64_0 by the Init function.
Window coeficients are placed in TRAM at TR_0_144_0 by the Init function.

Note

The kernel implementation guarantees no saturation for 0 output scaling factor. If other scaling is used, it's the user responsibility to prevent saturation.
The current implementation is ONLY to be used in combination with 512 Range FFT.
The input buffer CAN be reused for the output buffer.
All buffers MUST be 8 bytes aligned.

3.9.2.53 RsdkSptDoppler512smp128crp4chRcomp()

```
void * RsdkSptDoppler512smp128crp4chRcomp (
            void  )
```

Function for Doppler FFT 128, 4 channels(CH#0..CH#3).

Compute Doppler FFT, plus compensation for Range Adaptive scaling, storing the results in SRAM.

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Range FFT results), relative to SRAM start address RSDK_SPT_CUBE_BASE_ADDR. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * CH0_0-Im-Crp0 CH0_0-Re-Crp0 CH1_0-Im-Crp0 CH1_0-Re-Crp0 * CH2_0-Im-Crp0 CH2_0-Re-Crp0 CH3_0_Im-Crp0 CH3_0-Re-Crp0 * | | | | * CH0_0-Im-Crp127 CH0_0-Re-Crp127 CH1_0-Im-Crp127 CH1_0-Re-Crp127 * CH2_0-Im-Crp127 CH2_0-Re-Crp127 CH3_0_Im-Crp127 CH3_0-Re-Crp127 * ---------- * | | | | * | | | | * | | | | * ---------- * CH0_255-Im-Crp0 CH0_255-Re-Crp0 CH1_255-Im-Crp0 CH1_255-Re-Crp0 * CH2_255-Im-Crp0 CH2_255-Re-Crp0 CH3_255_Im-Crp0 CH3_255-Re-Crp0 * | | | | * CH0_255-Im-Crp127 CH0_255-Re-Crp127 CH1_255-Im-Crp127 CH1_255-Re-Crp1 * CH2_255-Im-Crp127 CH2_255-Re-Crp127 CH3_255_Im-Crp127 CH3_255-Re-Crp1 * ---------- *
out	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(Doppler FFT results), relative to SRAM start address RSDK_SPT_CUBE_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * CH0_0-Im-Crp0 CH0_0-Re-Crp0 CH1_0-Im-Crp0 CH1_0-Re-Crp0 * CH2_0-Im-Crp0 CH2_0-Re-Crp0 CH3_0_Im-Crp0 CH3_0-Re-Crp0 * | | | | * CH0_0-Im-Crp127 CH0_0-Re-Crp127 CH1_0-Im-Crp127 CH1_0-Re-Crp127 * CH2_0-Im-Crp127 CH2_0-Re-Crp127 CH3_0_Im-Crp127 CH3_0-Re-Crp127 * ---------- * | | | | * | | | | * | | | | * ---------- * CH0_255-Im-Crp0 CH0_255-Re-Crp0 CH1_255-Im-Crp0 CH1_255-Re-Crp0 * CH2_255-Im-Crp0 CH2_255-Re-Crp0 CH3_255_Im-Crp0 CH3_255-Re-Crp0 * | | | | * CH0_255-Im-Crp127 CH0_255-Re-Crp127 CH1_255-Im-Crp127 CH1_255-Re-Crp1 * CH2_255-Im-Crp127 CH2_255-Re-Crp127 CH3_255_Im-Crp127 CH3_255-Re-Crp1 * ---------- *
in	*WR↩ _3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Compensation factors list), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. These compensation factors are multiplication factors, not to be used directly from Range CF which are shift factors. Shall be computed on the CORE based on the Range CF: doppler_cf[i] = 0x7FFF >> range_cf[i]. * OUTPUT Memory layout: * SRAM, 16bit real * 63:48 47:32 31:16 15:0 * CF0_CH0CH1 CF1_CH0CH1 CF2_CH0CH1 CF3_CH0CH1 * | | | | * CF124_CH0CH1 CF125_CH0CH1 CF126_CH0CH1 CF127_CH0CH1 * CF0_CH2CH3 CF1_CH2CH3 CF2_CH2CH3 CF3_CH2CH3 * | | | | * CF124_CH2CH3 CF125_CH2CH3 CF126_CH2CH3 CF127_CH2CH3 *

Parameters

in	*WR_4*	24bit(RE): Scaling Factor, e.g. 1 for scale left with 1. 24bit(IM): 0.

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

Twiddle coeficients are placed in TRAM at TR_0_64_0 by the Init function.
Window coeficients are placed in TRAM at TR_0_144_0 by the Init function.
The kernel rely on the assumption that TRAM can be used as temp buffer from TR_0_162_0 to TR_0_194_0.

Note

The kernel implementation guarantees no saturation for 0 output scaling factor. If other scaling is used, it's the user responsibility to prevent saturation.
The current implementation is ONLY to be used in combination with 512 Range FFT.
The input buffer CAN be reused for the output buffer.
All buffers MUST be 8 bytes aligned.

3.9.2.54 RsdkSptDoppler512smp128crp8ch()

```
void * RsdkSptDoppler512smp128crp8ch (
            void  )
```

3.9.2.55 RsdkSptDoppler512smp128crp8chRcomp()

```
void * RsdkSptDoppler512smp128crp8chRcomp (
            void  )
```

3.9.2.56 RsdkSptDspExampleDirectBlocking()

```
void * RsdkSptDspExampleDirectBlocking (
            void  )
```

Sends a blocking "DSP" command which triggers execution of a preselected function on the BBE32 DSP. All command parameters are hardcoded at compile time. This routine will exit only after the DSP function has replied by pushing a response to the SPT Response Queue.

Note

3.9.2.57 RsdkSptDspExampleIndirectBlocking()

```
void * RsdkSptDspExampleIndirectBlocking (
            void  )
```

Sends a blocking "DSP" command which triggers execution of a preselected function on the BBE32 DSP. All command parameters are passed from the application at runtime, through Work Registers. This routine will exit only after the DSP function has replied by pushing a response to the SPT Response Queue.

Note

3.9.2.58 RsdkSptInit1024smp128crp4ch()

```
void * RsdkSptInit1024smp128crp4ch (
            void  )
```

Initialization function for 1024 samples and 128 chirps, 4 physical channels.

Copy twiddles, window, steering vector coefficients from SRAM to TRAM.

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT1024 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For twiddle generation algorithm, see S32R41_RM 68.9.5.3 / SAF85xx_RM 68.9.5.3. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * | | | | * | | | | * | | | | * tw128_Im tw128_Re tw128_Im tw128_Re * tw128_Im tw128_Re tw128_Im tw128_Re * tw128_Im tw128_Re tw128_Im tw128_Re * tw128_Im tw128_Re tw128_Im tw128_Re *
in	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT128 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For twiddle generation algorithm, see S32R41_RM 68.9.5.3 / SAF85xx_RM 68.9.5.3. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * | | | | * | | | | * | | | | * tw16_Im tw16_Re tw16_Im tw16_Re * tw16_Im tw16_Re tw16_Im tw16_Re * tw16_Im tw16_Re tw16_Im tw16_Re * tw16_Im tw16_Re tw16_Im tw16_Re *

Parameters

in	*WR↩ _3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Reordered Window coef for Range FFT1024), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For reorder algorithm, see S32R41_RM 68.9.9.7 / SAF85xx_RM 68.9.9.7. * INPUT Memory layout: * SRAM, 16bit Real Window Coef * 63:48 47:32 31:16 15:0 * win0 win256 win512 win768 * win64 win320 win576 win832 * | | | | * | | | | * win63 win319 win575 win831 * win127 win383 win639 win895 * win128 win384 win640 win896 * win192 win448 win704 win960 * | | | | * | | | | * win191 win447 win703 win959 * win255 win511 win767 win1023 *
in	*WR↩ _4*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Reordered Window coef for Doppler FFT128), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For reorder algorithm, see S32R41_RM 68.9.9.7 / SAF85xx_RM 68.9.9.7. * INPUT Memory layout: * SRAM, 16bit Real Window Coef * 63:48 47:32 31:16 15:0 * win0 win32 win64 win96 * win8 win40 win72 win104 * | | | | * | | | | * win7 win39 win71 win103 * win15 win47 win79 win111 * win16 win48 win80 win112 * win24 win56 win88 win120 * | | | | * | | | | * win23 win55 win87 win119 * win31 win63 win95 win127 *
in	*WR↩ _5*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT16 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For twiddle generation algorithm, see S32R41_RM 68.9.5.3 / SAF85xx_RM 68.9.5.3. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re *

Parameters

in	*WR_6*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Beamforming Steering Vector), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For steering coeficients generation algorithm, see rSDK_Auxiliary_vector_generation.docx. * INPUT Memory layout: * SRAM, 32 bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * SV0_Im-CH0 SV0_Re-CH0 SV0_Im-CH1 SV0_Re-CH1 * SV0_Im-CH2 SV0_Re-CH2 SV0_Im-CH3 SV0_Re-CH3 * SV1_Im-CH0 SV1_Re-CH0 SV1_Im-CH1 SV1_Re-CH1 * SV1_Im-CH2 SV1_Re-CH2 SV1_Im-CH3 SV1_Re-CH3 * | | | | * | | | | * | | | | * SV63_Im-CH0 SV63_Re-CH0 SV63_Im-CH1 SV63_Re-CH1 * SV63_Im-CH2 SV63_Re-CH2 SV63_Im-CH3 SV63_Re-CH3 *

Returns

WR_0: No value returned.

Return values

No	value returned.

Postcondition

Twiddle factors for FFT1024 placed in TRAM at TR_0_0_0.
Twiddle factors for FFT128 placed in TRAM at TR_0_128_0.
Window coeficients for Range FFT512 placed in TRAM at TR_0_144_0.
Window coeficients for Doppler FFT128 placed in TRAM at TR_0_272_0.
Twiddle factors for FFT16 placed in TRAM at TR_0_288_0.
Beamforming Steering Vector placed in TRAM at TR_0_448_0.
Free TRAM space TR_0_290_0 to TR_0_448_0.

Note

All buffers MUST be 8 bytes aligned.

3.9.2.59 RsdkSptInit1024smp128crp8ch()

```
void * RsdkSptInit1024smp128crp8ch (
            void  )
```

3.9.2.60 RsdkSptInit1024smp256crp4ch3TxDdMimo()

```
void * RsdkSptInit1024smp256crp4ch3TxDdMimo (
            void  )
```

Initialization function for 1024 samples and 256 chirps, 4 physical channels, 3TX DD-Mimo.

Copy twiddles and window coefficients from SRAM to TRAM.

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT1024 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For twiddle generation algorithm, see S32R41_RM 68.9.5.3 / SAF85xx_RM 68.9.5.3. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * | | | | * | | | | * | | | | * tw128_Im tw128_Re tw128_Im tw128_Re * tw128_Im tw128_Re tw128_Im tw128_Re * tw128_Im tw128_Re tw128_Im tw128_Re * tw128_Im tw128_Re tw128_Im tw128_Re *
in	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT256 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For twiddle generation algorithm, see S32R41_RM 68.9.5.3 / SAF85xx_RM 68.9.5.3. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * | | | | * | | | | * | | | | * tw32_Im tw32_Re tw32_Im tw32_Re * tw32_Im tw32_Re tw32_Im tw32_Re * tw32_Im tw32_Re tw32_Im tw32_Re * tw32_Im tw32_Re tw32_Im tw32_Re *

Parameters

in	*WR_3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Reordered Window coef for Range FFT1024), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For reorder algorithm, see S32R41_RM 68.9.9.7 / SAF85xx_RM 68.9.9.7. * INPUT Memory layout: * SRAM, 16bit Real Window Coef * 63:48 47:32 31:16 15:0 * win0 win256 win512 win768 * win64 win320 win576 win832 * | | | | * | | | | * win63 win319 win575 win831 * win127 win383 win639 win895 * win128 win384 win640 win896 * win192 win448 win704 win960 * | | | | * | | | | * win191 win447 win703 win959 * win255 win511 win767 win1023 *
in	*WR_4*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Reordered Window coef for Doppler FFT256), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For reorder algorithm, see S32R41_RM 68.9.9.7 / SAF85xx_RM 68.9.9.7. * INPUT Memory layout: * SRAM, 16bit Real Window Coef * 63:48 47:32 31:16 15:0 * win0 win64 win128 win192 * win16 win80 win144 win208 * | | | | * | | | | * win15 win79 win143 win207 * win31 win95 win159 win223 * win32 win96 win160 win224 * win48 win112 win176 win240 * | | | | * | | | | * win47 win111 win175 win239 * win63 win127 win191 win255 *

Parameters

in	*WR_5*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Beamforming Steering Vector), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For steering coeficients generation algorithm, see rSDK_Auxiliary_vector_generation.docx.

```
*                  INPUT Memory layout:
*                  SRAM, 32 bit complex(16b_Im + 16b_Re)
*                  63:48          47:32           31:16           15:0
*                  SV0_Im-CH0     SV0_Re-CH0      SV0_Im-CH1      SV0_Re-CH1
*                  SV0_Im-CH2     SV0_Re-CH2      SV0_Im-CH3      SV0_Re-CH3
*                  SV0_Im-CH4     SV0_Re-CH4      SV0_Im-CH5      SV0_Re-CH5
*                  SV0_Im-CH6     SV0_Re-CH6      SV0_Im-CH7      SV0_Re-CH7
*                  SV0_Im-CH8     SV0_Re-CH8      SV0_Im-CH9      SV0_Re-CH9
*                  SV0_Im-CH10    SV0_Re-CH10     SV0_Im-CH11     SV0_Re-CH11
*                  |              |               |               |
*                  |              |               |               |
*                  |              |               |               |
*                  SV63_Im-CH0    SV63_Re-CH0     SV63_Im-CH1     SV63_Re-CH1
*                  SV63_Im-CH2    SV63_Re-CH2     SV63_Im-CH3     SV63_Re-CH3
*                  SV63_Im-CH4    SV63_Re-CH4     SV63_Im-CH5     SV63_Re-CH5
*                  SV63_Im-CH6    SV63_Re-CH6     SV63_Im-CH7     SV63_Re-CH7
*                  SV63_Im-CH8    SV63_Re-CH8     SV63_Im-CH9     SV63_Re-CH9
*                  SV63_Im-CH10   SV63_Re-CH10    SV63_Im-CH11    SV63_Re-CH11
*
```

Returns

WR_0: No value returned.

Return values

No	value returned.

Postcondition

Twiddle factors for FFT1024 placed in TRAM at TR_0_0_0.
Twiddle factors for FFT256 placed in TRAM at TR_0_128_0.
Window coeficients for Range FFT1024 placed in TRAM at TR_0_160_0.
Window coeficients for Doppler FFT256 placed in TRAM at TR_0_288_0.
Beamforming Steering Vector placed in TRAM at TR_0_384_0.
Free TRAM space from TR_0_320_0 to TR_0_384_0.

Note

All buffers MUST be 8 bytes aligned.

3.9.2.61 RsdkSptInit1024smp64crp4ch()

```
void * RsdkSptInit1024smp64crp4ch (
            void  )
```

Initialization function for 1024 samples and 64 chirps, 4 physical channels.

Copy twiddles, window, steering vector coefficients from SRAM to TRAM.

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT1024 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For twiddle generation algorithm, see S32R41_RM 68.9.5.3 / SAF85xx_RM 68.9.5.3. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * | | | | * | | | | * | | | | * tw128_Im tw128_Re tw128_Im tw128_Re * tw128_Im tw128_Re tw128_Im tw128_Re * tw128_Im tw128_Re tw128_Im tw128_Re * tw128_Im tw128_Re tw128_Im tw128_Re *
in	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT64 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For twiddle generation algorithm, see S32R41_RM 68.9.5.3 / SAF85xx_RM 68.9.5.3. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * | | | | * | | | | * | | | | * tw8_Im tw8_Re tw8_Im tw8_Re * tw8_Im tw8_Re tw8_Im tw8_Re * tw8_Im tw8_Re tw8_Im tw8_Re * tw8_Im tw8_Re tw8_Im tw8_Re *

Parameters

in	*WR↩ _3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Reordered Window coef for Range FFT1024), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For reorder algorithm, see S32R41_RM 68.9.9.7 / SAF85xx_RM 68.9.9.7. * INPUT Memory layout: * SRAM, 16bit Real Window Coef * 63:48 47:32 31:16 15:0 * win0 win256 win512 win768 * win64 win320 win576 win832 * | | | | * | | | | * win63 win319 win575 win831 * win127 win383 win639 win895 * win128 win384 win640 win896 * win192 win448 win704 win960 * | | | | * | | | | * win191 win447 win703 win959 * win255 win511 win767 win1023 *
in	*WR↩ _4*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Reordered Window coef for Doppler FFT64), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For reorder algorithm, see S32R41_RM 68.9.9.7 / SAF85xx_RM 68.9.9.7. * INPUT Memory layout: * SRAM, 16bit Real Window Coef * 63:48 47:32 31:16 15:0 * win0 win16 win32 win48 * win4 win20 win36 win52 * | | | | * | | | | * win3 win19 win35 win51 * win7 win23 win39 win55 * win8 win24 win40 win56 * win12 win28 win44 win60 * | | | | * | | | | * win11 win27 win43 win59 * win15 win31 win47 win63 *
in	*WR↩ _5*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT16 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For twiddle generation algorithm, see S32R41_RM 68.9.5.3 / SAF85xx_RM 68.9.5.3. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re *

Parameters

in	*WR_6*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Beamforming Steering Vector), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For steering coeficients generation algorithm, see rSDK_Auxiliary_vector_generation.docx. * INPUT Memory layout: * SRAM, 32 bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * SV0_Im-CH0 SV0_Re-CH0 SV0_Im-CH1 SV0_Re-CH1 * SV0_Im-CH2 SV0_Re-CH2 SV0_Im-CH3 SV0_Re-CH3 * SV1_Im-CH0 SV1_Re-CH0 SV1_Im-CH1 SV1_Re-CH1 * SV1_Im-CH2 SV1_Re-CH2 SV1_Im-CH3 SV1_Re-CH3 * | | | | * | | | | * | | | | * SV63_Im-CH0 SV63_Re-CH0 SV63_Im-CH1 SV63_Re-CH1 * SV63_Im-CH2 SV63_Re-CH2 SV63_Im-CH3 SV63_Re-CH3 *

Returns

WR_0: No value returned.

Return values

No	value returned.

Postcondition

Twiddle factors for FFT1024 placed in TRAM at TR_0_0_0.
Twiddle factors for FFT64 placed in TRAM at TR_0_128_0.
Window coeficients for Range FFT1024 placed in TRAM at TR_0_136_0.
Window coeficients for Doppler FFT64 placed in TRAM at TR_0_264_0.
Twiddle factors for FFT16 placed in TRAM at TR_0_272_0.
Beamforming Steering Vector placed in TRAM at TR_0_448_0.
Free TRAM space TR_0_274_0 to TR_0_448_0.

Note

All buffers MUST be 8 bytes aligned.

3.9.2.62 RsdkSptInit1024smp64crp4ch4TxTdMimo()

```
void * RsdkSptInit1024smp64crp4ch4TxTdMimo (
            void  )
```

Initialization function for 1024 samples and 64 chirps, 4 physical channels/TX slot, 4TX TD-Mimo.

Copy twiddles, window, steering vector, derotation phase coefficients from SRAM to TRAM.

Parameters

in	*WR_1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT1024 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For twiddle generation algorithm, see S32R41_RM 68.9.5.3 / SAF85xx_RM 68.9.5.3. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * | | | | * | | | | * | | | | * tw128_Im tw128_Re tw128_Im tw128_Re * tw128_Im tw128_Re tw128_Im tw128_Re * tw128_Im tw128_Re tw128_Im tw128_Re * tw128_Im tw128_Re tw128_Im tw128_Re *
in	*WR_2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT64 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For twiddle generation algorithm, see S32R41_RM 68.9.5.3 / SAF85xx_RM 68.9.5.3. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * | | | | * | | | | * | | | | * tw8_Im tw8_Re tw8_Im tw8_Re * tw8_Im tw8_Re tw8_Im tw8_Re * tw8_Im tw8_Re tw8_Im tw8_Re * tw8_Im tw8_Re tw8_Im tw8_Re *

Parameters

in	*WR_3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Reordered Window coef for Range FFT 1024), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For reorder algorithm, see S32R41_RM 68.9.9.7 / SAF85xx_RM 68.9.9.7. * INPUT Memory layout: * SRAM, 16bit Real Window Coef * 63:48 47:32 31:16 15:0 * win0 win256 win512 win768 * win64 win320 win576 win832 * | | | | * | | | | * win63 win319 win575 win831 * win127 win383 win639 win895 * win128 win384 win640 win896 * win192 win448 win704 win960 * | | | | * | | | | * win191 win447 win703 win959 * win255 win511 win767 win1023 *
in	*WR_4*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Reordered Window coef for Doppler FFT 64), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For reorder algorithm, see see S32R41_RM 68.9.9.7 / SAF85xx_RM 68.9.9.7. * INPUT Memory layout: * SRAM, 16bit Real Window Coef * 63:48 47:32 31:16 15:0 * win0 win16 win32 win48 * win4 win20 win36 win52 * | | | | * | | | | * win3 win19 win35 win51 * win7 win23 win39 win55 * win8 win24 win40 win56 * win12 win28 win44 win60 * | | | | * | | | | * win11 win27 win43 win59 * win15 win31 win47 win63 *

Parameters

in	*WR↩ _5*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT32 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For twiddle generation algorithm, see S32R41_RM 68.9.5.3 / SAF85xx_RM 68.9.5.3. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Im tw1_Im tw1_Im * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw3_Im tw3_Re tw3_Im tw3_Re * tw3_Im tw3_Re tw3_Im tw3_Re * tw3_Im tw3_Re tw3_Im tw3_Re * tw3_Im tw3_Re tw3_Im tw3_Re * tw4_Im tw4_Re tw4_Im tw4_Re * tw4_Im tw4_Re tw4_Im tw4_Re * tw4_Im tw4_Re tw4_Im tw4_Re * tw4_Im tw4_Re tw4_Im tw4_Re *
in	*WR↩ _6*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Derotation Phased Vector), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For derotation coeficients generation algorithm, see rSDK_Auxiliary_vector_generation.docx. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * DeRotPh0_Im-TX1 DeRotPh0_Re-TX1 DeRotPh1_Im-TX1 DeRotPh1_Re-TX1 * DeRotPh2_Im-TX1 DeRotPh2_Re-TX1 DeRotPh3_Im-TX1 DeRotPh3_Re-TX1 * | | | | * | | | | * | | | | * DeRotPh60_Im-TX1 DeRotPh60_Re-TX1 DeRotPh61_Im-TX1 DeRotPh61_Re-TX1 * DeRotPh62_Im-TX1 DeRotPh62_Re-TX1 DeRotPh63_Im-TX1 DeRotPh63_Re-TX1 * DeRotPh0_Im-TX2 DeRotPh0_Re-TX2 DeRotPh1_Im-TX2 DeRotPh1_Re-TX2 * DeRotPh2_Im-TX2 DeRotPh2_Re-TX2 DeRotPh3_Im-TX2 DeRotPh3_Re-TX2 * | | | | * | | | | * | | | | * DeRotPh60_Im-TX2 DeRotPh60_Re-TX2 DeRotPh61_Im-TX2 DeRotPh61_Re-TX2 * DeRotPh62_Im-TX2 DeRotPh62_Re-TX2 DeRotPh63_Im-TX2 DeRotPh63_Re-TX2 * DeRotPh0_Im-TX3 DeRotPh0_Re-TX3 DeRotPh1_Im-TX3 DeRotPh1_Re-TX3 * DeRotPh2_Im-TX3 DeRotPh2_Re-TX3 DeRotPh3_Im-TX3 DeRotPh3_Re-TX3 * | | | | * | | | | * | | | | * DeRotPh60_Im-TX3 DeRotPh60_Re-TX3 DeRotPh61_Im-TX3 DeRotPh61_Re-TX3 * DeRotPh62_Im-TX3 DeRotPh62_Re-TX3 DeRotPh63_Im-TX3 DeRotPh63_Re-TX3 *

Parameters

in	*WR_7*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Beamforming Steering Vector), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For steering coeficients generation algorithm, see rSDK_Auxiliary_vector_generation.docx.

```
*                  INPUT Memory layout:
*                  SRAM, 32 bit complex(16b_Im + 16b_Re)
*                  63:48          47:32           31:16           15:0
*                  SV0_Im-CH0     SV0_Re-CH0      SV0_Im-CH1      SV0_Re-CH1
*                  SV0_Im-CH2     SV0_Re-CH2      SV0_Im-CH3      SV0_Re-CH3
*                  SV0_Im-CH4     SV0_Re-CH4      SV0_Im-CH5      SV0_Re-CH5
*                  SV0_Im-CH6     SV0_Re-CH6      SV0_Im-CH7      SV0_Re-CH7
*                  SV0_Im-CH8     SV0_Re-CH8      SV0_Im-CH9      SV0_Re-CH9
*                  SV0_Im-CH10    SV0_Re-CH10     SV0_Im-CH11     SV0_Re-CH11
*                  SV0_Im-CH12    SV0_Re-CH12     SV0_Im-CH13     SV0_Re-CH13
*                  SV0_Im-CH14    SV0_Re-CH14     SV0_Im-CH15     SV0_Re-CH15
*                  SV1_Im-CH0     SV1_Re-CH0      SV1_Im-CH1      SV1_Re-CH1
*                  SV1_Im-CH2     SV1_Re-CH2      SV1_Im-CH3      SV1_Re-CH3
*                  SV1_Im-CH4     SV1_Re-CH4      SV1_Im-CH5      SV1_Re-CH5
*                  SV1_Im-CH6     SV1_Re-CH6      SV1_Im-CH7      SV1_Re-CH7
*                  SV1_Im-CH8     SV1_Re-CH8      SV1_Im-CH9      SV1_Re-CH9
*                  SV1_Im-CH10    SV1_Re-CH10     SV1_Im-CH11     SV1_Re-CH11
*                  SV1_Im-CH12    SV1_Re-CH12     SV1_Im-CH13     SV1_Re-CH13
*                  SV1_Im-CH14    SV1_Re-CH14     SV1_Im-CH15     SV1_Re-CH15
*                  |              |               |               |
*                  |              |               |               |
*                  |              |               |               |
*                  SV63_Im-CH0    SV63_Re-CH0     SV63_Im-CH1     SV63_Re-CH1
*                  SV63_Im-CH2    SV63_Re-CH2     SV63_Im-CH3     SV63_Re-CH3
*                  SV63_Im-CH4    SV63_Re-CH4     SV63_Im-CH5     SV63_Re-CH5
*                  SV63_Im-CH6    SV63_Re-CH6     SV63_Im-CH7     SV63_Re-CH7
*                  SV63_Im-CH8    SV63_Re-CH8     SV63_Im-CH9     SV63_Re-CH9
*                  SV63_Im-CH10   SV63_Re-CH10    SV63_Im-CH11    SV63_Re-CH11
*                  SV63_Im-CH12   SV63_Re-CH12    SV63_Im-CH13    SV63_Re-CH13
*                  SV63_Im-CH14   SV63_Re-CH14    SV63_Im-CH15    SV63_Re-CH15
*
```

Returns

WR_0: No value returned.

Return values

No	value returned.

Postcondition

Twiddle factors for FFT1024 placed in TRAM at TR_0_0_0.
Twiddle factors for FFT64 placed in TRAM at TR_0_128_0.
Window coeficients for Range FFT1024 placed in TRAM at TR_0_136_0.
Window coeficients for Doppler FFT64 placed in TRAM at TR_0_264_0.
Twiddle factors for FFT32 placed in TRAM at TR_0_272_0.
Derotation Phase Vector placed in TRAM at TR_0_276_0.
Beamforming Steering Vector placed in TRAM at TR_0_384_0.
Free TRAM space TR_0_300_0 to TR_0_384_0.

Note

All buffers MUST be 8 bytes aligned.

3.9.2.63 RsdkSptInit2048smp256crp4ch()

```
void * RsdkSptInit2048smp256crp4ch (
            void  )
```

Initialization function for 2048 samples and 256 chirps, 4 physical channels.

Copy twiddles, window, steering vector coefficients from SRAM to TRAM.

Parameters

in	*WR_1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT2048 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For twiddle generation algorithm, see S32R41_RM 68.9.5.3 / SAF85xx_RM 68.9.5.3. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * | | | | * | | | | * | | | | * tw256_Im tw256_Re tw256_Im tw256_Re * tw256_Im tw256_Re tw256_Im tw256_Re * tw256_Im tw256_Re tw256_Im tw256_Re * tw256_Im tw256_Re tw256_Im tw256_Re *
in	*WR_2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT256 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For twiddle generation algorithm, see S32R41_RM 68.9.5.3 / SAF85xx_RM 68.9.5.3. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * | | | | * | | | | * | | | | * tw32_Im tw32_Re tw32_Im tw32_Re * tw32_Im tw32_Re tw32_Im tw32_Re * tw32_Im tw32_Re tw32_Im tw32_Re * tw32_Im tw32_Re tw32_Im tw32_Re *

Parameters

in	*WR↩ _3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Reordered Window coef for Range FFT2048), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For reorder algorithm, see S32R41_RM 68.9.9.7 / SAF85xx_RM 68.9.9.7. * INPUT Memory layout: * SRAM, 16bit Real Window Coef * 63:48 47:32 31:16 15:0 * win0 win512 win1024 win1536 * win128 win640 win1152 win1664 * | | | | * | | | | * win127 win639 win1151 win1663 * win255 win767 win1279 win1791 * win256 win768 win1280 win1792 * win384 win896 win1408 win1920 * | | | | * | | | | * win383 win895 win1407 win1919 * win511 win1023 win1535 win2047 *
in	*WR↩ _4*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Reordered Window coef for Doppler FFT 256), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For reorder algorithm, S32R41_RM 68.9.9.7 / SAF85xx_RM 68.9.9.7. * INPUT Memory layout: * SRAM, 16bit Real Window Coef * 63:48 47:32 31:16 15:0 * win0 win64 win128 win192 * win16 win80 win144 win208 * | | | | * | | | | * win15 win79 win143 win207 * win31 win95 win159 win223 * win32 win96 win160 win224 * win48 win112 win176 win240 * | | | | * | | | | * win47 win111 win175 win239 * win63 win127 win191 win255 *
in	*WR↩ _5*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT16 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For twiddle generation algorithm, see S32R41_RM 68.9.5.3 / SAF85xx_RM 68.9.5.3. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Im tw1_Im tw1_Im * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re *

Parameters

in	*WR↩ _6*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Beamforming Steering Vector), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For steering coeficients generation algorithm, see rSDK_Auxiliary_vector_generation.docx. * INPUT Memory layout: * SRAM, 32 bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * SV0_Im-CH0 SV0_Re-CH0 SV0_Im-CH1 SV0_Re-CH1 * SV0_Im-CH2 SV0_Re-CH2 SV0_Im-CH3 SV0_Re-CH3 * SV1_Im-CH0 SV1_Re-CH0 SV1_Im-CH1 SV1_Re-CH1 * SV1_Im-CH2 SV1_Re-CH2 SV1_Im-CH3 SV1_Re-CH3 * | | | | * | | | | * | | | | * SV63_Im-CH0 SV63_Re-CH0 SV63_Im-CH1 SV63_Re-CH1 * SV63_Im-CH2 SV63_Re-CH2 SV63_Im-CH3 SV63_Re-CH3 *

Returns

WR_0: No value returned.

Return values

No	value returned.

Postcondition

Twiddle factors for FFT2048 placed in TRAM at TR_0_0_0.
Twiddle factors for FFT256 placed in TRAM at TR_0_256_0.
Window coeficients for Range FFT2048 placed in TRAM at TR_1_0_0.
Window coeficients for Doppler FFT256 placed in TRAM at TR_0_288_0.
Twiddle factors for FFT16 placed in TRAM at TR_0_320_0.
Beamforming Steering Vector placed in TRAM at TR_0_448_0.
Free TRAM space TR_0_322_0 to TR_0_448_0. TRAM bank 1 used for Range FFT2048 window coefficients.

Note

All buffers MUST be 8 bytes aligned.

3.9.2.64 RsdkSptInit256smp128crp4ch3TxTdMimo()

```
void * RsdkSptInit256smp128crp4ch3TxTdMimo (
            void  )
```

Initialization function for 256 samples and 128 chirps, 4 physical channels/TX slot, 3TX TD-Mimo.

Copy twiddles, window, steering vector, derotation phase coefficients from SRAM to TRAM.

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT256 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For twiddle generation algorithm, see S32R41_RM 68.9.5.3 / SAF85xx_RM 68.9.5.3. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * | | | | * | | | | * | | | | * tw32_Im tw32_Re tw32_Im tw32_Re * tw32_Im tw32_Re tw32_Im tw32_Re * tw32_Im tw32_Re tw32_Im tw32_Re * tw32_Im tw32_Re tw32_Im tw32_Re *
in	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT128 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For twiddle generation algorithm, see S32R41_RM 68.9.5.3 / SAF85xx_RM 68.9.5.3. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * | | | | * | | | | * | | | | * tw16_Im tw16_Re tw16_Im tw16_Re * tw16_Im tw16_Re tw16_Im tw16_Re * tw16_Im tw16_Re tw16_Im tw16_Re * tw16_Im tw16_Re tw16_Im tw16_Re *

Parameters

in	*WR_3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Reordered Window coef for Range FFT256), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For reorder algorithm, see S32R41_RM 68.9.9.7 / SAF85xx_RM 68.9.9.7. * INPUT Memory layout: * SRAM, 16bit Real Window Coef * 63:48 47:32 31:16 15:0 * win0 win64 win128 win192 * win16 win80 win144 win208 * | | | | * | | | | * win15 win79 win143 win207 * win31 win95 win159 win223 * win32 win96 win160 win224 * win48 win112 win176 win240 * | | | | * | | | | * win47 win111 win175 win239 * win63 win127 win191 win255 *
in	*WR_4*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Reordered Window coef for Doppler FFT128), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For reorder algorithm, see S32R41_RM 68.9.9.7 / SAF85xx_RM 68.9.9.7. * INPUT Memory layout: * SRAM, 16bit Real Window Coef * 63:48 47:32 31:16 15:0 * win0 win32 win64 win96 * win8 win40 win72 win104 * | | | | * | | | | * win7 win39 win71 win103 * win15 win47 win79 win111 * win16 win48 win80 win112 * win24 win56 win88 win120 * | | | | * | | | | * win23 win55 win87 win119 * win31 win63 win95 win127 *

Parameters

in	*WR_5*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT32 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For twiddle generation algorithm, see S32R41_RM 68.9.5.3 / SAF85xx_RM 68.9.5.3. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw3_Im tw3_Re tw3_Im tw3_Re * tw3_Im tw3_Re tw3_Im tw3_Re * tw3_Im tw3_Re tw3_Im tw3_Re * tw3_Im tw3_Re tw3_Im tw3_Re * tw4_Im tw4_Re tw4_Im tw4_Re * tw4_Im tw4_Re tw4_Im tw4_Re * tw4_Im tw4_Re tw4_Im tw4_Re * tw4_Im tw4_Re tw4_Im tw4_Re *
in	*WR_6*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Derotation Phased Vector), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For derotation coeficients generation algorithm, see rSDK_Auxiliary_vector_generation.docx. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * DeRotPh0_Im-TX1 DeRotPh0_Re-TX1 DeRotPh1_Im-TX1 DeRotPh1_Re-TX1 * DeRotPh2_Im-TX1 DeRotPh2_Re-TX1 DeRotPh3_Im-TX1 DeRotPh3_Re-TX1 * | | | | * | | | | * | | | | * DeRotPh124_Im-TX1 DeRotPh124_Re-TX1 DeRotPh125_Im-TX1 DeRotPh125_Re-TX1 * DeRotPh126_Im-TX1 DeRotPh126_Re-TX1 DeRotPh127_Im-TX1 DeRotPh127_Re-TX1 * DeRotPh0_Im-TX2 DeRotPh0_Re-TX2 DeRotPh1_Im-TX2 DeRotPh1_Re-TX2 * DeRotPh2_Im-TX2 DeRotPh2_Re-TX2 DeRotPh3_Im-TX2 DeRotPh3_Re-TX2 * | | | | * | | | | * | | | | * DeRotPh124_Im-TX2 DeRotPh124_Re-TX2 DeRotPh125_Im-TX2 DeRotPh125_Re-TX2 * DeRotPh126_Im-TX2 DeRotPh126_Re-TX2 DeRotPh127_Im-TX2 DeRotPh127_Re-TX2 *

Parameters

in	*WR_7*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Beamforming Steering Vector), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For steering coeficients generation algorithm, see rSDK_Auxiliary_vector_generation.docx.

```
*                  INPUT Memory layout:
*                  SRAM, 32 bit complex(16b_Im + 16b_Re)
*                  63:48          47:32           31:16           15:0
*                  SV0_Im-CH0     SV0_Re-CH0      SV0_Im-CH1      SV0_Re-CH1
*                  SV0_Im-CH2     SV0_Re-CH2      SV0_Im-CH3      SV0_Re-CH3
*                  SV0_Im-CH4     SV0_Re-CH4      SV0_Im-CH5      SV0_Re-CH5
*                  SV0_Im-CH6     SV0_Re-CH6      SV0_Im-CH7      SV0_Re-CH7
*                  SV0_Im-CH8     SV0_Re-CH8      SV0_Im-CH9      SV0_Re-CH9
*                  SV0_Im-CH10    SV0_Re-CH10     SV0_Im-CH11     SV0_Re-CH11
*                  |              |               |               |
*                  |              |               |               |
*                  |              |               |               |
*                  SV63_Im-CH0    SV63_Re-CH0     SV63_Im-CH1     SV63_Re-CH1
*                  SV63_Im-CH2    SV63_Re-CH2     SV63_Im-CH3     SV63_Re-CH3
*                  SV63_Im-CH4    SV63_Re-CH4     SV63_Im-CH5     SV63_Re-CH5
*                  SV63_Im-CH6    SV63_Re-CH6     SV63_Im-CH7     SV63_Re-CH7
*                  SV63_Im-CH8    SV63_Re-CH8     SV63_Im-CH9     SV63_Re-CH9
*                  SV63_Im-CH10   SV63_Re-CH10    SV63_Im-CH11    SV63_Re-CH11
*
```

Returns

WR_0: No value returned.

Return values

No	value returned.

Postcondition

Twiddle factors for FFT256 placed in TRAM at TR_0_0_0.
Twiddle factors for FFT128 placed in TRAM at TR_0_32_0.
Window coeficients for Range FFT256 placed in TRAM at TR_0_48_0.
Window coeficients for Doppler FFT128 placed in TRAM at TR_0_80_0.
Twiddle factors for FFT32 placed in TRAM at TR_0_96_0.
Derotation Phase Vector, placed in TRAM at TR_0_100_0.
Beamforming Steering Vector placed in TRAM at TR_0_384_0.
Free TRAM space TR_0_132_0 to TR_0_384_0.

Note

All buffers MUST be 8 bytes aligned.

3.9.2.65 RsdkSptInit256smp128crp8ch6TxTdMimo()

```
void * RsdkSptInit256smp128crp8ch6TxTdMimo (
            void  )
```

3.9.2.66 RsdkSptInit256smp256crp4ch()

```
void * RsdkSptInit256smp256crp4ch (
            void  )
```

Initialization function for 256 samples and 256 chirps, 4 physical channels.

Copy twiddles, window, steering vector coefficients from SRAM to TRAM.

Parameters

in	*WR_1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT256 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For twiddle generation algorithm, see S32R41_RM 68.9.5.3 / SAF85xx_RM 68.9.5.3. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * | | | | * | | | | * | | | | * tw32_Im tw32_Re tw32_Im tw32_Re * tw32_Im tw32_Re tw32_Im tw32_Re * tw32_Im tw32_Re tw32_Im tw32_Re * tw32_Im tw32_Re tw32_Im tw32_Re *
in	*WR_2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Reordered Window coef for Range FFT 256), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For reorder algorithm, see S32R41_RM 68.9.9.7 / SAF85xx_RM 68.9.9.7. * INPUT Memory layout: * SRAM, 16bit Real Window Coef * 63:48 47:32 31:16 15:0 * win0 win64 win128 win192 * win16 win80 win144 win208 * | | | | * | | | | * win15 win79 win143 win207 * win31 win95 win159 win223 * win32 win96 win160 win224 * win48 win112 win176 win240 * | | | | * | | | | * win47 win111 win175 win239 * win63 win127 win191 win255 *

Parameters

in	*WR↩ _3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Reordered Window coef for Doppler FFT 256), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For reorder algorithm, S32R41_RM 68.9.9.7 / SAF85xx_RM 68.9.9.7. * INPUT Memory layout: * SRAM, 16bit Real Window Coef * 63:48 47:32 31:16 15:0 * win0 win64 win128 win192 * win16 win80 win144 win208 * | | | | * | | | | * win15 win79 win143 win207 * win31 win95 win159 win223 * win32 win96 win160 win224 * win48 win112 win176 win240 * | | | | * | | | | * win47 win111 win175 win239 * win63 win127 win191 win255 *
in	*WR↩ _4*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT16 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For twiddle generation algorithm, see S32R41_RM 68.9.5.3 / SAF85xx_RM 68.9.5.3. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Im tw1_Im tw1_Im * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re *
in	*WR↩ _5*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Beamforming Steering Vector), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For steering coeficients generation algorithm, see rSDK_Auxiliary_vector_generation.docx. * INPUT Memory layout: * SRAM, 32 bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * SV0_Im-CH0 SV0_Re-CH0 SV0_Im-CH1 SV0_Re-CH1 * SV0_Im-CH2 SV0_Re-CH2 SV0_Im-CH3 SV0_Re-CH3 * SV1_Im-CH0 SV1_Re-CH0 SV1_Im-CH1 SV1_Re-CH1 * SV1_Im-CH2 SV1_Re-CH2 SV1_Im-CH3 SV1_Re-CH3 * | | | | * | | | | * | | | | * SV63_Im-CH0 SV63_Re-CH0 SV63_Im-CH1 SV63_Re-CH1 * SV63_Im-CH2 SV63_Re-CH2 SV63_Im-CH3 SV63_Re-CH3 *

Returns

WR_0: No value returned.

Return values

No	value returned.

Postcondition

Twiddle factors for FFT256 placed in TRAM at TR_0_0_0.
Window coeficients for Range FFT256 placed in TRAM at TR_0_32_0.
Window coeficients for Doppler FFT256 placed in TRAM at TR_0_64_0.
Twiddle factors for FFT16 placed in TRAM at TR_0_96_0.
Beamforming Steering Vector placed in TRAM at TR_0_448_0.
Free TRAM space TR_0_98_0 to TR_0_448_0.

Note

All buffers MUST be 8 bytes aligned.

3.9.2.67 RsdkSptInit256smp256crp4ch2TxTdMimo()

```
void * RsdkSptInit256smp256crp4ch2TxTdMimo (
            void  )
```

Initialization function for 256 samples and 256 chirps, 4 physical channels/TX slot, 2TX TD-Mimo.

Copy twiddles, window, steering vector, derotation phase coefficients from SRAM to TRAM.

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT256 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For twiddle generation algorithm, see S32R41_RM 68.9.5.3 / SAF85xx_RM 68.9.5.3.

```
*                 INPUT Memory layout:
*                 SRAM, 32bit complex(16b_Im + 16b_Re)
*                 63:48    47:32    31:16    15:0
*                 tw1_Im   tw1_Re   tw1_Im   tw1_Re
*                 tw1_Im   tw1_Re   tw1_Im   tw1_Re
*                 tw1_Im   tw1_Re   tw1_Im   tw1_Re
*                 tw1_Im   tw1_Re   tw1_Im   tw1_Re
*                 tw2_Im   tw2_Re   tw2_Im   tw2_Re
*                 tw2_Im   tw2_Re   tw2_Im   tw2_Re
*                 tw2_Im   tw2_Re   tw2_Im   tw2_Re
*                 tw2_Im   tw2_Re   tw2_Im   tw2_Re
*                 |        |        |        |
*                 |        |        |        |
*                 |        |        |        |
*                 tw32_Im  tw32_Re  tw32_Im  tw32_Re
*                 tw32_Im  tw32_Re  tw32_Im  tw32_Re
*                 tw32_Im  tw32_Re  tw32_Im  tw32_Re
*                 tw32_Im  tw32_Re  tw32_Im  tw32_Re
*
```

Parameters

in	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Reordered Window coef for Range FFT 256), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For reorder algorithm, see S32R41_RM 68.9.9.7 / SAF85xx_RM 68.9.9.7. * INPUT Memory layout: * SRAM, 16bit Real Window Coef * 63:48 47:32 31:16 15:0 * win0 win64 win128 win192 * win16 win80 win144 win208 * | | | | * | | | | * win15 win79 win143 win207 * win31 win95 win159 win223 * win32 win96 win160 win224 * win48 win112 win176 win240 * | | | | * | | | | * win47 win111 win175 win239 * win63 win127 win191 win255 *
in	*WR↩ _3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Reordered Window coef for Doppler FFT 256), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For reorder algorithm, see S32R41_RM 68.9.9.7 / SAF85xx_RM 68.9.9.7. * INPUT Memory layout: * SRAM, 16bit Real Window Coef * 63:48 47:32 31:16 15:0 * win0 win64 win128 win192 * win16 win80 win144 win208 * | | | | * | | | | * win15 win79 win143 win207 * win31 win95 win159 win223 * win32 win96 win160 win224 * win48 win112 win176 win240 * | | | | * | | | | * win47 win111 win175 win239 * win63 win127 win191 win255 *
in	*WR↩ _4*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT16 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For twiddle generation algorithm, see S32R41_RM 68.9.5.3 / SAF85xx_RM 68.9.5.3. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Im tw1_Im tw1_Im * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re *

Parameters

in	*WR_5*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Derotation Phased Vector), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For derotation coeficients generation algorithm, see rSDK_Auxiliary_vector_generation.docx. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * DeRotPh0_Im DeRotPh0_Re DeRotPh1_Im DeRotPh1_Re * DeRotPh2_Im DeRotPh2_Re DeRotPh3_Im DeRotPh3_Re * | | | | * | | | | * | | | | * DeRotPh252_Im DeRotPh252_Re DeRotPh253_Im DeRotPh253_Re * DeRotPh254_Im DeRotPh254_Re DeRotPh255_Im DeRotPh255_Re *
in	*WR_6*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Beamforming Steering Vector), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For steering coeficients generation algorithm, see rSDK_Auxiliary_vector_generation.docx. * INPUT Memory layout: * SRAM, 32 bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * SV0_Im-CH0 SV0_Re-CH0 SV0_Im-CH1 SV0_Re-CH1 * SV0_Im-CH2 SV0_Re-CH2 SV0_Im-CH3 SV0_Re-CH3 * SV0_Im-CH4 SV0_Re-CH4 SV0_Im-CH5 SV0_Re-CH5 * SV0_Im-CH6 SV0_Re-CH6 SV0_Im-CH7 SV0_Re-CH7 * SV1_Im-CH0 SV1_Re-CH0 SV1_Im-CH1 SV1_Re-CH1 * SV1_Im-CH2 SV1_Re-CH2 SV1_Im-CH3 SV1_Re-CH3 * SV1_Im-CH4 SV1_Re-CH4 SV1_Im-CH5 SV1_Re-CH5 * SV1_Im-CH6 SV1_Re-CH6 SV1_Im-CH7 SV1_Re-CH7 * | | | | * | | | | * | | | | * SV63_Im-CH0 SV63_Re-CH0 SV63_Im-CH1 SV63_Re-CH1 * SV63_Im-CH2 SV63_Re-CH2 SV63_Im-CH3 SV63_Re-CH3 * SV63_Im-CH4 SV63_Re-CH4 SV63_Im-CH5 SV63_Re-CH5 * SV63_Im-CH6 SV63_Re-CH6 SV63_Im-CH7 SV63_Re-CH7 *

Returns

WR_0: No value returned.

Return values

No	value returned.

Postcondition

Twiddle factors for FFT256 placed in TRAM at TR_0_0_0.
Window coeficients for Range FFT256 placed in TRAM at TR_0_32_0.
Window coeficients for Doppler FFT256 placed in TRAM at TR_0_64_0.
Twiddle factors for FFT16 placed in TRAM at TR_0_96_0.
Derotation Phase Vector placed in TRAM at TR_0_98_0.

Beamforming Steering Vector placed in TRAM at TR_0_448_0.
Free TRAM space TR_0_130_0 to TR_0_448_0.

Note

All buffers MUST be 8 bytes aligned.

3.9.2.68 RsdkSptInit256smp256crp8ch()

```
void * RsdkSptInit256smp256crp8ch (
            void  )
```

3.9.2.69 RsdkSptInit256smp256crp8ch2TxTdMimo()

```
void * RsdkSptInit256smp256crp8ch2TxTdMimo (
            void  )
```

3.9.2.70 RsdkSptInit512smp128crp4ch()

```
void * RsdkSptInit512smp128crp4ch (
            void  )
```

Initialization function for 512 samples and 128 chirps, 4 physical channels.

Copy twiddles, window, steering vector coefficients from SRAM to TRAM.

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT512 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For twiddle generation algorithm, see S32R41_RM 68.9.5.3 / SAF85xx_RM 68.9.5.3. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * | | | | * | | | | * | | | | * tw64_Im tw64_Re tw64_Im tw64_Re * tw64_Im tw64_Re tw64_Im tw64_Re * tw64_Im tw64_Re tw64_Im tw64_Re * tw64_Im tw64_Re tw64_Im tw64_Re *
in	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT128 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For twiddle generation algorithm, see S32R41_RM 68.9.5.3 / SAF85xx_RM 68.9.5.3. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * | | | | * | | | | * | | | | * tw16_Im tw16_Re tw16_Im tw16_Re * tw16_Im tw16_Re tw16_Im tw16_Re * tw16_Im tw16_Re tw16_Im tw16_Re * tw16_Im tw16_Re tw16_Im tw16_Re *

Parameters

in	*WR↩ _3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Reordered Window coef for Range FFT512), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For reorder algorithm, see S32R41_RM 68.9.9.7 / SAF85xx_RM 68.9.9.7. * INPUT Memory layout: * SRAM, 16bit Real Window Coef * 63:48 47:32 31:16 15:0 * win0 win128 win256 win384 * win32 win160 win288 win416 * | | | | * | | | | * win31 win159 win287 win415 * win63 win191 win319 win447 * win64 win192 win320 win448 * win96 win224 win352 win480 * | | | | * | | | | * win95 win223 win351 win479 * win127 win255 win383 win511 *
in	*WR↩ _4*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Reordered Window coef for Doppler FFT128), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For reorder algorithm, see S32R41_RM 68.9.9.7 / SAF85xx_RM 68.9.9.7. * INPUT Memory layout: * SRAM, 16bit Real Window Coef * 63:48 47:32 31:16 15:0 * win0 win32 win64 win96 * win8 win40 win72 win104 * | | | | * | | | | * win7 win39 win71 win103 * win15 win47 win79 win111 * win16 win48 win80 win112 * win24 win56 win88 win120 * | | | | * | | | | * win23 win55 win87 win119 * win31 win63 win95 win127 *
in	*WR↩ _5*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT16 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For twiddle generation algorithm, see S32R41_RM 68.9.5.3 / SAF85xx_RM 68.9.5.3. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re *

Parameters

in	*WR_6*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Beamforming Steering Vector), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For steering coeficients generation algorithm, see rSDK_Auxiliary_vector_generation.docx. * INPUT Memory layout: * SRAM, 32 bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * SV0_Im-CH0 SV0_Re-CH0 SV0_Im-CH1 SV0_Re-CH1 * SV0_Im-CH2 SV0_Re-CH2 SV0_Im-CH3 SV0_Re-CH3 * SV1_Im-CH0 SV1_Re-CH0 SV1_Im-CH1 SV1_Re-CH1 * SV1_Im-CH2 SV1_Re-CH2 SV1_Im-CH3 SV1_Re-CH3 * | | | | * | | | | * | | | | * SV63_Im-CH0 SV63_Re-CH0 SV63_Im-CH1 SV63_Re-CH1 * SV63_Im-CH2 SV63_Re-CH2 SV63_Im-CH3 SV63_Re-CH3 *

Returns

WR_0: No value returned.

Return values

No	value returned.

Postcondition

Twiddle factors for FFT512 placed in TRAM at TR_0_0_0.
Twiddle factors for FFT128 placed in TRAM at TR_0_64_0.
Window coeficients for Range FFT512 placed in TRAM at TR_0_80_0.
Window coeficients for Doppler FFT128 placed in TRAM at TR_0_144_0.
Twiddle factors for FFT16 placed in TRAM at TR_0_160_0.
Beamforming Steering Vector placed in TRAM at TR_0_448_0.
Free TRAM space TR_0_162_0 to TR_0_448_0.

Note

All buffers MUST be 8 bytes aligned.

3.9.2.71 RsdkSptInit512smp128crp8ch()

```
void * RsdkSptInit512smp128crp8ch (
            void  )
```

3.9.2.72 RsdkSptMemRwErrInject()

```
void * RsdkSptMemRwErrInject (
            void  )
```

This kernel does a useless read-after-write on one OPRAM bank, to generate parity errors, then trigger a memory error interrupt on the SPT ECS interrupt line, when MEM_ERR_INJECT_CTRL register is set.

Parameters

in	*none*	

Returns

WR_0: Will return the 'RSDK' kernel watermarking pattern.

Postcondition

If MEM_ERR_INJECT_CTRL register is set, then parity bits for opram bank OR_0_0_0 will be corrupted. Otherwise OR_0_0_0 and OR_1_0_0 get cleared with no other side-effects

3.9.2.73 RsdkSptNcc1024smp128crp4chCp4d()

```
void * RsdkSptNcc1024smp128crp4chCp4d (
            void  )
```

Function for Non coherent combining, 128 chirps, 1024 sample/chirp, 4 channels(CH#0..CH#3)

Compute Non Coherent Combining, storing results in SRAM.

Parameters

in	*WR_1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Doppler FFT compressed results), relative to SRAM start address, RSDK_SPT_CUBE_BASE_ADDR.

```
*                  INPUT Memory layout:
*                  SRAM, 64bit compressed data
*                  63:0
*                  CH0_0-CH1_0-CH2_0-CH3_0_Crp0
*                  |
*                  CH0_0-CH1_0-CH2_0-CH3_0-Crp127           ---> RangeBin ##0
*                  ------------------------------------------------------------
*                  |
*                  |
*                  |
*                  ------------------------------------------------------------
*                  CH0_511-CH1_511-CH2_511-CH3_511-Crp0
*                  |
*                  CH0_511-CH1_511-CH2_511-CH3_511-Crp127   ---> RangeBin ##511
*                  ------------------------------------------------------------
*
```

Parameters

out	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the RDM OUTPUT buffer(NCC matrix), relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 16 bit real Log2 Mag2 format * 63:48 47:32 31:16 15:0 * R0_D0_MAG R0_D1_MAG R0_D2_MAG R0_D3_MAG * | | | | * R0_D124_MAG R0_D125_MAG R0_D126_MAG R0_D127_MAG * R1_D0_MAG R1_D1_MAG R1_D2_MAG R1_D3_MAG * | | | | * R1_D124_MAG R1_D125_MAG R1_D126_MAG R1_D127_MAG * | | | | * | | | | * | | | | * | | | | * | | | | * R511_D0_MAG R511_D1_MAG R511_D2_MAG R511_D3_MAG * | | | | * R511_D124_MAG R511_D125_MAG R511_D126_MAG R511_D127_MAG *
out	*WR↩ _3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the Histogram OUTPUT buffer, relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 16 bit values histogram counters * 0x0 bin1 bin0 * 0x4 bin3 bin2 * | | * bin61 bin60 * bin63 bin62 -> Range Bin 0 * ---------- ---------- * bin1 bin0 * bin3 bin2 * | | * bin61 bin60 * bin63 bin62 -> Range Bin 1 * ---------- ---------- * | | * | | * | | * ---------- ---------- * bin1 bin0 * bin3 bin2 * | | * bin61 bin60 * bin63 bin62 -> Range Bin 511 * ---------- ---------- *

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

The kernel rely on the assumption that TRAM can be used as temp buffer from TR_0_290_0 to TR_0_448_0.

Note

The current implementation is ONLY to be used in combination with 1024 Range FFT and 128 Doppler FFT.
The input buffer MUST NOT be reused for the histogram or RDM output.
All buffers MUST be 8 bytes aligned.

3.9.2.74 RsdkSptNcc1024smp128crp8chCp4d()

```
void * RsdkSptNcc1024smp128crp8chCp4d (
            void  )
```

3.9.2.75 RsdkSptNcc1024smp64crp4ch3TxDdMimoCp4d()

```
void * RsdkSptNcc1024smp64crp4ch3TxDdMimoCp4d (
            void  )
```

Function for Non coherent combining, 64 chirps/TX DD-MIMO slot(from 256chirps/frame), 1024 samples, 4 channels(CH#0..CH#3).

Compute Non Coherent Combining, storing results in SRAM.

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Doppler FFT CP4D compressed results), relative to SRAM start address, RSDK_SPT_CUBE_BASE_ADDR. * INPUT Memory layout: * SRAM, 64bit compressed data * 63:0 * CH0_0-CH1_0-CH2_0-CH3_0-Crp0 -> TX1 * CH4_0-CH5_0-CH6_0-CH7_0-Crp0 -> TX2 * CH8_0-CH9_0-CH10_0-CH11_0-Crp0 -> TX3 * | * CH0_0-CH1_0-CH2_0-CH3_0-Crp63 -> TX1 * CH4_0-CH5_0-CH6_0-CH7_0-Crp63 -> TX2 * CH8_0-CH9_0-CH10_0-CH11_0-Crp63 -> TX3 ---> RangeBin ##0 * ---------- * | * | * | * ---------- * CH0_511-CH1_511-CH2_511-CH3_511-Crp0 -> TX1 * CH4_511-CH5_511-CH6_511-CH7_511-Crp0 -> TX2 * CH8_511-CH9_511-CH10_511-CH11_511-Crp0 -> TX3 * | * CH0_511-CH1_511-CH2_511-CH3_511-Crp63 -> TX1 * CH4_511-CH5_511-CH6_511-CH7_511-Crp63 -> TX2 * CH8_511-CH9_511-CH10_511-CH11_511-Crp63 -> TX3 ---> RangeBin ##511 * ---------- *
out	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the RDM OUTPUT buffer(NCC matrix), relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 16 bit real Log2 Mag2 format * 63:48 47:32 31:16 15:0 * R0_D0_MAG R0_D1_MAG R0_D2_MAG R0_D3_MAG * | | | | * R0_D60_MAG R0_D61_MAG R0_D62_MAG R0_D63_MAG * R1_D0_MAG R1_D1_MAG R1_D2_MAG R1_D3_MAG * | | | | * R1_D60_MAG R1_D61_MAG R1_D62_MAG R1_D63_MAG * | | | | * | | | | * | | | | * R511_D0_MAG R511_D1_MAG R511_D2_MAG R511_D3_MAG * | | | | * R511_D60_MAG R511_D61_MAG R511_D62_MAG R511_D63_MAG *

Parameters

out	*WR↩ _3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the Histogram OUTPUT buffer, relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 16 bit values histogram counters * 0x0 bin1 bin0 * 0x4 bin3 bin2 * | | * bin61 bin60 * bin63 bin62 -> Range Bin 0 * ---------- * bin1 bin0 * bin3 bin2 * | | * bin61 bin60 * bin63 bin62 -> Range Bin 1 * ---------- * | | * | | * | | * ---------- * bin1 bin0 * bin3 bin2 * | | * bin61 bin60 * bin63 bin62 -> Range Bin 511 * ---------- *

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

The kernel rely on the assumption that TRAM can be used as temp buffer from TR_0_320_0 to TR_0_322_1.

Note

The current implementation is ONLY to be used in combination with 1024 Range FFT and 256 Doppler FFT.
The input buffer MUST NOT be reused for the histogram or RDM output.
All buffers MUST be 8 bytes aligned.

3.9.2.76 RsdkSptNcc1024smp64crp4ch4TxTdMimoCp4d()

```
void * RsdkSptNcc1024smp64crp4ch4TxTdMimoCp4d (
            void  )
```

Function for Non coherent combining, 64 chirps, 1024 sample/chirp, 4 physical channels/TX slot, 4TX TD-MIMO.

Compute Non Coherent Combining, storing results in SRAM.

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Doppler FFT compressed results), relative to SRAM start address, RSDK_SPT_CUBE_BASE_ADDR. * INPUT Memory layout: * SRAM, 64bit compressed data * 63:0 * CH0_0-CH1_0-CH2_0-CH3_0-TX0_Crp0 * CH0_0-CH1_0-CH2_0-CH3_0-TX1_Crp0 * CH0_0-CH1_0-CH2_0-CH3_0-TX2_Crp0 * CH0_0-CH1_0-CH2_0-CH3_0-TX3_Crp0 * | * CH0_0-CH1_0-CH2_0-CH3_0-TX0_Crp63 * CH0_0-CH1_0-CH2_0-CH3_0-TX1_Crp63 * CH0_0-CH1_0-CH2_0-CH3_0-TX2_Crp63 * CH0_0-CH1_0-CH2_0-CH3_0-TX3_Crp63 ---> RangeBin ##0 * ---------- * | * | * | * ---------- * CH0_511-CH1_511-CH2_511-CH3_511-TX0_Crp0 * CH0_511-CH1_511-CH2_511-CH3_511-TX1_Crp0 * CH0_511-CH1_511-CH2_511-CH3_511-TX2_Crp0 * CH0_511-CH1_511-CH2_511-CH3_511-TX3_Crp0 * | * CH0_511-CH1_511-CH2_511-CH3_511-TX0_Crp63 * CH0_511-CH1_511-CH2_511-CH3_511-TX1_Crp63 * CH0_511-CH1_511-CH2_511-CH3_511-TX2_Crp63 * CH0_511-CH1_511-CH2_511-CH3_511-TX3_Crp63 ---> RangeBin ##511 * ---------- *
out	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the RDM OUTPUT buffer(NCC matrix), relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 16 bit real Log2 Mag2 format * 63:48 47:32 31:16 15:0 * R0_D0_MAG R0_D1_MAG R0_D2_MAG R0_D3_MAG * | | | | * R0_D60_MAG R0_D61_MAG R0_D62_MAG R0_D63_MAG * R1_D0_MAG R1_D1_MAG R1_D2_MAG R1_D3_MAG * | | | | * R1_D60_MAG R1_D61_MAG R1_D62_MAG R1_D63_MAG * | | | | * | | | | * | | | | * | | | | * | | | | * R511_D0_MAG R511_D1_MAG R511_D2_MAG R511_D3_MAG * | | | | * R511_D60_MAG R511_D61_MAG R511_D62_MAG R511_D63_MAG *

Parameters

out	*WR↩ _3*	24bit(RE): 0
		24bit(IM): Offset(restricted to 24bit value!) of the Histogram OUTPUT buffer, relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR.
		
		`*  OUTPUT Memory layout:`
		`*  SRAM, 16 bit values histogram counters`
		`*  0x0  bin1  bin0`
		`*  0x4  bin3  bin2`
		`*  |  |`
		`*  bin61 bin60`
		`*  bin63 bin62 -> Range Bin 0`
		`*  --------------------------`
		`*  bin1  bin0`
		`*  bin3  bin2`
		`*  |  |`
		`*  bin61 bin60`
		`*  bin63 bin62 -> Range Bin 1`
		`*  --------------------------`
		`*  |  |`
		`*  |  |`
		`*  |  |`
		`*  --------------------------`
		`*  bin1  bin0`
		`*  bin3  bin2`
		`*  |  |`
		`*  bin61 bin60`
		`*  bin63 bin62 -> Range Bin 511`
		`*  --------------------------`
		`*`

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

The kernel rely on the assumption that TRAM can be used as temp buffer from TR_0_300_0 to TR_0_384_0.

Note

The current implementation is ONLY to be used in combination with 1024 Range FFT and 64 Doppler FFT, 4TX TD-MIMO.
The input buffer MUST NOT be reused for the histogram or RDM output.
All buffers MUST be 8 bytes aligned.

3.9.2.77 RsdkSptNcc1024smp64crp4chCp4d()

```
void * RsdkSptNcc1024smp64crp4chCp4d (
            void  )
```

Function for Non coherent combining, 64 chirps, 1024 sample/chirp, 4 channels(CH#0..CH#3)

Compute Non Coherent Combining, storing results in SRAM.

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Doppler FFT compressed results), relative to SRAM start address, RSDK_SPT_CUBE_BASE_ADDR. * INPUT Memory layout: * SRAM, 64bit compressed data * 63:0 * CH0_0-CH1_0-CH2_0-CH3_0_Crp0 * | * CH0_0-CH1_0-CH2_0-CH3_0-Crp63 ---> RangeBin ##0 * ---------- * | * | * | * ---------- * CH0_511-CH1_511-CH2_511-CH3_511-Crp0 * | * CH0_511-CH1_511-CH2_511-CH3_511-Crp63 ---> RangeBin ##511 * ---------- *
out	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the RDM OUTPUT buffer(NCC matrix), relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 16 bit real Log2 Mag2 format * 63:48 47:32 31:16 15:0 * R0_D0_MAG R0_D1_MAG R0_D2_MAG R0_D3_MAG * | | | | * R0_D60_MAG R0_D61_MAG R0_D62_MAG R0_D63_MAG * R1_D0_MAG R1_D1_MAG R1_D2_MAG R1_D3_MAG * | | | | * R1_D60_MAG R1_D61_MAG R1_D62_MAG R1_63_MAG * | | | | * | | | | * | | | | * | | | | * | | | | * R511_D0_MAG R511_D1_MAG R511_D2_MAG R511_D3_MAG * | | | | * R511_D60_MAG R511_D61_MAG R511_D62_MAG R511_D63_MAG *

Parameters

out	*WR_3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the Histogram OUTPUT buffer, relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 16 bit values histogram counters * 0x0 bin1 bin0 * 0x4 bin3 bin2 * | | * bin61 bin60 * bin63 bin62 -> Range Bin 0 * ---------- ---------- * bin1 bin0 * bin3 bin2 * | | * bin61 bin60 * bin63 bin62 -> Range Bin 1 * ---------- ---------- * | | * | | * | | * ---------- ---------- * bin1 bin0 * bin3 bin2 * | | * bin61 bin60 * bin63 bin62 -> Range Bin 511 * ---------- ---------- *

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

The kernel rely on the assumption that TRAM can be used as temp buffer from TR_0_274_0 to TR_0_448_0.

Note

The current implementation is ONLY to be used in combination with 1024 Range FFT and 64 Doppler FFT.
The input buffer MUST NOT be reused for the histogram or RDM output.
All buffers MUST be 8 bytes aligned.

3.9.2.78 RsdkSptNcc2048smp256crp4chCp4d()

```
void * RsdkSptNcc2048smp256crp4chCp4d (
            void  )
```

Function for Non coherent combining, 256 chirps, 2048 sample/chirp, 4 channels(CH#0..CH#3)

Compute Non Coherent Combining, storing results in SRAM.

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Doppler FFT compressed results), relative to SRAM start address, RSDK_SPT_CUBE_BASE_ADDR. * INPUT Memory layout: * SRAM, 64bit compressed data * 63:0 * CH0_0-CH1_0-CH2_0-CH3_0_Crp0 * | * CH0_0-CH1_0-CH2_0-CH3_0-Crp255 ---> RangeBin ##0 * ---------- * | * | * | * ---------- * CH0_1023-CH1_1023-CH2_1023-CH3_1023-Crp0 * | * CH0_1023-CH1_1023-CH2_1023-CH3_1023-Crp255 ---> RangeBin ##1023 * ---------- *
out	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the RDM OUTPUT buffer(NCC matrix), relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 16 bit real Log2 Mag2 format * 63:48 47:32 31:16 15:0 * R0_D0_MAG R0_D1_MAG R0_D2_MAG R0_D3_MAG * | | | | * R0_D252_MAG R0_D253_MAG R0_D254_MAG R0_D255_MAG * R1_D0_MAG R1_D1_MAG R1_D2_MAG R1_D3_MAG * | | | | * R1_D252_MAG R1_D253_MAG R1_D254_MAG R1_D255_MAG * | | | | * | | | | * | | | | * | | | | * | | | | * R1023_D0_MAG R1023_D1_MAG R1023_D2_MAG R1023_D3_MAG * | | | | * R1023_D252_MAG R1023_D253_MAG R1023_D254_MAG R1023_D255_MAG *

Parameters

out	*WR↩ _3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the Histogram OUTPUT buffer, relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 16 bit values histogram counters * 0x0 bin1 bin0 * 0x4 bin3 bin2 * | | * bin61 bin60 * bin63 bin62 -> Range Bin 0 * ---------- ---------- * bin1 bin0 * bin3 bin2 * | | * bin61 bin60 * bin63 bin62 -> Range Bin 1 * ---------- ---------- * | | * | | * | | * ---------- ---------- * bin1 bin0 * bin3 bin2 * | | * bin61 bin60 * bin63 bin62 -> Range Bin 1023 * ---------- ---------- *

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

The kernel rely on the assumption that TRAM can be used as temp buffer from TR_0_322_0 to TR_0_448_0.

Note

The current implementation is ONLY to be used in combination with 2048 Range FFT and 256 Doppler FFT.
The input buffer MUST NOT be reused for the histogram or RDM output.
All buffers MUST be 8 bytes aligned.

3.9.2.79 RsdkSptNcc256smp128crp4ch3TxTdMimo()

```
void * RsdkSptNcc256smp128crp4ch3TxTdMimo (
            void  )
```

Function for Non coherent combining, 128 chirps, 256 sample/chirp, 4 physical channels/TX slot, 3TX TD-MIMO.

Compute Non Coherent Combining, storing results in SRAM.

Parameters

in	*WR_1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Doppler FFT results), relative to SRAM start address, RSDK_SPT_CUBE_BASE_ADDR.

```
*                 INPUT Memory layout:
*                 SRAM, 32bit complex(16b_Im + 16b_Re)
*                 63:48                 47:32                 31:16                 15:0
*                 CH0_0_Im-Crp0         CH0_0_Re-Crp0         CH1_0_Im-Crp0         CH1_0_Re-Crp0
*                 CH2_0_Im-Crp0         CH2_0_Re-Crp0         CH3_0_Im-Crp0         CH3_0_Re-Crp0
*                 ---------------------------------------------------------------------------
*                 CH0_0_Im-Crp0         CH0_0_Re-Crp0         CH1_0_Im-Crp0         CH1_0_Re-Crp0
*                 CH2_0_Im-Crp0         CH2_0_Re-Crp0         CH3_0_Im-Crp0         CH3_0_Re-Crp0
*                 ---------------------------------------------------------------------------
*                 CH0_0_Im-Crp0         CH0_0_Re-Crp0         CH1_0_Im-Crp0         CH1_0_Re-Crp0
*                 CH2_0_Im-Crp0         CH2_0_Re-Crp0         CH3_0_Im-Crp0         CH3_0_Re-Crp0
*                 ---------------------------------------------------------------------------
*                 |                     |                     |                     |
*                 |                     |                     |                     |
*                 |                     |                     |                     |
*                 CH0_0_Im-Crp127       CH0_0_Re-Crp127       CH1_0_Im-Crp127       CH1_0_Re-Crp127
*                 CH2_0_Im-Crp127       CH2_0_Re-Crp127       CH3_0_Im-Crp127       CH3_0_Re-Crp127
*                 ---------------------------------------------------------------------------
*                 CH0_0_Im-Crp127       CH0_0_Re-Crp127       CH1_0_Im-Crp127       CH1_0_Re-Crp127
*                 CH2_0_Im-Crp127       CH2_0_Re-Crp127       CH3_0_Im-Crp127       CH3_0_Re-Crp127
*                 ---------------------------------------------------------------------------
*                 CH0_0_Im-Crp127       CH0_0_Re-Crp127       CH1_0_Im-Crp127       CH1_0_Re-Crp127
*                 CH2_0_Im-Crp127       CH2_0_Re-Crp127       CH3_0_Im-Crp127       CH3_0_Re-Crp127
*                 ---------------------------------------------------------------------------
*                 |                     |                     |                     |
*                 |                     |                     |                     |
*                 |                     |                     |                     |
*                 |                     |                     |                     |
*                 |                     |                     |                     |
*                 |                     |                     |                     |
*                 CH0_127_Im-Crp0       CH0_127_Re-Crp0       CH1_127_Im-Crp0       CH1_127_Re-Crp0
*                 CH2_127_Im-Crp0       CH2_127_Re-Crp0       CH3_127_Im-Crp0       CH3_127_Re-Crp0
*                 ---------------------------------------------------------------------------
*                 CH0_127_Im-Crp0       CH0_127_Re-Crp0       CH1_127_Im-Crp0       CH1_127_Re-Crp0
*                 CH2_127_Im-Crp0       CH2_127_Re-Crp0       CH3_127_Im-Crp0       CH3_127_Re-Crp0
*                 ---------------------------------------------------------------------------
*                 CH0_127_Im-Crp0       CH0_127_Re-Crp0       CH1_127_Im-Crp0       CH1_127_Re-Crp0
*                 CH2_127_Im-Crp0       CH2_127_Re-Crp0       CH3_127_Im-Crp0       CH3_127_Re-Crp0
*                 ---------------------------------------------------------------------------
*                 |                     |                     |                     |
*                 |                     |                     |                     |
*                 |                     |                     |                     |
*                 CH0_127_Im-Crp127     CH0_127_Re-Crp127     CH1_127_Im-Crp127     CH1_127_Re-Crp127
*                 CH2_127_Im-Crp127     CH2_127_Re-Crp127     CH3_127_Im-Crp127     CH3_127_Re-Crp127
*                 ---------------------------------------------------------------------------
*                 CH0_127_Im-Crp127     CH0_127_Re-Crp127     CH1_127_Im-Crp127     CH1_127_Re-Crp127
*                 CH2_127_Im-Crp127     CH2_127_Re-Crp127     CH3_127_Im-Crp127     CH3_127_Re-Crp127
*                 ---------------------------------------------------------------------------
*                 CH0_127_Im-Crp127     CH0_127_Re-Crp127     CH1_127_Im-Crp127     CH1_127_Re-Crp127
*                 CH2_127_Im-Crp127     CH2_127_Re-Crp127     CH3_127_Im-Crp127     CH3_127_Re-Crp127
*                 ---------------------------------------------------------------------------
*
```

Parameters

out	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the RDM OUTPUT buffer(NCC matrix), relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 16 bit real Log2 Mag2 format * 63:48 47:32 31:16 15:0 * R0_D0_MAG R0_D1_MAG R0_D2_MAG R0_D3_MAG * | | | | * R0_D124_MAG R0_D125_MAG R0_D126_MAG R0_D127_MAG * R1_D0_MAG R1_D1_MAG R1_D2_MAG R1_D3_MAG * | | | | * R1_D124_MAG R1_D125_MAG R1_D126_MAG R1_D127_MAG * | | | | * | | | | * | | | | * | | | | * | | | | * R127_D0_MAG R127_D1_MAG R127_D2_MAG R127_D3_MAG * | | | | * R127_D124_MAG R127_D125_MAG R127_D126_MAG R127_D127_MAG *
out	*WR↩ _3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the Histogram OUTPUT buffer, relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 16 bit values histogram counters * 0x0 bin1 bin0 * 0x4 bin3 bin2 * | | * bin61 bin60 * bin63 bin62 -> Range Bin 0 * ---------- * bin1 bin0 * bin3 bin2 * | | * bin61 bin60 * bin63 bin62 -> Range Bin 1 * ---------- * | | * | | * | | * ---------- * bin1 bin0 * bin3 bin2 * | | * bin61 bin60 * bin63 bin62 -> Range Bin 127 * ---------- *

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

The kernel rely on the assumption that TRAM can be used as temp buffer from TR_0_132_0 to TR_0_384_0.

Note

The current implementation is ONLY to be used in combination with 256 Range FFT and 128 Doppler FFT, 3TX TD-MIMO.
The input buffer MUST NOT be reused for the histogram or RDM output.
All buffers MUST be 8 bytes aligned.

3.9.2.80 RsdkSptNcc256smp128crp8ch6TxTdMimoCp4d()

```
void * RsdkSptNcc256smp128crp8ch6TxTdMimoCp4d (
            void  )
```

3.9.2.81 RsdkSptNcc256smp256crp4ch()

```
void * RsdkSptNcc256smp256crp4ch (
            void  )
```

Function for Non coherent combining, 256 chirps, 256 sample/chirp, 4 channels(CH#0..CH#3)

Compute Non Coherent Combining, storing results in SRAM.

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Doppler FFT results), relative to SRAM start address, RSDK_SPT_CUBE_BASE_ADDR.

```
*                 INPUT Memory layout:
*                 SRAM, 32bit complex(16b_Im + 16b_Re)
*                 63:48               47:32               31:16               15:0
*                 CH0_0-Im-Crp0       CH0_0-Re-Crp0       CH1_0-Im-Crp0       CH1_0-Re-Crp0
*                 CH2_0-Im-Crp0       CH2_0-Re-Crp0       CH3_0-Im-Crp0       CH3_0-Re-Crp0
*                 |                   |                   |                   |
*                 CH0_0-Im-Crp255     CH0_0-Re-Crp255     CH1_0-Im-Crp255     CH1_0-Re-Crp255
*                 CH2_0-Im-Crp255     CH2_0-Re-Crp255     CH3_0-Im-Crp255     CH3_0-Re-Crp255
*                 ------------------------------------------------------------------------------
*                 |                   |                   |                   |
*                 |                   |                   |                   |
*                 |                   |                   |                   |
*                 ------------------------------------------------------------------------------
*                 CH0_127-Im-Crp0     CH0_127-Re-Crp0     CH1_127-Im-Crp0     CH1_127-Re-Crp0
*                 CH2_127-Im-Crp0     CH2_127-Re-Crp0     CH3_127_Im-Crp0     CH3_127-Re-Crp0
*                 |                   |                   |                   |
*                 CH0_127-Im-Crp255   CH0_127-Re-Crp255   CH1_127-Im-Crp255   CH1_127-Re-Crp2
*                 CH2_127-Im-Crp255   CH2_127-Re-Crp255   CH3_127_Im-Crp255   CH3_127-Re-Crp2
*                 ------------------------------------------------------------------------------
*
```

Parameters

out	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the RDM OUTPUT buffer(NCC matrix), relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 16 bit real Log2 Mag2 format * 63:48 47:32 31:16 15:0 * R0_D0_MAG R0_D1_MAG R0_D2_MAG R0_D3_MAG * | | | | * R0_D252_MAG R0_D253_MAG R0_D254_MAG R0_D255_MAG * R1_D0_MAG R1_D1_MAG R1_D2_MAG R1_D3_MAG * | | | | * R1_D252_MAG R1_D253_MAG R1_D254_MAG R1_D255_MAG * | | | | * | | | | * | | | | * | | | | * | | | | * R127_D0_MAG R127_D1_MAG R127_D2_MAG R127_D3_MAG * | | | | * R127_D252_MAG R127_D253_MAG R127_D254_MAG R127_D255_MAG *
out	*WR↩ _3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the Histogram OUTPUT buffer, relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 16 bit values histogram counters * 0x0 bin1 bin0 * 0x4 bin3 bin2 * | | * bin61 bin60 * bin63 bin62 -> Range Bin 0 * ---------- ---------- * bin1 bin0 * bin3 bin2 * | | * bin61 bin60 * bin63 bin62 -> Range Bin 1 * ---------- ---------- * | | * | | * | | * ---------- ---------- * bin1 bin0 * bin3 bin2 * | | * bin61 bin60 * bin63 bin62 -> Range Bin 127 * ---------- ---------- *

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

The kernel rely on the assumption that TRAM can be used as temp buffer from TR_0_98_0 to TR_0_448_0.

Note

The current implementation is ONLY to be used in combination with 256 Range FFT and 256 Doppler FFT.
The input buffer MUST NOT be reused for the histogram or RDM output.
All buffers MUST be 8 bytes aligned.

3.9.2.82 RsdkSptNcc256smp256crp4ch2TxTdMimoCp4d()

```
void * RsdkSptNcc256smp256crp4ch2TxTdMimoCp4d (
            void  )
```

Function for Non coherent combining, 256 chirps, 256 sample/chirp, 4 physical channels/TX slot, 2TX TD-MIMO.

Compute Non Coherent Combining, storing results in SRAM.

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Doppler FFT compressed results), relative to SRAM start address, RSDK_SPT_CUBE_BASE_ADDR.

```
*                  INPUT Memory layout:
*                  SRAM, 64bit compressed data
*                  63:0
*                  CH0_0-CH1_0-CH2_0-CH3_0-TX0_Crp0
*                  CH0_0-CH1_0-CH2_0-CH3_0-TX1_Crp0
*                  |
*                  CH0_0-CH1_0-CH2_0-CH3_0-TX0_Crp255
*                  CH0_0-CH1_0-CH2_0-CH3_0-TX1_Crp255           ---> RangeBin ##0
*                  ---------------------------------------------------------------
*                  |
*                  |
*                  |
*                  ---------------------------------------------------------------
*                  CH0_127-CH1_127-CH2_127-CH3_127-TX0_Crp0
*                  CH0_127-CH1_127-CH2_127-CH3_127-TX1_Crp0
*                  |
*                  CH0_127-CH1_127-CH2_127-CH3_127-TX0_Crp255
*                  CH0_127-CH1_127-CH2_127-CH3_127-TX1_Crp255   ---> RangeBin ##127
*                  ---------------------------------------------------------------
*
```

Parameters

out	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the RDM OUTPUT buffer(NCC matrix), relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 16 bit real Log2 Mag2 format * 63:48 47:32 31:16 15:0 * R0_D0_MAG R0_D1_MAG R0_D2_MAG R0_D3_MAG * | | | | * R0_D252_MAG R0_D253_MAG R0_D254_MAG R0_D255_MAG * R1_D0_MAG R1_D1_MAG R1_D2_MAG R1_D3_MAG * | | | | * R1_D252_MAG R1_D253_MAG R1_D254_MAG R1_D255_MAG * | | | | * | | | | * | | | | * | | | | * | | | | * R127_D0_MAG R127_D1_MAG R127_D2_MAG R127_D3_MAG * | | | | * R127_D252_MAG R127_D253_MAG R127_D254_MAG R127_D255_MAG *
out	*WR↩ _3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the Histogram OUTPUT buffer, relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 16 bit values histogram counters * 0x0 bin1 bin0 * 0x4 bin3 bin2 * | | * bin61 bin60 * bin63 bin62 -> Range Bin 0 * ---------- ---------- * bin1 bin0 * bin3 bin2 * | | * bin61 bin60 * bin63 bin62 -> Range Bin 1 * ---------- ---------- * | | * | | * | | * ---------- ---------- * bin1 bin0 * bin3 bin2 * | | * bin61 bin60 * bin63 bin62 -> Range Bin 127 * ---------- ---------- *

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

The kernel rely on the assumption that TRAM can be used as temp buffer from TR_0_130_0 to TR_0_448_0.

Note

The current implementation is ONLY to be used in combination with 256 Range FFT and 256 Doppler FFT, 2TX TD-MIMO.
The input buffer MUST NOT be reused for the histogram or RDM output.
All buffers MUST be 8 bytes aligned.

3.9.2.83 RsdkSptNcc256smp256crp8ch()

```
void * RsdkSptNcc256smp256crp8ch (
            void  )
```

3.9.2.84 RsdkSptNcc256smp256crp8ch2TxTdMimoCp4d()

```
void * RsdkSptNcc256smp256crp8ch2TxTdMimoCp4d (
            void  )
```

3.9.2.85 RsdkSptNcc512smp128crp4ch()

```
void * RsdkSptNcc512smp128crp4ch (
            void  )
```

Function for Non coherent combining, 128 chirps, 512 sample/chirp, 4 channels(CH#0..CH#3).

Compute Non Coherent Combining, storing results in SRAM.

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Doppler FFT results), relative to SRAM start address, RSDK_SPT_CUBE_BASE_ADDR. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * CH0_0-Im-Crp0 CH0_0-Re-Crp0 CH1_0-Im-Crp0 CH1_0-Re-Crp0 * CH2_0-Im-Crp0 CH2_0-Re-Crp0 CH3_0-Im-Crp0 CH3_0-Re-Crp0 * | | | | * CH0_0-Im-Crp127 CH0_0-Re-Crp127 CH1_0-Im-Crp127 CH1_0-Re-Crp127 * CH2_0-Im-Crp127 CH2_0-Re-Crp127 CH3_0-Im-Crp127 CH3_0-Re-Crp127 * ---------- * | | | | * | | | | * | | | | * ---------- * CH0_255-Im-Crp0 CH0_255-Re-Crp0 CH1_255-Im-Crp0 CH1_255-Re-Crp0 * CH2_255-Im-Crp0 CH2_255-Re-Crp0 CH3_255_Im-Crp0 CH3_255-Re-Crp0 * | | | | * CH0_255-Im-Crp127 CH0_255-Re-Crp127 CH1_255-Im-Crp127 CH1_255-Re-Crp1 * CH2_255-Im-Crp127 CH2_255-Re-Crp127 CH3_255_Im-Crp127 CH3_255-Re-Crp1 * ---------- *
out	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the RDM OUTPUT buffer(NCC matrix), relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 16 bit real Log2 Mag2 format * 63:48 47:32 31:16 15:0 * R0_D0_MAG R0_D1_MAG R0_D2_MAG R0_D3_MAG * | | | | * R0_D124_MAG R0_D125_MAG R0_D126_MAG R0_D127_MAG * R1_D0_MAG R1_D1_MAG R1_D2_MAG R1_D3_MAG * | | | | * R1_D124_MAG R1_D125_MAG R1_D126_MAG R1_D127_MAG * | | | | * | | | | * | | | | * | | | | * | | | | * R255_D0_MAG R255_D1_MAG R255_D2_MAG R255_D3_MAG * | | | | * R255_D124_MAG R255_D125_MAG R255_D126_MAG R255_D127_MAG *

Parameters

out	*WR↩ _3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the Histogram OUTPUT buffer, relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 16 bit values histogram counters * 0x0 bin1 bin0 * 0x4 bin3 bin2 * | | * bin61 bin60 * bin63 bin62 -> Range Bin 0 * ---------- ---------- * bin1 bin0 * bin3 bin2 * | | * bin61 bin60 * bin63 bin62 -> Range Bin 1 * ---------- ---------- * | | * | | * | | * ---------- ---------- * bin1 bin0 * bin3 bin2 * | | * bin61 bin60 * bin63 bin62 -> Range Bin 255 * ---------- ---------- *

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

The kernel rely on the assumption that TRAM can be used as temp buffer from TR_0_162_0 to TR_0_448_0.

Note

The current implementation is ONLY to be used in combination with 512 Range FFT and 128 Doppler FFT.
The input buffer MUST NOT be reused for the histogram or RDM output.
All buffers MUST be 8 bytes aligned.

3.9.2.86 RsdkSptNcc512smp128crp8ch()

```
void * RsdkSptNcc512smp128crp8ch (
            void  )
```

3.9.2.87 RsdkSptOsCfar()

```
void * RsdkSptOsCfar (
            void  )
```

Function for Sliding OS-CFAR, any number of Range Bins and maximum 4096 Doppler Bins.

Compute Sliding OS-CFAR, storing results in SRAM/DDR.

Parameters

in	*WR↩_1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Range-Doppler Matrix) relative to SRAM/DDR start address, RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM/DDR, 16 bit real Log2 Mag2 format * 63:48 47:32 31:16 15:0 * R0_D0_MAG R0_D1_MAG R0_D2_MAG R0_D3_MAG * | | | | * R0_Dm-4_MAG R0_Dm-3_MAG R0_Dm-2_MAG R0_m-1_MAG * R1_D0_MAG R1_D1_MAG R1_D2_MAG R1_D3_MAG * | | | | * R1_Dm-4_MAG R1_Dm-3_MAG R1_Dm-2_MAG R1_m-1_MAG * | | | | * | | | | * | | | | * | | | | * | | | | * Rn_D0_MAG Rn_D1_MAG Rn_D2_MAG Rn_D3_MAG * | | | | * Rn-1_Dm-4_MAG Rn-1_Dm-3_MAG Rn-1_Dm-2_MAG Rn-1_m-1_MAG \m * OBS: n is the number of range bins and m is the number of doppler bins
out	*WR↩_2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(bitfiels with peaks postions), relative to SRAM/DDR start address, RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout (BIG ENDIAN representation): * SRAM/DDR, bitfield, P<n> = bit 1(peak), bit 0(non peak) * 0x0 P31P30......P17P16 P15P14........P1P0 * 0x4 P63P62......P49P48 P47P46......P33P32 * 0x8 P95P94......P81P80 P79P78......P65P64 * ..... * Pm-1Pm-2.....Pm-32 -> Range_bin0 * ----- * P31P30......P17P16 P15P14........P1P0 * P63P62......P49P48 P47P46......P33P32 * P95P94......P81P80 P79P78......P65P64 * ..... * Pm-1Pm-2.....Pm-32 -> Range_bin_1 * ----- * | | * | | * | | * ----- * P31P30......P17P16 P15P14........P1P0 * P63P62......P49P48 P47P46......P33P32 * P95P94......P81P80 P79P78......P65P64 * ..... * Pm-1Pm-2.....Pm-32 -> Range_bin_n-1 * ----- *
in	*WR↩_3*	24bit(RE): Threshold Scaling Factor (mag2log2 format). 24bit(IM): 0.

Parameters

in	*WR_4*	24bit(RE): Number of loops. 24bit(IM): 0. ∗Computed using the RSDK_SPT_OSCFAR_NUM_OF_LOOPS(rangeBins, dopplerBins) macro in rsdk_spt_kernels_s32r45.h
in	*WR_5*	24bit(RE): Pointer to rsdkBbe32OsCfarParams_t structure (RESERVED FOR DSP). 24bit(IM): 0. ∗Use the rsdkDspCmd_t struct.
in	*WR_6*	24bit(RE): Pointer to function ID (RESERVED FOR DSP). 24bit(IM): 0. ∗The function ID is returned by the macro RSDK_DSP_GET_FUNC_ID(RsdkBbe32OsCfar) ∗Use the rsdkDspCmd_t struct.
in	*WR_7*	24bit(RE): CRC (RESERVED FOR DSP). 24bit(IM): 0. ∗Computed with the RSDK_SPT_CMD_GEN_DSP_CMD_CRC command after populating the rsdkDspCmd_t struct with the pointer and id.

Returns

WR_0: No value returned.

Return values

No	value returned.

Note

Example code for computing CRC:

```
rsdkStatus_t sptStatus = RSDK_SUCCESS; \n
rsdkDspCmd_t dspCmd; \n
rsdkSptDriverCommand_t sptCmd; \n

AppMemAllocBuffer(heapMem, &osCfarParamsBufH, sizeof(rsdkBbe32OsCfarParams_t), CACHE_LINE_SIZE,
RSDK_OALMEM_CHUNK_ID_SRAM); \n

dspCmd.id = (uint16_t)RSDK_DSP_GET_FUNC_ID(RsdkBbe32OsCfar); \n
dspCmd.arg = (uint32_t)(uintptr_t)osCfarParamsBufH.phyAddr; \n
sptCmd.cmdId = RSDK_SPT_CMD_GEN_DSP_CMD_CRC; \n
sptCmd.cmdParam = (rsdkSptDriverCommandParam_t)&dspCmd; \n
sptStatus = RsdkSptCommand(&sptCmd, &sptCmdResult); \n \n
```

THIS KERNEL IS ONLY USED IN CONJUNCTION WITH THE DSP FUNCTION "RsdkBbe32OsCfar"
The current implementation works for any number of Range Bins >= 16 and Doppler Bins >= 16 and <= 4096.
The product of Range Bins nad Doppler Bins SHOULD be a multiple of 8192.
The user MUST PREVENT saturation when using the Threshold Scaling Factor in WR_3
The memory map is fixed. Input samples: OR_0_0_0, Input thresholds for BBE32: OR_2_0_0, Output: OR_0_0_0.
All buffers MUST be 16 bytes aligned.

3.9.2.88 RsdkSptPeakSearch1024smp128crp()

```
void * RsdkSptPeakSearch1024smp128crp (
            void  )
```

Function for Peak Search, 128 chirps (doppler bins), 1024 sample/chirp (512 range bins).

Compute Peaks, storing results in SRAM as packed bitfields, to be used togheter with Radar Data Cube by more advanced algorithms.

Parameters

<table>
<tr><td>in</td><td>WR↩
_1</td><td>24bit(RE): 0
24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Range-Doppler Matrix), relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR.

* INPUT Memory layout:
* SRAM, 16 bit real Log2 Mag2 format
* 63:48 47:32 31:16 15:0
* R0_D0_MAG R0_D1_MAG R0_D2_MAG R0_D3_MAG
* | | | |
* R0_D124_MAG R0_D125_MAG R0_D126_MAG R0_D127_MAG
* R1_D0_MAG R1_D1_MAG R1_D2_MAG R1_D3_MAG
* | | | |
* R1_D124_MAG R1_D125_MAG R1_D126_MAG R1_D127_MAG
* | | | |
* | | | |
* | | | |
* | | | |
* | | | |
* R511_D0_MAG R511_D1_MAG R511_D2_MAG R511_D3_MAG
* | | | |
* R511_D124_MAG R511_D125_MAG R511_D126_MAG R511_D127_MAG
*</td></tr>
<tr><td>in</td><td>WR↩
_2</td><td>24bit(RE): 0
24bit(IM): Offset(restricted to 24bit value!) of the Threshold buffer, log2 format, relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR.

* Threshold Memory layout:
* SRAM, 16bit log2 format
* 63:48 47:32 31:16 15:0
* Thr_RangeBin0 Thr_RangeBin1 Thr_RangeBin2 Thr_RangeBin3
* Thr_RangeBin4 Thr_RangeBin5 Thr_RangeBin6 Thr_RangeBin7
* | | | |
* | | | |
* | | | |
* Thr_RangeBin508 Thr_RangeBin509 Thr_RangeBin510 Thr_RangeBin511
*</td></tr>
<tr><td>out</td><td>WR↩
_3</td><td>24bit(RE): 0
24bit(IM): Offset(restricted to 24bit value!) of the SCRATCH buffer(data not needed after exiting the kernel), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR.</td></tr>
</table>

Parameters

out	*WR↩ _4*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(bitfiels with peaks postions), relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR.

```
*                  OUTPUT Memory layout:
*                  SRAM, bitfield, P<n> = bit 1(peak), bit 0(non peak)
*             0x0  P31P30......P17P16  P15P14........P1P0
*             0x4  P63P62......P49P48  P47P46......P33P32
*             0x8  P95P94......P81P80  P79P78......P65P64
*             0xC  P127P126..P113P112  P111P110....P97P96
*            0x10  P159P158..P145P144  P143P142..P129P128
*            0x14  P191P190..P177P176  P175P174..P161P160
*            0x18  P223P222..P209P208  P207P206..P193P192
*            0x1C  P255P254..P241P240  P239P238..P225P224
*                  |                   |
*                  P511P510..P497P496  P495P494..P481P480 -> chirp 0
*                  ------------------------------------------------
*                  P31P30......P17P16  P15P14........P1P0
*                  P63P62......P49P48  P47P46......P33P32
*                  P95P94......P81P80  P79P78......P65P64
*                  P127P126..P113P112  P111P110....P97P96
*                  P159P158..P145P144  P143P142..P129P128
*                  P191P190..P177P176  P175P174..P161P160
*                  P223P222..P209P208  P207P206..P193P192
*                  P255P254..P241P240  P239P238..P225P224
*                  |                   |
*                  P511P510..P497P496  P495P494..P481P480 -> chirp 1
*                  ------------------------------------------------
*                  |                   |
*                  |                   |
*                  |                   |
*                  ------------------------------------------------
*                  P31P30......P17P16  P15P14........P1P0
*                  P63P62......P49P48  P47P46......P33P32
*                  P95P94......P81P80  P79P78......P65P64
*                  P127P126..P113P112  P111P110....P97P96
*                  P159P158..P145P144  P143P142..P129P128
*                  P191P190..P177P176  P175P174..P161P160
*                  P223P222..P209P208  P207P206..P193P192
*                  P255P254..P241P240  P239P238..P225P224
*                  |                   |
*                  P511P510..P497P496  P495P494..P481P480 -> chirp 127
*                  ------------------------------------------------
*
```

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

N/A.

Note

The current implementation is ONLY to be used in combination with 1024 Range FFT and 128 Doppler FFT.
The input buffer MUST NOT be reused for either scratch or peaks output.
All buffers MUST be 8 bytes aligned.

3.9.2.89 RsdkSptPeakSearch1024smp64crp()

```
void * RsdkSptPeakSearch1024smp64crp (
            void  )
```

Function for Peak Search, 64 chirps (doppler bins), 1024 sample/chirp (512 range bins).

Compute Peaks, storing results in SRAM as packed bitfields, to be used togheter with Radar Data Cube by more advanced algorithms.

Parameters

in	*WR↩_1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Range-Doppler Matrix), relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 16 bit real Log2 Mag2 format * 63:48 47:32 31:16 15:0 * R0_D0_MAG R0_D1_MAG R0_D2_MAG R0_D3_MAG * | | | | * R0_D60_MAG R0_D61_MAG R0_D62_MAG R0_D63_MAG * R1_D0_MAG R1_D1_MAG R1_D2_MAG R1_D3_MAG * | | | | * R1_D60_MAG R1_D61_MAG R1_D62_MAG R1_D63_MAG * | | | | * | | | | * | | | | * R511_D0_MAG R511_D1_MAG R511_D2_MAG R511_D3_MAG * | | | | * R511_D60_MAG R511_D61_MAG R511_D62_MAG R511_D63_MAG *
in	*WR↩_2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the Threshold buffer, log2 format, relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * Threshold Memory layout: * SRAM, 16bit log2 format * 63:48 47:32 31:16 15:0 * Thr_RangeBin0 Thr_RangeBin1 Thr_RangeBin2 Thr_RangeBin3 * Thr_RangeBin4 Thr_RangeBin5 Thr_RangeBin6 Thr_RangeBin7 * | | | | * | | | | * | | | | * Thr_RangeBin508 Thr_RangeBin509 Thr_RangeBin510 Thr_RangeBin511 *
out	*WR↩_3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the SCRATCH buffer(data not needed after exiting the kernel), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR.

Parameters

out	*WR↩ _4*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(bitfiels with peaks postions), relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR.

```
*                  OUTPUT Memory layout:
*                  SRAM, bitfield, P<n> = bit 1(peak), bit 0(non peak)
*             0x0  P31P30......P17P16  P15P14........P1P0
*             0x4  P63P62......P49P48  P47P46......P33P32
*             0x8  P95P94......P81P80  P79P78......P65P64
*             0xC  P127P126..P113P112  P111P110....P97P96
*            0x10  P159P158..P145P144  P143P142..P129P128
*            0x14  P191P190..P177P176  P175P174..P161P160
*            0x18  P223P222..P209P208  P207P206..P193P192
*            0x1C  P255P254..P241P240  P239P238..P225P224
*                  |                   |
*                  P511P510..P497P496  P495P494..P481P480 -> chirp 0
*                  ------------------------------------------------
*                  P31P30......P17P16  P15P14........P1P0
*                  P63P62......P49P48  P47P46......P33P32
*                  P95P94......P81P80  P79P78......P65P64
*                  P127P126..P113P112  P111P110....P97P96
*                  P159P158..P145P144  P143P142..P129P128
*                  P191P190..P177P176  P175P174..P161P160
*                  P223P222..P209P208  P207P206..P193P192
*                  P255P254..P241P240  P239P238..P225P224
*                  |                   |
*                  P511P510..P497P496  P495P494..P481P480 -> chirp 1
*                  ------------------------------------------------
*                  |                   |
*                  |                   |
*                  |                   |
*                  ------------------------------------------------
*                  P31P30......P17P16  P15P14........P1P0
*                  P63P62......P49P48  P47P46......P33P32
*                  P95P94......P81P80  P79P78......P65P64
*                  P127P126..P113P112  P111P110....P97P96
*                  P159P158..P145P144  P143P142..P129P128
*                  P191P190..P177P176  P175P174..P161P160
*                  P223P222..P209P208  P207P206..P193P192
*                  P255P254..P241P240  P239P238..P225P224
*                  |                   |
*                  P511P510..P497P496  P495P494..P481P480 -> chirp 63
*                  ------------------------------------------------
*
```

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

N/A.

Note

The input buffer MUST NOT be reused for either scratch or peaks output.
All buffers MUST be 8 bytes aligned.

3.9.2.90 RsdkSptPeakSearch1024smp64crp1D()

```
void * RsdkSptPeakSearch1024smp64crp1D (
            void  )
```

Function for Peak Search, 64 chirps (doppler bins), 1024 sample/chirp (512 range bins).

Compute Peaks, storing results in SRAM as packed bitfields, to be used togheter with Radar Data Cube by more advanced algorithms.

Parameters

in	*WR↩_1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Range-Doppler Matrix), relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 16 bit real Log2 Mag2 format * 63:48 47:32 31:16 15:0 * R0_D0_MAG R0_D1_MAG R0_D2_MAG R0_D3_MAG * | | | | * R0_D60_MAG R0_D61_MAG R0_D62_MAG R0_D63_MAG * R1_D0_MAG R1_D1_MAG R1_D2_MAG R1_D3_MAG * | | | | * R1_D60_MAG R1_D61_MAG R1_D62_MAG R1_D63_MAG * | | | | * | | | | * | | | | * R511_D0_MAG R511_D1_MAG R511_D2_MAG R511_D3_MAG * | | | | * R511_D60_MAG R511_D61_MAG R511_D62_MAG R511_D63_MAG *
in	*WR↩_2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the Threshold buffer, log2 format, relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * Threshold Memory layout: * SRAM, 16bit log2 format * 63:48 47:32 31:16 15:0 * Thr_RangeBin0 Thr_RangeBin1 Thr_RangeBin2 Thr_RangeBin3 * Thr_RangeBin4 Thr_RangeBin5 Thr_RangeBin6 Thr_RangeBin7 * | | | | * | | | | * | | | | * Thr_RangeBin508 Thr_RangeBin509 Thr_RangeBin510 Thr_RangeBin511 *
out	*WR↩_3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(bitfiels with peaks postions), relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * Big Endian representation, internal to SPT. PDMA to SRAM swaps endian consi * SRAM, bitfield, P<n> = bit 1(peak), bit 0(non peak) * 0x0 P31P30......P17P16 P15P14........P1P0 * 0x4 P63P62......P49P48 P47P46......P33P32 -> RangeBin 0 * ---------- * P31P30......P17P16 P15P14........P1P0 * P63P62......P49P48 P47P46......P33P32 -> RangeBin 1 * ---------- * | | * | | * | | * ---------- * P31P30......P17P16 P15P14........P1P0 * P63P62......P49P48 P47P46......P33P32 -> RangeBin 511 * ---------- *

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

N/A.

Note

The input buffer MUST NOT be reused for peaks output.
All buffers MUST be 8 bytes aligned.

3.9.2.91 RsdkSptPeakSearch2048smp256crp()

```
void * RsdkSptPeakSearch2048smp256crp (
            void  )
```

Function for Peak Search, 256 chirps (doppler bins), 2048 sample/chirp (1024 range bins).

Compute Peaks, storing results in SRAM as packed bitfields, to be used togheter with Radar Data Cube by more advanced algorithms.

Parameters

in	*WR_1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Range-Doppler Matrix), relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR.

```
*                INPUT Memory layout:
*                SRAM, 16 bit real Log2 Mag2 format
*                63:48           47:32           31:16           15:0
*                R0_D0_MAG       R0_D1_MAG       R0_D2_MAG       R0_D3_MAG
*                |               |               |               |
*                R0_D252_MAG     R0_D253_MAG     R0_D254_MAG     R0_D255_MAG
*                R1_D0_MAG       R1_D1_MAG       R1_D2_MAG       R1_D3_MAG
*                |               |               |               |
*                R1_D252_MAG     R1_D253_MAG     R1_D254_MAG     R1_D255_MAG
*                |               |               |               |
*                |               |               |               |
*                |               |               |               |
*                |               |               |               |
*                |               |               |               |
*                R1023_D0_MAG    R1023_D1_MAG    R1023_D2_MAG    R1023_D3_MAG
*                |               |               |               |
*                R1023_D252_MAG  R1023_D253_MAG  R1023_D254_MAG  R1023_D255_MAG
*
```

Parameters

in	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the Threshold buffer, log2 format, relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * Threshold Memory layout: * SRAM, 16bit log2 format * 63:48 47:32 31:16 15:0 * Thr_RangeBin0 Thr_RangeBin1 Thr_RangeBin2 Thr_RangeBin3 * Thr_RangeBin4 Thr_RangeBin5 Thr_RangeBin6 Thr_RangeBin7 * | | | | * | | | | * | | | | * Thr_RangeBin1020 Thr_RangeBin1021 Thr_RangeBin1022 Thr_RangeBin1023 *
out	*WR↩ _3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the SCRATCH buffer(data not needed after exiting the kernel), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR.

Parameters

out	*WR↩ _4*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(bitfiels with peaks postions), relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR.

```
*                  OUTPUT Memory layout:
*                  SRAM, bitfield, P<n> = bit 1(peak), bit 0(non peak)
*             0x0  P31P30......P17P16  P15P14........P1P0
*             0x4  P63P62......P49P48  P47P46......P33P32
*             0x8  P95P94......P81P80  P79P78......P65P64
*             0xC  P127P126..P113P112  P111P110....P97P96
*            0x10  P159P158..P145P144  P143P142..P129P128
*            0x14  P191P190..P177P176  P175P174..P161P160
*            0x18  P223P222..P209P208  P207P206..P193P192
*            0x1C  P255P254..P241P240  P239P238..P225P224
*            0x20  P287P286..P273P272  P271P270..P257P256
*            0x24  P319P318..P305P304  P303P302..P289P288
*            0x28  P351P350..P337P336  P335P334..P321P320
*            0x2C  P383P382..P369P368  P367P366..P353P352
*            0x30  P415P414..P401P400  P399P398..P385P384
*            0x34  P447P446..P433P432  P431P430..P417P416
*            0x38  P479P478..P465P464  P463P462..P449P448
*            0x3C  P511P510..P241P496  P495P494..P481P480
*`                 ....................................
*            0x7C  P1023P1022...P1008  P1007P1006....P992 -> chirp 0
*                  ------------------------------------------------
*                  P31P30......P17P16  P15P14........P1P0
*                  P63P62......P49P48  P47P46......P33P32
*                  P95P94......P81P80  P79P78......P65P64
*                  P127P126..P113P112  P111P110....P97P96
*                  P159P158..P145P144  P143P142..P129P128
*                  P191P190..P177P176  P175P174..P161P160
*                  P223P222..P209P208  P207P206..P193P192
*                  P255P254..P241P240  P239P238..P225P224
*                  P287P286..P273P272  P271P270..P257P256
*                  P319P318..P305P304  P303P302..P289P288
*                  P351P350..P337P336  P335P334..P321P320
*                  P383P382..P369P368  P367P366..P353P352
*                  P415P414..P401P400  P399P398..P385P384
*                  P447P446..P433P432  P431P430..P417P416
*                  P479P478..P465P464  P463P462..P449P448
*                  P511P510..P241P496  P495P494..P481P480
*`                 ....................................
*                  P1023P1022...P1008  P1007P1006....P992 -> chirp 1
*                  ------------------------------------------------
*                  |                   |
*                  |                   |
*                  |                   |
*                  ------------------------------------------------
*                  P31P30......P17P16  P15P14........P1P0
*                  P63P62......P49P48  P47P46......P33P32
*                  P95P94......P81P80  P79P78......P65P64
*                  P127P126..P113P112  P111P110....P97P96
*                  P159P158..P145P144  P143P142..P129P128
*                  P191P190..P177P176  P175P174..P161P160
*                  P223P222..P209P208  P207P206..P193P192
*                  P255P254..P241P240  P239P238..P225P224
*                  P287P286..P273P272  P271P270..P257P256
*                  P319P318..P305P304  P303P302..P289P288
*                  P351P350..P337P336  P335P334..P321P320
*                  P383P382..P369P368  P367P366..P353P352
*                  P415P414..P401P400  P399P398..P385P384
*                  P447P446..P433P432  P431P430..P417P416
*                  P479P478..P465P464  P463P462..P449P448
*                  P511P510..P241P496  P495P494..P481P480
*`                 ....................................
*                  P1023P1022...P1008  P1007P1006....P992 -> chirp 255
*                  ------------------------------------------------
*
```

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

N/A.

Note

The current implementation is ONLY to be used in combination with 2048 Range FFT and 256 Doppler FFT.
The input buffer MUST NOT be reused for either scratch or peaks output.
All buffers MUST be 8 bytes aligned.

3.9.2.92 RsdkSptPeakSearch2048smp256crp1D()

```
void * RsdkSptPeakSearch2048smp256crp1D (
            void  )
```

Function for Peak Search 1D, 256 chirps (doppler bins), 2048 sample/chirp (1024 range bins).

Compute Peaks, storing results in SRAM as packed bitfields, to be used togheter with Radar Data Cube by more advanced algorithms.

Parameters

in	*WR_1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Range-Doppler Matrix), relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR.

```
*                 INPUT Memory layout:
*                 SRAM, 16 bit real Log2 Mag2 format
*                 63:48           47:32           31:16           15:0
*                 R0_D0_MAG       R0_D1_MAG       R0_D2_MAG       R0_D3_MAG
*                 |               |               |               |
*                 R0_D252_MAG     R0_D253_MAG     R0_D254_MAG     R0_D255_MAG
*                 R1_D0_MAG       R1_D1_MAG       R1_D2_MAG       R1_D3_MAG
*                 |               |               |               |
*                 R1_D252_MAG     R1_D253_MAG     R1_D254_MAG     R1_D255_MAG
*                 |               |               |               |
*                 |               |               |               |
*                 |               |               |               |
*                 |               |               |               |
*                 |               |               |               |
*                 R1023_D0_MAG    R1023_D1_MAG    R1023_D2_MAG    R1023_D3_MAG
*                 |               |               |               |
*                 R1023_D252_MAG  R1023_D253_MAG  R1023_D254_MAG  R1023_D255_MAG
*
```

Parameters

in	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the Threshold buffer, log2 format, relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * Threshold Memory layout: * SRAM, 16bit log2 format * 63:48 47:32 31:16 15:0 * Thr_RangeBin0 Thr_RangeBin1 Thr_RangeBin2 Thr_RangeBin3 * Thr_RangeBin4 Thr_RangeBin5 Thr_RangeBin6 Thr_RangeBin7 * | | | | * | | | | * | | | | * Thr_RangeBin1020 Thr_RangeBin1021 Thr_RangeBin1022 Thr_RangeBin1023 *
out	*WR↩ _3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(bitfiels with peaks postions), relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * Big Endian representation, internal to SPT. PDMA to SRAM swaps endian consi * SRAM, bitfield, P<n> = bit 1(peak), bit 0(non peak) * 0x0 P31P30......P17P16 P15P14........P1P0 * 0x4 P63P62......P49P48 P47P46......P33P32 * 0x8 P95P94......P81P80 P79P78......P65P64 * 0xC P127P126..P113P112 P111P110....P97P96 * 0x10 P159P158..P145P144 P143P142..P129P128 * 0x14 P191P190..P177P176 P175P174..P161P160 * 0x18 P223P222..P209P208 P207P206..P193P192 * 0x1C P255P254..P241P240 P239P238..P225P224 -> RangeBin 0 * ---------- * P31P30......P17P16 P15P14........P1P0 * P63P62......P49P48 P47P46......P33P32 * P95P94......P81P80 P79P78......P65P64 * P127P126..P113P112 P111P110....P97P96 * P159P158..P145P144 P143P142..P129P128 * P191P190..P177P176 P175P174..P161P160 * P223P222..P209P208 P207P206..P193P192 * P255P254..P241P240 P239P238..P225P224 -> RangeBin 1 * ---------- * | | * | | * | | * ---------- * P31P30......P17P16 P15P14........P1P0 * P63P62......P49P48 P47P46......P33P32 * P95P94......P81P80 P79P78......P65P64 * P127P126..P113P112 P111P110....P97P96 * P159P158..P145P144 P143P142..P129P128 * P191P190..P177P176 P175P174..P161P160 * P223P222..P209P208 P207P206..P193P192 * P255P254..P241P240 P239P238..P225P224 -> RangeBin 1023 * ---------- *

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

N/A.

Note

The current implementation is ONLY to be used in combination with 2048 Range FFT and 256 Doppler FFT.
The input buffer MUST NOT be reused for peaks output.
All buffers MUST be 8 bytes aligned.

3.9.2.93 RsdkSptPeakSearch256smp128crp()

```
void * RsdkSptPeakSearch256smp128crp (
            void  )
```

Function for Peak Search, 128 chirps (doppler bins), 256 sample/chirp (128 range bins).

Compute Peaks, storing results in SRAM as packed bitfields, to be used togheter with Radar Data Cube by more advanced algorithms.

Parameters

in	*WR↩_1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Range-Doppler Matrix), relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 16 bit real Log2 Mag2 format * 63:48 47:32 31:16 15:0 * R0_D0_MAG R0_D1_MAG R0_D2_MAG R0_D3_MAG * | | | | * R0_D124_MAG R0_D125_MAG R0_D126_MAG R0_D127_MAG * R1_D0_MAG R1_D1_MAG R1_D2_MAG R1_D3_MAG * | | | | * R1_D124_MAG R1_D125_MAG R1_D126_MAG R1_D127_MAG * | | | | * | | | | * | | | | * | | | | * | | | | * R127_D0_MAG R127_D1_MAG R127_D2_MAG R127_D3_MAG * | | | | * R127_D124_MAG R127_D125_MAG R127_D126_MAG R127_D127_MAG *
in	*WR↩_2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the Threshold buffer, log2 format, relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * Threshold Memory layout: * SRAM, 16bit log2 format * 63:48 47:32 31:16 15:0 * Thr_RangeBin0 Thr_RangeBin1 Thr_RangeBin2 Thr_RangeBin3 * Thr_RangeBin4 Thr_RangeBin5 Thr_RangeBin6 Thr_RangeBin7 * | | | | * | | | | * | | | | * Thr_RangeBin124 Thr_RangeBin125 Thr_RangeBin126 Thr_RangeBin127 *

Parameters

out	*WR↩ _3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the SCRATCH buffer(data not needed after exiting the kernel), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR.
out	*WR↩ _4*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(bitfiels with peaks postions), relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * SRAM, bitfield, P<n> = bit 1(peak), bit 0(non peak) * 0x0 P31P30......P17P16 P15P14........P1P0 * 0x4 P63P62......P49P48 P47P46......P33P32 * 0x8 P95P94......P81P80 P79P78......P65P64 * 0xC P127P126..P113P112 P111P110....P97P96 -> chirp 0 * ---------- * P31P30......P17P16 P15P14........P1P0 * P63P62......P49P48 P47P46......P33P32 * P95P94......P81P80 P79P78......P65P64 * P127P126..P113P112 P111P110....P97P96 -> chirp 1 * ---------- * | | * | | * | | * ---------- * P31P30......P17P16 P15P14........P1P0 * P63P62......P49P48 P47P46......P33P32 * P95P94......P81P80 P79P78......P65P64 * P127P126..P113P112 P111P110....P97P96 -> chirp 127 * ---------- *

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

N/A.

Note

The current implementation is ONLY to be used in combination with 256 Range FFT and 128 Doppler FFT.
The input buffer MUST NOT be reused for either scratch or peaks output.
All buffers MUST be 8 bytes aligned.

3.9.2.94 RsdkSptPeakSearch256smp256crp()

```
void * RsdkSptPeakSearch256smp256crp (
            void  )
```

Function for Peak Search, 256 chirps (doppler bins), 256 sample/chirp (128 range bins).

Compute Peaks, storing results in SRAM as packed bitfields, to be used togheter with Radar Data Cube by more advanced algorithms.

Parameters

<table>
<tr><td>in</td><td>WR↩
_1</td><td>24bit(RE): 0
24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Range-Doppler Matrix), relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR.

<pre>
*   INPUT Memory layout:
*   SRAM, 16 bit real Log2 Mag2 format
*   63:48   47:32   31:16   15:0
*   R0_D0_MAG   R0_D1_MAG   R0_D2_MAG   R0_D3_MAG
* | | | |
*   R0_D252_MAG   R0_D253_MAG   R0_D254_MAG   R0_D255_MAG
*   R1_D0_MAG   R1_D1_MAG   R1_D2_MAG   R1_D3_MAG
* | | | |
*   R1_D252_MAG   R1_D253_MAG   R1_D254_MAG   R1_D255_MAG
* | | | |
* | | | |
* | | | |
* | | | |
* | | | |
*   R127_D0_MAG   R127_D1_MAG   R127_D2_MAG   R127_D3_MAG
* | | | |
*   R127_D252_MAG   R127_D253_MAG   R127_D254_MAG   R127_D255_MAG
*
</pre></td></tr>
<tr><td>in</td><td>WR↩
_2</td><td>24bit(RE): 0
24bit(IM): Offset(restricted to 24bit value!) of the Threshold buffer, log2 format, relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR.

<pre>
*   Threshold Memory layout:
*   SRAM, 16bit log2 format
*   63:48   47:32   31:16   15:0
*   Thr_RangeBin0   Thr_RangeBin1   Thr_RangeBin2   Thr_RangeBin3
*   Thr_RangeBin4   Thr_RangeBin5   Thr_RangeBin6   Thr_RangeBin7
* | | | |
* | | | |
* | | | |
*   Thr_RangeBin124 Thr_RangeBin125 Thr_RangeBin126 Thr_RangeBin127
*
</pre></td></tr>
<tr><td>out</td><td>WR↩
_3</td><td>24bit(RE): 0
24bit(IM): Offset(restricted to 24bit value!) of the SCRATCH buffer(data not needed after exiting the kernel), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR.</td></tr>
<tr><td>out</td><td>WR↩
_4</td><td>24bit(RE): 0
24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(bitfiels with peaks postions), relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR.

<pre>
*   OUTPUT Memory layout:
*   SRAM, bitfield, P<n> = bit 1(peak), bit 0(non peak)
*   0x0 P31P30......P17P16 P15P14........P1P0
*   0x4 P63P62......P49P48 P47P46......P33P32
*   0x8 P95P94......P81P80 P79P78......P65P64
*   0xC P127P126..P113P112 P111P110....P97P96 -> chirp 0
*   --
*   P31P30......P17P16 P15P14........P1P0
*   P63P62......P49P48 P47P46......P33P32
*   P95P94......P81P80 P79P78......P65P64
*   P127P126..P113P112 P111P110....P97P96 -> chirp 1
*   --
* | |
* | |
* | |
*   --
*   P31P30......P17P16 P15P14........P1P0
*   P63P62......P49P48 P47P46......P33P32
*   P95P94......P81P80 P79P78......P65P64
*   P127P126..P113P112 P111P110....P97P96 -> chirp 255
*   --
*
</pre></td></tr>
</table>

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

N/A.

Note

The current implementation is ONLY to be used in combination with 256 Range FFT and 256 Doppler FFT.
The input buffer MUST NOT be reused for either scratch or peaks output.
All buffers MUST be 8 bytes aligned.

3.9.2.95 RsdkSptPeakSearch512smp128crp()

```
void * RsdkSptPeakSearch512smp128crp (
            void  )
```

Function for Peak Search, 128 chirps (doppler bins), 512 sample/chirp (256 range bins).

Compute Peaks, storing results in SRAM as packed bitfields, to be used togheter with Radar Data Cube by more advanced algorithms.

Parameters

in	*WR_1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(Range-Doppler Matrix), relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR.

```
*                 INPUT Memory layout:
*                 SRAM, 16 bit real Log2 Mag2 format
*                 63:48           47:32           31:16           15:0
*                 R0_D0_MAG       R0_D1_MAG       R0_D2_MAG       R0_D3_MAG
*                 |               |               |               |
*                 R0_D124_MAG     R0_D125_MAG     R0_D126_MAG     R0_D127_MAG
*                 R1_D0_MAG       R1_D1_MAG       R1_D2_MAG       R1_D3_MAG
*                 |               |               |               |
*                 R1_D124_MAG     R1_D125_MAG     R1_D126_MAG     R1_D127_MAG
*                 |               |               |               |
*                 |               |               |               |
*                 |               |               |               |
*                 |               |               |               |
*                 |               |               |               |
*                 R255_D0_MAG     R255_D1_MAG     R255_D2_MAG     R255_D3_MAG
*                 |               |               |               |
*                 R255_D124_MAG   R255_D125_MAG   R255_D126_MAG   R255_D127_MAG
*
```

Parameters

in	*WR_2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the Threshold buffer, log2 format, relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * Threshold Memory layout: * SRAM, 16bit log2 format * 63:48 47:32 31:16 15:0 * Thr_RangeBin0 Thr_RangeBin1 Thr_RangeBin2 Thr_RangeBin3 * Thr_RangeBin4 Thr_RangeBin5 Thr_RangeBin6 Thr_RangeBin7 * | | | | * | | | | * | | | | * Thr_RangeBin252 Thr_RangeBin253 Thr_RangeBin254 Thr_RangeBin255 *
out	*WR_3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the SCRATCH buffer(data not needed after exiting the kernel), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR.
out	*WR_4*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(bitfiels with peaks postions), relative to SRAM start address, RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout (BIG ENDIAN representation): * SRAM, bitfield, P<n> = bit 1(peak), bit 0(non peak) * 0x0 P31P30......P17P16 P15P14........P1P0 * 0x4 P63P62......P49P48 P47P46......P33P32 * 0x8 P95P94......P81P80 P79P78......P65P64 * 0xC P127P126..P113P112 P111P110....P97P96 * 0x10 P159P158..P145P144 P143P142..P129P128 * 0x14 P191P190..P177P176 P175P174..P161P160 * 0x18 P223P222..P209P208 P207P206..P193P192 * 0x1C P255P254..P241P240 P239P238..P225P224 -> chirp 0 * -- * P31P30......P17P16 P15P14........P1P0 * P63P62......P49P48 P47P46......P33P32 * P95P94......P81P80 P79P78......P65P64 * P127P126..P113P112 P111P110....P97P96 * P159P158..P145P144 P143P142..P129P128 * P191P190..P177P176 P175P174..P161P160 * P223P222..P209P208 P207P206..P193P192 * P255P254..P241P240 P239P238..P225P224 -> chirp 1 * -- * | | * | | * | | * -- * P31P30......P17P16 P15P14........P1P0 * P63P62......P49P48 P47P46......P33P32 * P95P94......P81P80 P79P78......P65P64 * P127P126..P113P112 P111P110....P97P96 * P159P158..P145P144 P143P142..P129P128 * P191P190..P177P176 P175P174..P161P160 * P223P222..P209P208 P207P206..P193P192 * P255P254..P241P240 P239P238..P225P224 -> chirp 127 * -- *

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

N/A.

Note

The current implementation is ONLY to be used in combination with 512 Range FFT and 128 Doppler FFT.
The input buffer MUST NOT be reused for either scratch or peaks output.
All buffers MUST be 8 bytes aligned.

3.9.2.96 RsdkSptRange1024smp128crp4chCp4d()

```
void * RsdkSptRange1024smp128crp4chCp4d (
            void  )
```

Function for Range FFT 1024, 4 channels(CH#0..CH#3)

Compute Range FFT storing compressed results(CP4D) in SRAM.

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(sample ADC), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR.

```
*                  INPUT Memory layout:
*                  SRAM, 16bit Real ADC sample
*                  63:48     47:32     31:16     15:0
*                  CH0_0     CH0_1     CH0_2     CH0_3
*                  CH0_4     CH0_5     CH0_6     CH0_7
*                  CH1_0     CH1_1     CH1_2     CH1_3
*                  CH1_4     CH1_5     CH1_6     CH1_7
*                  CH2_0     CH2_1     CH2_2     CH2_3
*                  CH2_4     CH2_5     CH2_6     CH2_7
*                  CH3_0     CH3_1     CH3_2     CH3_3
*                  CH3_4     CH3_5     CH3_6     CH3_7
*                  |         |         |         |
*                  |         |         |         |
*                  |         |         |         |
*                  CH0_1016  CH0_1017  CH0_1018  CH0_1019
*                  CH0_1020  CH0_1021  CH0_1022  CH0_1023
*                  CH1_1016  CH1_1017  CH1_1018  CH1_1019
*                  CH1_1020  CH1_1021  CH1_1022  CH1_1023
*                  CH2_1016  CH2_1017  CH2_1018  CH2_1019
*                  CH2_1020  CH2_1021  CH2_1022  CH2_1023
*                  CH3_1016  CH3_1017  CH3_1018  CH3_1019
*                  CH3_1020  CH3_1021  CH3_1022  CH3_1023
*                  ---------------------------------------
*                  <RESERVED FOR CSI2 STATS: ADDR LINE 1>
*                  <RESERVED FOR CSI2 STATS: ADDR LINE 2>
*                  |
*                  <RESERVED FOR CSI2 STATS: ADDR LINE 10>
*                  ---------------------------------------
*
```

Parameters

out	*WR↩ _2*	24bit(RE): 0
		24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(Range FFT compressed results), relative to SRAM start address RSDK_SPT_CUBE_BASE_ADDR.
		
		`* OUTPUT Memory layout:`
		`* SRAM, 64bit compressed data`
		`* SRAM, 64bit compressed data`
		`* 63:0`
		`* CH0_0-CH1_0-CH2_0-CH3_0_Crp0`
		`* |`
		`* CH0_0-CH1_0-CH2_0-CH3_0-Crp127 ---> RangeBin ##0`
		`* ----------`
		`* |`
		`* |`
		`* |`
		`* ----------`
		`* CH0_511-CH1_511-CH2_511-CH3_511-Crp0`
		`* |`
		`* CH0_511-CH1_511-CH2_511-CH3_511-Crp127 ---> RangeBin ##511`
		`* ----------`
		`*`

in	*WR↩ _3*	24bit(RE): Scaling Factor, e.g. 1 for scale left with 1. 24bit(IM): 0.

Returns

WR_0: Return Global Max over the range output.

Return values

Return	Global Max amoung the absolute values of the real or imaginary part of the range output. Return value = max(max(abs(Re(k))), max(abs(Im(k)))), 24 bit format(1Q23).

Precondition

Twiddle coeficients are placed in TRAM at TR_0_0_0 by the Init function.
Window coeficients are placed in TRAM at TR_0_144_0 by the Init function.
MIPI_EVENT_LINE is RFE HW connection dependent. Can be configured from spt_kernel_internals.inc.

Note

This kernel implementation assumes that each ADC buffer is followed by BYTES_CSI2_STATS [80B] reserved for MIPI-CSI2 statistics.
The current implementation is ONLY to be used in combination with 128 Doppler FFT.
The input buffer CANNOT be reused for the output buffer.
All buffers MUST be 8 bytes aligned.

3.9.2.97 RsdkSptRange1024smp128crp8chCp4d()

```
void * RsdkSptRange1024smp128crp8chCp4d (
            void  )
```

3.9.2.98 RsdkSptRange1024smp256crp4ch3TxDdMimoCp4d()

```
void * RsdkSptRange1024smp256crp4ch3TxDdMimoCp4d (
            void  )
```

Function for Range FFT 1024, 4 channels(CH#0..CH#3), DD-Mimo use case.

Compute Range FFT storing compressed(CP4D) results in SRAM.

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the first INPUT buffer (sample ADC, CH#0..CH#3), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 16bit Real ADC sample * 63:48 47:32 31:16 15:0 * CH0_0 CH0_1 CH0_2 CH0_3 * CH0_4 CH0_5 CH0_6 CH0_7 * CH1_0 CH1_1 CH1_2 CH1_3 * CH1_4 CH1_5 CH1_6 CH1_7 * CH2_0 CH2_1 CH2_2 CH2_3 * CH2_4 CH2_5 CH2_6 CH2_7 * CH3_0 CH3_1 CH3_2 CH3_3 * CH3_4 CH3_5 CH3_6 CH3_7 * | | | | * | | | | * | | | | * CH0_1016 CH0_1017 CH0_1018 CH0_1019 * CH0_1020 CH0_1021 CH0_1022 CH0_1023 * CH1_1016 CH1_1017 CH1_1018 CH1_1019 * CH1_1020 CH1_1021 CH1_1022 CH1_1023 * CH2_1016 CH2_1017 CH2_1018 CH2_1019 * CH2_1020 CH2_1021 CH2_1022 CH2_1023 * CH3_1016 CH3_1017 CH3_1018 CH3_1019 * CH3_1020 CH3_1021 CH3_1022 CH3_1023 * ---------- * <RESERVED FOR CSI2 STATS: ADDR LINE 1> * <RESERVED FOR CSI2 STATS: ADDR LINE 2> * | * <RESERVED FOR CSI2 STATS: ADDR LINE 10> * ---------- *
out	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer (Range FFT compressed results), relative to SRAM start address RSDK_SPT_CUBE_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 64bit compressed data * 63:0 * CH0_0-CH1_0-CH2_0-CH3_0_Crp0 * | * CH0_0-CH1_0-CH2_0-CH3_0-Crp255 ---> RangeBin ##0 * ---------- * | * | * | * ---------- * CH0_511-CH1_511-CH2_511-CH3_511-Crp0 * | * CH0_511-CH1_511-CH2_511-CH3_511-Crp255 ---> RangeBin ##511 * ---------- *
in	*WR↩ _3*	24bit(RE): Scaling Factor, e.g. 1 for scale left with 1. 24bit(IM): 0.

Returns

WR_0: Return Global Max over the range output.

Return values

Return	Global Max amoung the absolute values of the real or imaginary part of the range output. Return value = max(max(abs(Re(k))), max(abs(Im(k)))), 24 bit format(1Q23).

Precondition

Twiddle coeficients are placed in TRAM at TR_0_0_0 by the Init function.
Window coeficients are placed in TRAM at TR_0_160_0 by the Init function.
MIPI_EVENT_LINE is RFE HW connection dependent. Can be configured from spt_kernel_internals.inc.

Note

The kernel implementation guarantees no saturation, for 0 output scaling factor. If other scaling is used, it's the user responsibility to prevent saturation.
This kernel implementation assumes that each ADC buffer is followed by BYTES_CSI2_STATS [80B] reserved for MIPI-CSI2 statistics.
The current implementation is ONLY to be used in combination with 256 CP4D Doppler FFT.
The input buffer CANNOT be reused for the output buffer.
All buffers MUST be 8 bytes aligned.

3.9.2.99 RsdkSptRange1024smp64crp4ch4TxTdMimoCp4d()

```
void * RsdkSptRange1024smp64crp4ch4TxTdMimoCp4d (
            void  )
```

Function for Range FFT 1024, 4 physical channels/TX slot, 4TX TD-MIMO.

Compute Range FFT storing compressed results(CP4D) in SRAM.

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(sample ADC), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout * SRAM, 16bit Real ADC sample * 63:48 47:32 31:16 15:0 * CH0_0 CH0_1 CH0_2 CH0_3 * CH0_4 CH0_5 CH0_6 CH0_7 * CH1_0 CH1_1 CH1_2 CH1_3 * CH1_4 CH1_5 CH1_6 CH1_7 * CH2_0 CH2_1 CH2_2 CH2_3 * CH2_4 CH2_5 CH2_6 CH2_7 * CH3_0 CH3_1 CH3_2 CH3_3 * CH3_4 CH3_5 CH3_6 CH3_7 * | | | | * | | | | * | | | | * CH0_1016 CH0_1017 CH0_1018 CH0_1019 * CH0_1020 CH0_1021 CH0_1022 CH0_1023 * CH1_1016 CH1_1017 CH1_1018 CH1_1019 * CH1_1020 CH1_1021 CH1_1022 CH1_1023 * CH2_1016 CH2_1017 CH2_1018 CH2_1019 * CH2_1020 CH2_1021 CH2_1022 CH2_1023 * CH3_1016 CH3_1017 CH3_1018 CH3_1019 * CH3_1020 CH3_1021 CH3_1022 CH3_1023 * ---------- * <RESERVED FOR CSI2 STATS: ADDR LINE 1> * <RESERVED FOR CSI2 STATS: ADDR LINE 2> * | * <RESERVED FOR CSI2 STATS: ADDR LINE 10> * ---------- *
out	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(Range FFT compressed results), relative to SRAM start address RSDK_SPT_CUBE_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 64bit compressed data * 63:0 * CH0_0-CH1_0-CH2_0-CH3_0-TX0_Crp0 * CH0_0-CH1_0-CH2_0-CH3_0-TX1_Crp0 * CH0_0-CH1_0-CH2_0-CH3_0-TX2_Crp0 * CH0_0-CH1_0-CH2_0-CH3_0-TX3_Crp0 * | * CH0_0-CH1_0-CH2_0-CH3_0-TX0_Crp63 * CH0_0-CH1_0-CH2_0-CH3_0-TX1_Crp63 * CH0_0-CH1_0-CH2_0-CH3_0-TX2_Crp63 * CH0_0-CH1_0-CH2_0-CH3_0-TX3_Crp63 ---> RangeBin ##0 * ---------- * | * | * | * ---------- * CH0_511-CH1_511-CH2_511-CH3_511-TX0_Crp0 * CH0_511-CH1_511-CH2_511-CH3_511-TX1_Crp0 * CH0_511-CH1_511-CH2_511-CH3_511-TX2_Crp0 * CH0_511-CH1_511-CH2_511-CH3_511-TX3_Crp0 * | * CH0_511-CH1_511-CH2_511-CH3_511-TX0_Crp63 * CH0_511-CH1_511-CH2_511-CH3_511-TX1_Crp63 * CH0_511-CH1_511-CH2_511-CH3_511-TX2_Crp63 * CH0_511-CH1_511-CH2_511-CH3_511-TX3_Crp63 ---> RangeBin ##511 * ---------- *
in	*WR↩ _3*	24bit(RE): Scaling Factor, e.g. 1 for scale left with 1. 24bit(IM): 0.

Returns

WR_0: Return Global Max over the range output.

Return values

Return	Global Max amoung the absolute values of the real or imaginary part of the range output. Return value = max(max(abs(Re(k))), max(abs(Im(k)))), 24 bit format(1Q23).

Precondition

Twiddle coeficients are placed in TRAM at TR_0_0_0 by the Init function.
Window coeficients are placed in TRAM at TR_0_136_0 by the Init function.
MIPI_EVENT_LINE is RFE HW connection dependent. Can be configured from spt_kernel_internals.inc.

Note

This kernel implementation assumes that each ADC buffer is followed by BYTES_CSI2_STATS [80B] reserved for MIPI-CSI2 statistics.
The current implementation is ONLY to be used in combination with 64 Doppler FFT 4TX TD-MIMO.
The input buffer CANNOT be reused for the output buffer.
All buffers MUST be 8 bytes aligned.

3.9.2.100 RsdkSptRange1024smp64crp4chCp4d()

```
void * RsdkSptRange1024smp64crp4chCp4d (
            void  )
```

Function for Range FFT 1024, 4 physical channels(CH#0..CH#3)

Compute Range FFT storing compressed results(CP4D) in SRAM. Register address for the CSI channel VC1 is hardcoded in this kernel, MIPI_EVENT_LINE_VC1.

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(sample ADC), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout * SRAM, 16bit Real ADC sample * 63:48 47:32 31:16 15:0 * CH0_0 CH0_1 CH0_2 CH0_3 * CH0_4 CH0_5 CH0_6 CH0_7 * CH1_0 CH1_1 CH1_2 CH1_3 * CH1_4 CH1_5 CH1_6 CH1_7 * CH2_0 CH2_1 CH2_2 CH2_3 * CH2_4 CH2_5 CH2_6 CH2_7 * CH3_0 CH3_1 CH3_2 CH3_3 * CH3_4 CH3_5 CH3_6 CH3_7 * | | | | * | | | | * | | | | * CH0_1016 CH0_1017 CH0_1018 CH0_1019 * CH0_1020 CH0_1021 CH0_1022 CH0_1023 * CH1_1016 CH1_1017 CH1_1018 CH1_1019 * CH1_1020 CH1_1021 CH1_1022 CH1_1023 * CH2_1016 CH2_1017 CH2_1018 CH2_1019 * CH2_1020 CH2_1021 CH2_1022 CH2_1023 * CH3_1016 CH3_1017 CH3_1018 CH3_1019 * CH3_1020 CH3_1021 CH3_1022 CH3_1023 * ---------- * <RESERVED FOR CSI2 STATS: ADDR LINE 1> * <RESERVED FOR CSI2 STATS: ADDR LINE 2> * | * <RESERVED FOR CSI2 STATS: ADDR LINE 10> * ---------- *
out	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(Range FFT compressed results), relative to SRAM start address RSDK_SPT_CUBE_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 64bit compressed data * 63:0 * CH0_0-CH1_0-CH2_0-CH3_0-Crp0 * | * CH0_0-CH1_0-CH2_0-CH3_0-_Crp64 ---> RangeBin ##0 * ---------- * | * | * | * ---------- * CH0_511-CH1_511-CH2_511-CH3_511-Crp0 * | * CH0_511-CH1_511-CH2_511-CH3_511-Crp64 ---> RangeBin ##511 * ---------- *
in	*WR↩ _3*	24bit(RE): Scaling Factor, e.g. 1 for scale left with 1. 24bit(IM): 0.

Returns

WR_0: Return Global Max over the range output.

Return values

Return	Global Max amoung the absolute values of the real or imaginary part of the range output. Return value = max(max(abs(Re(k))), max(abs(Im(k)))), 24 bit format(1Q23).

Precondition

Twiddle coeficients are placed in TRAM at TR_0_0_0 by the Init function.
Window coeficients are placed in TRAM at TR_0_136_0 by the Init function.
MIPI_EVENT_LINE is RFE HW connection dependent. Can be configured from spt_kernel_internals.inc.

Note

This kernel implementation assumes that each ADC buffer is followed by BYTES_CSI2_STATS [80B] reserved for MIPI-CSI2 statistics.
The current implementation is ONLY to be used in combination with 64 Doppler FFT.
The input buffer CANNOT be reused for the output buffer.
All buffers MUST be 8 bytes aligned.

3.9.2.101 RsdkSptRange2048smp256crp4chCp4d()

```
void * RsdkSptRange2048smp256crp4chCp4d (
            void  )
```

Function for Range FFT 2048, 4 channels(CH#0..CH#3)

Compute Range FFT storing the results in SRAM.

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(sample ADC), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout * SRAM, 16bit Real ADC sample * 63:48 47:32 31:16 15:0 * CH0_0 CH0_1 CH0_2 CH0_3 * CH0_4 CH0_5 CH0_6 CH0_7 * CH1_0 CH1_1 CH1_2 CH1_3 * CH1_4 CH1_5 CH1_6 CH1_7 * CH2_0 CH2_1 CH2_2 CH2_3 * CH2_4 CH2_5 CH2_6 CH2_7 * CH3_0 CH3_1 CH3_2 CH3_3 * CH3_4 CH3_5 CH3_6 CH3_7 * | | | | * | | | | * | | | | * CH0_2040 CH0_2041 CH0_2042 CH0_2043 * CH0_2044 CH0_2045 CH0_2046 CH0_2047 * CH1_2040 CH1_2041 CH1_2042 CH1_2043 * CH1_2044 CH1_2045 CH1_2046 CH1_2047 * CH2_2040 CH2_2041 CH2_2042 CH2_2043 * CH2_2044 CH2_2045 CH2_2046 CH2_2047 * CH3_2040 CH3_2041 CH3_2042 CH3_2043 * CH3_2044 CH3_2045 CH3_2046 CH3_2047 * ---------- * <RESERVED FOR CSI2 STATS: ADDR LINE 1> * <RESERVED FOR CSI2 STATS: ADDR LINE 2> * | * <RESERVED FOR CSI2 STATS: ADDR LINE 10> * ---------- *
out	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(Range FFT compressed results), relative to SRAM start address RSDK_SPT_CUBE_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 64bit compressed data * 63:0 * CH0_0-CH1_0-CH2_0-CH3_0_Crp0 * | * CH0_0-CH1_0-CH2_0-CH3_0-Crp255 ---> RangeBin ##0 * ---------- * | * | * | * ---------- * CH0_1023-CH1_1023-CH2_1023-CH3_1023-Crp0 * | * CH0_1023-CH1_1023-CH2_1023-CH3_1023-Crp255 ---> RangeBin ##1023 * ---------- *
in	*WR↩ _3*	24bit(RE): Scaling Factor, e.g. 1 for scale left with 1. 24bit(IM): 0.

Returns

WR_0: Return Global Max over the range output.

Return values

Return	Global Max amoung the absolute values of the real or imaginary part of the range output. Return value = max(max(abs(Re(k))), max(abs(Im(k)))), 24 bit format(1Q23).

Precondition

Twiddle coeficients are placed in TRAM at TR_0_0_0 by the Init function.
Window coeficients are placed in TRAM at TR_1_0_0 by the Init function.
MIPI_EVENT_LINE is RFE HW connection dependent. Can be configured from spt_kernel_internals.inc.

Note

This kernel implementation assumes that each ADC buffer is followed by BYTES_CSI2_STATS [80B] reserved for MIPI-CSI2 statistics.
The current implementation is ONLY to be used in combination with 256 Doppler FFT.
The input buffer CANNOT be reused for the output buffer.
All buffers MUST be 8 bytes aligned.

3.9.2.102 RsdkSptRange256smp128crp4ch3TxTdMimo()

```
void * RsdkSptRange256smp128crp4ch3TxTdMimo (
            void  )
```

Function for Range FFT 256, 4 physical channels/TX slot, 3TX TD-MIMO.

Compute Range FFT storing the results in SRAM.

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(sample ADC), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR.

```
*                   INPUT Memory layout
*                   SRAM, 16bit Real ADC sample
*                   63:48     47:32     31:16     15:0
*                   CH0_0     CH0_1     CH0_2     CH0_3
*                   CH0_4     CH0_5     CH0_6     CH0_7
*                   CH1_0     CH1_1     CH1_2     CH1_3
*                   CH1_4     CH1_5     CH1_6     CH1_7
*                   CH2_0     CH2_1     CH2_2     CH2_3
*                   CH2_4     CH2_5     CH2_6     CH2_7
*                   CH3_0     CH3_1     CH3_2     CH3_3
*                   CH3_4     CH3_5     CH3_6     CH3_7
*                   |         |         |         |
*                   |         |         |         |
*                   |         |         |         |
*                   CH0_248   CH0_249   CH0_250   CH0_251
*                   CH0_252   CH0_253   CH0_254   CH0_255
*                   CH1_248   CH1_249   CH1_250   CH1_251
*                   CH1_252   CH1_253   CH1_254   CH1_255
*                   CH2_248   CH2_249   CH2_250   CH2_251
*                   CH2_252   CH2_253   CH2_254   CH2_255
*                   CH3_248   CH3_249   CH3_250   CH3_251
*                   CH3_252   CH3_253   CH3_254   CH3_255
*                   ---------------------------------------
*                   <RESERVED FOR CSI2 STATS: ADDR LINE 1>
*                   <RESERVED FOR CSI2 STATS: ADDR LINE 2>
*                   |
*                   <RESERVED FOR CSI2 STATS: ADDR LINE 10>
*                   ---------------------------------------
*
```

Parameters

out	*WR_2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(Range FFT results), relative to SRAM start address RSDK_SPT_CUBE_BASE_ADDR.
in	*WR_3*	24bit(RE): Scaling Factor, e.g. 1 for scale left with 1. 24bit(IM): 0.

```
*                 OUTPUT Memory layout:
*                 SRAM, 32bit complex(16b_Im + 16b_Re)
*                 63:48               47:32               31:16               15:0
*                 CH0_0_Im-Crp0       CH0_0_Re-Crp0       CH1_0_Im-Crp0       CH1_0_Re-Crp0
*                 CH2_0_Im-Crp0       CH2_0_Re-Crp0       CH3_0_Im-Crp0       CH3_0_Re-Crp0
*                 ------------------------------------------------------------------------------
*                 CH0_0_Im-Crp0       CH0_0_Re-Crp0       CH1_0_Im-Crp0       CH1_0_Re-Crp0
*                 CH2_0_Im-Crp0       CH2_0_Re-Crp0       CH3_0_Im-Crp0       CH3_0_Re-Crp0
*                 ------------------------------------------------------------------------------
*                 CH0_0_Im-Crp0       CH0_0_Re-Crp0       CH1_0_Im-Crp0       CH1_0_Re-Crp0
*                 CH2_0_Im-Crp0       CH2_0_Re-Crp0       CH3_0_Im-Crp0       CH3_0_Re-Crp0
*                 ------------------------------------------------------------------------------
*                 |                   |                   |                   |
*                 |                   |                   |                   |
*                 |                   |                   |                   |
*                 CH0_0_Im-Crp127     CH0_0_Re-Crp127     CH1_0_Im-Crp127     CH1_0_Re-Crp127
*                 CH2_0_Im-Crp127     CH2_0_Re-Crp127     CH3_0_Im-Crp127     CH3_0_Re-Crp127
*                 ------------------------------------------------------------------------------
*                 CH0_0_Im-Crp127     CH0_0_Re-Crp127     CH1_0_Im-Crp127     CH1_0_Re-Crp127
*                 CH2_0_Im-Crp127     CH2_0_Re-Crp127     CH3_0_Im-Crp127     CH3_0_Re-Crp127
*                 ------------------------------------------------------------------------------
*                 CH0_0_Im-Crp127     CH0_0_Re-Crp127     CH1_0_Im-Crp127     CH1_0_Re-Crp127
*                 CH2_0_Im-Crp127     CH2_0_Re-Crp127     CH3_0_Im-Crp127     CH3_0_Re-Crp127
*                 ------------------------------------------------------------------------------
*                 |                   |                   |                   |
*                 |                   |                   |                   |
*                 |                   |                   |                   |
*                 |                   |                   |                   |
*                 |                   |                   |                   |
*                 |                   |                   |                   |
*                 CH0_127_Im-Crp0     CH0_127_Re-Crp0     CH1_127_Im-Crp0     CH1_127_Re-Crp0
*                 CH2_127_Im-Crp0     CH2_127_Re-Crp0     CH3_127_Im-Crp0     CH3_127_Re-Crp0
*                 ------------------------------------------------------------------------------
*                 CH0_127_Im-Crp0     CH0_127_Re-Crp0     CH1_127_Im-Crp0     CH1_127_Re-Crp0
*                 CH2_127_Im-Crp0     CH2_127_Re-Crp0     CH3_127_Im-Crp0     CH3_127_Re-Crp0
*                 ------------------------------------------------------------------------------
*                 CH0_127_Im-Crp0     CH0_127_Re-Crp0     CH1_127_Im-Crp0     CH1_127_Re-Crp0
*                 CH2_127_Im-Crp0     CH2_127_Re-Crp0     CH3_127_Im-Crp0     CH3_127_Re-Crp0
*                 ------------------------------------------------------------------------------
*                 |                   |                   |                   |
*                 |                   |                   |                   |
*                 |                   |                   |                   |
*                 CH0_127_Im-Crp127   CH0_127_Re-Crp127   CH1_127_Im-Crp127   CH1_127_Re-Crp127
*                 CH2_127_Im-Crp127   CH2_127_Re-Crp127   CH3_127_Im-Crp127   CH3_127_Re-Crp127
*                 ------------------------------------------------------------------------------
*                 CH0_127_Im-Crp127   CH0_127_Re-Crp127   CH1_127_Im-Crp127   CH1_127_Re-Crp127
*                 CH2_127_Im-Crp127   CH2_127_Re-Crp127   CH3_127_Im-Crp127   CH3_127_Re-Crp127
*                 ------------------------------------------------------------------------------
*                 CH0_127_Im-Crp127   CH0_127_Re-Crp127   CH1_127_Im-Crp127   CH1_127_Re-Crp127
*                 CH2_127_Im-Crp127   CH2_127_Re-Crp127   CH3_127_Im-Crp127   CH3_127_Re-Crp127
*                 ------------------------------------------------------------------------------
*
*
```

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

Twiddle coeficients are placed in TRAM at TR_0_0_0 by the Init function.
Window coeficients are placed in TRAM at TR_0_48_0 by the Init function.
MIPI_EVENT_LINE is RFE HW connection dependent. Can be configured from spt_kernel_internals.inc.

Note

The kernel implementation guarantees no saturation for 0 output scaling factor. If other scaling is used, it's the user responsibility to prevent saturation.
This kernel implementation assumes that each ADC buffer is followed by BYTES_CSI2_STATS [80B] reserved for MIPI-CSI2 statistics.
The current implementation is ONLY to be used in combination with 128 Doppler FFT 3TX TD-MIMO.
The input buffer CANNOT be reused for the output buffer.
All buffers MUST be 8 bytes aligned.

3.9.2.103 RsdkSptRange256smp128crp4ch3TxTdMimoAdptv()

```
void * RsdkSptRange256smp128crp4ch3TxTdMimoAdptv (
            void  )
```

Function for Range FFT 256, 4 physical channels/TX slot, 3TX TD-MIMO, Adaptive scaling.

Compute Range FFT and compensation factor list, storing the results in SRAM.

Parameters

in	*WR_1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(sample ADC), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout * SRAM, 16bit Real ADC sample * 63:48 47:32 31:16 15:0 * CH0_0 CH0_1 CH0_2 CH0_3 * CH0_4 CH0_5 CH0_6 CH0_7 * CH1_0 CH1_1 CH1_2 CH1_3 * CH1_4 CH1_5 CH1_6 CH1_7 * CH2_0 CH2_1 CH2_2 CH2_3 * CH2_4 CH2_5 CH2_6 CH2_7 * CH3_0 CH3_1 CH3_2 CH3_3 * CH3_4 CH3_5 CH3_6 CH3_7 * | | | | * | | | | * | | | | * CH0_248 CH0_249 CH0_250 CH0_251 * CH0_252 CH0_253 CH0_254 CH0_255 * CH1_248 CH1_249 CH1_250 CH1_251 * CH1_252 CH1_253 CH1_254 CH1_255 * CH2_248 CH2_249 CH2_250 CH2_251 * CH2_252 CH2_253 CH2_254 CH2_255 * CH3_248 CH3_249 CH3_250 CH3_251 * CH3_252 CH3_253 CH3_254 CH3_255 * ---------- * <RESERVED FOR CSI2 STATS: ADDR LINE 1> * <RESERVED FOR CSI2 STATS: ADDR LINE 2> * | * <RESERVED FOR CSI2 STATS: ADDR LINE 10> * ---------- *

Parameters

out	*WR_2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(Range FFT results), relative to SRAM start address RSDK_SPT_CUBE_BASE_ADDR.

```
*                 OUTPUT Memory layout:
*                 SRAM, 32bit complex(16b_Im + 16b_Re)
*                 63:48              47:32              31:16              15:0
*                 CH0_0_Im-Crp0      CH0_0_Re-Crp0      CH1_0_Im-Crp0      CH1_0_Re-Crp0
*                 CH2_0_Im-Crp0      CH2_0_Re-Crp0      CH3_0_Im-Crp0      CH3_0_Re-Crp0
*                 -----------------------------------------------------------------------
*                 CH0_0_Im-Crp0      CH0_0_Re-Crp0      CH1_0_Im-Crp0      CH1_0_Re-Crp0
*                 CH2_0_Im-Crp0      CH2_0_Re-Crp0      CH3_0_Im-Crp0      CH3_0_Re-Crp0
*                 -----------------------------------------------------------------------
*                 CH0_0_Im-Crp0      CH0_0_Re-Crp0      CH1_0_Im-Crp0      CH1_0_Re-Crp0
*                 CH2_0_Im-Crp0      CH2_0_Re-Crp0      CH3_0_Im-Crp0      CH3_0_Re-Crp0
*                 -----------------------------------------------------------------------
*                 |                  |                  |                  |
*                 |                  |                  |                  |
*                 |                  |                  |                  |
*                 CH0_0_Im-Crp127    CH0_0_Re-Crp127    CH1_0_Im-Crp127    CH1_0_Re-Crp127
*                 CH2_0_Im-Crp127    CH2_0_Re-Crp127    CH3_0_Im-Crp127    CH3_0_Re-Crp127
*                 -----------------------------------------------------------------------
*                 CH0_0_Im-Crp127    CH0_0_Re-Crp127    CH1_0_Im-Crp127    CH1_0_Re-Crp127
*                 CH2_0_Im-Crp127    CH2_0_Re-Crp127    CH3_0_Im-Crp127    CH3_0_Re-Crp127
*                 -----------------------------------------------------------------------
*                 CH0_0_Im-Crp127    CH0_0_Re-Crp127    CH1_0_Im-Crp127    CH1_0_Re-Crp127
*                 CH2_0_Im-Crp127    CH2_0_Re-Crp127    CH3_0_Im-Crp127    CH3_0_Re-Crp127
*                 -----------------------------------------------------------------------
*                 |                  |                  |                  |
*                 |                  |                  |                  |
*                 |                  |                  |                  |
*                 |                  |                  |                  |
*                 |                  |                  |                  |
*                 |                  |                  |                  |
*                 CH0_127_Im-Crp0    CH0_127_Re-Crp0    CH1_127_Im-Crp0    CH1_127_Re-Crp0
*                 CH2_127_Im-Crp0    CH2_127_Re-Crp0    CH3_127_Im-Crp0    CH3_127_Re-Crp0
*                 -----------------------------------------------------------------------
*                 CH0_127_Im-Crp0    CH0_127_Re-Crp0    CH1_127_Im-Crp0    CH1_127_Re-Crp0
*                 CH2_127_Im-Crp0    CH2_127_Re-Crp0    CH3_127_Im-Crp0    CH3_127_Re-Crp0
*                 -----------------------------------------------------------------------
*                 CH0_127_Im-Crp0    CH0_127_Re-Crp0    CH1_127_Im-Crp0    CH1_127_Re-Crp0
*                 CH2_127_Im-Crp0    CH2_127_Re-Crp0    CH3_127_Im-Crp0    CH3_127_Re-Crp0
*                 -----------------------------------------------------------------------
*                 |                  |                  |                  |
*                 |                  |                  |                  |
*                 |                  |                  |                  |
*                 CH0_127_Im-Crp127  CH0_127_Re-Crp127  CH1_127_Im-Crp127  CH1_127_Re-Crp127
*                 CH2_127_Im-Crp127  CH2_127_Re-Crp127  CH3_127_Im-Crp127  CH3_127_Re-Crp127
*                 -----------------------------------------------------------------------
*                 CH0_127_Im-Crp127  CH0_127_Re-Crp127  CH1_127_Im-Crp127  CH1_127_Re-Crp127
*                 CH2_127_Im-Crp127  CH2_127_Re-Crp127  CH3_127_Im-Crp127  CH3_127_Re-Crp127
*                 -----------------------------------------------------------------------
*                 CH0_127_Im-Crp127  CH0_127_Re-Crp127  CH1_127_Im-Crp127  CH1_127_Re-Crp127
*                 CH2_127_Im-Crp127  CH2_127_Re-Crp127  CH3_127_Im-Crp127  CH3_127_Re-Crp127
*                 -----------------------------------------------------------------------
*
```

Parameters

out	*WR↩ _3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(Compensation factors list), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 16bit real * 63:48 47:32 31:16 15:0 * CF0_CH0CH1 CF0_CH2CH3 CF1_CH0CH1 CF1_CH2CH3 * CF2_CH0CH1 CF2_CH2CH3 CF3_CH0CH1 CF3_CH2CH3 * | | | | * CF380_CH0CH1 CF380_CH2CH3 CF381_CH0CH1 CF381_CH2CH3 * CF382_CH0CH1 CF382_CH2CH3 CF383_CH0CH1 CF383_CH2CH3 *

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

Twiddle coeficients are placed in TRAM at TR_0_0_0 by the Init function.
Window coeficients are placed in TRAM at TR_0_48_0 by the Init function.
MIPI_EVENT_LINE is RFE HW connection dependent. Can be configured from spt_kernel_internals.inc.

Note

This kernel implementation assumes that each ADC buffer is followed by BYTES_CSI2_STATS [80B] reserved for MIPI-CSI2 statistics.
The current implementation is ONLY to be used in combination with 128 Doppler FFT 3TX TD-MIMO.
The input buffer CANNOT be reused for the output buffer.
All buffers MUST be 8 bytes aligned.

3.9.2.104 RsdkSptRange256smp128crp8ch6TxTdMimoCp4d()

```
void * RsdkSptRange256smp128crp8ch6TxTdMimoCp4d (
            void  )
```

3.9.2.105 RsdkSptRange256smp256crp4ch()

```
void * RsdkSptRange256smp256crp4ch (
            void  )
```

Function for Range FFT 256, 4 channels(CH#0..CH#3)

Compute Range FFT storing the results in SRAM.

Parameters

in	*WR_1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(sample ADC), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout * SRAM, 16bit Real ADC sample * 63:48 47:32 31:16 15:0 * CH0_0 CH0_1 CH0_2 CH0_3 * CH0_4 CH0_5 CH0_6 CH0_7 * CH1_0 CH1_1 CH1_2 CH1_3 * CH1_4 CH1_5 CH1_6 CH1_7 * CH2_0 CH2_1 CH2_2 CH2_3 * CH2_4 CH2_5 CH2_6 CH2_7 * CH3_0 CH3_1 CH3_2 CH3_3 * CH3_4 CH3_5 CH3_6 CH3_7 * | | | | * | | | | * | | | | * CH0_248 CH0_249 CH0_250 CH0_251 * CH0_252 CH0_253 CH0_254 CH0_255 * CH1_248 CH1_249 CH1_250 CH1_251 * CH1_252 CH1_253 CH1_254 CH1_255 * CH2_248 CH2_249 CH2_250 CH2_251 * CH2_252 CH2_253 CH2_254 CH2_255 * CH3_248 CH3_249 CH3_250 CH3_251 * CH3_252 CH3_253 CH3_254 CH3_255 * ---------- * <RESERVED FOR CSI2 STATS: ADDR LINE 1> * <RESERVED FOR CSI2 STATS: ADDR LINE 2> * | * <RESERVED FOR CSI2 STATS: ADDR LINE 10> * ---------- *
out	*WR_2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(Range FFT results), relative to SRAM start address RSDK_SPT_CUBE_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * CH0_0-Im-Crp0 CH0_0-Re-Crp0 CH1_0-Im-Crp0 CH1_0-Re-Crp0 * CH2_0-Im-Crp0 CH2_0-Re-Crp0 CH3_0_Im-Crp0 CH3_0-Re-Crp0 * | | | | * CH0_0-Im-Crp255 CH0_0-Re-Crp255 CH1_0-Im-Crp255 CH1_0-Re-Crp255 * CH2_0-Im-Crp255 CH2_0-Re-Crp255 CH3_0_Im-Crp255 CH3_0-Re-Crp255 * ---------- * | | | | * | | | | * | | | | * ---------- * CH0_127-Im-Crp0 CH0_127-Re-Crp0 CH1_127-Im-Crp0 CH1_127-Re-Crp0 * CH2_127-Im-Crp0 CH2_127-Re-Crp0 CH3_127_Im-Crp0 CH3_127-Re-Crp0 * | | | | * CH0_127-Im-Crp255 CH0_127-Re-Crp255 CH1_127-Im-Crp255 CH1_127-Re-Crp2 * CH2_127-Im-Crp255 CH2_127-Re-Crp255 CH3_127_Im-Crp255 CH3_127-Re-Crp2 * ---------- *
in	*WR_3*	24bit(RE): Scaling Factor, e.g. 1 for scale left with 1. 24bit(IM): 0.

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

Twiddle coeficients are placed in TRAM at TR_0_0_0 by the Init function.
Window coeficients are placed in TRAM at TR_0_32_0 by the Init function.
MIPI_EVENT_LINE is RFE HW connection dependent. Can be configured from spt_kernel_internals.inc.

Note

The kernel implementation guarantees no saturation for 0 output scaling factor. If other scaling is used, it's the user responsibility to prevent saturation.
This kernel implementation assumes that each ADC buffer is followed by BYTES_CSI2_STATS [80B] reserved for MIPI-CSI2 statistics.
The current implementation is ONLY to be used in combination with 256 Doppler FFT.
The input buffer CANNOT be reused for the output buffer.
All buffers MUST be 8 bytes aligned.

3.9.2.106 RsdkSptRange256smp256crp4ch2TxTdMimoCp4d()

```
void * RsdkSptRange256smp256crp4ch2TxTdMimoCp4d (
            void  )
```

Function for Range FFT 256, 4 physical channels/TX slot, 2TX TD-MIMO.

Compute Range FFT storing compressed results(CP4D) in SRAM.

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(sample ADC), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 16bit Real ADC sample * 63:48 47:32 31:16 15:0 * CH0_0 CH0_1 CH0_2 CH0_3 * CH0_4 CH0_5 CH0_6 CH0_7 * CH1_0 CH1_1 CH1_2 CH1_3 * CH1_4 CH1_5 CH1_6 CH1_7 * CH2_0 CH2_1 CH2_2 CH2_3 * CH2_4 CH2_5 CH2_6 CH2_7 * CH3_0 CH3_1 CH3_2 CH3_3 * CH3_4 CH3_5 CH3_6 CH3_7 * | | | | * | | | | * | | | | * CH0_248 CH0_249 CH0_250 CH0_251 * CH0_252 CH0_253 CH0_254 CH0_255 * CH1_248 CH1_249 CH1_250 CH1_251 * CH1_252 CH1_253 CH1_254 CH1_255 * CH2_248 CH2_249 CH2_250 CH2_251 * CH2_252 CH2_253 CH2_254 CH2_255 * CH3_248 CH3_249 CH3_250 CH3_251 * CH3_252 CH3_253 CH3_254 CH3_255 * ---------- * <RESERVED FOR CSI2 STATS: ADDR LINE 1> * <RESERVED FOR CSI2 STATS: ADDR LINE 2> * | * <RESERVED FOR CSI2 STATS: ADDR LINE 10> * ---------- *
out	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(Range FFT compressed results), relative to SRAM start address RSDK_SPT_CUBE_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 64bit compressed data * 63:0 * CH0_0-CH1_0-CH2_0-CH3_0-TX0_Crp0 * CH0_0-CH1_0-CH2_0-CH3_0-TX1_Crp0 * | * CH0_0-CH1_0-CH2_0-CH3_0-TX0_Crp255 * CH0_0-CH1_0-CH2_0-CH3_0-TX1_Crp255 ---> RangeBin ##0 * ---------- * | * | * | * ---------- * CH0_127-CH1_127-CH2_127-CH3_127-TX0_Crp0 * CH0_127-CH1_127-CH2_127-CH3_127-TX1_Crp0 * | * CH0_127-CH1_127-CH2_127-CH3_127-TX0_Crp255 * CH0_127-CH1_127-CH2_127-CH3_127-TX1_Crp255 ---> RangeBin ##127 * ---------- *
in	*WR↩ _3*	24bit(RE): Scaling Factor, e.g. 1 for scale left with 1. 24bit(IM): 0.

Returns

WR_0: Return Global Max over the range output.

Return values

Return	Global Max amoung the absolute values of the real or imaginary part of the range output. Return value = max(max(abs(Re(k))), max(abs(Im(k)))), 24 bit format(1Q23).

Precondition

Twiddle coeficients are placed in TRAM at TR_0_0_0 by the Init function.
Window coeficients are placed in TRAM at TR_0_32_0 by the Init function.
MIPI_EVENT_LINE is RFE HW connection dependent. Can be configured from spt_kernel_internals.inc.

Note

This kernel implementation assumes that each ADC buffer is followed by BYTES_CSI2_STATS [80B] reserved for MIPI-CSI2 statistics.
The current implementation is ONLY to be used in combination with 256 Doppler FFT 2TX TD-MIMO.
The input buffer CANNOT be reused for the output buffer.
All buffers MUST be 8 bytes aligned.

3.9.2.107 RsdkSptRange256smp256crp4chAdptv()

```
void * RsdkSptRange256smp256crp4chAdptv (
            void  )
```

Function for Range FFT 256, 4 channels(CH#0..CH#3), Adaptive scaling.

Compute Range FFT and compensation factor list, storing the results in SRAM.

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(sample ADC), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout * SRAM, 16bit Real ADC sample * 63:48 47:32 31:16 15:0 * CH0_0 CH0_1 CH0_2 CH0_3 * CH0_4 CH0_5 CH0_6 CH0_7 * CH1_0 CH1_1 CH1_2 CH1_3 * CH1_4 CH1_5 CH1_6 CH1_7 * CH2_0 CH2_1 CH2_2 CH2_3 * CH2_4 CH2_5 CH2_6 CH2_7 * CH3_0 CH3_1 CH3_2 CH3_3 * CH3_4 CH3_5 CH3_6 CH3_7 * | | | | * | | | | * | | | | * CH0_248 CH0_249 CH0_250 CH0_251 * CH0_252 CH0_253 CH0_254 CH0_255 * CH1_248 CH1_249 CH1_250 CH1_251 * CH1_252 CH1_253 CH1_254 CH1_255 * CH2_248 CH2_249 CH2_250 CH2_251 * CH2_252 CH2_253 CH2_254 CH2_255 * CH3_248 CH3_249 CH3_250 CH3_251 * CH3_252 CH3_253 CH3_254 CH3_255 * ---------- * <RESERVED FOR CSI2 STATS: ADDR LINE 1> * <RESERVED FOR CSI2 STATS: ADDR LINE 2> * | * <RESERVED FOR CSI2 STATS: ADDR LINE 10> * ---------- *
out	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(Range FFT results), relative to SRAM start address RSDK_SPT_CUBE_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * CH0_0-Im-Crp0 CH0_0-Re-Crp0 CH1_0-Im-Crp0 CH1_0-Re-Crp0 * CH2_0-Im-Crp0 CH2_0-Re-Crp0 CH3_0_Im-Crp0 CH3_0-Re-Crp0 * | | | | * CH0_0-Im-Crp255 CH0_0-Re-Crp255 CH1_0-Im-Crp255 CH1_0-Re-Crp255 * CH2_0-Im-Crp255 CH2_0-Re-Crp255 CH3_0_Im-Crp255 CH3_0-Re-Crp255 * ---------- * | | | | * | | | | * | | | | * ---------- * CH0_127-Im-Crp0 CH0_127-Re-Crp0 CH1_127-Im-Crp0 CH1_127-Re-Crp0 * CH2_127-Im-Crp0 CH2_127-Re-Crp0 CH3_127_Im-Crp0 CH3_127-Re-Crp0 * | | | | * CH0_127-Im-Crp255 CH0_127-Re-Crp255 CH1_127-Im-Crp255 CH1_127-Re-Crp2 * CH2_127-Im-Crp255 CH2_127-Re-Crp255 CH3_127_Im-Crp255 CH3_127-Re-Crp2 * ---------- *

Parameters

out	*WR↩ _3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(Compensation factors list), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 16bit real * 63:48 47:32 31:16 15:0 * CF0_CH0CH1 CF1_CH0CH1 CF2_CH0CH1 CF3_CH0CH1 * | | | | * CF252_CH0CH1 CF253_CH0CH1 CF254_CH0CH1 CF255_CH0CH1 * CF0_CH2CH3 CF1_CH2CH3 CF2_CH2CH3 CF3_CH2CH3 * | | | | * CF252_CH2CH3 CF253_CH2CH3 CF254_CH2CH3 CF255_CH2CH3 *

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

Twiddle coeficients are placed in TRAM at TR_0_0_0 by the Init function.
Window coeficients are placed in TRAM at TR_0_32_0 by the Init function.
MIPI_EVENT_LINE is RFE HW connection dependent. Can be configured from spt_kernel_internals.inc.

Note

This kernel implementation assumes that each ADC buffer is followed by BYTES_CSI2_STATS [80B] reserved for MIPI-CSI2 statistics.
The current implementation is ONLY to be used in combination with 256 Doppler FFT.
The input buffer CANNOT be reused for the output buffer.
All buffers MUST be 8 bytes aligned.

3.9.2.108 RsdkSptRange256smp256crp8ch()

```
void * RsdkSptRange256smp256crp8ch (
            void  )
```

3.9.2.109 RsdkSptRange256smp256crp8ch2TxTdMimoCp4d()

```
void * RsdkSptRange256smp256crp8ch2TxTdMimoCp4d (
            void  )
```

3.9.2.110 RsdkSptRange256smp256crp8chAdptv()

```
void * RsdkSptRange256smp256crp8chAdptv (
            void  )
```

3.9.2.111 RsdkSptRange512smp128crp4ch()

```
void * RsdkSptRange512smp128crp4ch (
            void  )
```

Function for Range FFT 512, 4 channels(CH#0..CH#3)

Compute Range FFT storing the results in SRAM.

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the first INPUT buffer (sample ADC, CH#0..CH#3), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR.

```
*                  INPUT Memory layout:
*                  SRAM, 16bit Real ADC sample
*                  63:48     47:32     31:16     15:0
*                  CH0_0     CH0_1     CH0_2     CH0_3
*                  CH0_4     CH0_5     CH0_6     CH0_7
*                  CH1_0     CH1_1     CH1_2     CH1_3
*                  CH1_4     CH1_5     CH1_6     CH1_7
*                  CH2_0     CH2_1     CH2_2     CH2_3
*                  CH2_4     CH2_5     CH2_6     CH2_7
*                  CH3_0     CH3_1     CH3_2     CH3_3
*                  CH3_4     CH3_5     CH3_6     CH3_7
*                  |         |         |         |
*                  |         |         |         |
*                  |         |         |         |
*                  CH0_504   CH0_505   CH0_506   CH0_507
*                  CH0_508   CH0_509   CH0_510   CH0_511
*                  CH1_504   CH1_505   CH1_506   CH1_507
*                  CH1_508   CH1_509   CH1_510   CH1_511
*                  CH2_504   CH2_505   CH2_506   CH2_507
*                  CH2_508   CH2_509   CH2_510   CH2_511
*                  CH3_504   CH3_505   CH3_506   CH3_507
*                  CH3_508   CH3_509   CH3_510   CH3_511
*                  ---------------------------------------
*                  <RESERVED FOR CSI2 STATS: ADDR LINE 1>
*                  <RESERVED FOR CSI2 STATS: ADDR LINE 2>
*                  |
*                  <RESERVED FOR CSI2 STATS: ADDR LINE 10>
*                  ---------------------------------------
*
```

Parameters

out	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer (Range FFT results), relative to SRAM start address RSDK_SPT_CUBE_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * CH0_0-Im-Crp0 CH0_0-Re-Crp0 CH1_0-Im-Crp0 CH1_0-Re-Crp0 * CH2_0-Im-Crp0 CH2_0-Re-Crp0 CH3_0-Im-Crp0 CH3_0-Re-Crp0 * | | | | * CH0_0-Im-Crp127 CH0_0-Re-Crp127 CH1_0-Im-Crp127 CH1_0-Re-Crp127 * CH2_0-Im-Crp127 CH2_0-Re-Crp127 CH3_0-Im-Crp127 CH3_0-Re-Crp127 * ---------- * | | | | * | | | | * | | | | * ---------- * CH0_255-Im-Crp0 CH0_255-Re-Crp0 CH1_255-Im-Crp0 CH1_255-Re-Crp0 * CH2_255-Im-Crp0 CH2_255-Re-Crp0 CH3_255_Im-Crp0 CH3_255-Re-Crp0 * | | | | * CH0_255-Im-Crp127 CH0_255-Re-Crp127 CH1_255-Im-Crp127 CH1_255-Re-Crp1 * CH2_255-Im-Crp127 CH2_255-Re-Crp127 CH3_255_Im-Crp127 CH3_255-Re-Crp1 * ---------- *
in	*WR↩ _3*	24bit(RE): Scaling Factor, e.g. 1 for scale left with 1. 24bit(IM): 0.

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

Twiddle coeficients are placed in TRAM at TR_0_0_0 by the Init function.
Window coeficients are placed in TRAM at TR_0_80_0 by the Init function.
MIPI_EVENT_LINE is RFE HW connection dependent. Can be configured from spt_kernel_internals.inc.

Note

The kernel implementation guarantees no saturation for 0 output scaling factor. If other scaling is used, it's the user responsibility to prevent saturation.
This kernel implementation assumes that each ADC buffer is followed by BYTES_CSI2_STATS [80B] reserved for MIPI-CSI2 statistics.
The current implementation is ONLY to be used in combination with 128 Doppler FFT.
The input buffer CANNOT be reused for the output buffer.
All buffers MUST be 8 bytes aligned.

3.9.2.112 RsdkSptRange512smp128crp4chAdptv()

```
void * RsdkSptRange512smp128crp4chAdptv (
            void  )
```

Function for Range FFT 512, 4 channels(CH#0..CH#3), Adaptive scaling.

Compute Range FFT and compensation factor list, storing results in SRAM.

Parameters

in	*WR↩_1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(sample ADC), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 16bit Real ADC sample * 63:48 47:32 31:16 15:0 * CH0_0 CH0_1 CH0_2 CH0_3 * CH0_4 CH0_5 CH0_6 CH0_7 * CH1_0 CH1_1 CH1_2 CH1_3 * CH1_4 CH1_5 CH1_6 CH1_7 * CH2_0 CH2_1 CH2_2 CH2_3 * CH2_4 CH2_5 CH2_6 CH2_7 * CH3_0 CH3_1 CH3_2 CH3_3 * CH3_4 CH3_5 CH3_6 CH3_7 * | | | | * | | | | * | | | | * CH0_504 CH0_505 CH0_506 CH0_507 * CH0_508 CH0_509 CH0_510 CH0_511 * CH1_504 CH1_505 CH1_506 CH1_507 * CH1_508 CH1_509 CH1_510 CH1_511 * CH2_504 CH2_505 CH2_506 CH2_507 * CH2_508 CH2_509 CH2_510 CH2_511 * CH3_504 CH3_505 CH3_506 CH3_507 * CH3_508 CH3_509 CH3_510 CH3_511 * ---------- * <RESERVED FOR CSI2 STATS: ADDR LINE 1> * <RESERVED FOR CSI2 STATS: ADDR LINE 2> * | * <RESERVED FOR CSI2 STATS: ADDR LINE 10> * ---------- *
out	*WR↩_2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(Range FFT results), relative to SRAM start address RSDK_SPT_CUBE_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * CH0_0-Im-Crp0 CH0_0-Re-Crp0 CH1_0-Im-Crp0 CH1_0-Re-Crp0 * CH2_0-Im-Crp0 CH2_0-Re-Crp0 CH3_0_Im-Crp0 CH3_0-Re-Crp0 * | | | | * CH0_0-Im-Crp127 CH0_0-Re-Crp127 CH1_0-Im-Crp127 CH1_0-Re-Crp127 * CH2_0-Im-Crp127 CH2_0-Re-Crp127 CH3_0_Im-Crp127 CH3_0-Re-Crp127 * ---------- * | | | | * | | | | * | | | | * ---------- * CH0_255-Im-Crp0 CH0_255-Re-Crp0 CH1_255-Im-Crp0 CH1_255-Re-Crp0 * CH2_255-Im-Crp0 CH2_255-Re-Crp0 CH3_255_Im-Crp0 CH3_255-Re-Crp0 * | | | | * CH0_255-Im-Crp127 CH0_255-Re-Crp127 CH1_255-Im-Crp127 CH1_255-Re-Crp1 * CH2_255-Im-Crp127 CH2_255-Re-Crp127 CH3_255_Im-Crp127 CH3_255-Re-Crp1 * ---------- *

Parameters

out	*WR_3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(Compensation factors list), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 16bit real * 63:48 47:32 31:16 15:0 * CF0_CH0CH1 CF1_CH0CH1 CF2_CH0CH1 CF3_CH0CH1 * | | | | * CF124_CH0CH1 CF125_CH0CH1 CF126_CH0CH1 CF127_CH0CH1 * CF0_CH2CH3 CF1_CH2CH3 CF2_CH2CH3 CF3_CH2CH3 * | | | | * CF124_CH2CH3 CF125_CH2CH3 CF126_CH2CH3 CF127_CH2CH3 *

Returns

WR_0: No value returned.

Return values

No	value returned.

Precondition

Twiddle coeficients are placed in TRAM at TR_0_0_0 by the Init function.
Window coeficients are placed in TRAM at TR_0_80_0 by the Init function.
MIPI_EVENT_LINE is RFE HW connection dependent. Can be configured from spt_kernel_internals.inc.

Note

This kernel implementation assumes that each ADC buffer is followed by BYTES_CSI2_STATS [80B] reserved for MIPI-CSI2 statistics.
The current implementation is ONLY to be used in combination with 128 Doppler FFT.
The input buffer CANNOT be reused for the output buffer.
All buffers MUST be 8 bytes aligned.

3.9.2.113 RsdkSptRange512smp128crp8ch()

```
void * RsdkSptRange512smp128crp8ch (
            void  )
```

3.9.2.114 RsdkSptRange512smp128crp8chAdptv()

```
void * RsdkSptRange512smp128crp8chAdptv (
            void  )
```

3.9.2.115 RsdkSptRfBist128smp1crp4ch()

```
void * RsdkSptRfBist128smp1crp4ch (
            void  )
```

Function for RF Build-in Self Test 128 samples, 1 chirp, 4 channels(CH#0..CH#3). Offline version only.

Compute FFT on ADC samples, amplitude and max amplitude on each RX channel, storing results in SRAM.

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT128 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For twiddle generation algorithm, see S32R41_RM 68.9.5.3 / SAF85xx_RM 68.9.5.3. * INPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw1_Im tw1_Re tw1_Im tw1_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * tw2_Im tw2_Re tw2_Im tw2_Re * | | | | * | | | | * | | | | * tw16_Im tw16_Re tw16_Im tw16_Re * tw16_Im tw16_Re tw16_Im tw16_Re * tw16_Im tw16_Re tw16_Im tw16_Re * tw16_Im tw16_Re tw16_Im tw16_Re *
in	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(sample ADC), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * INPUT Memory layout: * SRAM, 16bit Real ADC sample * 63:48 47:32 31:16 15:0 * CH0_0 CH0_1 CH0_2 CH0_3 * CH0_4 CH0_5 CH0_6 CH0_7 * CH1_0 CH1_1 CH1_2 CH1_3 * CH1_4 CH1_5 CH1_6 CH1_7 * CH2_0 CH2_1 CH2_2 CH2_3 * CH2_4 CH2_5 CH2_6 CH2_7 * CH3_0 CH3_1 CH3_2 CH3_3 * CH3_4 CH3_5 CH3_6 CH3_7 * | | | | * CH0_120 CH0_121 CH0_122 CH0_123 * CH0_124 CH0_125 CH0_126 CH0_127 * CH1_120 CH1_121 CH1_122 CH1_123 * CH1_124 CH1_125 CH1_126 CH1_127 * CH2_120 CH2_121 CH2_122 CH2_123 * CH2_124 CH2_125 CH2_126 CH2_127 * CH3_120 CH3_121 CH3_122 CH3_123 * CH3_124 CH3_125 CH3_126 CH3_127 *

Parameters

out	*WR_3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(FFT results), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * CH0_0_Im CH0_0_Re CH1_0_Im CH1_0_Re * CH2_0_Im CH2_0_Re CH3_0_Im CH3_0_Re * CH0_1_Im CH0_1_Re CH1_1_Im CH1_1_Re * CH2_1_Im CH2_1_Re CH3_1_Im CH3_1_Re * | | | | * | | | | * | | | | * CH0_62_Im CH0_62_Re CH1_62_Im CH1_62_Re * CH2_62_Im CH2_62_Re CH3_62_Im CH3_62_Re * CH0_63_Im CH0_63_Re CH1_63_Im CH1_63_Re * CH2_63_Im CH2_63_Re CH3_63_Im CH3_63_Re *
out	*WR_4*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(Mag2+Log2 results), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 16bit real * 63:48 47:32 31:16 15:0 * CH0_0_Mag CH1_0_Mag CH2_0_Mag CH3_0_Mag * CH0_1_Mag CH1_1_Mag CH2_1_Mag CH3_1_Mag * | | | | * | | | | * | | | | * CH0_62_Mag CH1_62_Mag CH2_62_Mag CH3_62_Mag * CH0_63_Mag CH1_63_Mag CH2_63_Mag CH3_63_Mag *
in	*WR_5*	24bit(RE): Scaling Factor, e.g. 1 for scale left with 1. 24bit(IM): 0.

Returns

WR_0: Global maxima(amplitude) for channel CH#0.

Return values

Will	indicate the Mag2+Log2 value of the global maxima for the channel CH#0.

Precondition

MIPI_EVENT_LINE is RFE HW connection dependent. Can be configured from spt_kernel_internals.inc.

Note

The kernel implementation guarantees no saturation for 0 output scaling factor. If other scaling is used, it's the user responsibility to prevent saturation.
All buffers MUST be 8 bytes aligned.

3.9.2.116 RsdkSptRfBist2048smp1crp4ch()

```
void * RsdkSptRfBist2048smp1crp4ch (
            void  )
```

Function for RF Build-in Self Test 2048 samples, 1 chirp, 4 channels(CH_0..CH_3). Offline version only.

Compute FFT on ADC samples, amplitude and max amplitude on each RX channel, storing results in SRAM.

Parameters

in	*WR↩ _1*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(FFT2048 Twiddle Factors), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. For twiddle generation algorithm, see S32R41_RM 68.9.5.3 / SAF85xx_RM 68.9.5.3.
in	*WR↩ _2*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the INPUT buffer(sample ADC), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR.

```
*                  INPUT Memory layout:
*                  SRAM, 32bit complex(16b_Im + 16b_Re)
*                  63:48      47:32      31:16      15:0
*                  tw1_Im     tw1_Re     tw1_Im     tw1_Re
*                  tw1_Im     tw1_Re     tw1_Im     tw1_Re
*                  tw1_Im     tw1_Re     tw1_Im     tw1_Re
*                  tw1_Im     tw1_Re     tw1_Im     tw1_Re
*                  tw2_Im     tw2_Re     tw2_Im     tw2_Re
*                  tw2_Im     tw2_Re     tw2_Im     tw2_Re
*                  tw2_Im     tw2_Re     tw2_Im     tw2_Re
*                  tw2_Im     tw2_Re     tw2_Im     tw2_Re
*                  |          |          |          |
*                  |          |          |          |
*                  |          |          |          |
*                  tw256_Im   tw256_Re   tw265_Im   tw256_Re
*                  tw256_Im   tw256_Re   tw265_Im   tw256_Re
*                  tw256_Im   tw256_Re   tw265_Im   tw256_Re
*                  tw256_Im   tw256_Re   tw265_Im   tw256_Re
*
```

```
*                  INPUT Memory layout:
*                  SRAM, 16bit Real ADC sample
*                  63:48      47:32      31:16      15:0
*                  CH0_0      CH0_1      CH0_2      CH0_3
*                  CH0_4      CH0_5      CH0_6      CH0_7
*                  CH1_0      CH1_1      CH1_2      CH1_3
*                  CH1_4      CH1_5      CH1_6      CH1_7
*                  CH2_0      CH2_1      CH2_2      CH2_3
*                  CH2_4      CH2_5      CH2_6      CH2_7
*                  CH3_0      CH3_1      CH3_2      CH3_3
*                  CH3_4      CH3_5      CH3_6      CH3_7
*                  |          |          |          |
*                  CH0_2040   CH0_2041   CH0_2042   CH0_2043
*                  CH0_2044   CH0_2045   CH0_2046   CH0_2047
*                  CH1_2040   CH1_2041   CH1_2042   CH1_2043
*                  CH1_2044   CH1_2045   CH1_2046   CH1_2047
*                  CH2_2040   CH2_2041   CH2_2042   CH2_2043
*                  CH2_2044   CH2_2045   CH2_2046   CH2_2047
*                  CH3_2040   CH3_2041   CH3_2042   CH3_2043
*                  CH3_2044   CH3_2045   CH3_2046   CH3_2047
*
```

Parameters

out	*WR↩ _3*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(FFT results), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 32bit complex(16b_Im + 16b_Re) * 63:48 47:32 31:16 15:0 * CH0_0_Im CH0_0_Re CH1_0_Im CH1_0_Re * CH2_0_Im CH2_0_Re CH3_0_Im CH3_0_Re * CH0_1_Im CH0_1_Re CH1_1_Im CH1_1_Re * CH2_1_Im CH2_1_Re CH3_1_Im CH3_1_Re * | | | | * | | | | * | | | | * CH0_1022_Im CH0_1022_Re CH1_1022_Im CH1_1022_Re * CH2_1022_Im CH2_1022_Re CH3_1022_Im CH3_1022_Re * CH0_1023_Im CH0_1023_Re CH1_1023_Im CH1_1023_Re * CH2_1023_Im CH2_1023_Re CH3_1023_Im CH3_1023_Re *
out	*WR↩ _4*	24bit(RE): 0 24bit(IM): Offset(restricted to 24bit value!) of the OUTPUT buffer(Mag2+Log2 results), relative to SRAM start address RSDK_SPT_OTHER_BASE_ADDR. * OUTPUT Memory layout: * SRAM, 16bit real * 63:48 47:32 31:16 15:0 * CH0_0_Mag CH1_0_Mag CH2_0_Mag CH3_0_Mag * CH0_1_Mag CH1_1_Mag CH2_1_Mag CH3_1_Mag * | | | | * | | | | * | | | | * CH0_1022_Mag CH1_1022_Mag CH2_1022_Mag CH3_1022_Mag * CH0_1023_Mag CH1_1023_Mag CH2_1023_Mag CH3_1023_Mag *
in	*WR↩ _5*	24bit(RE): Scaling Factor, e.g. 1 for scale left with 1. 24bit(IM): 0.

Returns

WR_0: Global maxima(amplitude) for channel CH#0.

Return values

Will	indicate the Mag2+Log2 value of the global maxima for the channel CH#0.

Precondition

MIPI_EVENT_LINE is RFE HW connection dependent. Can be configured from spt_kernel_internals.inc.

Note

The kernel implementation guarantees no saturation for 0 output scaling factor. If other scaling is used, it's the user responsibility to prevent saturation.
All buffers MUST be 8 bytes aligned.

3.10 SPT Error codes

Enumerations

- enum Spt_ErrStatusType {
  SPT_E_INVALID_PARAM = RSDK_SPT_RET_ERR_INVALID_PARAM - RSDK_SPT_STATUS_BASE ,
  SPT_E_INVALID_STATE , SPT_E_TIMEOUT_BLOCK , SPT_E_WARN_HW_BUSY ,
  SPT_E_MEM , SPT_E_DMA , SPT_E_HW_ACC , SPT_E_ILLOP ,
  SPT_E_TIMEOUT_START , SPT_E_TIMEOUT_STOP , SPT_E_WARN_UNEXPECTED_STOP ,
  SPT_E_HIST_OVF0 ,
  SPT_E_HIST_OVF1 , SPT_E_IRQ_REG , SPT_E_UNMAP_SPT_MEM , SPT_E_WR ,
  SPT_E_WARN_DRV_BUSY , SPT_E_API_INIT_LOCK_FAIL , SPT_E_API_ENTER_LOCK_FAIL ,
  SPT_E_API_ENTER_UNLOCK_FAIL ,
  SPT_E_API_EXIT_LOCK_FAIL , SPT_E_API_EXIT_UNLOCK_FAIL , SPT_E_THR_CREATE , SPT_E_THR_TERM
  ,
  SPT_E_OAL_COMM_INIT , SPT_E_CHECK_WATERMARK , SPT_E_INIT_Q_FAIL , SPT_E_BBE32_REBOOT
  ,
  SPT_E_INVALID_KERNEL , SPT_E_HW_RST , SPT_E_OTHER = 0xFFU }

  Return datatype for SPT API calls.

3.10.1 Detailed Description

3.10.2 Enumeration Type Documentation

3.10.2.1 Spt_ErrStatusType

```
enum Spt_ErrStatusType
```

Return datatype for SPT API calls.

API functions either return a Spt_ErrStatusType value or void. These error codes are derived from rsdkStatus_t.

Enumerator

SPT_E_INVALID_PARAM	Driver API error: Parameter value or combination of values not supported.
SPT_E_INVALID_STATE	Driver API error: operation not supported in current state.
SPT_E_TIMEOUT_BLOCK	Driver error: SPT blocking call did not finish in due time and did not report any specific error. This is treated as a faulty state and imposes re-initialization of the driver.
SPT_E_WARN_HW_BUSY	Driver API warning: Input parameters are valid but no action was done by current function call because the SPT hardware is busy.
SPT_E_MEM	SPT hw error: internal memory handling, triggers an ECS interrupt. MEM_ERR_STATUS register value is passed to the user callback (rsdkSptIsrCb_t)
SPT_E_DMA	SPT hw error: SDMA operation, triggers an ECS interrupt. GBL_STATUS register value is passed to the user callback (rsdkSptIsrCb_t)

Enumerator

SPT_E_HW_ACC	SPT hw error: tried to execute an instruction with illegal operands or configuration, triggered an ECS interrupt. HW_ACC_ERR_STATUS register value is passed to the user callback (rsdkSptIsrCb_t)
SPT_E_ILLOP	SPT hw error: Illegal SPT instruction, triggers an ECS interrupt. CS_STATUS1 register value is passed to the user callback (rsdkSptIsrCb_t)
SPT_E_TIMEOUT_START	Driver error: SPT kernel launcher function was not able to verify that all SPT hw status bits were properly cleared. This is treated as a faulty state and imposes re-initialization of the driver
SPT_E_TIMEOUT_STOP	Driver error: RsdkSptStop function was not able to verify that SPT hw state machine has been reset to "START" state. This signals a hardware fault.
SPT_E_WARN_UNEXPECTED_STOP	Driver API warning: Spurious SPT_IRQ_ECS interrupt was triggered by the CS_STATUS0[PS_STOP] bit, while the SPT command sequencer did not reach a STOP instruction, or the SPT Driver had not launched any kernel.
SPT_E_HIST_OVF0	SPT hw error: HIST overflow. HIST_OVF_STATUS0 register value is passed to the user callback (rsdkSptIsrCb_t)
SPT_E_HIST_OVF1	SPT hw error: HIST overflow. HIST_OVF_STATUS1 register value is passed to the user callback (rsdkSptIsrCb_t)
SPT_E_IRQ_REG	SPT error: the irq handler was not registered.
SPT_E_UNMAP_SPT_MEM	Driver error: Failed to unmap SPT registers' addresses.
SPT_E_WR	SPT hw error: WR or SPR access error. WR_ACCESS_ERR_REG register value is passed to the user callback (rsdkSptIsrCb_t)
SPT_E_WARN_DRV_BUSY	Driver API warning: an API call is already in progress on another thread
SPT_E_API_INIT_LOCK_FAIL	Driver API error: could not init mutex for controlling multithreaded API call sequence
SPT_E_API_ENTER_LOCK_FAIL	Driver API error: could not lock mutex for controlling multithreaded API call sequence
SPT_E_API_ENTER_UNLOCK_FAIL	Driver API error: could not unlock mutex for controlling multithreaded API call sequence
SPT_E_API_EXIT_LOCK_FAIL	Driver API error: could not lock mutex for controlling multithreaded API call sequence
SPT_E_API_EXIT_UNLOCK_FAIL	Driver API error: could not unlock mutex for controlling multithreaded API call sequence
SPT_E_THR_CREATE	Driver API error: could not create a thread for detecting OS kernel event in user space
SPT_E_THR_TERM	Driver API error: could not terminate the thread used for detecting OS kernel event in user space
SPT_E_OAL_COMM_INIT	Driver API error: could not initialize OAL communication channel for transmitting OS kernel events to user space
SPT_E_CHECK_WATERMARK	Driver error: Failed to check if the watermark instruction is placed at the start of the kernel code.
SPT_E_INIT_Q_FAIL	Driver error: Failed to init the queue used to handle irq data processing on separate thread.
SPT_E_BBE32_REBOOT	Driver API error: could not reboot the BBE32
SPT_E_INVALID_KERNEL	Driver error: detected invalid SPT kernel code, which does not start with the mandatory watermarking instruction. See also SPT_KERNEL_WATERMARK
SPT_E_HW_RST	SPT error: hardware is in unexpected RST state

Enumerator

SPT_E_OTHER	Any other return status not covered above. No SPT error codes should be defined with a value greater than this one.

Chapter 4

Data Structure Documentation

4.1 Spt_DriverCmdResType Struct Reference

Typedef for returning information from Spt_Command(); to be used as argument to the function.

`#include <Spt_Types.h>`

Data Fields

- uint8 bytes [SPT_CMD_RESULT_SIZE]

4.1.1 Detailed Description

Typedef for returning information from Spt_Command(); to be used as argument to the function.

This structure is only used by Spt_Command(). Must be allocated by the caller. Its content is modified by the SPT Driver.

4.1.2 Field Documentation

4.1.2.1 bytes

`uint8 bytes[SPT_CMD_RESULT_SIZE]`

4.2 Spt_DriverCommandType Struct Reference

Structure containing the ID of the command to be served and additional parameters that may be needed for Spt_Command().

`#include <Spt_Types.h>`

Data Fields

- Spt_DriverCommandIdType cmdId
- Spt_DriverCommandParamType cmdParam

4.2.1 Detailed Description

Structure containing the ID of the command to be served and additional parameters that may be needed for Spt_Command().

4.2.2 Field Documentation

4.2.2.1 cmdId

```
Spt_DriverCommandIdType cmdId
```

Command ID

4.2.2.2 cmdParam

```
Spt_DriverCommandParamType cmdParam
```

Additional command parameter

4.3 Spt_DriverContextType Struct Reference

Contains runtime parameters for the SPT Driver.

```
#include <Spt_Types.h>
```

Data Fields

- Spt_DriverOpModeType opMode
- Spt_IsrCbType ecsIsrCb
- Spt_IsrCbType evtIsrCb
- uintptr_t kernelCodeAddr
- Spt_DrvParamType kernelParList [SPT_MAX_N_PAR]
- volatile sint32 * kernelRetPar
- uint8 checkKernelWatermark

4.3.1 Detailed Description

Contains runtime parameters for the SPT Driver.

This structure is only used by Spt_Run(). Must be allocated and initialized by the caller. Its content is not modified by the SPT Driver. Some fields are used for implementation of the SPT calling convention (see spt_call_conv for details). The intended direction of data flow is specified in the description of each field.

4.3.2 Field Documentation

4.3.2.1 checkKernelWatermark

```
uint8 checkKernelWatermark
```

Check if the watermark instruction exists in the SPT kernel at kernelCodeAddr address, to guard against passing invalid pointers to the SPT. Valid values: 0=disable, 1=enable. If this option is enabled in the SPT Driver, then all SPT kernels must start with the predefined instruction "set #SPT_KERNEL_WATERMARK, WR_0", otherwise the driver will not launch them.

4.3.2.2 ecsIsrCb

```
Spt_IsrCbType ecsIsrCb
```

[in] Callback function to be called from the SPT ECS interrupt handler. If no callback is required then it can be initialized to NULL.

4.3.2.3 evtIsrCb

```
Spt_IsrCbType evtIsrCb
```

[in] Callback function to be called from the SPT EVT interrupt handler. If no callback is required then it can be initialized to NULL.

4.3.2.4 kernelCodeAddr

```
uintptr_t kernelCodeAddr
```

[in] Starting address of the SPT executable code, must be aligned to an SPT_CODE_ADDR_ALIGN_BYTES boundary

4.3.2.5 kernelParList

```
Spt_DrvParamType kernelParList[SPT_MAX_N_PAR]
```

[in] List of parameters to be passed to the SPT kernel.

4.3.2.6 kernelRetPar

```
volatile sint32* kernelRetPar
```

[out] Return parameter from the SPT kernel. The SPT driver creates this 32-bit value by concatenating8 bits from WR0_IM with 24 bits from WR0_RE. This pointer must be initialized with a global address which is visible in the contexts of Spt_Run() and SPT interrupts. If no return parameter is expected from the SPT kernel, then it can be initialized to NULL.

4.3.2.7 opMode

```
Spt_DriverOpModeType opMode
```

[in] Operating mode for Spt_Run() .

4.4 Spt_DriverInitPlatSpecificType Struct Reference

Init parameters which are specific to the hardware platform.

```
#include <Spt_Types.h>
```

Data Fields

- uint8 dspEn
- Spt_IsrCbType dspIsrCb
- void(* dspBootloaderCb)(void)

4.4.1 Detailed Description

Init parameters which are specific to the hardware platform.

4.4.2 Field Documentation

4.4.2.1 dspBootloaderCb

```
void(* dspBootloaderCb) (void)
```

[in] External function called for loading the DSP boot image into memory. This functions is called by the driver in the middle of the boot procedure, because the DSP internal memory (IRAM, DRAM) can only be accessed after releasing the DSP from reset. This callback is required when dspEn != 0.

4.4.2.2 dspEn

```
uint8 dspEn
```

Enable the BBE32 DSP to boot-up and start running the "DSP Dispatcher": 0=disable, "non-0"=enable.

4.4.2.3 dspIsrCb

```
Spt_IsrCbType dspIsrCb
```

[in] Callback function to be called from the SPT DSP interrupt handler. Used to pass BBE32 status & error information to the user application. If no callback is required then it can be initialized to NULL.

4.5 Spt_DriverInitType Struct Reference

Contains initialization parameters for the SPT Driver.

```
#include <Spt_Types.h>
```

Data Fields

- Spt_DriverInitPlatSpecificType hwPlatSpec

4.5.1 Detailed Description

Contains initialization parameters for the SPT Driver.

This structure is only used by Spt_Setup(). Must be allocated and initialized by the caller. Its content is not modified by the SPT Driver.

4.5.2 Field Documentation

4.5.2.1 hwPlatSpec

```
Spt_DriverInitPlatSpecificType hwPlatSpec
```

Init parameters which are specific to a particular hardware platform.

4.6 Spt_DrvParamType Struct Reference

Datatype describing the parameters to be passed to the SPT kernel. See spt_call_conv for details.

```
#include <Spt_Types.h>
```

Data Fields

- Spt_ParamType paramType
- uintptr_t paramValue

4.6.1 Detailed Description

Datatype describing the parameters to be passed to the SPT kernel. See spt_call_conv for details.

Used to initialize Spt_DriverContextType::kernelParList with information for the SPT kernels.

4.6.2 Field Documentation

4.6.2.1 paramType

```
Spt_ParamType paramType
```

Indicates the intended usage of paramValue as an immediate value or a memory address

4.6.2.2 paramValue

```
uintptr_t paramValue
```

Can contain either an immediate integer value or a memory address

4.7 Spt_DspCmdType Struct Reference

This structure packs the information needed to call a function on the BBE32 .

```
#include <Spt_Types.h>
```

Data Fields

- uint8 id
- uint32 arg
- uint8 crc

4.7.1 Detailed Description

This structure packs the information needed to call a function on the BBE32 .

4.7.2 Field Documentation

4.7.2.1 arg

```
uint32 arg
```

Command argument. Can be either an immediate value or a 32-bit pointer to a command-specific data structure.

4.7.2.2 crc

```
uint8 crc
```

CRC checksum of the id and arg fields, used to verify command integrity on BBE32 side

4.7.2.3 id

```
uint8 id
```

Command ID returned from RSDK_DSP_GET_FUNC_ID(function_name)

Index

How to Reach Us:

Home Page:
nxp.com

Web Support:
nxp.com/support